

角色动画与运动仿真 课程笔记

AutoPku

2026-06-04

Contents

课程概览	6
课程脉络	7
知识图谱（全课程）	9
各讲速览	10
全局术语中英对照	11
Lecture 01 — 导论 (Introduction)	14
0. 名词热身：读本讲前需要知道的 6 个词	14
1. 3D 计算机动画的全景	14
2. 为什么专门研究 Character Animation	16
3. 人体运动的生理链条——方法论的起点	16
4. 方法学坐标系：三层控制层级	17
5. Kinematic（运动学）方法谱系	18
6. Physics-based（基于物理）方法谱系	19
7. 两类方法的全景对比	23
8. 课程路线图	23
9. Worked Examples 例题	24
10. Anki 记忆卡片	25
11. 中英术语速查表	26
Lecture 02 — 数学基础 (Math Background)	28
0. 概念导图	29
1. 线性代数基础	29
2. 刚体变换： $SE(3)$	32
3. Rodrigues 公式：从轴角到旋转矩阵	33
4. 旋转表示 1：旋转矩阵	34
5. 旋转表示 2：Euler 角	35
6. 旋转表示 3：旋转向量 / 轴角	36
7. 旋转表示 4：四元数	37
8. 指数映射与对数映射	40
9. 四种表示的对比与转换	42
10. 插值基础	43
11. Worked Examples	44
12. Anki 卡片	46
13. 中英术语速查表	47
Lecture 03 — 角色运动学 (Character Kinematics)	49
0. 名词热身：本讲反复出现的 7 个概念	49
1. 骨骼与单链正向运动学	50
2. 角色 FK（树状骨架）	52
3. Retargeting：在不同参考姿态间迁移动作	53

4. 逆运动学 (IK): 问题与启发式解法	54
5. IK 作为优化问题 (全推导)	56
6. Jacobian 矩阵: 定义与构造	56
7. IK 求解方法 (三大类)	59
8. 全身 IK (Full-Body IK)	61
9. Worked Examples (例题)	62
10. Anki 记忆卡片	63
11. 中英术语速查表	64
12. 复习要点	65
Lecture 04 一关键帧动画 (Keyframe Animation)	66
0. 从走马灯到关键帧: 问题从哪里来?	66
1. 插值问题的形式化	66
2. 平滑性 (Smoothness): 我们在追求什么?	67
3. 多项式插值与 Runge 现象	69
4. 样条插值 (Spline Interpolation): 核心工具	69
5. 三次样条 (Cubic Spline)	70
6. 三次 Hermite 样条 (Cubic Hermite Spline)	71
7. Catmull-Rom 样条	73
8. Bezier 样条 (简介)	74
9. 三类样条综合对比	74
10. 旋转的插值 (Interpolation of Rotations)	75
11. 多维与散点插值	77
12. 插值方法综合谱系图	80
13. Worked Examples (例题)	80
14. Anki 记忆卡片	82
15. 中英术语速查表	83
16. 复习要点	85
Lecture 05 一数据驱动动画 (Data-driven Animation)	86
0. 名词预热	86
1. 从关键帧到数据驱动: 为什么需要 MoCap?	87
2. 数据驱动方法谱系	87
3. Motion Capture 动作捕捉	87
4. 动作数据表示 (Motion Data Representation)	90
5. Motion Retargeting 动作重定向	92
6. Motion Editing 动作编辑	94
7. Motion Transition 动作过渡与混合	94
8. Motion Graph 动作图	96
9. 动作组合范式比较	99
10. Motion Fields 动作场	99
11. Motion Matching 动作匹配	99
12. 横向方法总览	103
13. Worked Examples 例题	104
14. Anki 记忆卡片	104
15. 中英术语速查表	105
16. 复习要点	107
Lecture 06 一基于学习的动画 (Learning-based Animation)	108
0. 名词预热: 5 个必须先懂的词	108
1. 人体运动的低维流形	109
2. 主成分分析 (PCA)	109
3. 高斯模型族	111
4. 运动序列的两种建模视角	112
5. 自回归模型与歧义问题	113
6. PFNN: 相位函数神经网络	114
7. 从 PCA 到 Autoencoder	115

8. 变分自编码器 (VAE)	116
9. VQ-VAE: 向量量化变分自编码器	117
10. 扩散模型 (Diffusion Models)	118
11. 生成模型大比较	120
12. 学习式运动控制	120
13. 其他应用	121
14. 总结	122
15. Worked Examples (例题)	122
16. Anki 记忆卡片	123
17. 中英术语速查表	124
18. 复习要点	126
Lecture 07 一蒙皮与表情动画 (Skinning & Facial Animation)	127
0. 角色动画流水线总览	127
1. Skinning Deformation 几何推导	127
2. Linear Blend Skinning (LBS / 线性混合蒙皮)	128
3. Dual Quaternion Skinning (DQS / 对偶四元数蒙皮)	131
4. 蒙皮方法对比	133
5. Pose Space Deformation (PSD)	133
6. SMPL 人体模型	134
7. Facial Animation (面部动画)	135
8. 面部动画完整流水线	138
9. 总结: 核心公式速查	139
10. Worked Examples (例题)	139
11. Anki 记忆卡片	140
12. 中英术语速查表	141
13. 复习要点	142
Lecture 08 一基于物理的仿真 (Simulation)	143
0. Motivation: 为什么需要物理仿真?	143
1. 质点动力学与时间离散	143
2. 数值积分方法	144
3. 刚体运动学: 位姿、速度与积分	146
4. 刚体动力学: Newton-Euler 方程	147
5. 铰接刚体: 关节约束与 Lagrange 乘子	149
6. 接触建模	152
7. 投影高斯-赛德尔 (PGS) 求解器	155
8. 完整仿真管线	156
9. Worked Examples (例题)	157
10. 中英术语速查表	158
11. Anki 复习卡片	159
12. 复习要点	161
Lecture 09 一驱动角色 (Actuating Characters)	162
0. 核心问题 / Motivation	162
1. 两种坐标体系	163
2. 关节力矩: 物理本质与施加方式	165
3. 全驱动与欠驱动	166
4. 正动力学与逆动力学	166
5. PD / PID 控制: 最简的反馈律	169
6. Jacobian Transpose 控制 (虚拟力控制)	171
7. 跟踪控制与残差力	172
8. 静态平衡 (Static Balance)	174
9. 控制方法谱系与平衡策略总览	176
10. Worked Examples (例题)	176
11. Anki 卡片	178
12. 中英术语速查表	179

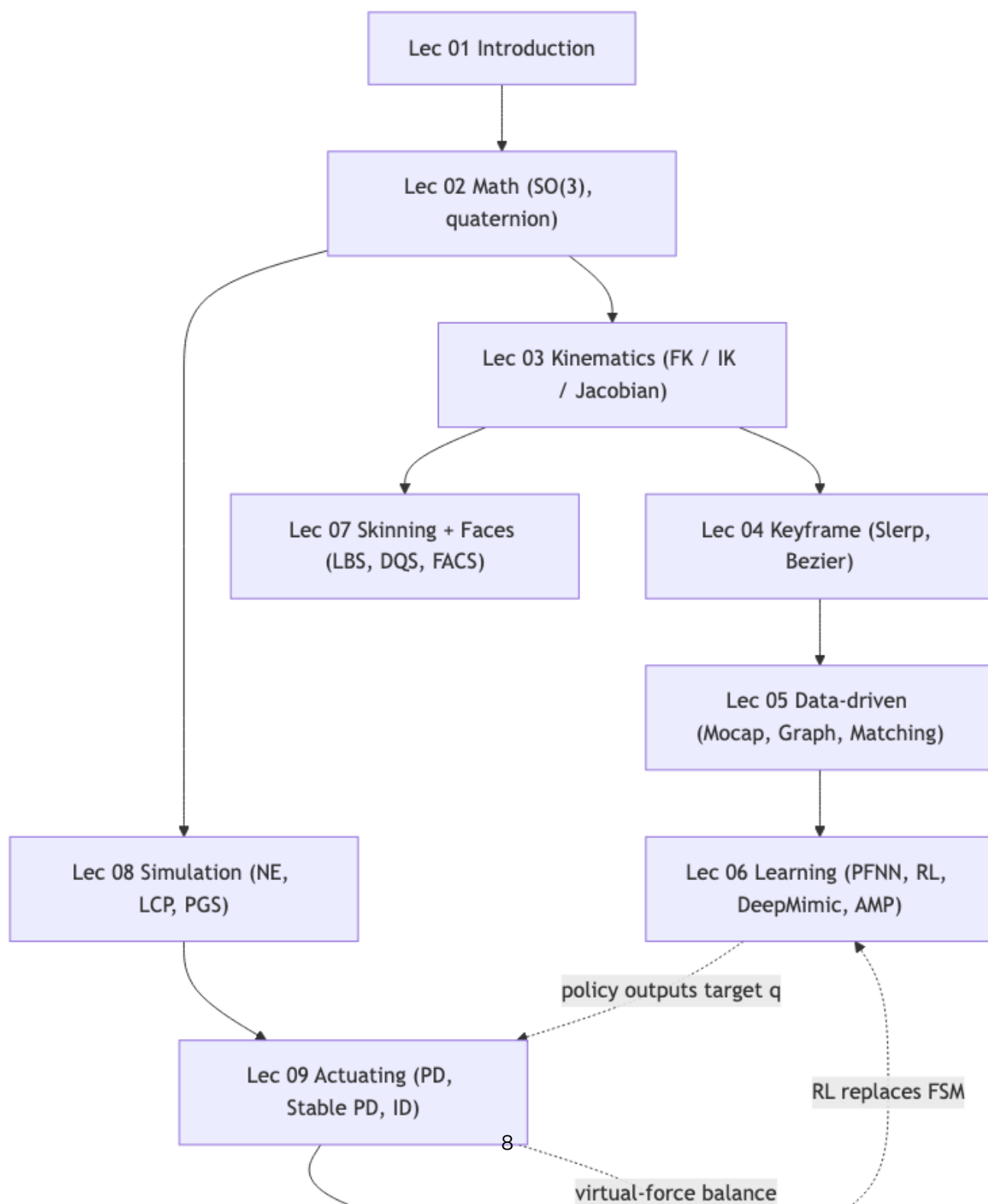
13. 复习要点	180
Lecture 10 —学习走路 (Learning to Walk)	181
0. 名词预热	181
1. 步态周期与相位分析	181
2. 零力矩点 ZMP	183
3. 简化模型总论	184
4. 倒立摆模型 IPM	186
5. 线性倒立摆模型 LIPM	186
6. Cart-Table 模型与 Preview Control	188
7. SIMBICON: 有限状态机走路控制	190
8. 概念架构图	192
9. 三种简化模型对比	194
10. 走路控制的层级结构	196
11. 与学习派方法的衔接	197
12. Worked Examples (例题)	197
13. Anki 记忆卡片	198
14. 中英术语速查表	199
15. 复习要点	200
16. 关键文献	200
Lecture 11 —最优控制与强化学习 (Optimal Control & Reinforcement Learning)	201
0. 全景鸟瞰	201
1. 轨迹优化 (Trajectory Optimization)	202
2. 无导数优化: CMA-ES 与 SAMCON	204
3. 反馈控制与动态规划	205
4. 线性二次型调节器 (LQR)	206
5. 模型法 vs 无模型法	208
6. 马尔可夫决策过程 (MDP)	208
7. 值方法: TD 学习、Q-Learning 与 DQN	210
8. 策略梯度与 PPO	212
9. 基于模型的强化学习 (Model-Based RL)	213
10. 生成式控制模型	214
11. 完整方法谱系对比	215
Worked Examples (例题解析)	216
Anki 卡片 (15+ 张, 复习用)	218
中英术语速查表	219
复习要点	221
Lecture 12 —肌肉驱动的角色 (Muscle-actuated Characters)	222
0. 全景鸟瞰	223
1. 角色运动是怎么产生的?	224
2. 肌肉与肌腱: 解剖学速通	224
3. Hill 型肌肉模型	226
4. 给定 α 解出肌肉力: 收缩动力学	227
5. 激活动力学 (Activation Dynamics)	231
6. 刚性肌腱简化 (Rigid Tendon Simplification)	232
7. 肌肉路径与关节力矩 (Musculoskeletal Geometry)	233
8. 工具与应用	234
9. 记号速查表	234
10. 中英术语对照表	235
11. 进一步阅读	236
Lecture 13 —运动规划与群体仿真 (Motion Planning & Crowds)	237
0. 全景鸟瞰	237
1. 运动规划问题: Workspace 与 Configuration Space	238
2. 图搜索基石	239

3. 几何路标方法	240
4. 基于采样的方法 (Sampling-based Approaches)	242
5. 在角色动画中的应用	247
6. 群体仿真 (Crowds)	247
7. 记号速查表	252
8. 中英术语对照表	252
9. 进一步阅读	253

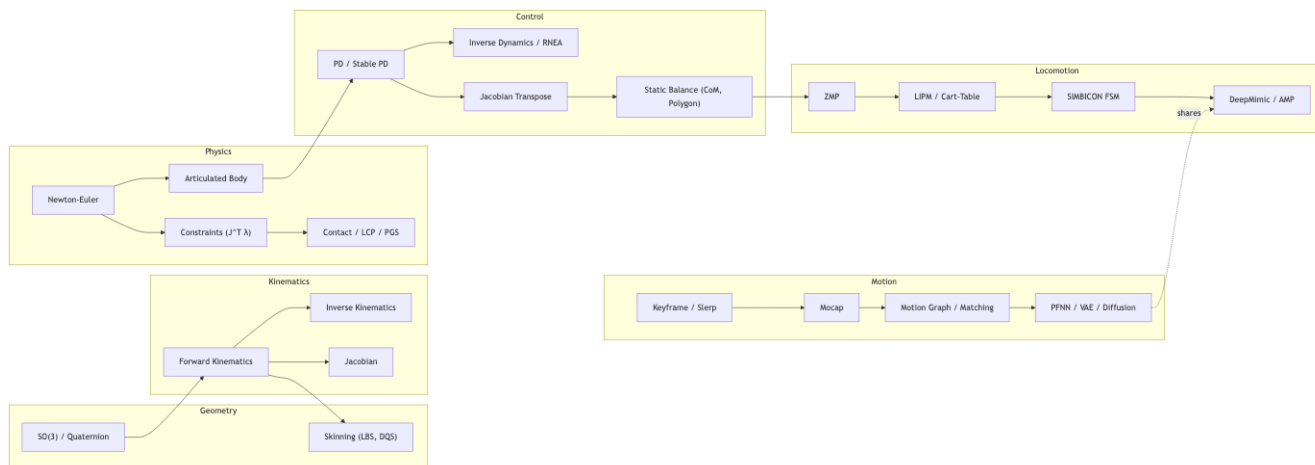
课程概览

北京大学《角色动画与运动仿真》(刘利斌, MOCCA 2026) 2025–2026 学年第 2 学期。课程从几何与运动学起步, 依次介绍关键帧动画、数据驱动方法、学习方法, 再到蒙皮与表情动画、基于物理的仿真, 再到角色驱动与走路控制, 统一到最优控制与强化学习框架, 最后由肌肉驱动角色 (Muscle-driven Characters) 与运动规划 (Motion Planning) 收束。本笔记覆盖全部 13 讲。

课程脉络



知识图谱（全课程）



各讲速览

Lec	主题	关键概念	文件
01	Introduction	课程定位、动画产业链、计算机动画的研究问题	lec01_introduction.md
02	Math Background	旋转表示、 $SO(3)$ 、四元数、指数映射、 $\dot{R} = [\omega]_{\times} R$	lec02_math.md
03	Character Kinematics	骨骼层级、Forward / Inverse Kinematics、Jacobian、CCD、FABRIK	lec03_kinematics.md
04	Keyframe Animation	插值 (线性/Hermite/Bezier/Catmull-Rom)、Slerp/Squad、动画曲线编辑	lec04_keyframe.md
05	Data-driven Animation	Motion capture、Motion graph、Motion matching、Motion blending	lec05_data_driven.md
06	Learning-based Animation	PFNN/MANN、Motion VAE/Diffusion、DeepMimic/AMP/ControlVAE	lec06_learning.md
07	Skinning & Facial Animation	Linear blend skinning、Dual quaternion、Blendshapes、FACS	lec07_skinning.md
08	Simulation	数值积分、Newton-Euler、铰接刚体、约束、Penalty/LCP 接触、PGS	lec08_simulation.md
09	Actuating Characters	极大/广义坐标、关节力矩、正/逆动力学、RNEA、PD/Stable PD、Jacobian Transpose、静态平衡	lec09_actuating.md
10	Learning to Walk	动态平衡、ZMP、LIPM、轨道能量、Cart-Table、Preview Control、SIMBICON	lec10_learning_to_walk.md
11	Optimal Control & RL	Trajectory Opt、Lagrange/KKT、Pontryagin、Bellman、LQR/iLQR、MDP、Q-Learning/DQN、Policy Gradient/PPO、Model-based RL、ControlVAE/VQ-VAE/MoConVQ	lec11_optimal_control_rl.md
12	Muscle-driven Characters	解剖学 (MTU / sarcomere / tendon)、Hill 三元件模型、激活/收缩动力学、刚性肌腱简化、肌肉路径与关节力矩、Muscle VAE	lec12_muscle.md
13	Motion Planning	Configuration space、A*/Dijkstra、Bug 算法、栅格/可视图/单元分解、PRM/RRT/RRT*/RRT-Connect、NavMesh、Motion Graph + 搜索、Boids/SFM 群体仿真	lec13_motion_planning.md

全局术语中英对照

中文	English	首次出现
自由度	Degrees of Freedom (DoF)	Lec 03
前向运动学	Forward Kinematics (FK)	Lec 03
逆运动学	Inverse Kinematics (IK)	Lec 03
雅可比矩阵	Jacobian matrix	Lec 03
阻尼最小二乘	Damped Least Squares (DLS)	Lec 03
四元数	Quaternion	Lec 02
球面线性插值	Spherical Linear Interpolation (Slerp)	Lec 04
动作捕捉	Motion Capture (MoCap)	Lec 05
动作图	Motion Graph	Lec 05
动作匹配	Motion Matching	Lec 05
相位函数神经网络	Phase-Functioned Neural Network (PFNN)	Lec 06
强化学习	Reinforcement Learning (RL)	Lec 06
对抗动作先验	Adversarial Motion Prior (AMP)	Lec 06
线性混合蒙皮	Linear Blend Skinning (LBS)	Lec 07
双四元数蒙皮	Dual Quaternion Skinning (DQS)	Lec 07
形状混合	Blendshape	Lec 07
显式/隐式/半隐式 Euler	Explicit / Implicit / Semi-implicit Euler	Lec 08
角速度	Angular velocity	Lec 02/08
反对称矩阵	Skew-symmetric matrix	Lec 02/08
角动量	Angular momentum	Lec 08
惯量张量	Moment of inertia tensor	Lec 08
牛顿-欧拉方程	Newton-Euler equations	Lec 08
铰接刚体	Articulated rigid body	Lec 08
拉格朗日乘子	Lagrange multiplier	Lec 08
库仑摩擦律	Coulomb's law of friction	Lec 08
线性互补问题	Linear Complementary Problem (LCP)	Lec 08
摩擦锥	Friction cone	Lec 08
投影高斯-赛德尔法	Projected Gauss-Seidel	Lec 08
极大坐标	Maximized Coordinates	Lec 09
广义坐标	Generalized Coordinates	Lec 09
关节力矩	Joint Torque	Lec 09
正动力学	Forward Dynamics (FD)	Lec 09
逆动力学	Inverse Dynamics (ID)	Lec 09
递归牛顿欧拉算法	Recursive Newton-Euler Algorithm (RNEA)	Lec 09
比例-微分控制	Proportional-Derivative Control (PD)	Lec 09
比例-积分-微分控制	PID Control	Lec 09
稳定 PD 控制	Stable PD Control	Lec 09
重力补偿	Gravity Compensation	Lec 09

中文	English	首次出现
雅可比转置控制	Jacobian Transpose Control	Lec 09
虚拟力控制	Virtual Model Control / Virtual Force	Lec 09
残差力/力矩	Residual Force / Torque	Lec 09
欠驱动系统	Underactuated System	Lec 09
全驱动系统	Fully-actuated System	Lec 09
质心	Center of Mass (CoM)	Lec 09
支撑多边形	Support Polygon	Lec 09
踝/髌平衡策略	Ankle / Hip Strategy	Lec 09
动量控制	Momentum Control	Lec 09
单/双支撑相	Single / Double Stance Phase	Lec 10
飞行相	Flight Phase	Lec 10
零力矩点	Zero-Moment Point (ZMP)	Lec 10
压力中心	Center of Pressure (CoP)	Lec 10
地面反作用力	Ground Reaction Force (GRF)	Lec 10
倒立摆模型	Inverted Pendulum Model (IPM)	Lec 10
线性倒立摆模型	Linear Inverted Pendulum Model (LIPM)	Lec 10
弹簧负载倒立摆	Spring-Loaded Inverted Pendulum (SLIP)	Lec 10
轨道能量	Orbital Energy	Lec 10
桌-车模型	Cart-Table Model	Lec 10
预览控制	Preview Control	Lec 10
被动动态行走	Passive Dynamic Walking	Lec 10
SIMBICON	SIMple Blped LOCOMOTION CONTROL	Lec 10
有限状态机	Finite State Machine (FSM)	Lec 10
跨步策略	Stepping Strategy	Lec 10
轨迹优化	Trajectory Optimization	Lec 11
时空约束	Spacetime Constraints	Lec 11
开环/闭环控制	Open-loop / Closed-loop Control	Lec 11
前馈/反馈控制	Feedforward / Feedback Control	Lec 11
拉格朗日乘子	Lagrange Multiplier	Lec 11
KKT 条件	Karush-Kuhn-Tucker Conditions	Lec 11
协态/伴随变量	Co-state / Adjoint Variable	Lec 11
庞特里亚金极小值原理	Pontryagin's Minimum Principle (PMP)	Lec 11
打靶法	Shooting Method	Lec 11
无导数优化	Derivative-Free Optimization	Lec 11
协方差矩阵自适应进化策略	CMA-ES	Lec 11
基于采样的运动控制	SAMCON	Lec 11
动态规划	Dynamic Programming (DP)	Lec 11
贝尔曼最优性原理	Bellman's Principle of Optimality	Lec 11
值函数	Value Function	Lec 11
状态-动作值函数	Q-function / State-Action Value	Lec 11
线性二次调节器	Linear Quadratic Regulator (LQR)	Lec 11
黎卡提递推	Riccati Recursion	Lec 11
迭代 LQR / 微分动态规划	iLQR / DDP	Lec 11
马尔可夫决策过程	Markov Decision Process (MDP)	Lec 11
折扣因子	Discount Factor	Lec 11
回报	Return	Lec 11
时序差分学习	Temporal Difference (TD) Learning	Lec 11
Q 学习	Q-Learning	Lec 11
深度 Q 网络	Deep Q-Network (DQN)	Lec 11
经验回放	Replay Buffer	Lec 11

中文	English	首次出现
目标网络	Target Network	Lec 11
策略梯度	Policy Gradient	Lec 11
行动者-评论家	Actor-Critic	Lec 11
近端策略优化	Proximal Policy Optimization (PPO)	Lec 11
基于模型的强化学习	Model-Based Reinforcement Learning	Lec 11
变分自编码器	Variational Autoencoder (VAE)	Lec 11
向量量化	Vector Quantization (VQ-VAE)	Lec 11
码本	Codebook	Lec 11

Lecture 01 一导论 (Introduction)

[TIP] 学习指南

本节核心：这是整门课的地图讲。目标是建立一个“坐标系”：3D 计算机动画研究什么、为什么角色动画 (character animation) 是最复杂的子领域、解决它的方法族谱 (运动学方法 vs. 物理方法) 是什么、以及本课 13 讲的知识路线图。

前置知识：无；本讲假设读者零基础。后续讲次才正式依赖线性代数、微积分、Python、概率、力学、ML/RL。

重点掌握：1. 3D 计算机图形学的三大支柱 (Geometry / Animation / Rendering)，角色动画的定位 2. 为什么角色动画“难”——高自由度 + 物理约束 + 智能行为 3. 人体运动的生理链条 → 两类方法 (Kinematic vs. Dynamic) 的分叉点 4. 所有主流方法在“high-level goals low-level control physics”坐标轴上的位置 5. 课程 13 讲的主题顺序，以及每讲在这张全图中的对应位置

0. 名词热身：读本讲前需要知道的 6 个词

名词	一句话解释
角色 (Character)	能做出目标导向行为的可动实体——人、动物、虚拟生物、手、机器人、NPC 人群等
关节 (Joint)	骨骼中两段骨骼的铰接点，每个关节有若干自由度 (DOF)。人体约 20+ 主要关节
自由度 (DOF, Degree of Freedom)	一个关节能独立运动的维度数。肩关节约 3 DOF (可旋转三个轴)，膝关节约 1 DOF (屈伸)
Pose (姿态)	某一时刻所有关节的角度/位置组合，是 character 的“快照”
Kinematics (运动学)	只研究位置、速度、加速度， 不考虑力 。像描述一个舞蹈，不管肌肉发了多少力
Dynamics (动力学)	$\text{力} = \text{质量} \times \text{加速度}$ (Newton)，把力/扭矩纳入方程。仿真会考虑摔倒、惯性、接触

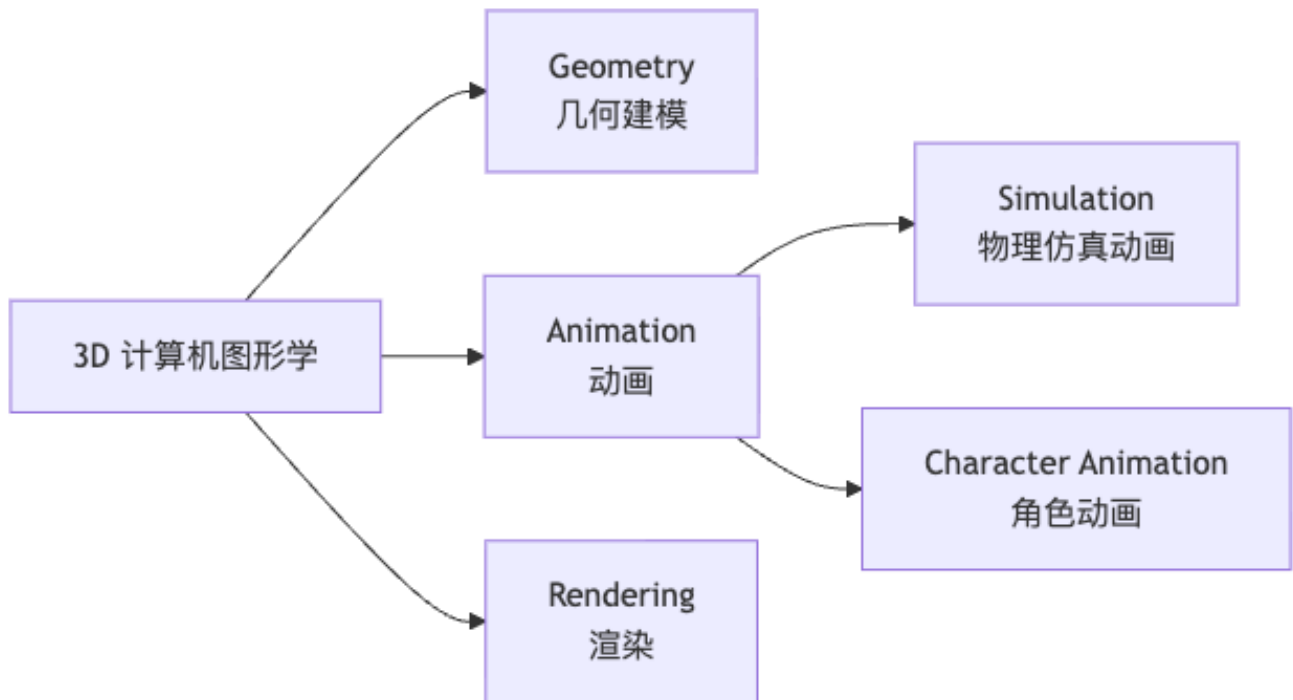
[NOTE] 直觉理解：Kinematics vs. Dynamics

想象两种控制玩偶的方式：(A) 直接抓手臂拧到某个角度——这是 Kinematics，你忽略了玩偶有没有内骨架、接触地面的反力；(B) 让玩偶内置电机驱动关节，电机输出扭矩作用于骨架，由物理引擎算出身体位置——这是 Dynamics。(A) 更直接可控，(B) 更真实但极难保证不摔倒。

1. 3D 计算机动画的全景

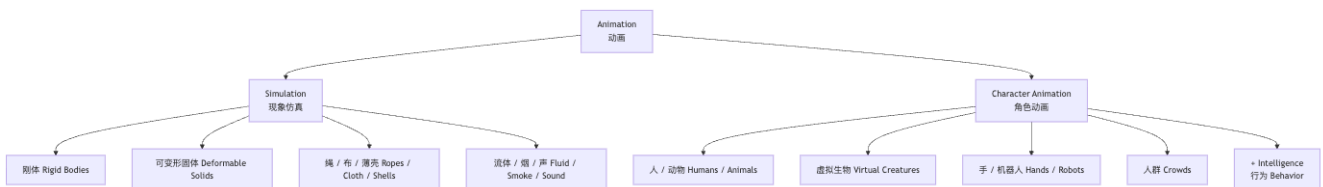
1.1 3D 计算机图形学的三大支柱

3D 计算机图形学研究如何在计算机中创建、描述、渲染虚拟世界，传统上分三个模块：



- **Geometry**: 描述物体的形状（网格、NURBS、隐式曲面等）
- **Rendering**: 把 3D 场景投影成 2D 图像，处理光照、材质、阴影
- **Animation**: 让物体随时间运动

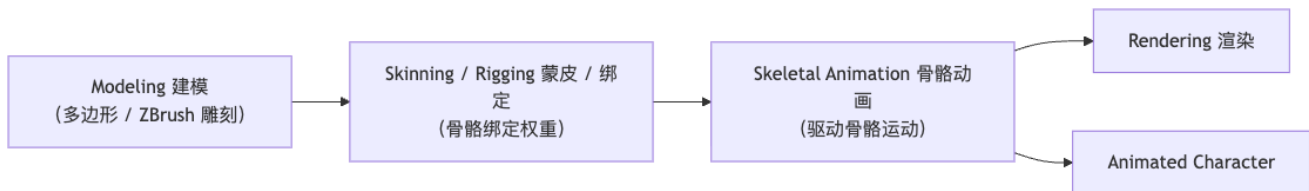
1.2 Animation 的内部分野



关键差异: Simulation 动画模拟现象 (Phenomenon), 结果由初始状态 + 物理规律决定; Character Animation 模拟行为 (Behavior), 角色要主动实现目标, 系统里有一层”控制”。这个”+ Intelligence”让 character animation 远比普通物理仿真复杂。

1.3 Character Animation 在工业流水线中的位置

现代 3D 制作管线（电影、游戏）中，character animation 处于如下位置：



- **Modeling (建模)**: 制作角色的外观几何——用 ZBrush 雕刻高模，或直接建低模。
- **Rigging & Skinning (绑定与蒙皮)**: 在模型内部放置骨骼，定义骨骼如何带动外皮网格变形（蒙皮权重）。
- **Skeletal Animation (骨骼动画)**: 驱动骨骼，整个外皮随之变形——这就是本课的核心研究对象。

2. 为什么专门研究 Character Animation

2.1 问题的规模

一个人体角色：– 20+ 主要关节 (Hip, Spine, Shoulder, Elbow, Wrist, Knee, Ankle...) – 每个关节 1–3 DOF，总 DOF 50–100+ – 这不算”超高维”——手工摆关键帧理论上可行，但代价高昂

2.2 纯手工 K 帧的局限

- 劳动密集：电影级动画 1 秒 24 帧，全靠手工摆太贵
- 不适合交互：游戏/VR 需要实时响应玩家输入，不能预先录制所有可能动作
- 无法泛化：换一个角色体型，所有动画要重做（除非有 retargeting 技术）

2.3 Character Animation 的研究问题

从建模角度，character animation 要构造一个 Motion Model：

$$(\text{current state, control signal}) \xrightarrow{\text{Motion Model}} \text{next state} \tag{1}$$

control signal 可以是：– 交互输入（手柄摇杆方向）– 音频 / 文本（语音驱动手势；文字描述驱动动作）– 抽象任务（“去那里，帮我拿杯水”）

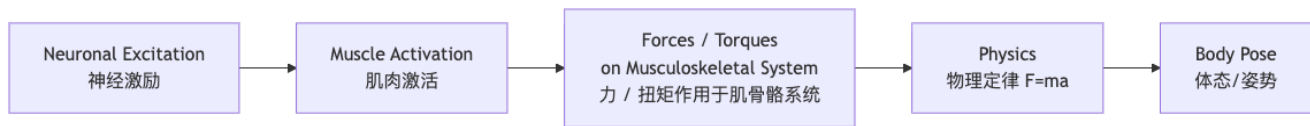
这个 Motion Model 的构造方式——是查表、做插值、学神经网络还是物理仿真——决定了不同的方法学。

2.4 Character Animation 的三大应用方向

1. 理解运动机制 (Understanding the mechanism behind motions and behaviors)
2. 智能编辑 / 复用 / 生成动画 (Smart editing / Reuse / Generate new animation)
3. 迁移到新身体 (Embodiment Transfer)：数字化身 (Digital Avatar)、仿人机器人 (Humanoid Robot)

3. 人体运动的生理链条——方法论的起点

理解”为什么有两类方法”需要先看清楚人是怎么动的：



切割点决定方法类别：

方法类	进入链条的位置	含义
Kinematic（运动学）方法	直接更新 Body Pose（跳过力和物理）	pose/velocity/acceleration 由控制器直接写入
Physics-based（动力学）方法	在 Forces/Torques 处介入，让物理决定 pose	控制器输出关节力矩， 物理引擎积分出下一帧

[NOTE] 直觉理解：为什么 kinematic 方法会脚滑、穿模

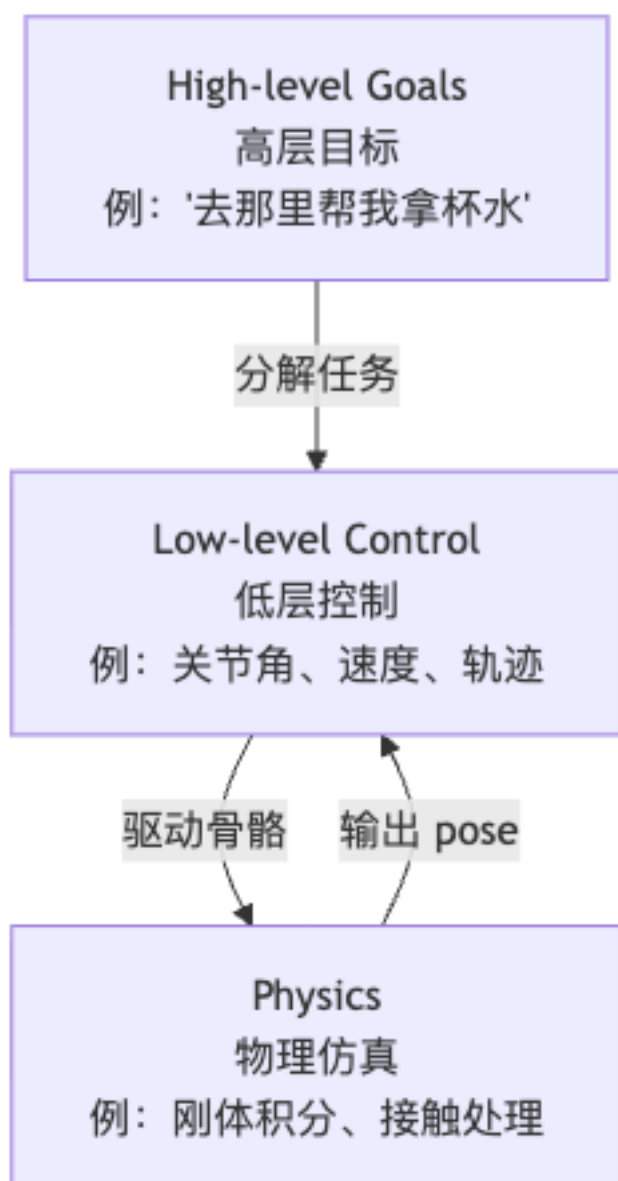
直接更新 pose 时，你可以命令脚“出现在地面下 5cm”——物理引擎不参与，不会抗议。这就导致脚滑 (foot sliding)、穿模 (interpenetration) 等伪影。Physics-based 方法天然不会有这个问题，但难点转移到了“如何输出合理的力矩让角色保持平衡”。

[WARN] 易错点：Simplified Physics

实用的 physics-based 方法并非完全用肌肉骨骼模型——肌肉力学建模极其复杂。课程主要用**简化物理**：骨骼（刚体 rigid body）+ 关节（铰链 joint）+ 关节力矩（joint torques），由此驱动 Ragdoll 仿真。

4. 方法学坐标系：三层控制层级

所有 character animation 方法都可以放进这个坐标轴：



- **Kinematic 方法**：主要活动在 High-level Goals → Low-level Control 区间，不碰物理层
- **Physics-based 方法**：Low-level Control 输出力矩，由 Physics 出 pose；问题在于“如何给出正确的力矩”

5. Kinematic（运动学）方法谱系

以下所有方法都按“逐步解决上一代遗留问题”的逻辑叠加。

5.1 基础：Keyframe Animation 关键帧动画

机制：动画师在关键时刻（keyframe）手动摆出角色 pose，工具自动补算中间帧（in-betweening）。

历史：– **Rotoscoping**（约 1914 年）：Max Fleischer 发明，用投影仪把真人表演逐帧投到赛璐珞片上描绘——最早的“参考真实动作”技术。– **Disney 12 Principles of Animation (1981)**：Squash & Stretch、Anticipation、Staging 等，至今是 keyframe 动画的美学准则。– **3D Keyframe**：在 3D 软件（Maya 等）中摆骨骼，F 曲线控制插值，原理与 2D 相同。

工具支持：– **Forward Kinematics (FK)**：给出每个关节旋转角 → 计算末端执行器（end-effector）位置 – **Inverse Kinematics (IK)**：给出末端位置 → 反推关节旋转角（数学上欠定，有多解）

[NOTE] FK vs. IK 的直觉

FK 像“我要把肘弯 45 度、手腕弯 30 度”——从根到末端逐层计算；IK 像“我要把手放到某个位置”——让系统自动算每个关节转多少。游戏引擎里角色的脚踩不平地面时自动适应，就是 IK 在工作。

5.2 Motion Capture 动作捕捉

机制：用设备（光学标记点、IMU、视频姿态估计）记录真人或物体的 3D 运动，直接当动画数据用。

类型：– **光学 Mocap**：演播室里穿反光标记服，多台红外相机重建 3D 轨迹（高精度，现仍是影视主流）– **IMU Mocap**：惯性测量单元贴在身体各段，无需专用场地（精度稍低，但便携）– **视频 Pose Estimation**：OpenPose 等工具直接从普通 RGB 视频估计 2D/3D 姿态（最廉价，但噪声大）

Motion Retargeting（动作重定向）：把源角色动作迁移到目标角色。目标角色可能：骨骼比例不同、骨骼数量不同、拓扑不同。这是独立的研究问题（Aberman et al. 2020, SIGGRAPH）。

5.3 Motion Graphs / State Machines 动作图 / 状态机

动机：Mocap 数据库有大量不同动作片段（走路、跑步、跳跃……）。如何让角色流畅地在片段之间切换并响应实时控制？

Motion Graph：– 节点 = 动作帧；边 = 可以“切换”的帧对（两帧 pose 足够相似）– 实时播放时，沿图的边切换，实现无缝拼接

State Machine：– 更工程化：每个节点是一段动画，边上有触发条件（例：速度 > 阈值则切到跑步动画）– 复杂项目中状态机会膨胀成“意大利面”——可维护性是痛点

Parametric Motion Graphs（Heck & Gleicher 2007）：在边上插值，让动作参数（速度、风格）连续变化。

5.4 Motion Matching 动作匹配

动机：状态机维护成本太高；希望动作切换逻辑能“自动”从数据中涌现。

机制：在每帧，从整个 Mocap 库中用**查询特征**（当前 pose + 速度 + 未来轨迹预测）检索最接近的帧，直接跳到那一帧继续播放。

- 不需要手工设计状态机——动作库越大效果越好

- GDC 2016 首次系统介绍，现已是 AAA 游戏标配（如《最后生还者 Part II》）

5.5 Learning-based / Generative Models 学习型方法

动机：手工设计规则（Motion Graphs 的转移条件、Motion Matching 的特征权重）仍需大量调参；希望从数据端到端学习运动模型。

通用框架：

Generative Model : (current pose, control signal) $\rightarrow$ next pose

代表工作：– **Local Motion Phases** (Starke et al. 2020): 用相位变量编码运动节律, MLP 生成下一帧 – **Motion VAE** (Ling et al. 2021): 变分自编码器学习动作的低维隐空间, 在隐空间做控制 – **Diffusion-based** (Chen et al. 2024, Taming Diffusion Probabilistic Models): 扩散模型生成动作序列

5.6 Cross-Modal Motion Synthesis 跨模态动作合成

动机：让角色的动作由其他模态信号（音频、文本）驱动，而非人工设计的控制器。

两类代表：– **Audio-driven:** 音乐 $\rightarrow$ 舞蹈 (Music-to-Dance); 语音 $\rightarrow$ 说话手势 (Co-speech Gesture) —— Rhythmic Gesticulator (Ao et al. SIGGRAPH Asia 2022) – **Text-driven:** 文字描述 $\rightarrow$ 动作序列——MotionDiffuse (Zhang et al. 2022)、多种后续工作

5.7 LLM 驱动的高层规划

动机：前面所有方法都假设 control signal 是底层的（方向向量、速度）或中层的（“走”/“跑”）。如何处理“去拿那把钥匙，开门，坐到椅子上”这类语义指令？

方案：Large Language Model 把高层任务分解成有序的基元动作序列（每个基元有对应的 motion index），再由下层控制器执行。

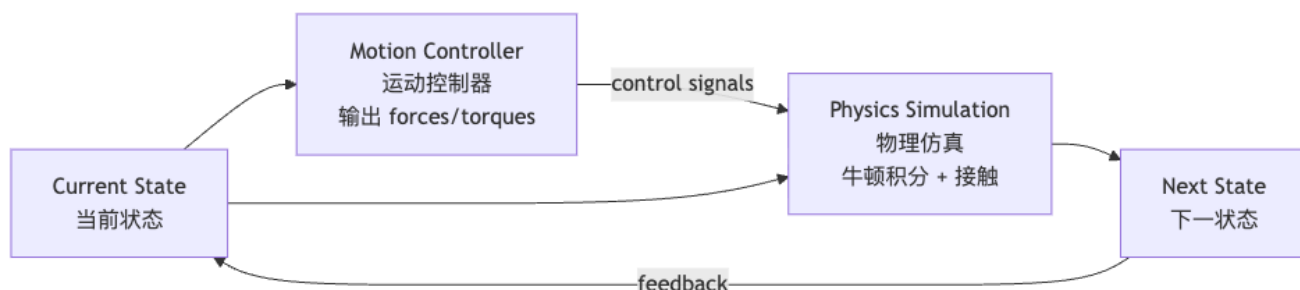
"钥匙在前方，我要开门然后坐下" $\rightarrow$ LLM $\rightarrow$ [walk(297), bend_pickup(246), insert_key(129), v

[WARN] Kinematic 方法的两大根本缺陷

1. **Physical Plausibility (物理可信度):** 脚滑、穿模、空中悬停、惯性不对——因为没有物理约束
 2. **Environment Interaction (环境交互):** 被动响应障碍、被推倒后站起来、在不平地形上保持平衡——纯数据生成几乎无法泛化到未见过的场景
- 这两点缺陷正是物理方法的入口。

6. Physics-based (基于物理) 方法谱系

6.1 核心架构



与 Kinematic 方法的对比：

方面	Kinematic	Physics-based
输出	直接写 pose	输出力矩 → 物理决定 pose
物理可信	否（可脚滑、穿模）	是（自然保持接触约束）
环境交互	差	天然支持
控制难度	低	高（稳定性难以保证）
实现成本	低	高

6.2 Ragdoll Simulation 破布娃娃仿真

最简极端：不加任何 Motion Controller，角色完全被动地被重力和接触力驱动，像一个布娃娃”瘫倒”。

用途：游戏中角色死亡/被击飞后的物理响应（Unreal Engine Ragdoll Physics）。

问题：它只会”倒”，不会”站”。要做 locomotion，需要控制器。

6.3 Tracking Controllers 跟踪控制器

动机：有了参考动作（Keyframe / Mocap），让仿真角色”跟着”参考动作运动，同时由物理保证接触、平衡。

代表：Hodgins & Wooten 1995（最早的完整 physics-based 跑跳动作仿真）。

控制量：关节 PD 控制（比例-微分）：

$$\tau_i = k_p(\theta_i^{\text{ref}} - \theta_i) - k_d\dot{\theta}_i \quad (2)$$

θ_i^{ref} 是参考角度， θ_i 是当前角度， k_p/k_d 是刚度/阻尼系数。

6.4 力与力矩（基础公式）

物理仿真的基本量：

多个力 F_i 作用在位置 r_i ，对参考点的合力矩：

$$\tau = \sum_i r_i \times F_i \quad (3)$$

其中 $\times$ 是向量叉积（cross product）。这是 Lec07-08 展开 Newton-Euler 方程的起点。

[NOTE] 扭矩直觉

用扳手拧螺丝：力越大（ F 大）、力臂越长（ r 大）、力垂直于力臂（叉积最大），扭矩越大。关节力矩控制就是”决定每个关节’拧’多少”。

6.5 “控制为什么难”——QWOP 直觉

[EX] QWOP 游戏

QWOP 是一个用 Q/W/O/P 四个键独立控制双腿四个肌肉群的跑步游戏。玩家通常立刻摔倒。这直观呈现了 physics-based 控制的核心困难：**状态空间高维**（每个关节位置、速度）+ **动态不稳定**（双足本质上是倒立摆）+ **接触不连续**（脚离地/着地是离散事件）+ **误差累积**（一个时刻的控制误差会在下一时刻放大）。

6.6 “Keyframe Control”/ Trajectory Crafting 轨迹设计

动机：把动画师的”手工 K 帧”思路引入物理仿真——让动画师设计力矩轨迹而非 pose 轨迹。

代表：NaturalMotion Endorphin（商业软件，“生物力学仿真中间件”）。

6.7 Spacetime / Trajectory Optimization 空时优化

动机：手工设计力矩轨迹太费力；能否自动求解？

方法：把整段运动写成优化问题：

$$\min_{u(t)} J(x(t), u(t)) \quad \text{s.t. } \dot{x} = f(x, u), g(x, u) \leq 0 \quad (4)$$

其中 $u(t)$ 是控制轨迹（力矩序列）， J 是目标函数（如跑得快 + 能量最小），约束包含物理方程和接触条件。

代表：– SAMCON (Liu et al. 2010)：Sample-based Motion Control，采样 + 局部优化 – Wampler & Popovic’ 2009：最优步态与动物运动形态

[WARN] 易错点：Trajectory Optimization $\neq$ Reinforcement Learning

Trajectory Optimization 一次解出整段轨迹（offline），适合短时间动作设计；Reinforcement Learning 学习一个策略（policy），能在任意状态下给出动作（online / reactive）。两者互补，课程在 Lec10、Lec11 分别深入。

6.8 Abstract Models 抽象简化模型

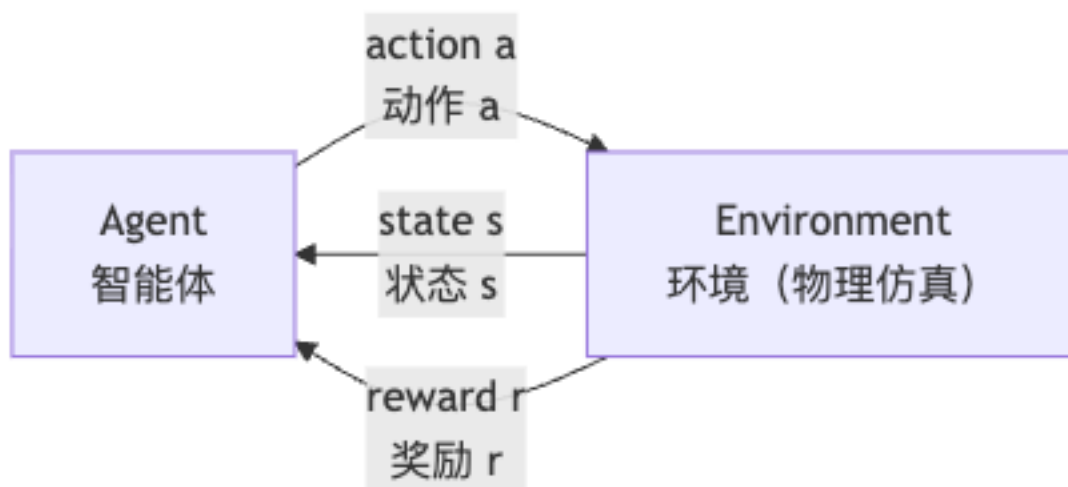
动机：完整骨骼仿真计算量大、优化困难；对步态等问题能否用更简单模型？

两类代表：– 倒立摆模型 (Inverted Pendulum)：把人体简化为一根摆，研究重心 (CoM) 轨迹，足够分析走路稳定性 – SIMBICON (Yin et al. 2007)：线性反馈控制 + 有限状态机 (FSM) ——状态定义为步态阶段（左脚支撑、右脚支撑……），每阶段用简单线性反馈调整关节角

现代应用：人形机器人（如 Unitree H1）的步态控制仍大量借鉴抽象模型。

6.9 Reinforcement Learning / Deep RL 强化学习

强化学习框架 (RL)：



策略 $\pi(a|s)$ ：给定当前状态 s ，输出动作 a （这里是关节力矩）的概率分布。

目标：最大化累积折扣奖励 $G = \sum_{t=0}^{\infty} \gamma^t r_t$, $0 < \gamma < 1$ 。

Deep RL：用深度神经网络（如 MLP）表示策略 $\pi_{\theta}(a|s)$ ，用梯度上升优化参数 θ 。

分水岭工作——DeepMimic (Peng et al. 2018)：把参考动作（Mocap）纳入奖励函数：

$$r_t = r_t^{\text{imitation}} + r_t^{\text{task}} \quad (5)$$

前一项奖励角色”像参考动作”，后一项奖励完成任务（如保持站立）。这使 RL 角色能学会有风格的跳跃、翻滚、武术等 athletic motion。

6.10 Hierarchical Controllers 分层控制器

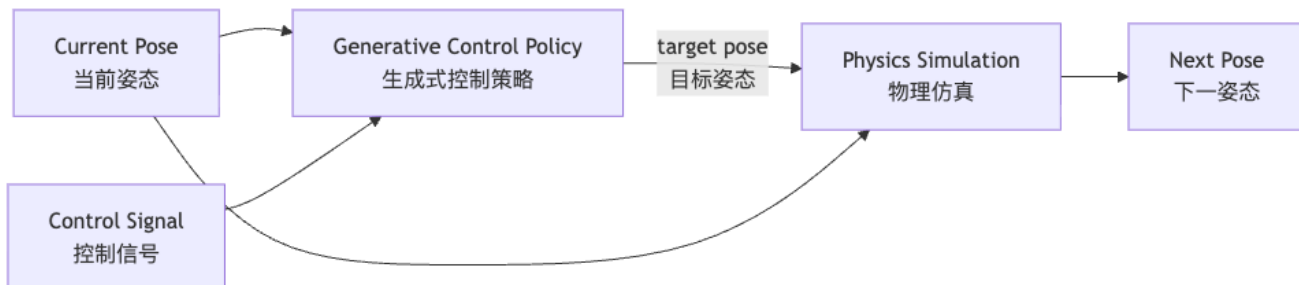
动机：DeepMimic 的一个 tracking controller 只能跟踪一段参考动作。怎么做多技能角色？

结构：– 底层 (Low-level)：多个跟踪控制器，每个擅长一类动作 – 上层 (High-level)：调度器决定”现在该运行哪个底层控制器”

代表：Liu et al. 2017 《Learning to Schedule Control Fragments》——上层控制器是用 RL 训练的调度器，能让角色完成高层运动目标。

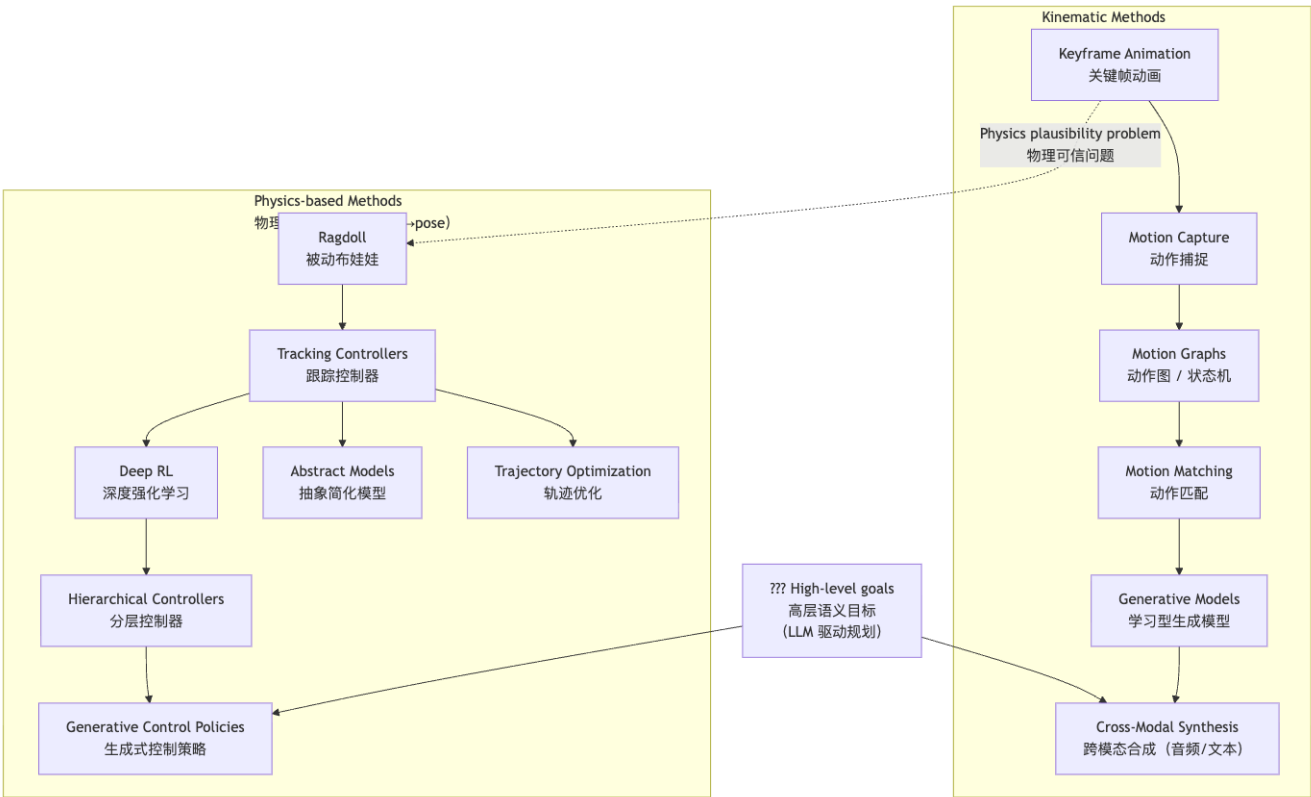
6.11 Generative Control Policies 生成式控制策略

动机：把”生成模型的多样性”和”物理仿真的真实感”结合。



与纯 Kinematic 生成模型的区别：生成模型给出”想去的 pose”，物理仿真负责”真正去到那个 pose”——接触、惯性、平衡由物理保证。

7. 两类方法的全景对比



[TIP] 复习要点

上图是全课的”索引”：每一讲都在深入这张图里的某个节点。记住：– 所有 Kinematic 方法在同一条纵轴上，各自解决前一个方法的短板 – 物理方法的核心问题是”控制”，而控制的核心工具是优化（轨迹优化）和学习（RL）– 两类方法的交叉点（Generative Control Policies、LLM+physics）是当前研究前沿

8. 课程路线图

8.1 课程定位

本课不讲：– 如何使用 Maya / MotionBuilder / Houdini / Unity / Unreal Engine – 如何成为一名职业动画师
本课讲：– 动作背后的方法、理论、技术 – 如何从头构建：机器狗、人形机器人.....的运动控制器

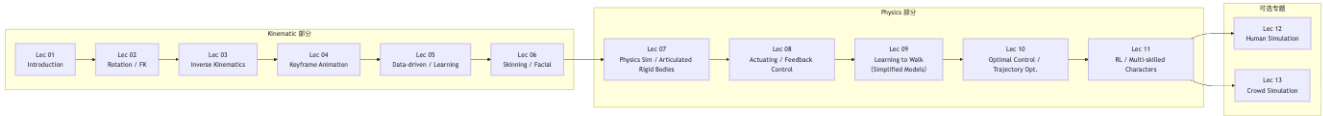
8.2 考核结构

组成	权重	内容
Class Participation	15%	包括课堂测试
Programming Exercises	60%	4 个编程作业（禁止抄袭）
Project	25%	按难度、完成度、性能、技术与艺术评分（禁止一稿多投）

编程作业预告：1. Exercise 1: FK / IK 2. Exercise 2: Play with motion data（动作数据处理）3. Exercise

3: Simulation and ragdoll (物理仿真 + 布娃娃) 4. **Exercise 4:** Make your robot move! (让机器人动起来) 5. **Project S:** Interactive & simulated character (交互式仿真角色)

8.3 讲次规划



Lec	主题	对应方法
01	Introduction (本讲)	全局地图
02	Rotation, Transformation, FK	数学基础 + FK
03	Inverse Kinematics	IK 理论与算法
04	Keyframe Character Animation	Keyframe + 插值
05	Data-driven / Learning-based Animation	Motion Graphs, Matching, Generative
06	Skinning & Facial Animation	蒙皮、混合形变
07	Physics-based Simulation & Articulated Rigid Bodies	物理仿真基础
08	Actuating Character & Feedback Control	关节驱动 + PD 控制
09	Learning to Walk with Simplified Models	抽象模型 (SIMBICON, 倒立摆)
10	Optimal Control & Trajectory Optimization	优化方法
11	Reinforcement Learning & Multi-skilled Characters	RL, DeepMimic, 分层控制
12	Human Simulation (可选)	肌肉骨骼仿真
13	Crowd Simulation (可选)	人群 Agent 行为

9. Worked Examples 例题

[EX] 例题 1: 概念辨析——Kinematic 方法 vs. Physics-based 方法

问：以下哪种场景最适合用 physics-based 方法而非 kinematic 方法？请说明理由。(a) 制作一段电影中角色跳舞的动画，所有动作预先确定；(b) 游戏中角色被爆炸冲击波推倒后需要自然地倒地；(c) 用文字描述”向前走”驱动角色运动。

答：(b) 最适合 physics-based 方法。被冲击波推倒是与环境的**被动交互**，需要接触力、惯性、关约束共同决定身体如何运动，纯 kinematic 方法无法在未预设的冲击方向和强度下给出自然结果。

(a) 是典型 keyframe 动画场景，预先确定的 pose 序列无需物理；(c) 是 cross-modal synthesis 或 LLM 规划的范畴，不需要 physics-based 方法（虽然后者可以与 physics 结合）。

[EX] 例题 2: 方法定位——把方法放进坐标系

问：将以下方法标注在”high-level goals low-level control physics”坐标轴上：(A) Motion Matching；(B) DeepMimic；(C) LLM-driven 任务分解；(D) Ragdoll Simulation。

答：- (A) Motion Matching：处于 low-level control 层，根据实时控制信号（速度方向）检索 pose，无物理 - (B) DeepMimic：横跨 low-level control → physics, RL 策略输出力矩，物理仿真出 pose - (C) LLM-driven 任务分解：处于 high-level goals 层，把语义任务分解为动作基元 - (D) Ragdoll Simulation：纯 physics 层，无 controller，只有被动物理响应

[EX] 例题 3: FK 与 IK 的计算方向

问：一条简单机械臂有 2 个旋转关节，各旋转角为 θ_1, θ_2 ，臂长为 l_1, l_2 。(a) FK 问题：给定 $\theta_1 = 30^\circ, \theta_2 = 45^\circ$ ，求末端位置；(b) IK 问题：给定目标末端位置 (x, y) ，如何反求 θ_1, θ_2 ？

答（定性）：(a) FK 是直接正向计算：

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2), \quad y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

代入数值得唯一解。

(b) IK 是反问题：给定 (x, y) ，上述方程有 2 个未知数，理论上有限个解（肘关节向上或向下两种构型），也可能无解（目标超出可达范围）。3D 中 DOF 更多时，IK 通常欠定（多解）。IK 算法（如 Jacobian 伪逆、CCD）是 Lec03 的核心。

[EX] 例题 4: 奖励函数设计——DeepMimic 的思路

问：DeepMimic 用 $r_t = r_t^{\text{imitation}} + r_t^{\text{task}}$ 作为奖励。如果只用 r_t^{task} （例如“角色不摔倒”），会有什么问题？如果只用 $r_t^{\text{imitation}}$ （pose 与参考完全匹配），又会有什么问题？

答：- 只用 r_t^{task} ：策略可能学会“原地不动”（永远不摔倒，但也不运动）或学到非自然的姿态（如趴地爬行），缺乏运动风格约束。- 只用 $r_t^{\text{imitation}}$ ：角色能精确跟踪参考动作，但对环境扰动（被撞了一下）极其脆弱——参考动作里没有被撞后恢复的帧，策略完全不知道如何应对。- 组合奖励：既有风格（像参考），又有任务鲁棒性（不摔倒）——两项互补。

10. Anki 记忆卡片

#	Q (正面)	A (背面)
1	3D 计算机图形学的三大支柱？	Geometry (几何建模) / Animation (动画) / Rendering (渲染)
2	Simulation Animation 和 Character Animation 的根本差异？	Simulation 模拟被动物理现象（风格），Character Animation 模拟主动行为（Behavior），后者有“+ Intelligence”
3	Character animation 的 Motion Model 抽象形式？	$(s_t, c_t) \xrightarrow{\text{Motion Model}} s_{t+1}$, s 是状态, c 是控制信号
4	人体运动的生理链条（从神经到动作）？	神经激励 → 肌肉激活 → 力矩作用于肌骨骼系统 → 物理 → 体态
5	Kinematic 方法在生理链条的哪里“切入”？	直接更新 Body Pose, 跳过力和物理
6	Physics-based 方法在生理链条的哪里“切入”？	在 Forces/Torques 处介入，输出关节力矩，物理引擎积分出 pose
7	FK 和 IK 的方向各是什么？	FK: 关节角 → 末端位置（正向，唯一解）；IK: 末端位置 → 关节角（反向，可能多解或无解）
8	Motion Matching 的核心机制？	每帧用查询特征（当前 pose + 速度 + 轨迹预测）在 Mocap 库中检索最近邻帧
9	Ragdoll Simulation 是什么？有什么局限？	无控制器，角色完全被动被物理驱动；只会“倒”，不能主动运动
10	Tracking Controller 的 PD 控制公式？	$\tau_i = k_p(\theta_i^{\text{ref}} - \theta_i) - k_d\dot{\theta}_i$, k_p 刚度, k_d 阻尼
11	力矩 (torque) 的计算公式？	$\tau = \sum_i r_i \times F_i$
12	DeepMimic 的奖励函数组成？	$r_t = r_t^{\text{imitation}}$ (模仿参考动作) + r_t^{task} (完成任务)
13	Trajectory Optimization 和 Reinforcement Learning 的关键区别？	Trajectory Opt. 离线求整段轨迹；RL 学在线策略 $\pi(a s)$ ，能实时响应任意状态

#	Q (正面)	A (背面)
14	Kinematic 方法的两大根本缺陷?	Physical plausibility (脚滑穿模) + Environment interaction (无法泛化到未见场景)
15	本课 Kinematic 部分对应哪几讲?	Lec 02–06 (FK/IK/Keyframe/Data-driven/Skinning)
16	本课 Physics 部分对应哪几讲?	Lec 07–11 (物理仿真/反馈控制/简化步态/优化/RL)
17	什么是 Motion Retargeting?	把源角色的动作迁移到骨骼比例/数量/拓扑不同的目标角色

11. 中英术语速查表

中文	English	首次出现
角色动画	Character Animation	§1.1
关节	Joint	§0
自由度	DOF (Degree of Freedom)	§0
姿态	Pose	§0
运动学	Kinematics	§0
动力学	Dynamics	§0
末端执行器	End-effector	§5.1
正运动学	Forward Kinematics (FK)	§5.1
逆运动学	Inverse Kinematics (IK)	§5.1
动作捕捉	Motion Capture (Mocap)	§5.2
动作重定向	Motion Retargeting	§5.2
动作图	Motion Graph	§5.3
状态机	State Machine	§5.3
动作匹配	Motion Matching	§5.4
跨模态合成	Cross-Modal Synthesis	§5.6
破布娃娃仿真	Ragdoll Simulation	§6.2
跟踪控制器	Tracking Controller	§6.3
比例-微分控制	PD Control	§6.3
空时优化 / 轨迹优化	Spacetime / Trajectory Optimization	§6.7
倒立摆	Inverted Pendulum	§6.8
有限状态机	FSM (Finite State Machine)	§6.8
强化学习	Reinforcement Learning (RL)	§6.9
累积折扣奖励	Cumulative Discounted Reward	§6.9
策略	Policy $\pi(a s)$	§6.9
分层控制器	Hierarchical Controller	§6.10
生成式控制策略	Generative Control Policy	§6.11
蒙皮	Skinning	§2 (pipeline)
骨骼绑定	Rigging	§2 (pipeline)
关键帧	Keyframe	§5.1
中间帧补算	In-betweening	§5.1
转描机 / 转描法	Rotoscoping	§5.1

[TIP] 期末复习要点

1. **3D 动画的分野**: Simulation (现象) vs. Character Animation (行为 + Intelligence), 后者是本课主题 2. **两类方法的分叉点**: 在人体运动链条的哪里”切入”——Kinematic 直接写 pose, Physics-based 输出力矩 3. **Kinematic 谱系** (按问题演进): Keyframe → Mocap + Retargeting → Motion Graphs → Motion Matching → Generative Models → Cross-Modal → LLM 规划 4. **Physics 谱系** (按控制复杂度): Ragdoll (无控制) → Tracking Controllers → Trajectory Optimization → Abstract Models → Deep RL → Hierarchical → Generative Control Policies 5. **两大根本缺陷**: Kinematic 方法缺乏 Physical Plausibility 和 Environment Interaction, 这是 Physics-based 方法存在的理由 6. **课程路线**: Lec02–06 深入 Kinematic; Lec07–11 深入 Physics; LLM 融合是当前前沿 (“???”节点)

Lecture 02 —数学基础 (Math Background)

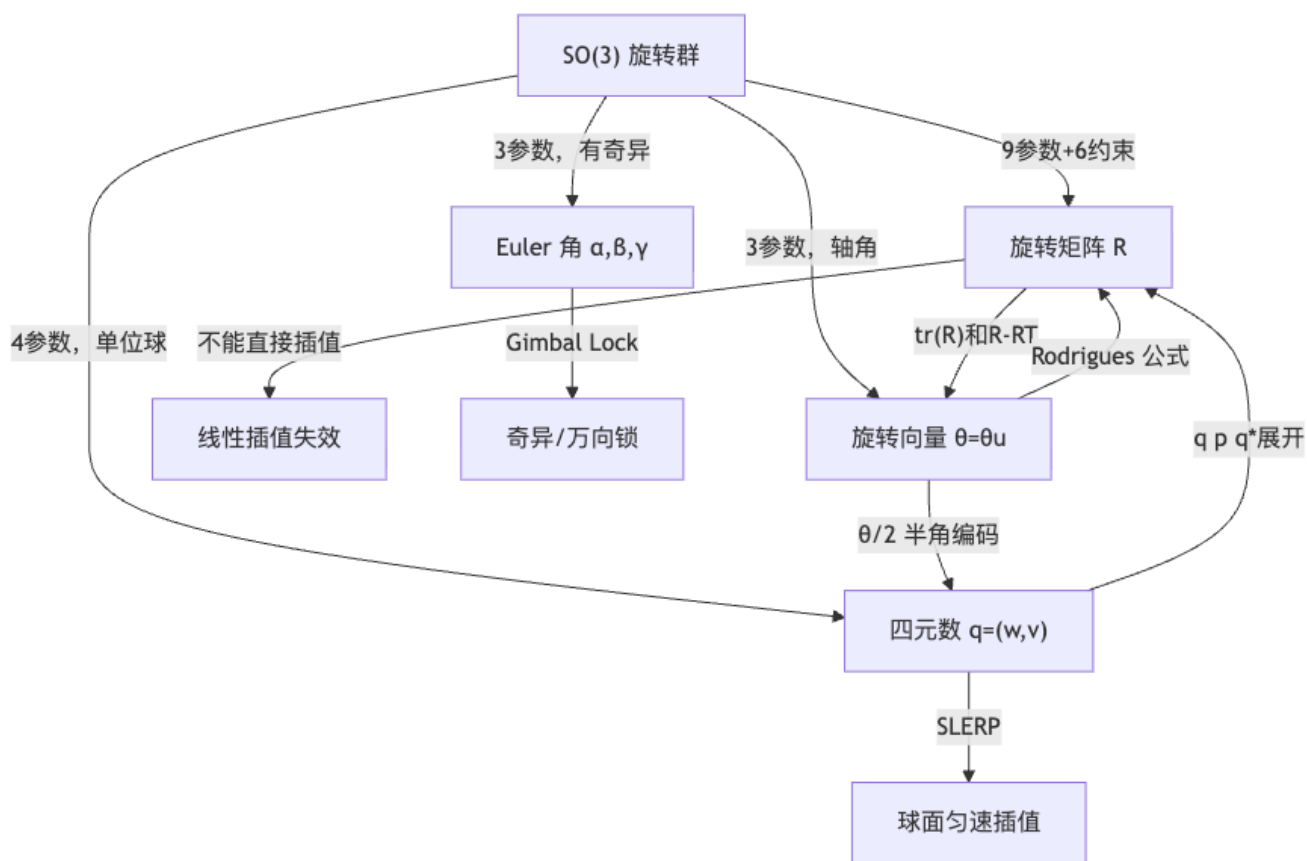
[TIP] 学习指南

本节核心：角色动画的“姿态”是 $SE(3)$ 元素——平移 $t \in \mathbb{R}^3$ 加旋转 $R \in SO(3)$ 。平移是平凡线性空间；**旋转所在流形非线性、非交换**，必须选一种参数化表示。本讲的主线有两条：(1) 打牢线性代数基础——向量/矩阵/点积/叉积/正交矩阵/特征值；(2) 系统学习四种 3D 旋转表示——旋转矩阵、Euler 角、旋转向量/轴角、四元数——各自的定义、几何直觉、相互转换、优缺点对比，以及插值问题。

前置知识：基础线性代数（矩阵乘法/转置/逆），三角函数，复数（选读）。

重点掌握：1. Rodrigues 公式的推导与 $R \leftrightarrow (u, \theta)$ 互转 2. $SO(3)$ 的代数结构与 6 个约束条件 3. 四元数乘法公式及旋转公式 $p' = q(0, p)q^*$ 的推导 4. SLERP 的推导与“双覆盖”陷阱 5. Gimbal Lock 的成因与轴角表示的不唯一性 6. 反对称矩阵 $[\omega]_{\times}$ 与角速度的关系 $\dot{R} = [\omega]_{\times} R$

0. 概念导图



1. 线性代数基础

1.1 向量与模

定义 1.1 (向量) n 维列向量 $a = (a_1, a_2, \dots, a_n)^T \in \mathbb{R}^n$ 具有大小与方向两个属性。

$$\|a\|_2 = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \quad (1.1)$$

单位化: $\hat{a} = a/\|a\|$, 满足 $\|\hat{a}\| = 1$ 。3D 向量 $a = (a_x, a_y, a_z)^T$ 在笛卡尔坐标系下分解为

$$a = a_x e_x + a_y e_y + a_z e_z \quad (1.2)$$

其中 e_x, e_y, e_z 构成标准正交基: $\|e_i\| = 1$, $e_i \cdot e_j = 0$ ($i \neq j$), $e_x \times e_y = e_z$ (右手系)。

1.2 点积 (Dot Product)

$$a \cdot b = \sum_{i=1}^n a_i b_i = a^T b = \|a\| \|b\| \cos \theta \quad (1.3)$$

三个主要用途：

用途	公式
夹角	$\theta = \arccos \frac{a \cdot b}{\ a\ \ b\ }$
投影 (b 在 a 方向的标量分量)	$b_a = \frac{b \cdot a}{\ a\ }$
垂直判定	$a \cdot b = 0 \Leftrightarrow a \perp b$

[NOTE] 直觉理解

点积是“两向量方向对齐程度”的度量：完全同向 $\cos \theta = 1$ ，垂直 $\cos \theta = 0$ ，反向 $\cos \theta = -1$ 。神经网络中的相似度打分本质上都是点积。

1.3 叉积 (Cross Product)

定义 1.2 (叉积) 3D 向量 a, b 的叉积：

$$c = a \times b = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix} = \|a\| \|b\| \sin \theta n \quad (1.4)$$

分量记忆规律： $x : yz, y : zx, z : xy$ (循环置换)。

性质 (完整列表)：

- $c \perp a, c \perp b$; n 由右手定则确定。
- 反交换： $a \times b = -b \times a$ 。
- 分配律： $a \times (b + d) = a \times b + a \times d$ 。
- 无结合律： $a \times (b \times c) \neq (a \times b) \times c$ (相差 BAC-CAB 项)。
- 平行判定： $a \times b = 0 \Leftrightarrow a \parallel b$ (含零向量情形)。

也可表示为行列式展开：

$$a \times b = \det \begin{pmatrix} i & j & k \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{pmatrix} \quad (1.5)$$

两向量间的最小旋转：给定非平行 a, b ，最短弧旋转的轴与角为

$$u = \frac{a \times b}{\|a \times b\|}, \quad \theta = \arccos \frac{a \cdot b}{\|a\| \|b\|} \quad (1.6)$$

[WARN] 易错点

若 $a \parallel b$ (同向或反向)，则 $a \times b = 0$ ，无法用式 (1.6) 求轴——轴在垂直于 a 的平面内任意选取 (反向时 $\theta = \pi$ ，轴自由)。

1.4 叉积的矩阵形式：反对称矩阵

定义 1.3 (反对称矩阵 / 叉积矩阵) 对 $a = (a_x, a_y, a_z)^T$, 定义

$$[a]_{\times} = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix} \quad (1.7)$$

则 $a \times b = [a]_{\times} b$ 。矩阵满足 $[a]_{\times}^T = -[a]_{\times}$ (反对称性)。

重要恒等式：

$$[a]_{\times} b = -[b]_{\times} a \quad (1.8)$$

$$a \times (b \times c) = [a]_{\times} [b]_{\times} c \quad (1.9)$$

$$[a]_{\times}^2 b = a \times (a \times b) = (aa^T - \|a\|^2 I) b \quad (1.10)$$

[NOTE] 直觉理解

把叉积写成矩阵乘法的好处：矩阵乘法满足结合律，可以把多次叉积的链式运算化为矩阵乘积 $[a]_{\times} [b]_{\times}$ ，这是后面 Rodrigues 公式推导的关键步骤。

[WARN] 易错点

$[a]_{\times} [b]_{\times} c$ 对应的是 $a \times (b \times c)$ ，**不是** $(a \times b) \times c$ 。后者等于 $[a \times b]_{\times} c = [a]_{\times} [b]_{\times} c - [b]_{\times} [a]_{\times} c$ (由 BAC-CAB 恒等式可验证)。

1.5 矩阵基础

特殊矩阵：对角矩阵、单位矩阵 I 、对称矩阵 $A^T = A$ 、反对称矩阵 $A^T = -A$ 。矩阵乘法满足结合律、分配律，不满足交换律 $AB \neq BA$ 。重要运算规律：

$$(AB)^T = B^T A^T, \quad (AB)^{-1} = B^{-1} A^{-1} \quad (1.11)$$

正交矩阵 (Orthogonal Matrix)：列 (或行) 两两单位正交。等价定义：

$$A^T A = A A^T = I \Leftrightarrow A^{-1} = A^T \quad (1.12)$$

行列式 $\det A = \pm 1$ 。正交变换保长度、保内积，是等距映射。

行列式： $\det(AB) = \det A \cdot \det B$, $\det(A^T) = \det A$, $\det(A^{-1}) = 1/\det A$ 。

特征值与特征向量：若 $Ax = \lambda x$, $x \neq 0$ ，则 λ 为特征值， x 为对应特征向量。

定理 1.1: 3×3 正交矩阵至少有一个实特征值 $\lambda = \det A = \pm 1$ 。

[NOTE] 直觉理解

正交矩阵的实特征值对应“不动向量”。旋转矩阵 ($\det R = +1$) 的实特征值为 $+1$ ，对应的特征向量就是旋转轴 u ——它在旋转中保持不动。这是后面求旋转轴的理论依据。

2. 刚体变换: $SE(3)$

定义 2.1 (刚体变换) 保形状、保距离的变换, 由平移 $t \in \mathbb{R}^3$ 和旋转 $R \in SO(3)$ 组合:

$$a' = Ra + t \quad (2.1)$$

缩放 (非刚体): $a' = \text{diag}(s_x, s_y, s_z)a$, 会改变形状。

平移的合成: $t = t' + t''$, 可交换。

旋转的合成: 先旋转 R_1 再旋转 R_2 , 合成为 $R = R_2 R_1$ (注意右乘顺序), 不可交换 $R_2 R_1 \neq R_1 R_2$ (一般情况)。

2.1 旋转矩阵的性质与 $SO(3)$

定义 2.2 (特殊正交群)

$$SO(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^T R = I, \det R = +1\} \quad (2.2)$$

9 个参数受 6 个正交约束 (3 个列单位化 + 3 个列两两正交), 实际自由度 = $9 - 6 = 3$ 。

旋转矩阵保长度: $\|Rx\| = \|x\|$ (由 $\|Rx\|^2 = x^T R^T R x = x^T x$ 即得)。 $\det R = +1$ 区别于反射 ($\det = -1$)。

[NOTE] 直觉理解

$SO(3)$ 是三维李群, 流形维度为 3, 嵌入于 $\mathbb{R}^9$ 中是弯曲的。这意味着旋转矩阵的简单线性插值 $(1 - t)R_0 + tR_1$ 不再在 $SO(3)$ 上, 插值结果不是合法旋转矩阵。

2.2 绕坐标轴的基本旋转

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}, \quad R_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \quad (2.3)$$

$$R_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (2.4)$$

验证 (以 R_z 为例): $R_z^T R_z = I$, $\det R_z = \cos^2 \gamma + \sin^2 \gamma = 1$ 。

2.3 坐标变换: 物体系 $\leftrightarrow$ 全局系

设物体局部正交基 e'_x, e'_y, e'_z 在世界系下排列为矩阵 $R = [e'_x | e'_y | e'_z]$, 则:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix}_{\text{world}} = R \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}_{\text{local}} + t \quad (2.5)$$

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}_{\text{local}} = R^T \left(\begin{pmatrix} x \\ y \\ z \end{pmatrix}_{\text{world}} - t \right) \quad (2.6)$$

[NOTE] 直觉理解

记忆口诀：“世界 = $R \cdot$ 局部 + t ”，逆变换用 R^T 。后续骨骼蒙皮、IK、刚体仿真全部基于此公式。

3. Rodrigues 公式：从轴角到旋转矩阵

3.1 几何推导

问题：给定单位轴 u ($\|u\| = 1$) 和角度 θ ，求 a 绕 u 旋转 θ 后的向量 b 。

推导步骤：

设 a 在旋转后变为 $b = a + v + t$ ，其中 v 是沿圆弧切向的分量， t 是沿圆心方向的分量。

在以 u 为法向的旋转平面内，圆周方向为 $u \times a$ ，向心方向为 $u \times (u \times a)$ 。由几何观察（在 $u^\perp$ 平面内的极坐标分析）：

$$v = \sin \theta (u \times a), \quad t = (1 - \cos \theta) u \times (u \times a) \quad (3.1)$$

因此

$$b = a + \sin \theta (u \times a) + (1 - \cos \theta) u \times (u \times a) \quad (3.2)$$

3.2 矩阵形式 (Rodrigues 公式)

将 $u \times = [u]_\times$ 代入式 (3.2)：

$$b = (I + \sin \theta [u]_\times + (1 - \cos \theta) [u]_\times^2) a = R a$$

[NOTE] 定理 3.1 (Rodrigues' Rotation Formula)

$$R(u, \theta) = I + \sin \theta [u]_\times + (1 - \cos \theta) [u]_\times^2 \quad (3.3)$$

这是将“绕单位轴 u 转角 θ ”显式写成 3×3 旋转矩阵的核心公式，后续 $\dot{R} = [\omega]_\times R$ 、指数映射 $\exp([\omega]_\times)$ 、四元数 $\leftrightarrow$ 矩阵的互转均建立于此。

验证 $R \in SO(3)$ ：因 $[u]_\times$ 反对称、 $\|u\| = 1$ ，可直接验证 $R^T R = I$ ， $\det R = 1$ （过程略，使用 Cayley-Hamilton 定理）。

[EX] 例题 3.1: 绕 z 轴转 90°

$$u = (0, 0, 1)^T, \theta = 90^\circ, \sin \theta = 1, \cos \theta = 0.$$

$$[u]_{\times} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, [u]_{\times}^2 = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

$$R = I + [u]_{\times} + [u]_{\times}^2 = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = R_z(90^\circ), \text{ 与式 (2.4) 一致。}$$

3.3 逆问题: 从 R 提取 (u, θ)

提取角度: 利用 Rodrigues 公式计算迹:

$$\begin{aligned} \text{tr}(R) &= \text{tr}(I) + \sin \theta \underbrace{\text{tr}([u]_{\times})}_{=0} + (1 - \cos \theta) \underbrace{\text{tr}([u]_{\times}^2)}_{=-2} \\ \text{tr}(R) &= 3 - 2(1 - \cos \theta) = 1 + 2 \cos \theta \end{aligned} \quad (3.4)$$

$$\boxed{\theta = \arccos \frac{\text{tr}(R) - 1}{2}} \quad (3.5)$$

提取轴: 由 $Ru = u$ 得 $(R - I)u = 0$ 。等价地, 注意 $R - R^T = 2 \sin \theta [u]_{\times}$ (对 Rodrigues 公式取反对称部分), 直接读出:

$$u' = \begin{pmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{pmatrix}, \quad \|u'\| = 2 \sin \theta \quad (3.6)$$

当 $\sin \theta \neq 0$ 时, 归一化 $u = u' / \|u'\|$ 。

[WARN] 易错点: 退化情形

- $\theta = 0$: $R = I$, $R - R^T = 0$, 轴任意选取, $u' = 0$ 无法归一化。
- $\theta = \pi$: $\sin \theta = 0$, 仍然 $u' = 0$, 改用 $R + R^T = 2(I + 2uu^T - 2I) + 2I = 2(2uu^T - I) + 2I$, 即 $R + I = 2uu^T$, 从对角元开方提轴 (注意符号歧义)。

4. 旋转表示 1: 旋转矩阵

旋转矩阵是最“自然”的表示, 直接来自 $SO(3)$ 定义。

操作	实现方式	评价
向量旋转	$b = Ra$	容易 (矩阵乘法)
合成两个旋转	$R = R_2 R_1$	容易
求逆旋转	$R^{-1} = R^T$	容易

操作	实现方式	评价
存储空间	9 个浮点数	冗余 (实际 DoF=3)
插值	$(1-t)R_0 + tR_1$ 不合法	困难
直观编辑	无直观语义	困难

[WARN] 易错点：旋转矩阵插值失效

$$\text{取 } R_0 = R_y(-90^\circ) = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}, R_1 = R_y(+90^\circ) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}.$$

则 $R_{0.5} = 0.5(R_0 + R_1) = \text{diag}(0, 1, 0)$ ，行列式为 0，根本不是旋转矩阵。说明 $SO(3)$ 在 $\mathbb{R}^9$ 中是弯曲流形，不能做欧氏插值。

实际应用场景：旋转矩阵常用于引擎内部计算（GPU 变换流水线）和骨骼蒙皮，不用于存储或插值。

5. 旋转表示 2：Euler 角

核心思想：任意旋转 = 三次绕坐标轴旋转的乘积。

$$R = R_{a_3}(\gamma)R_{a_2}(\beta)R_{a_1}(\alpha) \quad (5.1)$$

5.1 旋转顺序的 12 种约定

共 12 种顺序：– **Tait-Bryan 角**（三轴互不相同，6 种）：XYZ, XZY, YXZ, YZX, ZXY, ZYX。– **Proper Euler 角**（首尾轴相同，6 种）：XYX, XZX, YXY, YZY, ZXZ, ZYZ。

例：ZYX 顺序（航空惯例）： $R = R_z(\gamma)R_y(\beta)R_x(\alpha)$ ，先绕 x 转俯仰（pitch） α ，再绕 y 转偏航（yaw） β ，最后绕 z 转滚转（roll） γ 。

5.2 内旋（Intrinsic）与外旋（Extrinsic）

类型	旋转轴	矩阵形式
外旋（Extrinsic）	固定世界坐标轴，从左到右乘	$R_z(\gamma)R_y(\beta)R_x(\alpha)$
内旋（Intrinsic）	物体自身局部轴，从右到左乘	$R_x(\alpha)R_y(\beta)R_z(\gamma)$

代数关系：外旋顺序 $X \rightarrow Y \rightarrow Z$ 等价于内旋顺序 $Z \rightarrow Y \rightarrow X$ （反序相等）：

$$R_z^{\text{extrinsic}}(\gamma)R_y^{\text{extrinsic}}(\beta)R_x^{\text{extrinsic}}(\alpha) = R_x^{\text{intrinsic}}(\alpha)R_y^{\text{intrinsic}}(\beta)R_z^{\text{intrinsic}}(\gamma) \quad (5.2)$$

5.3 万向锁（Gimbal Lock）

定理 5.1 (Gimbal Lock)：在 ZYX 顺序中，当中间轴角 $\beta = \pm 90^\circ$ 时，三自由度退化为两自由度。

推导：以 $\beta = 90^\circ$ 为例， $R_y(90^\circ) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}$ ，代入后：

$$R_z(\gamma)R_y(90^\circ)R_x(\alpha) = R_z(\gamma - \alpha) \cdot (\text{fixed part}) \quad (5.3)$$

只剩 $\gamma - \alpha$ 一个自由参数，第三轴被“锁定”——无法独立控制 γ 和 α 。

[WARN] 易错点：万向锁的后果

1. 奇异性：在 $\beta = \pm 90^\circ$ 附近，Euler 角数值可能突变（“猛抽一下”），插值路径不平滑。
2. 插值非恒速：即使没到奇异点，三个 Euler 角分量独立线性插值也不等于旋转匀速。
3. 数值稳定性：逆运动学（IK）在奇异位形附近雅可比矩阵病态。

Euler 角优缺点汇总：

优点	缺点
参数少（3 个），直观（美术常用） 约定多（12 种可选） 便于用户界面编辑	Gimbal Lock 插值非恒速，需转矩阵合成 难以直接指定目标姿态

6. 旋转表示 3：旋转向量 / 轴角

6.1 定义

轴角 (Axis-Angle)：用单位轴 u ($\|u\| = 1$) 和旋转角 θ 联合表示，共 4 个参数，1 个约束 ($\|u\| = 1$)，DoF = 3。

旋转向量 (Rotation Vector)：更紧凑的 3 参数形式，令

$$\theta = \theta u \in \mathbb{R}^3 \quad (6.1)$$

则 $\theta = \|\theta\|$, $u = \theta / \|\theta\|$ (当 $\theta \neq 0$)。

6.2 应用旋转

通过 Rodrigues 公式 (3.3) 先转为旋转矩阵，再乘向量。或直接用向量形式：

$$x' = x + \sin \theta (u \times x) + (1 - \cos \theta) u \times (u \times x) \quad (6.2)$$

6.3 插值方法

方法一（直接线性插值）：

$$\theta_t = (1 - t)\theta_0 + t\theta_1 \quad (6.3)$$

结果 θ_t 合法（可用 Rodrigues 转为旋转矩阵），但旋转角速度**不恒定**（弦非弧）。

方法二（偏移旋转法，角速度恒定）：

1. 计算偏移旋转： $R_{\delta\theta} = R^T(\theta_0)R(\theta_1)$ ，提取其旋转向量 $\delta\theta$ ；
2. 线性插值偏移量： $\delta\theta_t = t \delta\theta$ ；
3. 重组： $R(\theta_t) = R(\theta_0)R(\delta\theta_t)$ 。

$$R(\theta_t) = R(\theta_0) R(t \delta\theta), \quad \delta\theta = \log(R^T(\theta_0)R(\theta_1)) \quad (6.4)$$

这等价于在旋转李群的指数映射下的测地线插值（见 §8 指数映射）。

[WARN] 易错点：表示不唯一

$(u, \theta) \sim (-u, -\theta) \sim (u, \theta + 2n\pi)$, $\theta = 0$ 时轴 u 未定义。工程中通常约定 $\theta \in [0, \pi]$ （最短弧），但插值时仍需检查是否翻转。

轴角/旋转向量优缺点：

优点	缺点
紧凑（3 个参数）	应用需先转矩阵
无 Gimbal Lock	合成无简单解析公式
几何直觉强	表示不唯一（ $\theta = 0$ 轴未定）
直接线性插值合法	直接插值角速度不恒定

7. 旋转表示 4：四元数

7.1 从复数到四元数

2D 旋转用单位复数表示： $z = e^{i\theta} = \cos \theta + i \sin \theta$ ，旋转 $z' = e^{i\alpha} z$ 。Hamilton 1843 年推广：引入两个新虚单位 j, k ，满足

$$i^2 = j^2 = k^2 = ijk = -1 \quad (7.1)$$

$$ij = k, ji = -k; \quad jk = i, kj = -i; \quad ki = j, ik = -j \quad (7.2)$$

[NOTE] 直觉理解

虚单位间的乘法规律与叉积完全对应： $ij = k$ 对应 $e_x \times e_y = e_z$ ，这是四元数能编码 3D 旋转的根本原因。

定义 7.1（四元数）一般四元数 $q = a + bi + cj + dk \in \mathbb{H}$, $a, b, c, d \in \mathbb{R}$ ，记为

$$q = \begin{pmatrix} w \\ v \end{pmatrix} = (w, v)^T, \quad w \in \mathbb{R} \text{ (实部)}, \quad v = (x, y, z)^T \in \mathbb{R}^3 \text{ (虚部)} \quad (7.3)$$

纯四元数 (pure quaternion): $w = 0$ ，记为 $(0, v)$ ——用于嵌入 3D 向量。

7.2 四元数代数

运算	定义	说明
加法	$q_1 + q_2 = (w_1 + w_2, v_1 + v_2)$	逐分量
数乘	$tq = (tw, tv)$	逐分量
点积	$q_1 \cdot q_2 = w_1 w_2 + v_1 \cdot v_2$	标量

运算	定义	说明
模	$\ q\ = \sqrt{q \cdot q} = \sqrt{w^2 + \ v\ ^2}$	
共轭	$q^* = (w, -v)$	翻转虚部符号

乘法 (Hamilton product): 按分配律展开, 利用虚单位规则 (7.1)(7.2) 整理:

$$q_1 q_2 = \left(w_1 w_2 - v_1 \cdot v_2 + w_1 v_2 + w_2 v_1 + v_1 \times v_2 \right) \quad (7.4)$$

[NOTE] 公式 (7.4) 推导草图

展开 $(w_1 + v_1)(w_2 + v_2)$: - 标量 $\times$ 标量 $\rightarrow w_1 w_2$ - 标量 $\times$ 向量 $\rightarrow w_1 v_2 + w_2 v_1$ - 向量 $\times$ 向量: $v_1 v_2 = -v_1 \cdot v_2 + v_1 \times v_2$ (对应 $i^2 = -1$ 和 $ij = k$ 等规则)
合并实部 $w_1 w_2 - v_1 \cdot v_2$, 虚部 $w_1 v_2 + w_2 v_1 + v_1 \times v_2$, 即得。

乘法性质 (完整):

$$q_1 q_2 \neq q_2 q_1 \quad (\text{非交换, 一般情形}) \quad (7.5)$$

$$(q_1 q_2) q_3 = q_1 (q_2 q_3) \quad (\text{结合律}) \quad (7.6)$$

$$(q_1 q_2)^* = q_2^* q_1^* \quad (7.7)$$

$$\|q\|^2 = q q^* = q^* q \quad (\text{纯标量}) \quad (7.8)$$

$$q^{-1} = \frac{q^*}{\|q\|^2} \quad (7.9)$$

7.3 单位四元数与旋转

定义 7.2 (单位四元数) $\|q\| = 1$, 此时 $q^{-1} = q^*$ 。全体单位四元数构成 4D 单位超球 S^3 。

编码旋转: 绕单位轴 u 转角 θ 的旋转编码为

$$q = \left(\cos \frac{\theta}{2}, u \sin \frac{\theta}{2} \right) \quad (7.10)$$

验证: $\|q\|^2 = \cos^2(\theta/2) + \sin^2(\theta/2)\|u\|^2 = 1$ 。

反向提取: $\theta = 2 \arccos w$, $u = v/\|v\|$ (当 $\|v\| \neq 0$)。

[NOTE] 直觉理解: 为什么用 $\theta/2$?

设纯四元数 $p = (0, p)$ 嵌入 3D 向量。要验证 $q(0, p)q^*$ 确实表示绕 u 转 θ : 展开四元数乘积后, $\cos(\theta/2)$ 和 $\sin(\theta/2)$ 恰好通过二倍角公式组合出 $\cos \theta$ 和 $\sin \theta$, 最终给出 Rodrigues 公式。角度“压缩”一半是这个映射的代价, 也是双覆盖的根源。

7.4 用单位四元数旋转向量

将 3D 向量 p 嵌入为纯四元数 $(0, p)$ 。

定理 7.1 (四元数旋转公式)

$$(0, p') = q(0, p)q^* \quad (7.11)$$

其中 p' 就是 p 绕 u 旋转 θ 后的向量。

证明思路 (展开验证):

将 $q = (c, su)$ ($c = \cos(\theta/2)$, $s = \sin(\theta/2)$) 代入 $q(0, p)q^*$:

第一步: $q(0, p) = (-s(u \cdot p), cp + su \times p)$ (用式 (7.4))。

第二步: 右乘 $q^* = (c, -su)$, 利用二倍角公式 $c^2 - s^2 = \cos \theta$, $2cs = \sin \theta$ 整理, 标量部分为 0, 矢量部分恰好等于 Rodrigues 公式 (3.2)。□

双覆盖: $(0, p') = q(0, p)q^* = (-q)(0, p)(-q)^*$, 故 q 与 $-q$ 表示同一个 3D 旋转。 $S^3 \rightarrow SO(3)$ 是 2:1 覆盖映射。

[WARN] 易错点: 双覆盖与 SLERP

在插值 $q_0 \rightarrow q_1$ 之前, 务必检查 $q_0 \cdot q_1 \geq 0$ 。若内积为负, 将 q_1 翻号 $q_1 \leftarrow -q_1$ 。否则 SLERP 沿”长弧”绕远路 (超过 180°), 产生”倒转”的视觉效果。

7.5 合成旋转

命题 7.1: 先施加旋转 q_1 , 再施加 q_2 , 合成四元数为

$$q = q_2 q_1 \quad (7.12)$$

证明:

$$(0, p'') = q_2[q_1(0, p)q_1^*]q_2^* = (q_2 q_1)(0, p)(q_2 q_1)^*$$

最后一步用了 $(AB)^* = B^* A^*$ 。□

[WARN] 易错点: 合成顺序与矩阵相同

四元数合成 $q_2 q_1$ 中, q_1 在右 (先执行), q_2 在左 (后执行), 与矩阵乘法 $R_2 R_1$ 一致。

7.6 SLERP: 球面线性插值

问题: 在 S^3 上从 q_0 到 q_1 以恒定角速度插值。

定理 7.2 (SLERP) 设 $\cos \theta = q_0 \cdot q_1$, $\theta \in [0, \pi]$, 则

$$\text{SLERP}(q_0, q_1, t) = \frac{\sin[(1-t)\theta]}{\sin \theta} q_0 + \frac{\sin(t\theta)}{\sin \theta} q_1 \quad (7.13)$$

推导： 设 $q_t = a(t)q_0 + b(t)q_1$ 在 S^3 上, 且 q_t 与 q_0 夹角为 $t\theta$, 与 q_1 夹角为 $(1-t)\theta$ 。对两边分别点乘 q_0 和 q_1 ：

$$\cos(t\theta) = a(t) + b(t) \cos \theta$$

$$\cos[(1-t)\theta] = a(t) \cos \theta + b(t)$$

解线性方程组：

$$a(t) = \frac{\cos(t\theta) - \cos[(1-t)\theta] \cos \theta}{\sin^2 \theta} = \frac{\sin[(1-t)\theta]}{\sin \theta}$$

$$b(t) = \frac{\cos[(1-t)\theta] - \cos(t\theta) \cos \theta}{\sin^2 \theta} = \frac{\sin(t\theta)}{\sin \theta}$$

(最后一步利用正弦差角公式化简)。□

边界验证： $t = 0 \Rightarrow q_t = q_0$, $t = 1 \Rightarrow q_t = q_1$ 。当 $\theta \rightarrow 0$ 时 $\sin \theta \rightarrow \theta$, 退化为普通 LERP。

LERP + 投影 (近似方案)： $\tilde{q}_t = (1-t)q_0 + tq_1$, 然后投影 $q_t = \tilde{q}_t / \|\tilde{q}_t\|$ 。结果合法但角速度**不恒定** (弦非大圆弧)。

7.7 四元数优缺点汇总

优缺点	说明
紧凑	4 个浮点数 (vs 矩阵 9 个)
合成高效	一次四元数乘法 (16 乘 + 12 加)
SLERP 恒速	球面测地线, 角速度恒定
无 Gimbal Lock	S^3 无全局奇异
数值稳定	只需维护归一化约束
双覆盖	q 与 $-q$ 同义, 需统一符号
数值漂移	积分/累乘后需归一化 $q \leftarrow q/\ q\ $
不直观	4 个参数无直接几何语义

8. 指数映射与对数映射

8.1 李代数 $\mathfrak{so}(3)$

$SO(3)$ 是三维李群, 其对应的李代数为反对称矩阵空间：

$$\mathfrak{so}(3) = \{A \in \mathbb{R}^{3 \times 3} \mid A^T = -A\} \quad (8.1)$$

$\mathfrak{so}(3)$ 与 $\mathbb{R}^3$ 同构, 同构映射由 $[\cdot]_{\times}$ 给出 (式 1.7)。

8.2 指数映射

定义 8.1 (矩阵指数) 对矩阵 A , $\exp(A) = \sum_{k=0}^{\infty} A^k/k!$.

定理 8.1 (指数映射 $\mathfrak{so}(3) \rightarrow SO(3)$) 对单位轴 u 和角度 θ :

$$\exp(\theta[u]_{\times}) = I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2 = R(u, \theta) \quad (8.2)$$

证明 (利用幂次规律): 对单位轴 u , 有 $[u]_{\times}^3 = -[u]_{\times}$, $[u]_{\times}^4 = -[u]_{\times}^2$ (循环), 因此

$$\begin{aligned} \exp(\theta[u]_{\times}) &= I + \sum_{k=1}^{\infty} \frac{\theta^k}{k!} [u]_{\times}^k = I + \left(\sum_{k=0}^{\infty} \frac{(-1)^k \theta^{2k+1}}{(2k+1)!} \right) [u]_{\times} + \left(\sum_{k=1}^{\infty} \frac{(-1)^{k-1} \theta^{2k}}{(2k)!} \right) [u]_{\times}^2 \\ &= I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2 \quad \square \end{aligned}$$

[NOTE] 直觉理解

指数映射把“李代数中的直线”映射到“李群上的测地线 (大圆弧)”。旋转向量 $\theta = \theta u \in \mathfrak{so}(3)$ 对应的旋转矩阵就是 $R = \exp([\theta]_{\times})$, 这正是 Rodrigues 公式的几何含义。

8.3 对数映射

定义 8.2 (对数映射) 指数映射的逆: 对 $R \in SO(3)$, $\theta \in [0, \pi)$:

$$\log(R) = \frac{\theta}{2 \sin \theta} (R - R^T) = \theta [u]_{\times} \quad (8.3)$$

其中 θ 由迹公式 (3.5) 给出, u 由式 (3.6) 给出。

对数映射把旋转矩阵映回旋转向量 (李代数元素), 是“旋转矩阵 $\rightarrow$ 旋转向量”转换的严格形式, 等价于 §3.3 的逆运算。

[WARN] 易错点

对数映射仅在 $\theta \in [0, \pi)$ 唯一。当 $\theta = \pi$ 时退化, 需特殊处理 (同 §3.3 的退化情形)。

8.4 角速度与旋转矩阵的时间导数

定理 8.2 设刚体旋转矩阵 $R(t) \in SO(3)$, 角速度向量为 $\omega(t)$, 则

$$\dot{R} = [\omega]_{\times} R \quad (8.4)$$

推导: 对 $R^T R = I$ 两边关于时间 t 求导:

$$\dot{R}^T R + R^T \dot{R} = 0 \implies R^T \dot{R} = -(R^T \dot{R})^T$$

令 $\Omega = R^T \dot{R}$, 则 $\Omega^T = -\Omega$ (反对称), 即 $\Omega \in \mathfrak{so}(3)$, 故存在 ω 使得 $\Omega = [\omega]_{\times}$:

$$R^T \dot{R} = [\omega]_{\times} \implies \dot{R} = R[\omega]_{\times}$$

注意这里 ω 是体固系 (body frame) 角速度。等价地, 若 ω 为世界系角速度, 则

$$\dot{R} = [\omega]_{\times} R \quad (8.5)$$

(课件采用此约定)。□

[NOTE] 直觉理解

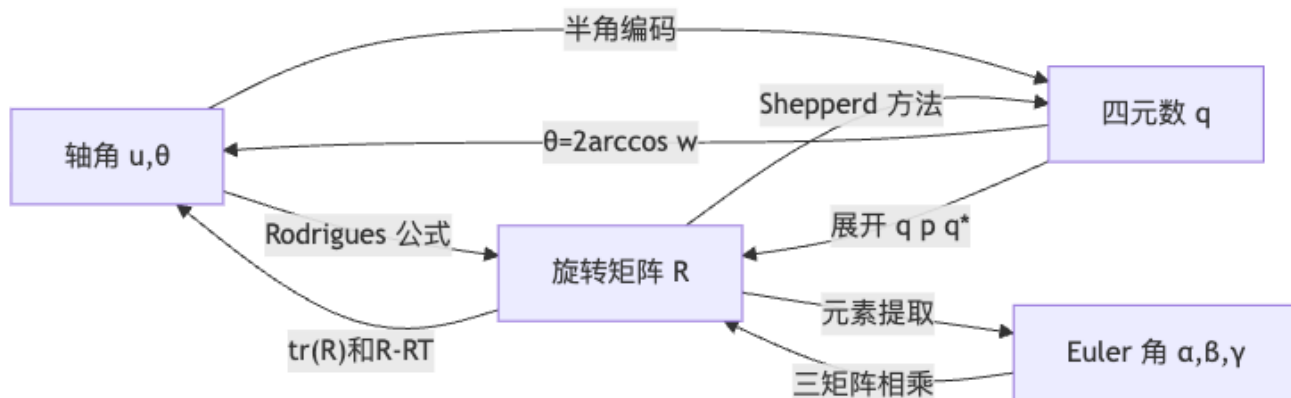
$[\omega]_{\times} R$ 的含义: ω 是刚体当前的角速度向量, $\dot{R}$ 是旋转矩阵的瞬时变化率。这个方程是刚体旋转动力学的基础, 后续积分器 (仿真) 和关节速度计算 (正/逆运动学) 都依赖于此。

[WARN] 易错点: 体固系 vs 世界系

- 世界系角速度 (spatial velocity): $\dot{R} = [\omega]_{\times} R$
 - 体固系角速度 (body velocity): $\dot{R} = R[\omega^b]_{\times}$
- 两者关系: $\omega = R\omega^b$ 。不同教材约定不同, 使用时务必明确。

9. 四种表示的对比与转换

9.1 转换关系图



9.2 关键转换公式

旋转矩阵 $\rightarrow$ 四元数 (Shepperd 方法, 数值稳定):

$$w = \frac{1}{2} \sqrt{1 + r_{11} + r_{22} + r_{33}}, \quad x = \frac{r_{32} - r_{23}}{4w}, \quad y = \frac{r_{13} - r_{31}}{4w}, \quad z = \frac{r_{21} - r_{12}}{4w} \quad (9.1)$$

(当 w 接近 0 时改用其他分母, 从 x, y, z 中选最大值, 避免除以小数)。

四元数 $\rightarrow$ 旋转矩阵 (展开 $q(0, p)q^*$):

$$R = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix} \quad (9.2)$$

其中 (w, x, y, z) 为单位四元数分量。

9.3 综合对比表

维度	旋转矩阵	Euler 角	轴角/旋转向量	四元数
参数数	9	3	3 (或 4)	4
约束数	6	0	0 (或 1)	1
应用 p	Rp	先转矩阵	先转矩阵	$q(0, p)q^*$
合成	R_2R_1	先转矩阵相乘	无解析公式	q_2q_1
插值	不能直接插	可插 (不恒速 +Gimbal Lock)	直插 (不恒速); 偏移法恒速	SLERP 恒速
奇异性	无	Gimbal Lock	$\theta = 0$ 轴未定	双覆盖 (非奇异)
实际用途	GPU 变换、蒙皮	美术编辑 UI	IK 增量、运动捕捉	存储、插值、网络传输

9.4 6D 旋转表示 (其他)

取旋转矩阵前两列 (r_x, r_y) , 第三列由叉积补全 $r_z = r_x \times r_y$ 。优点: 连续光滑 (全局无奇异), 神经网络回归时比四元数数值更稳定 (Zhou et al., CVPR 2019)。后续机器学习章节使用。

10. 插值基础

平移插值: 平凡线性插值 $x_t = (1 - t)x_0 + tx_1$ 始终合法。

旋转插值”好”的标准 (课件原文): 1. 任意 $t \in [0, 1]$ 时插值结果是合法旋转; 2. 旋转角速度恒定 (constant rotational speed)。

各表示的插值对比:

表示	直接线性插值合法?	角速度恒定?	推荐方案
旋转矩阵	否	—	不直接用
Euler 角	是 (3 个角各自线性)	否, 且有 Gimbal Lock	转四元数再 SLERP
旋转向量	是	否	偏移旋转法 (式 6.4)
四元数 LERP+	是	否	—
归一化			
四元数 SLERP	是	是	推荐

[TIP] 复习要点

1. $SO(3)$ 是弯曲流形, 旋转矩阵不能做线性插值。
2. SLERP 公式 (7.13) 在 S^3 上等弧长插值, 是旋转恒速插值的”黄金标准”。
3. SLERP 使用前必须保证 $q_0 \cdot q_1 \geq 0$ (双覆盖符号统一)。
4. $\dot{R} = [\omega]_{\times} R$ 是刚体动力学仿真的基础微分方程。

11. Worked Examples

例题 11.1: 给定轴角, 求四元数与旋转矩阵

题目: 旋转轴 $u = \frac{1}{\sqrt{3}}(1, 1, 1)^T$, 转角 $\theta = 120^\circ$, 求对应的单位四元数 q 和旋转矩阵 R 。

解:

$$\cos(60^\circ) = 1/2, \sin(60^\circ) = \sqrt{3}/2.$$

四元数:

$$q = \left(\frac{1}{2}, \frac{1}{\sqrt{3}} \cdot \frac{\sqrt{3}}{2} \cdot (1, 1, 1)^T \right) = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right)$$

$$\text{验证: } \|q\|^2 = (1/2)^2 \times 4 = 1.$$

旋转矩阵 (用 Rodrigues 公式): $\sin 120^\circ = \sqrt{3}/2$, $1 - \cos 120^\circ = 3/2$ 。

$$[u]_{\times} = \frac{1}{\sqrt{3}} \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}, \quad [u]_{\times}^2 = \frac{1}{3} \begin{pmatrix} -2 & 1 & 1 \\ 1 & -2 & 1 \\ 1 & 1 & -2 \end{pmatrix}$$

$$R = I + \frac{\sqrt{3}}{2} \cdot \frac{1}{\sqrt{3}} \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix} + \frac{3}{2} \cdot \frac{1}{3} \begin{pmatrix} -2 & 1 & 1 \\ 1 & -2 & 1 \\ 1 & 1 & -2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

直觉验证: 绕 $(1, 1, 1)$ 轴转 120° 是 $x \rightarrow y \rightarrow z \rightarrow x$ 的循环置换, $Re_x = e_y, Re_y = e_z, Re_z = e_x$, 与矩阵一致。

例题 11.2: 从旋转矩阵提取轴角

题目: 旋转矩阵 $R = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, 求旋转轴 u 和角度 θ 。

解:

用迹公式: $\text{tr}(R) = 0 + 0 + 1 = 1$, 故 $\cos \theta = (1 - 1)/2 = 0$, $\theta = 90^\circ$ 。

用式 (3.6): $u' = (r_{32} - r_{23}, r_{13} - r_{31}, r_{21} - r_{12})^T = (0 - 0, 0 - 0, 1 - (-1))^T = (0, 0, 2)^T$ 。

$$\|u'\| = 2 = 2 \sin 90^\circ = 2.$$

归一化: $u = (0, 0, 1)^T$ 。即绕 z 轴转 90° , 与 $R = R_z(90^\circ)$ 一致。

例题 11.3: 验证反对称矩阵 $[\omega]_{\times}$ 的性质

题目: 设 $\omega = (1, 0, 0)^T$, 验证: (a) $[\omega]_{\times}^T = -[\omega]_{\times}$; (b) $[\omega]_{\times} b = \omega \times b$; (c) $[\omega]_{\times}^2 = \omega \omega^T - \|\omega\|^2 I$ (当 $\|\omega\| = 1$)。

解:

$$[\omega]_{\times} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

$$(a) [\omega]_{\times}^T = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix} = -[\omega]_{\times}.$$

$$(b) \text{ 取 } b = (b_x, b_y, b_z)^T: [\omega]_{\times} b = (0, -b_z, b_y)^T = (1, 0, 0)^T \times (b_x, b_y, b_z)^T.$$

$$(c) [\omega]_{\times}^2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{pmatrix}, \text{ 而 } \omega \omega^T - I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{pmatrix}.$$

例题 11.4: SLERP 计算

题目: $q_0 = (1, 0, 0, 0)$ (恒等旋转), $q_1 = (0, 0, 0, 1)$ (绕 z 轴转 180°), 求 $\text{SLERP}(q_0, q_1, 0.5)$ 。

解:

首先检查符号: $q_0 \cdot q_1 = 1 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 1 = 0 \geq 0$ 。

$\cos \theta = 0$, 故 $\theta = 90^\circ$ 。

$$q_{0.5} = \frac{\sin(45^\circ)}{\sin(90^\circ)} q_0 + \frac{\sin(45^\circ)}{\sin(90^\circ)} q_1 = \frac{\sqrt{2}/2}{1} (1, 0, 0, 0) + \frac{\sqrt{2}/2}{1} (0, 0, 0, 1) = \left(\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2} \right)$$

对应轴角: $w = \sqrt{2}/2$, $\theta = 2 \arccos(\sqrt{2}/2) = 90^\circ$, 轴 $u = v/\|v\| = (0, 0, 1)^T$ 。即绕 z 轴转 90° ——恰好是 0° 和 180° 的中点。

例题 11.5: Gimbal Lock 的数值演示

题目: ZYX Euler 角 $(\alpha, \beta, \gamma) = (30^\circ, 90^\circ, 45^\circ)$, 说明自由度损失。

解:

$$R = R_z(45^\circ) R_y(90^\circ) R_x(30^\circ)$$

$$\text{代入 } R_y(90^\circ) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}, R_z(45^\circ) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & \sqrt{2} \end{pmatrix}, R_x(30^\circ) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ 0 & \frac{1}{2} & \frac{\sqrt{3}}{2} \end{pmatrix}.$$

计算 $R_y(90^\circ)R_x(30^\circ) = \begin{pmatrix} 0 & 1/2 & \sqrt{3}/2 \\ 0 & \sqrt{3}/2 & -1/2 \\ -1 & 0 & 0 \end{pmatrix}$ 。

注意整个乘积仅依赖 $\alpha - \gamma$ (可以验证, 修改 α 同时修改 γ 相同量, R 不变)。故在 $\beta = 90^\circ$ 处, 三个 Euler 角只有 **2 个独立自由度**, 万向锁发生。

12. Anki 卡片

[EX] Anki 1

Q: 反对称矩阵 $[a]_\times$ 的定义与性质?

A: $[a]_\times = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}$, 满足 $[a]_\times^T = -[a]_\times$, 且 $[a]_\times b = a \times b$ 。

[EX] Anki 2

Q: Rodrigues 公式是什么?

A: $R(u, \theta) = I + \sin \theta [u]_\times + (1 - \cos \theta) [u]_\times^2$, 其中 $\|u\| = 1$, θ 为旋转角。

[EX] Anki 3

Q: 已知旋转矩阵 R , 如何求旋转角 θ ?

A: $\theta = \arccos \frac{\text{tr}(R) - 1}{2}$, 由 $\text{tr}(R) = 1 + 2 \cos \theta$ 得出。

[EX] Anki 4

Q: 已知旋转矩阵 R ($\theta \notin \{0, \pi\}$), 如何求旋转轴 u ?

A: $u' = (r_{32} - r_{23}, r_{13} - r_{31}, r_{21} - r_{12})^T$, $\|u'\| = 2 \sin \theta$, 归一化得 $u = u' / \|u'\|$ 。

[EX] Anki 5

Q: 四元数乘法公式 (向量形式)?

A: $q_1 q_2 = \begin{pmatrix} w_1 w_2 - v_1 \cdot v_2 \\ w_1 v_2 + w_2 v_1 + v_1 \times v_2 \end{pmatrix}$, 非交换, 有结合律。

[EX] Anki 6

Q: 单位四元数如何编码轴角旋转?

A: $q = (\cos(\theta/2), u \sin(\theta/2))$, 其中 $\|u\| = 1$, θ 为旋转角。

[EX] Anki 7

Q: 用四元数旋转向量 p 的公式?

A: $(0, p') = q(0, p)q^*$, 其中 q 为单位四元数, $(0, p)$ 为纯四元数嵌入。

[EX] Anki 8

Q: 四元数双覆盖是什么意思? 有何实际影响?

A: q 与 $-q$ 表示同一个 3D 旋转 ($S^3 \rightarrow SO(3)$ 是 2:1 覆盖映射)。做 SLERP 时需检查 $q_0 \cdot q_1 \geq 0$, 否则沿长弧绕远路, 翻号 $q_1 \leftarrow -q_1$ 。

[EX] Anki 9

Q: SLERP 公式?

A: $\text{SLERP}(q_0, q_1, t) = \frac{\sin[(1-t)\theta]}{\sin \theta} q_0 + \frac{\sin(t\theta)}{\sin \theta} q_1$, 其中 $\cos \theta = q_0 \cdot q_1$ 。

[EX] Anki 10

Q: $SO(3)$ 的定义? 自由度是多少?

A: $SO(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^T R = I, \det R = +1\}$, 9 个参数减去 6 个正交约束, DoF = 3。

[EX] Anki 11

Q: $\dot{R} = [\omega]_{\times} R$ 的推导来源?

A: 对 $R^T R = I$ 求导得 $R^T \dot{R} + \dot{R}^T R = 0$, 故 $R^T \dot{R}$ 反对称, 记为 $[\omega]_{\times}$, 则 $\dot{R} = R[\omega]_{\times}$ (体固系) 或 $\dot{R} = [\omega]_{\times} R$ (世界系)。

[EX] Anki 12

Q: Gimbal Lock 的数学本质?

A: 在 ZYX Euler 角中, 当中间轴角 $\beta = \pm 90^\circ$ 时, $R_z(\gamma)R_y(\pm 90^\circ)R_x(\alpha)$ 仅依赖 γ 与 α 一个参数, 三自由度退化为两自由度, 无法独立控制三个角。

[EX] Anki 13

Q: 指数映射 $\exp(\theta[u]_{\times})$ 等于什么?

A: $\exp(\theta[u]_{\times}) = I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2 = R(u, \theta)$ (即 Rodrigues 公式)。利用 $[u]_{\times}^3 = -[u]_{\times}$ 的幂次循环性质推导。

[EX] Anki 14

Q: 四种旋转表示的插值能力对比?

A: 旋转矩阵不能直接线性插值 (结果不在 $SO(3)$); Euler 角可插但角速度不恒定且有 Gimbal Lock; 旋转向量直接线性插合法但不恒速, 偏移旋转法可恒速; 四元数 SLERP 在 S^3 上等弧长, 角速度恒定, 是推荐方案。

13. 中英术语速查表

中文	English	核心公式/说明
向量	Vector	$a \in \mathbb{R}^n$, 粗体小写
点积 / 内积	Dot product / Inner product	$a \cdot b = a^T b = \ a\ \ b\ \cos \theta$
叉积	Cross product	$a \times b = [a]_{\times} b$

中文	English	核心公式/说明
反对称矩阵	Skew-symmetric matrix	$A^T = -A$; 叉积的矩阵形式
叉积矩阵	Cross product matrix	$[a]_{\times}$, 3x3 反对称
正交矩阵	Orthogonal matrix	$A^T A = I$, $\det A = \pm 1$
特殊正交群	Special orthogonal group	$SO(3)$, 旋转矩阵全体
旋转矩阵	Rotation matrix	$R \in SO(3)$, 9 个参数, DoF=3
刚体变换	Rigid transformation	$a' = Ra + t$, 即 $SE(3)$
Rodrigues 公式	Rodrigues' rotation formula	$R = I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2$
Euler 角	Euler angles	三次绕轴旋转乘积, 12 种顺序
万向锁	Gimbal lock	Euler 角奇异, DoF 退化
内旋	Intrinsic rotation	绕物体自身局部轴旋转
外旋	Extrinsic rotation	绕固定世界轴旋转
轴角	Axis-angle	(u, θ) , $\ u\ = 1$
旋转向量	Rotation vector	$\theta = \theta u \in \mathbb{R}^3$
四元数	Quaternion	$q = (w, v) \in \mathbb{H}$
单位四元数	Unit quaternion	$\ q\ = 1$, $q^{-1} = q^*$
纯四元数	Pure quaternion	$w = 0$, $(0, p)$ 嵌入 3D 向量
四元数共轭	Quaternion conjugate	$q^* = (w, -v)$
双重覆盖	Double cover	q 与 $-q$ 表示同一旋转
球面线性插值	SLERP (Spherical Linear Interpolation)	式 (7.13), 恒速球面插值
角速度	Angular velocity	ω , $\dot{R} = [\omega]_{\times} R$
李群	Lie group	$SO(3)$: 3 维连续旋转群
李代数	Lie algebra	$\mathfrak{so}(3)$: 反对称矩阵空间
指数映射	Exponential map	$\exp([\theta]_{\times}) = R(u, \theta)$
对数映射	Logarithm map	$\log(R) = \theta [u]_{\times}$
6D 表示	6D rotation representation	旋转矩阵前两列, 连续无奇异

[TIP] 复习要点

1. 叉积矩阵: $[a]_{\times}$ 反对称, $a \times b = [a]_{\times} b$, $[u]_{\times}^3 = -[u]_{\times}$ 。
2. Rodrigues 公式 (必须会推导): $R = I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2$; 逆向用迹公式 $\theta = \arccos \frac{\text{tr} R - 1}{2}$ 提角, 用 $R - R^T$ 提轴。
3. 四元数乘法: $q_1 q_2 = (w_1 w_2 - v_1 \cdot v_2, w_1 v_2 + w_2 v_1 + v_1 \times v_2)$, 非交换。
4. 旋转公式: $(0, p') = q(0, p)q^*$; 合成 $q_2 q_1$ (q_1 先执行)。
5. 双覆盖: q 与 $-q$ 同义, SLERP 前需保证 $q_0 \cdot q_1 \geq 0$ 。
6. 角速度: $\dot{R} = [\omega]_{\times} R$ (世界系), 由 $R^T R = I$ 求导得来。
7. 插值: 四种表示中只有 SLERP 同时满足“合法”且“恒速”, 是旋转插值的首选。
8. Gimbal Lock: Euler 角固有奇异, $\beta = \pm 90^\circ$ 时 DoF 从 3 降为 2。

Lecture 03 —角色运动学 (Character Kinematics)

[TIP] 学习指南

本节核心：如何用关节角参数化一个角色的姿态，并在“给关节角求末端位置（正向运动学，FK）”和“给末端目标位置反求关节角（逆运动学，IK）”之间来回穿梭。这是角色动画最底层的数学基础——没有它，骨骼就不会动。

前置知识：– lec02 —旋转矩阵、四元数、坐标变换；知道 $R^T = R^{-1}$ 即可 – 链式法则 (Chain Rule)：对复合函数求导 – 线性代数基础：矩阵乘法、叉积、逆矩阵

重点掌握 (考试高频)：1. FK 递推公式： $Q_i = Q_{p_i} R_i$, $x_i = x_{p_i} + Q_{p_i} l_i$ 2. Retargeting 核心公式： $R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T$ 3. CCD 与 FABRIK 的算法步骤 4. Jacobian 定义与几何法构造： $\partial f / \partial \theta_i = a_i \times r_i$ 5. 三种 IK 更新公式：Jacobian 转置、伪逆、阻尼最小二乘 (DLS/LM)
本节从零开始，所有术语首次出现均有解释，无任何隐性假设。

0. 名词热身：本讲反复出现的 7 个概念

名词	英文	一句话解释
骨骼	Skeleton	角色的“内骨架”——一组刚性骨头通过关节相连，驱动角色表面变形
关节	Joint	相邻骨段之间的连接点，允许相对旋转（有时也允许平移）
link / bone / body	—	两个关节之间的刚性骨段，三词等价
根关节	Root Joint	骨骼树的根，在世界坐标系中有确定的位置和朝向
末端执行器	End-Effector	动画师关心其位置/朝向的末端关节，如手、脚、头
姿态	Pose	所有关节角的集合，完全决定角色的形状
运动学	Kinematics	只研究几何（位置、速度），不考虑质量与力的学科

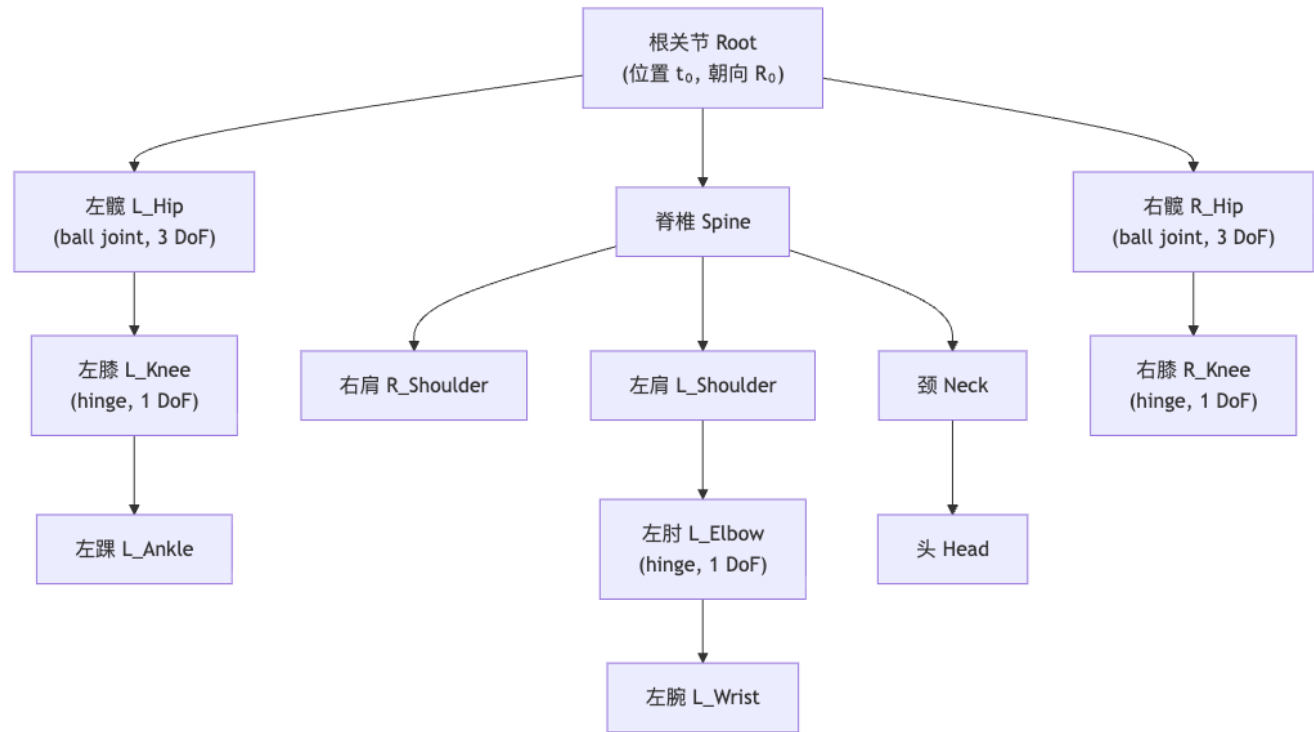
[NOTE] 运动学 vs 动力学

运动学 (Kinematics)：给定关节角，求末端在哪里——纯几何问题。**动力学 (Dynamics)：**给定力，求运动——需要质量、惯量等物理量（本讲后续 Lec 08 才讲）。本节只做运动学。

1. 骨骼与单链正向运动学

1.1 骨骼的树状结构

一个角色的骨骼是一棵**有根树 (rooted tree)**： – 每个节点 = 一个关节 i – 节点 i 的父亲记为 p_i – 根关节 p_0 没有父亲 – 叶子节点通常是末端执行器（手指尖、脚趾、头顶）



1.2 局部旋转 R_i vs 全局朝向 Q_i

每个关节 i 有两个量，初学者容易混淆：

量	符号	含义	类比
局部旋转	R_i	关节 i 相对其父亲坐标系的旋转	“手臂弯曲了多少度”
全局朝向	Q_i	关节 i 在世界坐标系中的朝向	“手臂指向世界的哪个方向”

T-pose (参考姿态)：所有 $R_i = I$ (单位矩阵)，此时 $Q_i = Q_{p_i}$ 。T-pose 是骨骼的“零点”，BVH、FBX 等格式以此为基准。

1.3 单链 FK 核心推导

设链有关节 $0, 1, 2, 3, 4$ ，根关节朝向 $Q_0 = R_0$ 。从根向末端逐步推导：

步骤	全局朝向
$Q_0 = R_0$ $Q_1 = Q_0 R_1$	根关节就是 R_0 在 Q_0 基础上再转 R_1

步骤	全局朝向
$Q_2 = Q_1 R_2 = Q_0 R_1 R_2$	继续累乘
$Q_3 = Q_2 R_3$	...
$Q_4 = Q_3 R_4 = R_0 R_1 R_2 R_3 R_4$	展开就是所有局部旋转的乘积

$$Q_i = Q_{p_i} R_i \quad (1.1)$$

[NOTE] 几何直觉

想象你在一节火车上转身——你的全局朝向 = 火车的朝向 (Q_{p_i}) $\times$ 你相对火车的朝向 (R_i)。矩阵乘法的顺序对应“先变换外层坐标系，再在其中叠加局部旋转”。

反向：从全局朝向还原局部旋转（利用旋转矩阵正交性 $Q^{-1} = Q^T$ ）：

$$R_i = Q_{p_i}^T Q_i \quad (1.2)$$

两关节间的相对旋转：

$$R_4^{(\text{相对于关节 } 1)} = Q_1^T Q_4 = R_2 R_3 R_4 \quad (1.3)$$

1.4 关节位置的递推

设 link i （从关节 i 到 $i+1$ ）在关节 i 的局部坐标系下的偏移向量为 l_i ，关节 i 的世界位置为 p_i ：

$$p_{i+1} = p_i + Q_i l_i \quad (1.4)$$

[NOTE] 为什么是 Q_i 而非 Q_i^T ？

l_i 是在关节 i 的局部坐标系下表示的骨段向量。把它变到世界坐标系，需要乘以关节 i 的全局朝向 Q_i 。

末端执行器上的某个点 x_0 （在末端关节 E 的局部坐标系下），其世界坐标为：

$$x = p_E + Q_E x_0 \quad (1.5)$$

1.5 单链 FK 算法（正向遍历）

输入：所有关节的局部旋转 $\{R_i\}$ ，根位置 p_0 ，各 link 偏移 $\{l_i\}$

输出：末端位置 x ，各关节位置 $\{p_i\}$

```

Q_0 = R_0
for i from 1 to E:
    Q_i = Q_{i-1} * R_i
    p_i = p_{i-1} + Q_{i-1} * l_{i-1}
x = p_E + Q_E * x_0

```

等价反向形式（从末端向根推，数学完全等价）：

```

x = x_0
for i from E down to 1:
    x = l_{i-1} + R_i * x
最后: x = p_0 + Q_0 * x

```

1.6 局部坐标查询

末端点 x 在关节 k 的局部坐标系下的坐标:

$$x_{Q_k} = Q_k^T(x - p_k) \quad (1.6)$$

对单链 ($k = 3$): $x_{Q_3} = l_3 + R_4 x_0$, 即在关节 3 看来, 末端点的局部位置就是再往前走 $R_4 x_0$ 再加 l_3 。

2. 角色 FK (树状骨架)

2.1 拓扑排序与通用 FK

骨架是树, 遍历时必须保证父节点先于子节点处理 (拓扑排序, BVH/FBX 格式天然满足此约定):

$$\text{for } i \text{ in joint_list (父先于子): } Q_i = Q_{p_i} R_i, \quad x_i = x_{p_i} + Q_{p_i} l_i \quad (2.1)$$

2.2 关节类型与自由度 (DoF)

自由度 (Degrees of Freedom, DoF): 完全描述系统状态所需独立参数的数量。

关节类型	典型部位	DoF	说明
Free / Floating (根关节)	整体	6	3D 位移 + 3D 旋转
Ball-and-socket (球窝关节)	髋、肩	3	任意方向旋转
Universal (万向节)	腕	2	两轴旋转
Hinge / Revolute (铰链/转动关节)	膝、肘	1	单轴旋转
Fixed / Welded (固定)	—	0	完全刚性连接

[NOTE] 关节角度限制 (Joint Limits)

真实人体不能任意旋转 (膝盖不能反折、肘弯不超 180°)。每个 DoF 设约束 $\theta_{\min} \leq \theta \leq \theta_{\max}$ 。IK 求解时必须将结果投影回该区间, 否则角色姿态不合理。

2.3 完整姿态参数向量

$$\theta = (t_0, R_0, R_1, R_2, \dots, R_n) \quad (2.2)$$

- 根关节: 给 6 DoF (位移 $t_0 \in \mathbb{R}^3$ + 朝向 $R_0 \in SO(3)$)
- 其余关节: 只给相对旋转 R_i (hinge = 1 参数, ball = 3 参数)

2.4 BVH 文件格式

BVH (BioVision Hierarchy) 是最常用的动捕数据格式：– **HIERARCHY 段**：定义 T-pose 拓扑，每个关节的 OFFSET (即 l_i) 和 CHANNELS (位置/旋转通道)。– **MOTION 段**：每帧记录根关节的位置 + 所有关节的 Euler 角。– BVH 默认 Euler 顺序为 外旋/固定轴 (extrinsic / fixed angles)，即 $R = R_z R_x R_y$ 。

2.5 T-pose 与 A-pose

同一段关节角序列 $\{R_i(t)\}$ 施加到 T-pose 角色和 A-pose 角色上，产生的全局朝向完全不同——这就是 **Retargeting** (动作迁移) 要解决的问题。

[WARN] 同样的局部旋转 $\neq$ 同样的全局姿势

例：“手臂上抬 90°”在 T-pose 下 (手臂水平出发) 结果是手臂竖直；在 A-pose 下 (手臂斜下出发) 结果是手臂斜上，看起来完全不同。直接复用 BVH 的关节角到不同 reference pose 的角色上会出现扭曲。

3. Retargeting：在不同参考姿态间迁移动作

3.1 问题定义

已知：– 角色 A (source, 如 T-pose) 有动作数据 $\{R_i^A(t)\}$ – 角色 B (target, 如 A-pose) 有与 A 不同的参考姿态 – 两个参考姿态之间的**全局朝向偏差** (offset) 为 $Q_i^{A \rightarrow B}$ (每个关节各自有一个，预先标定)

目标：求 B 上的关节旋转 R_i^B ，使得 B 的**全局朝向**与 A 相同 (即 $Q_i^B = Q_i^A (Q_i^{A \rightarrow B})^T$)。

3.2 单刚体 Retargeting

设只有一个刚体，A 上旋转为 R^A ，全局朝向偏差为 $Q^{A \rightarrow B}$ ，则 B 上等效旋转：

$$R^B = R^A (Q^{A \rightarrow B})^T \quad (3.1)$$

[NOTE] 直觉

B 想达到和 A 同样的全局朝向，但 B 的”出发点” (参考朝向) 比 A 偏转了 $Q^{A \rightarrow B}$ 。所以 B 要先”抵消”这个偏差 (乘以 $(Q^{A \rightarrow B})^T$)，再施加 A 的旋转 R^A 。

3.3 链式 Retargeting (完整推导)

对链中的关节 i ，已知：

$$Q_i^A = Q_{p_i}^A R_i^A, \quad Q_i^B = Q_{p_i}^B R_i^B \quad (3.2)$$

$$Q_i^B = Q_i^A (Q_i^{A \rightarrow B})^T \quad (3.3)$$

$$Q_{p_i}^B = Q_{p_i}^A (Q_{p_i}^{A \rightarrow B})^T \quad (3.4)$$

从 $R_i^B = (Q_{p_i}^B)^T Q_i^B$ 出发代入 (3.3)(3.4):

$$\begin{aligned}
 R_i^B &= [Q_{p_i}^A (Q_{p_i}^{A \rightarrow B})^T]^T \cdot Q_i^A (Q_i^{A \rightarrow B})^T \\
 &= Q_{p_i}^{A \rightarrow B} \cdot (Q_{p_i}^A)^T \cdot Q_i^A \cdot (Q_i^{A \rightarrow B})^T \\
 &= Q_{p_i}^{A \rightarrow B} \cdot R_i^A \cdot (Q_i^{A \rightarrow B})^T
 \end{aligned}$$

$$R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T \quad (3.5)$$

[NOTE] 记忆口诀

“父亲的偏差在前，自己的偏差（的转置）在后，中间夹着 A 的旋转”。 $Q_{p_i}^{A \rightarrow B}$ 把 B 父坐标系对齐到 A 父坐标系； $(Q_i^{A \rightarrow B})^T$ 把结果从 A 子坐标系映射回 B 子坐标系。

4. 逆运动学 (IK): 问题与启发式解法

4.1 FK vs IK 对比

方面	正向 $x = f(\theta)$	逆向: 给 $\tilde{x}$ 求 θ
方向	$\theta \rightarrow x$	$x \rightarrow \theta$
难度	直接代入, 闭合解唯一	解非线性方程组, 可能多解或无解
DoF	$\dim \theta \gg \dim x$ 没问题	$\dim \theta \gg \dim x \rightarrow$ 欠定/无穷多解
物理直觉	θ 不直观	$\tilde{x}$ 直观 (“手放在这里”)
应用	实时渲染	动画绑定、运动捕捉、机械臂控制

[WARN] IK 可能无解

对 n 个 link (长度 $l_0, l_1, \dots, l_{n-1}$), 若目标距根 $d > \sum l_i$ (够不着) 或 $d < |l_0 - \sum_{i>0} l_i|$ (太近), 则无精确解。启发式和优化方法此时给出最优近似解。

4.2 两关节解析 IK (Two-Joint IK)

这是手臂/腿这类短链最常用的闭合解法, 步骤如下:

1. **确定弯曲角**: 用余弦定理, 从目标距离 $d = \|\tilde{x} - p_0\|$ 计算 joint 1 的角度

$$\cos \angle(\text{joint } 1) = \frac{l_0^2 + l_1^2 - d^2}{2l_0l_1} \quad (4.1)$$

2. **旋转 joint 0**: 把整段链“指向”目标 $\tilde{x}$
3. **绕 l_{01} 轴旋转** (可选): 若需匹配朝向, 则绕第一段骨骼轴做额外旋转

优点: 闭合解、快、无迭代。 **缺点**: 只适用于两关节链, 无法处理长链或过约束。

4.3 CCD 一循环坐标下降 (Cyclic Coordinate Descent)

CCD 是最简单的启发式 IK。核心思想：每次只调一个关节，循环迭代直到末端到达目标。

算法步骤：

初始化：给定任意 pose θ

repeat until $||x - \tilde{x}|| < \epsilon$:

```
    for i = E-1 downto 0:      # 从离末端最近的关节开始，向根遍历
        v1 = 归一化(当前末端位置 x - 关节 i 的位置)
        v2 = 归一化(目标位置  $\tilde{x}$  - 关节 i 的位置)
        旋转轴 n = 归一化(v1  $\times$  v2)
        旋转角  $\phi = \arccos(v1 \cdot v2)$ 
        将关节 i 绕轴 n 旋转  $\phi$  (使 x 对准  $\tilde{x}$ )
        更新末端位置 x
```

[NOTE] CCD 的几何意义

每一步都在解一个“两向量对齐”的最小旋转问题——这恰好是关于单个 θ_i 的 $\arg \min F(\theta)$ ，因此 CCD 是 IK 优化问题上的坐标下降 (Coordinate Descent)。它不是 hack，而是有严格优化对应物的方法。

优点：- 实现极简 (约 10 行代码) - 无需计算导数 - 可扩展至伸缩 link (拉伸 link i 使 $(x - \tilde{x}) \perp l_{i,E}$)

缺点：- 离末端近的关节旋转量大 $\rightarrow$ 姿态不自然 (尤其是链的“根部”关节动得少) - 收敛慢，对初值敏感 - 无约束时收敛，加 joint limits 后不保证

4.4 FABRIK 一前向后向到达 IK

FABRIK (Forward And Backward Reaching IK, Aristidou & Lasenby, 2011) 直接在位置空间操作，完全不涉及旋转矩阵。

算法步骤 (无约束版)：

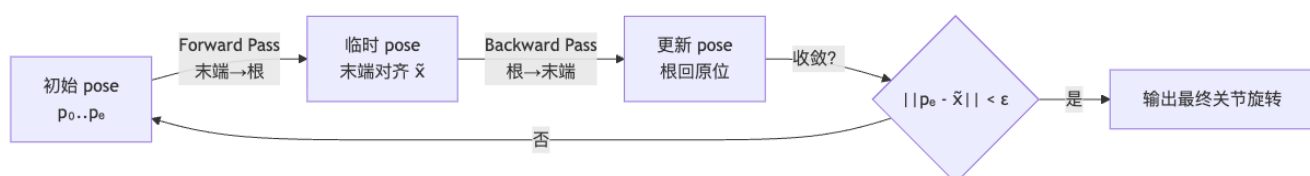
设关节位置 $p_0, p_1, \dots, p_E$ ，各 link 长度 $l_0, l_1, \dots, l_{\{E-1\}}$
设根位置 $root = p_0$ ，目标 $\tilde{x}$

repeat until $||p_E - \tilde{x}|| < \epsilon$:

```
# ===== Forward Pass (从末端向根拉伸) =====
p_E  $\leftarrow$   $\tilde{x}$                                 # 把末端强制拉到目标
for i = E-1 downto 0:
    方向 d = 归一化(p_i - p_{i+1})
    p_i  $\leftarrow$  p_{i+1} + l_i * d              # 保持 link 长度，沿方向放置关节 i
```

```
# ===== Backward Pass (从根向末端拉伸) =====
p_0  $\leftarrow$  root                             # 把根拉回原位
for i = 1 to E:
    方向 d = 归一化(p_i - p_{i-1})
    p_i  $\leftarrow$  p_{i-1} + l_{i-1} * d         # 保持 link 长度，沿方向放置关节 i
```

最后从相邻位置反算关节旋转 R_i



优点：- 简单、快，无需矩阵求逆 - 无约束情形收敛性有保证 - 可加 joint constraints (投影到允许锥上)

缺点：– 位置方法，需事后反算关节旋转（额外步骤）– 加约束后收敛不再保证

5. IK 作为优化问题（全推导）

5.1 目标函数形式化

给定 FK 函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ （输入关节角向量 $\theta \in \mathbb{R}^n$ ，输出末端位置 $x \in \mathbb{R}^m$ ），IK 问题写成非线性最小二乘：

$$\min_{\theta} F(\theta) = \frac{1}{2} \|f(\theta) - \tilde{x}\|_2^2 \quad (5.1)$$

其中 $\tilde{x}$ 是目标末端位置， $\Delta \triangleq f(\theta) - \tilde{x}$ 为位置误差。

[NOTE] 为什么是最小二乘而非精确求解？

精确求解 $f(\theta) = \tilde{x}$ 是非线性方程组，通常无解析解，且当 DoF 过多时有无穷多解。转为最小二乘后，既能处理无解情形（最优近似），又能通过加正则项从无穷多解中选出“自然”的解。

5.2 通用迭代框架

$$\theta_{i+1} = \theta_i + \alpha d \quad (5.2)$$

不同的 d 选取策略对应不同方法，下文逐一推导。

6. Jacobian 矩阵：定义与构造

6.1 定义

Jacobian 矩阵（雅可比矩阵） J 是 FK 函数 f 对参数 θ 的一阶偏导矩阵：

$$J = \frac{\partial f}{\partial \theta} \in \mathbb{R}^{m \times n}, \quad f: \mathbb{R}^n \mapsto \mathbb{R}^m \quad (6.1)$$

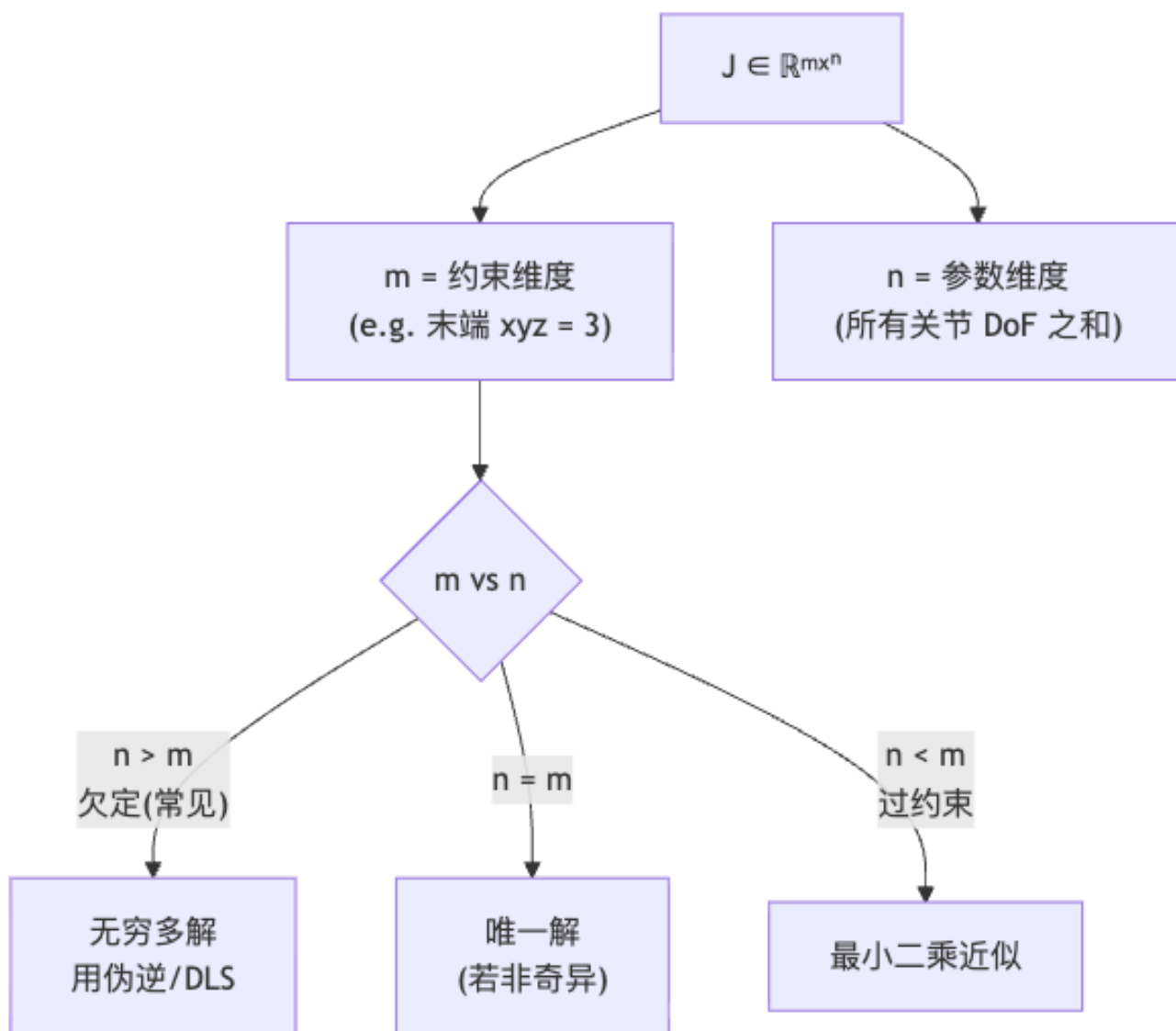
其中第 j 列为：

$$J_{:,j} = \frac{\partial f}{\partial \theta_j} = \begin{pmatrix} \partial f_x / \partial \theta_j \\ \partial f_y / \partial \theta_j \\ \partial f_z / \partial \theta_j \end{pmatrix} \in \mathbb{R}^3 \quad (6.2)$$

物理意义： $\partial f / \partial \theta_j$ 是“微小扰动第 j 个关节角时，末端位置向哪个方向移动、移动多快”。

[NOTE] Jacobian 的维度分析

设角色有 n 个 DoF，末端约束维度为 m （通常 $m = 3$ 或 6 ）：– $n > m$ ：欠定——无穷多解（IK 常见情形，需选最优解）– $n = m$ ：恰好 m 个方程 m 个未知数——有唯一解（或奇异时无解）– $n < m$ ：过约束——一般无精确解



6.2 几何法构造 Jacobian (Hinge 关节)

问题：如何不用符号微分直接得到 $\partial f / \partial \theta_i$?

设 hinge 关节 i 的旋转轴为单位向量 a_i (世界坐标系), 从关节 i 到末端 x 的向量为 $r_i = x - p_i$ 。

当关节 i 转动微小角 $\delta\theta_i$ 时, 末端位移由 **Rodrigues 旋转公式**给出:

$$x' - x = \sin(\delta\theta_i) (a_i \times r_i) + (1 - \cos(\delta\theta_i)) a_i \times (a_i \times r_i) \quad (6.3)$$

取极限 $\delta\theta_i \rightarrow 0$ (注意 $\sin(\delta\theta_i) \rightarrow \delta\theta_i$, $1 - \cos(\delta\theta_i) \rightarrow 0$):

$$\frac{\partial f}{\partial \theta_i} = \lim_{\delta\theta_i \rightarrow 0} \frac{x' - x}{\delta\theta_i} = a_i \times r_i \quad (6.4)$$

$$\boxed{\frac{\partial f}{\partial \theta_i} = a_i \times r_i} \quad (6.5)$$

[TIP] 几何法的优雅性

Jacobian 第 i 列 = 关节 i 的旋转轴 $\times$ 关节 i 到末端的向量。这个公式完全由当前 pose 下的几何量决定，不需要对 FK 函数进行符号求导。

实践中 Jacobian 的填写（以 4 个 hinge 关节、单末端为例）：

$$J = (a_0 \times r_0 \quad a_1 \times r_1 \quad a_2 \times r_2 \quad a_3 \times r_3) \in \mathbb{R}^{3 \times 4} \quad (6.6)$$

6.3 Ball Joint 的 Jacobian (Euler 角参数化)

Ball joint（腕、肩）有 3 个 DoF，用 Euler 角参数化 $R_i = R_{ix}R_{iy}R_{iz}$ ，将其视为三个串联 hinge 关节，旋转轴依次为（在世界坐标系下）：

$$a_{ix} = Q_{i-1} e_x \quad (6.7)$$

$$a_{iy} = Q_{i-1} R_{ix} e_y \quad (6.8)$$

$$a_{iz} = Q_{i-1} R_{ix} R_{iy} e_z \quad (6.9)$$

对应三列 Jacobian：

$$\frac{\partial f}{\partial \theta_{i*}} = a_{i*} \times r_i, \quad * \in \{x, y, z\} \quad (6.10)$$

[WARN] 不能用 Axis-Angle 直接求导

若用轴角 θu 参数化 ball joint，则 Jacobian 有非常复杂的解析表达式（涉及 u 对 θ 的变化）。推荐始终用 Euler 角 + 三 hinge 的分解策略。

6.4 数值法：有限差分与自动微分

方法	公式	优点	缺点
有限差分	$\frac{\partial f}{\partial \theta_i} \approx [f(\theta + \delta e_i) - f(\theta)] / \delta$	实现简单，无需推导	每列需做一次 FK ($O(n)$ 次 FK)，慢；有数值噪声
自动微分 (Autograd)	PyTorch/TensorFlow/JAX 自动计算精确梯度	精确，与代码无缝集成	需要将 f 纳入可微计算图
几何法	$a_i \times r_i$	最快，无额外 FK 调用	仅适用于旋转关节

7. IK 求解方法（三大类）

7.1 方法一：Jacobian 转置法（梯度下降）

推导：对 $F(\theta) = \frac{1}{2}\|f(\theta) - \tilde{x}\|^2$ ，用链式法则求梯度：

$$\nabla_{\theta} F = \left(\frac{\partial f}{\partial \theta} \right)^T (f(\theta) - \tilde{x}) = J^T \Delta \quad (7.1)$$

沿负梯度方向更新：

$$\theta_{i+1} = \theta_i - \alpha J^T \Delta \quad (7.2)$$

[NOTE] 几何直觉

$J^T \Delta$ 的第 j 分量为 $(a_j \times r_j) \cdot \Delta$ ——即关节 j 的运动方向与误差方向的点积。若关节 j 的运动能减小误差，它就动；若不能，它就不动。 J^T 相当于把末端误差“反传”回各关节。

特点：- 一阶方法，收敛慢（尤其接近目标时“抖动”）- 每迭代步需重新计算 J - 步长 α 难以自动调整： α 太大震荡， α 太小迟缓 - 无需求解线性方程，数值稳定

7.2 方法二：伪逆法（Jacobian Pseudoinverse / Gauss-Newton）

推导：将 $f(\theta)$ 在 θ_0 处做一阶 Taylor 展开：

$$f(\theta) \approx f(\theta_0) + J(\theta - \theta_0) \quad (7.3)$$

代入 $F(\theta)$ ，得二次近似：

$$F(\theta) \approx \frac{1}{2}\|f(\theta_0) + J(\theta - \theta_0) - \tilde{x}\|^2 \quad (7.4)$$

对 $(\theta - \theta_0)$ 求梯度并令其为零（一阶最优性条件）：

$$J^T J(\theta - \theta_0) + J^T (f(\theta_0) - \tilde{x}) = 0 \quad (7.5)$$

$$J^T J(\theta - \theta_0) = -J^T \Delta \quad (7.6)$$

情形 A： $J^T J$ 可逆（过约束， $m \geq n$ ）

$$\theta = \theta_0 - (J^T J)^{-1} J^T \Delta, \quad J^+ = (J^T J)^{-1} J^T \quad (7.7)$$

情形 B： $J^T J$ 奇异， $J J^T$ 可逆（欠定， $m < n$ ，IK 最常见情形）

直接对方程 $J(\theta - \theta_0) = -\Delta$ 两边左乘 $(J J^T)^{-1} J$ ：

$$\theta = \theta_0 - J^T (J J^T)^{-1} \Delta, \quad J^+ = J^T (J J^T)^{-1} \quad (7.8)$$

统一形式 (Moore–Penrose 伪逆):

$$\theta_{i+1} = \theta_i - \alpha J^+ \Delta \quad (7.9)$$

其中:

$$J^+ = \begin{cases} J^T (J J^T)^{-1} & \text{当 } m < n \text{ (欠定)} \\ (J^T J)^{-1} J^T & \text{当 } m \geq n \text{ (超定)} \end{cases} \quad (7.10)$$

[NOTE] 为什么欠定情形用 $J^T(JJ^T)^{-1}$?

$J \in \mathbb{R}^{3 \times n}$ ($n > 3$), $J^T J \in \mathbb{R}^{n \times n}$ 的秩最多为 3, 是奇异的 (不可逆)。但 $J J^T \in \mathbb{R}^{3 \times 3}$ 若满秩则可逆。Moore–Penrose 伪逆给出范数最小的解, 即关节角变化量最小的解。

与 Jacobian 转置法的比较: 伪逆法通常收敛快得多 (类似牛顿法 vs 梯度下降的差距), 但需要求解一个 $m \times m$ 的线性系统。

7.3 方法三: 阻尼最小二乘 (DLS) / Levenberg–Marquardt

问题: 当链伸直或处于奇异构型时, $J J^T$ (或 $J^T J$) 的最小特征值接近 0, $(J J^T)^{-1}$ 会使 θ 更新量爆炸。

解决: 在目标函数中加正则项 (惩罚关节角变化量):

$$F(\theta) = \frac{1}{2} \|f(\theta) - \tilde{x}\|^2 + \frac{\lambda}{2} \|\theta - \theta_i\|^2 \quad (7.11)$$

重新对 $(\theta - \theta_0)$ 求最优条件, 得到 DLS 更新公式:

$$\theta_{i+1} = \theta_i - \alpha J^T (J J^T + \lambda^2 I)^{-1} \Delta \quad (7.12)$$

等价地 (当 $m > n$ 时用另一形式):

$$\theta_{i+1} = \theta_i - \alpha (J^T J + \lambda^2 I)^{-1} J^T \Delta \quad (7.13)$$

阻尼参数 λ 的意义: $-\lambda \rightarrow 0$: 退化为纯伪逆 (收敛快但奇异处不稳) $-\lambda \rightarrow \infty$: 退化为梯度下降 (稳定但慢) $-\lambda$ 适中: 在速度和稳定性之间折中, 是工业标准

[TIP] Levenberg–Marquardt = 工业默认

DLS 也叫 Levenberg–Marquardt (LM) 算法, 在 Maya、Blender、Unity IK solver、机械臂控制器中无处不在。只需调一个旋钮 λ 即可在奇异处保持稳定, 是角色动画 IK 的主流方案。

7.4 加权阻尼 (Weighted DLS)

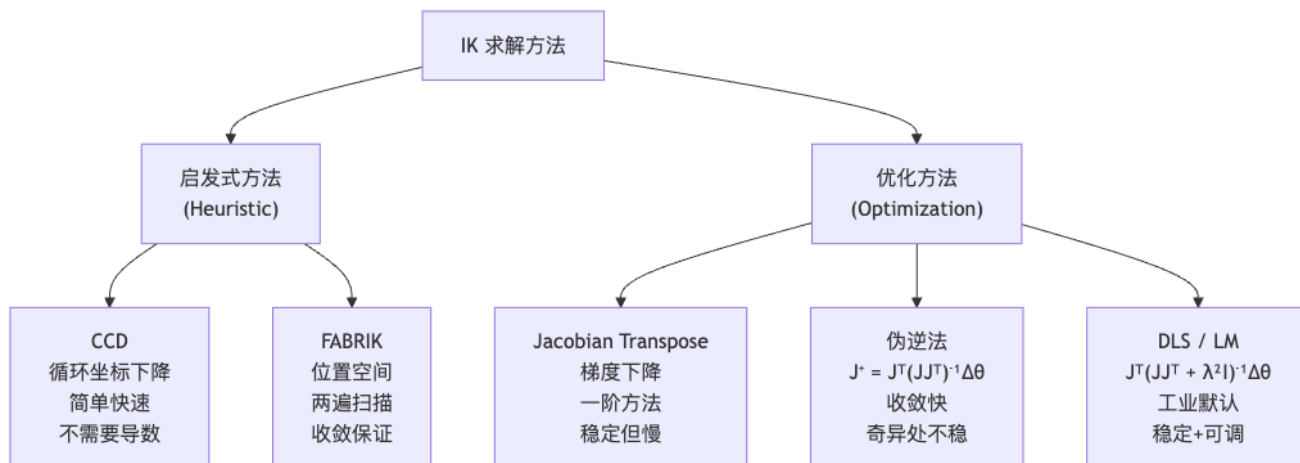
给不同关节设置不同权重矩阵 $W = \text{diag}(w_0, w_1, \dots, w_n)$, 偏好”不常动”的关节少动:

$$F = \frac{1}{2} \|f - \tilde{x}\|^2 + \frac{\lambda}{2} (\theta - \theta_i)^T W (\theta - \theta_i) \quad (7.14)$$

$$\theta_{i+1} = \theta_i - \alpha (J^T J + \lambda W)^{-1} J^T \Delta \quad (7.15)$$

用途：给脊椎、骨盆等核心关节大权重，使 IK 优先动末梢（手腕、肘），让姿态更自然。

7.5 四种方法汇总



8. 全身 IK (Full-Body IK)

8.1 链穿过根关节的处理

问题：想把脚踩到某点，但脚 → 骨盆 → 根关节这条链穿过了根，不是简单的单链。

标准处理（单约束）：1. 把“脚 → 根”这段的关节角全部反向旋转（视脚为新根，把链翻转）2. 在翻转链上跑 IK（脚固定，根当末端）3. 用翻转链的 FK 重新计算根在世界坐标系的变换

通用处理（多约束，优化方法）：直接把 t_0, R_0 放进 θ 里一起优化——根关节就是一个 6 DoF 的“虚拟浮动关节”。

8.2 多末端 IK (全身 IK)

同时约束多个末端（双脚 + 双手 + 头 = 5 个约束）：

$$F(\theta) = \frac{1}{2} \sum_k \|f_k(\theta) - \tilde{x}_k\|^2 + \frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2 \quad (8.1)$$

正则项 $\|\theta - \tilde{\theta}\|^2$ 把解拉向参考姿态 $\tilde{\theta}$ （如上一帧或用户设定的自然姿态），避免欠定带来的奇怪姿势（“鬼畜 pose”）。

[NOTE] Jacobian 的维度扩展

多末端时， f 的输出维度 $m = 3 \times (\text{末端数})$ （若含朝向则 $m = 6 \times (\text{末端数})$ ）， $J \in \mathbb{R}^{m \times n}$ 同样用几何法逐列构造。

8.3 商业 Rig 与多链 IK

商业软件（Blender、Maya、MotionBuilder）中的角色 Rig 通常由多条 IK 链组成：– 用户激活某些 IK 链（如“手抓住某物”），其余链保持 FK 控制 – 被 IK 链控制的关节固定；FK 链的关节自由运动 – 这是动画师的工作流，底层仍是以上 IK 算法

9. Worked Examples (例题)

[EX] 例题 1: 二连杆 FK

题: 二连杆机械臂, link 0 长 1, link 1 长 1, 关节 0 在原点 ($p_0 = (0, 0, 0)^T$), $R_0 = I$, joint 1 旋转角 $\theta_1 = 90^\circ$ (绕 z 轴), joint 2 无旋转 $R_2 = I$, 末端偏移 $x_0 = (0, 0, 0)^T$ 。求末端位置。

解: $-Q_0 = R_0 = I$, $p_0 = (0, 0, 0)^T - p_1 = p_0 + Q_0 l_0 = (0, 0, 0) + I \cdot (1, 0, 0)^T = (1, 0, 0)^T - Q_1 = Q_0 R_1 = I \cdot R_z(90^\circ) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} - p_2 = p_1 + Q_1 l_1 = (1, 0, 0) + Q_1(1, 0, 0)^T = (1, 0, 0) + (0, 1, 0) = (1, 1, 0)^T - x = p_2 + Q_2 x_0 = (1, 1, 0)^T$
末端坐标 = $(1, 1, 0)$ 。直觉: 关节 0 水平, 关节 1 转 90° 后第二段向上, 所以末端在 $x=1, y=1$ 。

[EX] 例题 2: 二连杆解析 IK (余弦定理)

题: 二连杆 $l_0 = l_1 = 1$, 目标位置 $\tilde{x} = (\sqrt{2}/2, \sqrt{2}/2, 0)^T$, 根在原点。求关节角。

解: 1. 目标距离 $d = \|\tilde{x}\| = 1$ 2. 余弦定理求 joint 1 角度 (肘关节弯折量):

$$\cos \phi_1 = \frac{l_0^2 + l_1^2 - d^2}{2l_0 l_1} = \frac{1 + 1 - 1}{2} = \frac{1}{2} \implies \phi_1 = 60^\circ$$

因此 joint 1 的关节角 $\theta_1 = 180^\circ - 60^\circ = 120^\circ$ (注意 joint 1 的弯折使两 link 夹角为 60°) 3. 求 joint 0 使链对准目标: 目标方向为 $(1, 1, 0)/\sqrt{2}$, 即 45° 。计算 joint 0 旋转到使整链末端对准目标。

最终: $\theta_0 = 45^\circ - \alpha$ (其中 α 由三角形几何确定), $\theta_1 = 120^\circ$ 。(注: 有两解, 取”肘朝上”还是”肘朝下”由用户选择。)

[EX] 例题 3: Jacobian 矩阵构造 (二连杆 hinge)

题: 二连杆 (2D, hinge 关节), link 长度均为 1, 当前 pose $\theta_0 = 0, \theta_1 = 90^\circ$, 关节轴均为 $e_z = (0, 0, 1)^T$, 末端在 $(1, 1, 0)^T$ (由例题 1)。构造 Jacobian $J \in \mathbb{R}^{3 \times 2}$ 。

解: $-J$ 第 0 列 (关节 0): $-$ 旋转轴 $a_0 = e_z = (0, 0, 1)^T$ $-$ 从关节 0 到末端: $r_0 = x - p_0 = (1, 1, 0)^T - \frac{\partial f}{\partial \theta_0} = a_0 \times r_0 = (0, 0, 1) \times (1, 1, 0) = (-1, 1, 0)^T - J$ 第 1 列 (关节 1): $-$ 旋转轴 $a_1 = Q_1 e_z = (0, 0, 1)^T$ (2D 中轴不变) $-$ 从关节 1 到末端: $r_1 = x - p_1 = (1, 1, 0) - (1, 0, 0) = (0, 1, 0)^T - \frac{\partial f}{\partial \theta_1} = a_1 \times r_1 = (0, 0, 1) \times (0, 1, 0) = (-1, 0, 0)^T$

$$J = \begin{pmatrix} -1 & -1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

[EX] 例题 4: DLS 更新步骤

题：承例题 3，目标 $\tilde{x} = (1, 1.5, 0)^T$ ，当前末端 $x = (1, 1, 0)^T$ ， $\lambda = 0.1$ ， $\alpha = 1$ 。用 DLS 计算一步更新量 $\Delta\theta$ 。

解： $\Delta = f(\theta) - \tilde{x} = (0, -0.5, 0)^T$

(取 J 的 xy 行， $J_{2D} = \begin{pmatrix} -1 & -1 \\ 1 & 0 \end{pmatrix}$)

$$JJ^T = \begin{pmatrix} -1 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -1 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$$

$$JJ^T + \lambda^2 I = \begin{pmatrix} 2.01 & -1 \\ -1 & 1.01 \end{pmatrix}$$

$$(JJ^T + \lambda^2 I)^{-1} = \frac{1}{\det} \begin{pmatrix} 1.01 & 1 \\ 1 & 2.01 \end{pmatrix}, \quad \det \approx 2.01 \times 1.01 - 1 \approx 1.03$$

$$\Delta\theta = -J^T (JJ^T + \lambda^2 I)^{-1} \Delta_{2D} \approx (\text{数值接近让末端向上移动的小角度})$$

关键结论：DLS 会给出一个有界的 $\Delta\theta$ ，即使在接近奇异时也不会爆炸。

[EX] 例题 5: Retargeting 公式应用

题：角色 A 的参考姿态 (T-pose) 与角色 B 的参考姿态 (A-pose) 中，关节 i (肩关节) 的全局朝向偏差：父 (脊椎) $Q_{p_i}^{A \rightarrow B} = R_z(30^\circ)$ ，自身 $Q_i^{A \rightarrow B} = R_z(30^\circ)R_x(-10^\circ)$ 。角色 A 上肩关节局部旋转为 $R_i^A = R_x(45^\circ)$ (手臂上举)。求角色 B 上等效旋转 R_i^B 。

解：

$$\begin{aligned} R_i^B &= Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T = R_z(30^\circ) \cdot R_x(45^\circ) \cdot [R_z(30^\circ)R_x(-10^\circ)]^T \\ &= R_z(30^\circ) \cdot R_x(45^\circ) \cdot R_x(10^\circ) \cdot R_z(-30^\circ) \end{aligned}$$

结论：不能简单把 A 的旋转直接用到 B，必须用 retargeting 公式加“偏差旋转”前后包夹。

10. Anki 记忆卡片

以下为 Q/A 形式，可直接导入 Anki。

#	Q (正面)	A (背面)
1	FK 核心递推公式? (全局朝向 + 位置)	$Q_i = Q_{p_i} R_i; x_i = x_{p_i} + Q_{p_i} l_i$
2	从全局朝向还原局部旋转的公式?	$R_i = Q_{p_i}^T Q_i$
3	Retargeting 核心公式?	$R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T$
4	Jacobian 矩阵维度? $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$	$J \in \mathbb{R}^{m \times n}$
5	Hinge 关节的 Jacobian 列公式?	$\partial f / \partial \theta_i = a_i \times r_i$ (轴 $\times$ 关节到末端向量)
6	CCD 算法一句话描述?	从末端最近关节开始，逐关节旋转使末端对准目标，循环迭代
7	FABRIK 两遍是哪两遍?	Forward Pass (末端拉到目标，向根反推) + Backward Pass (根回原位，向末端正推)
8	Jacobian 转置更新公式?	$\theta_{i+1} = \theta_i - \alpha J^T \Delta$
9	欠定情形 ($m < n$) 的 Moore-Penrose 伪逆公式?	$J^+ = J^T (J J^T)^{-1};$ $\theta_{i+1} = \theta_i - \alpha J^+ \Delta$

#	Q (正面)	A (背面)
10	DLS/LM 更新公式 (欠定形式)?	$\theta_{i+1} = \theta_i - \alpha J^T (J J^T + \lambda^2 I)^{-1} \Delta$
11	DLS 中 λ 的作用? 极端情形?	阻尼参数; $\lambda \rightarrow 0$ 退化为伪逆 (快但奇异处不稳), $\lambda \rightarrow \infty$ 退化为梯度下降 (稳但慢)
12	为什么不能用 Axis-Angle 参数化 ball joint 再用几何法求 Jacobian?	Axis-Angle 的 Jacobian 公式极复杂; 应改用 Euler 角 + 三 hinge 分解
13	CCD 收敛性如何?	无约束情形收敛, 但不保证收敛到全局最优; 加 joint limits 后收敛不保证
14	T-pose 的数学定义?	所有局部旋转 $R_i = I$ 时的姿态, 是骨骼的“零点”参考姿态
15	全身 IK 多末端目标函数?	$F(\theta) =$
16	链穿过根关节时的 IK 处理策略?	$\frac{1}{2} \sum_k \ f_k(\theta) - \tilde{x}_k\ ^2 + \frac{\lambda}{2} \ \theta - \tilde{\theta}\ ^2$ 将脚视为新根, 把脚 $\rightarrow$ 根的关节角反向旋转 (翻转链), 在翻转链上跑 IK, 最后由 FK 重算根变换

11. 中英术语速查表

中文	English	说明/公式
运动学	Kinematics	研究运动几何, 不涉及力/质量
骨骼	Skeleton	角色的关节层级结构
关节	Joint	相邻骨段连接点, 允许旋转
骨段 / 骨头	Link / Bone / Body	两关节间的刚性段
根关节	Root Joint	骨骼树的根, 有世界坐标
末端执行器	End-Effector	关心其位置的末端关节
局部旋转	Local Rotation	R_i , 相对父坐标系
全局朝向	Global Orientation	$Q_i = Q_{p_i} R_i$
自由度	DoF (Degrees of Freedom)	描述系统状态所需的独立参数数
铰链关节	Hinge / Revolute Joint	1 DoF, 如膝、肘
球窝关节	Ball-and-Socket Joint	3 DoF, 如髋、肩
关节角限制	Joint Limits	$\theta_{\min} \leq \theta \leq \theta_{\max}$
参考姿态 / T-pose	Reference Pose / T-pose	$R_i = I$ 时的零点姿态
正向运动学	Forward Kinematics (FK)	给 θ 求 x
逆运动学	Inverse Kinematics (IK)	给 $\tilde{x}$ 求 θ
动作迁移	Retargeting	在不同 reference pose 角色间迁移动作
雅可比矩阵	Jacobian Matrix	$J = \partial f / \partial \theta \in \mathbb{R}^{m \times n}$
伪逆	Pseudoinverse	$J^+ = J^T (J J^T)^{-1}$ (欠定) 或 $(J^T J)^{-1} J^T$ (超定)
阻尼最小二乘	Damped Least Squares (DLS)	$J^T (J J^T + \lambda^2 I)^{-1}$
Levenberg-Marquardt	LM 算法	DLS 的另一种叫法
循环坐标下降	CCD (Cyclic Coordinate Descent)	启发式 IK, 逐关节对准目标
前向后向到达 IK	FABRIK	位置空间两遍扫描
奇异构型	Singular Configuration	$J J^T$ 近奇异, 如链完全伸直
阻尼系数	Damping Parameter	λ , 调节 DLS 稳定性
欠定	Underdetermined	$n > m$, DoF 多于约束
过约束	Overdetermined	$n < m$, 约束多于 DoF

12. 复习要点

[TIP] 期末复习——本讲必记

1. FK = 链式矩阵乘法: $Q_i = Q_{p_i} R_i$, $x_i = x_{p_i} + Q_{p_i} l_i$ 。树状骨架按拓扑排序遍历。
2. Retargeting: 不同 reference pose 间动作迁移用 $R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T$, 父偏差在前、子偏差 (的转置) 在后。
3. IK 启发式:
 - CCD: 逐关节从末端到根循环旋转, 本质是坐标下降, 简单但可能不自然
 - FABRIK: 位置空间两遍 (Forward + Backward), 无约束时收敛保证
4. IK 优化: 目标 $F = \frac{1}{2} \|f(\theta) - \tilde{x}\|^2$, 关键工具是 Jacobian $J = \partial f / \partial \theta$, hinge 关节 $J_{:,i} = a_i \times r_i$ 。
5. 三大更新公式 (会推导 + 能说清物理意义):
 - Jacobian 转置: $\theta \leftarrow \theta - \alpha J^T \Delta$ (梯度下降, 稳定慢)
 - 伪逆: $\theta \leftarrow \theta - \alpha J^T (J J^T)^{-1} \Delta$ (快, 奇异处爆炸)
 - DLS: $\theta \leftarrow \theta - \alpha J^T (J J^T + \lambda^2 I)^{-1} \Delta$ (工业默认, λ 调节稳定性)
6. Ball joint 的 Jacobian: 分解为三个串联 hinge, 轴为 $Q_{i-1} e_x$, $Q_{i-1} R_{ix} e_y$, $Q_{i-1} R_{ix} R_{iy} e_z$, 切勿用 axis-angle 直接求导。
7. 全身 IK: 多末端用求和形式, 根关节作 6 DoF 浮动变量; 正则项 $\lambda \|\theta - \tilde{\theta}\|^2$ 防止奇怪姿态。

[NOTE] 数据来源

本笔记整理自 MOCCA 2026 (Fundamentals of Character Animation and Motion Control) 课件 “**Character Kinematics: Forward and Inverse Kinematics**” (Libin Liu, SIST @ PKU, 2026)。参考文献: Aristidou & Lasenby, “FABRIK: A fast, iterative solver for the Inverse Kinematics problem”, Graphical Models 73(5), 2011; Aristidou et al., “Inverse Kinematics Techniques in Computer Graphics: A Survey”, Computer Graphics Forum, 2018。

Lecture 04 —关键帧动画 (Keyframe Animation)

[TIP] 学习指南

本节核心：角色动画的本质是在**关键姿态之间做插值**。读完本讲你应能回答：为什么线性插值不够平滑？三次样条、Hermite、Catmull-Rom 各有什么取舍？旋转为什么不能直接线性插值？SLERP 怎么推导？Blend Space 和 RBF 又解决了什么问题？

前置知识：– 基本线性代数（矩阵乘法、逆矩阵）– 四元数基础（ $\mathbf{q} = w + \mathbf{v}$ ，模长，点积）——在 §4 会从零引导

重点掌握：1. Hermite 基函数的推导与矩阵形式（常考手推）2. Catmull-Rom 的切线公式及其与 Hermite 的关系 3. SLERP 公式的完整含义与使用前提 4. 连续性等级 $C^0/C^1/C^2$ 与各种条的对应关系 5. Blend Space 与重心坐标、RBF 在动画混合中的用途

阅读策略：第一遍跳过所有公式推导，只看“几何直觉”和对比表；第二遍细读推导，对照 Worked Examples 自己算一遍。

0. 从走马灯到关键帧：问题从哪里来？

19 世纪的走马灯 (Zoetrope) 揭示了动画的基本原理：人眼的视觉暂留使快速切换的静止图像产生运动感。传统手绘动画需要每秒 24 帧，每帧都要画师手绘。

关键帧动画 (Keyframe Animation) 是工业界对这一问题的聪明回答：

动画师只画（或设置）少数几个**关键帧 (Keyframes)**——如起始姿态、高潮姿态、结束姿态；其余**中间帧 (In-betweens / Tweenes)** 由计算机通过**插值 (Interpolation)** 自动生成。

在 3D 动画（如 Pixar 的影片）中，“关键帧”通常记录的是若干离散时刻的**关节角度、骨骼位置、形态混合权重**等。因此，关键帧动画的数学核心就是：

给定时间轴上若干“关键姿态”数据点，找一条平滑曲线穿过它们。

这正是**插值问题**的一般形式。

1. 插值问题的形式化

1.1 基本定义

输入：数据对集合

$$D = \{(x_i, y_i) \mid i = 0, 1, \dots, N\} \quad (1.1)$$

其中 x_i 通常是时间, y_i 可以是标量 (如单个关节角度) 或向量 (如三维位置、完整骨骼姿态)。

目标: 求函数 $f(x)$ 使得

$$f(x_i) = y_i, \quad \forall (x_i, y_i) \in D \quad (1.2)$$

[NOTE] 插值 vs 拟合 vs 外推

– **插值 (Interpolation)**: f 严格通过所有数据点。动画关键帧必须用插值——动画师指定的姿态必须精确出现在对应时间帧上。– **回归/拟合 (Regression/Fitting)**: f 最小化与数据点的整体误差, 不要求精确通过。适合带噪声的实验数据。– **外推 (Extrapolation)**: 在数据范围 $[x_0, x_N]$ 之外求 f 的值, 风险大, 误差可能剧增。

1.2 阶梯插值 (Stepped Interpolation)

最简单的策略: 在每个区间 $[x_i, x_{i+1})$ 内保持常数值。

$$f(x) = y_i, \quad x \in [x_i, x_{i+1}) \quad (1.3)$$

- 完全不连续 (无 C^0)。
- 在动画中用于离散切换的参数, 如“可见性”开关、表情动作集的切换。

1.3 线性插值 (Linear Interpolation, LERP)

在区间 $[x_1, x_2]$ 内, f 是连接 (x_1, y_1) 和 (x_2, y_2) 的直线段:

$$f(x) = y_1 + \frac{x - x_1}{x_2 - x_1}(y_2 - y_1) \quad (1.4)$$

引入归一化参数 $t = (x - x_1)/(x_2 - x_1) \in [0, 1]$, 得到更简洁的形式:

$$f(t) = (1 - t)y_1 + ty_2 \quad (1.5)$$

- 整体只有 C^0 (每段内部导数为常数, 不同段在节点处导数不连续)。
- 直觉: 关键帧之间的动作沿直线“突变速度”——角色运动会出现机器人般的顿挫感。

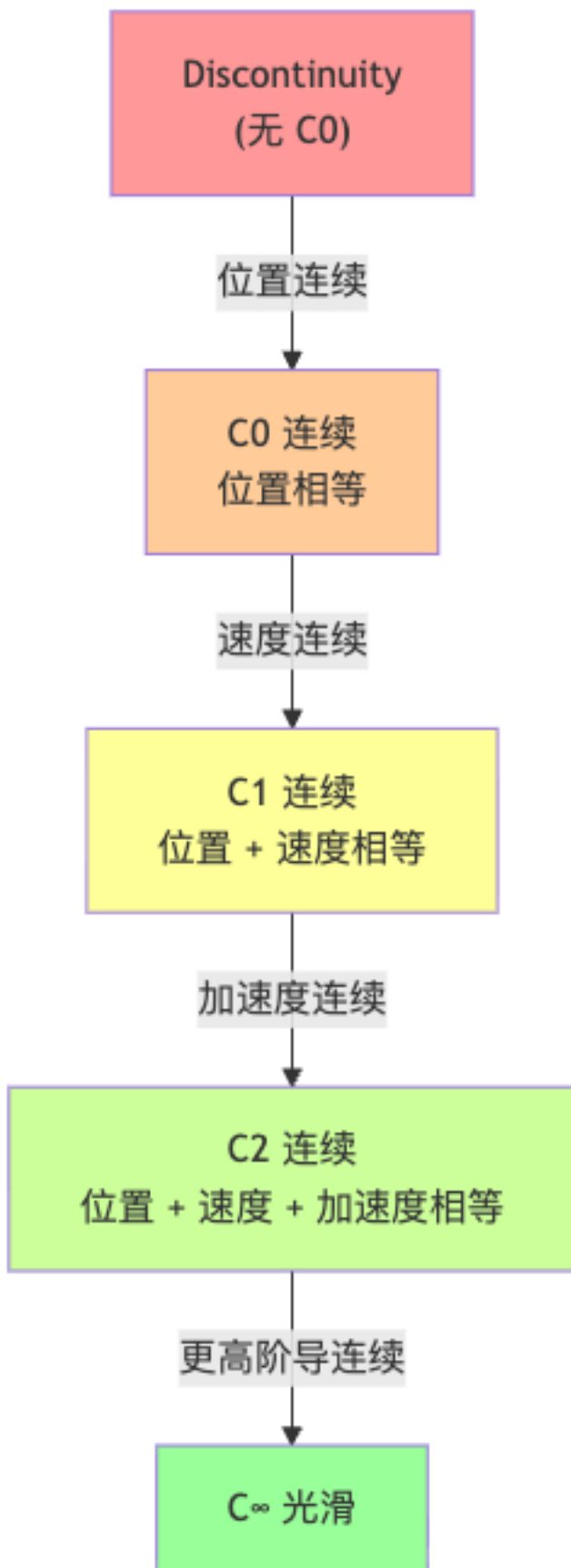
2. 平滑性 (Smoothness): 我们在追求什么?

在深入样条之前, 需要建立衡量曲线质量的语言。

[NOTE] 连续性层级定义

设两段曲线 $S_{i-1}(x)$ 和 $S_i(x)$ 在节点 x_i 处拼接:

- C^0 连续 (位置连续): $S_{i-1}(x_i) = S_i(x_i)$
- C^1 连续 (速度连续): C^0 且 $S'_{i-1}(x_i) = S'_i(x_i)$
- C^2 连续 (加速度连续): C^1 且 $S''_{i-1}(x_i) = S''_i(x_i)$



[WARN] 为什么动画对平滑性有要求?

– C^0 不满足: 画面位置跳变, 肉眼立刻察觉。– C^0 满足但 C^1 不满足: 角色速度突变, 看起来像被无形力拉扯。– C^1 满足但 C^2 不满足: 加速度突变, 物理上对应“冲击力”, 轻微的人眼察觉。– 一般动画要求至少 C^1 ; 电影级动画追求 C^2 甚至更高。

另有 G^1 (几何连续): 不要求导数向量大小相等, 只要求方向相同——允许在节点改变“参数化速度”, Bezier 曲线拼接时常见。

3. 多项式插值与 Runge 现象

3.1 全局多项式的想法

最自然的高阶插值: 设

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \quad (3.1)$$

对 $N + 1$ 个数据点各代入, 得到 **Vandermonde 线性方程组**:

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^n \end{bmatrix}}_{\text{Vandermonde 矩阵}} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix} \quad (3.2)$$

若 x_i 互不相同, 此系统有唯一解。要穿过 $N + 1$ 个点, 需要 $n = N$ 次多项式。

3.2 Runge 现象 (Runge's Phenomenon)

[WARN] 高阶全局多项式的致命缺陷

高次多项式插值在区间端点附近会出现剧烈振荡——即使原始数据点完全平滑。

经典例子: 在 $[-1, 1]$ 上对 $f(x) = 1/(1 + 25x^2)$ 取 16 个等距节点做 15 次多项式插值, 端点附近的振荡幅度远超函数本身。

结论: 对于动画插值, 高阶全局多项式不可用。

解决方案: 坚持用低阶多项式, 但将定义域分段, 在每段上各用一个低阶多项式 → **样条 (Spline)**。

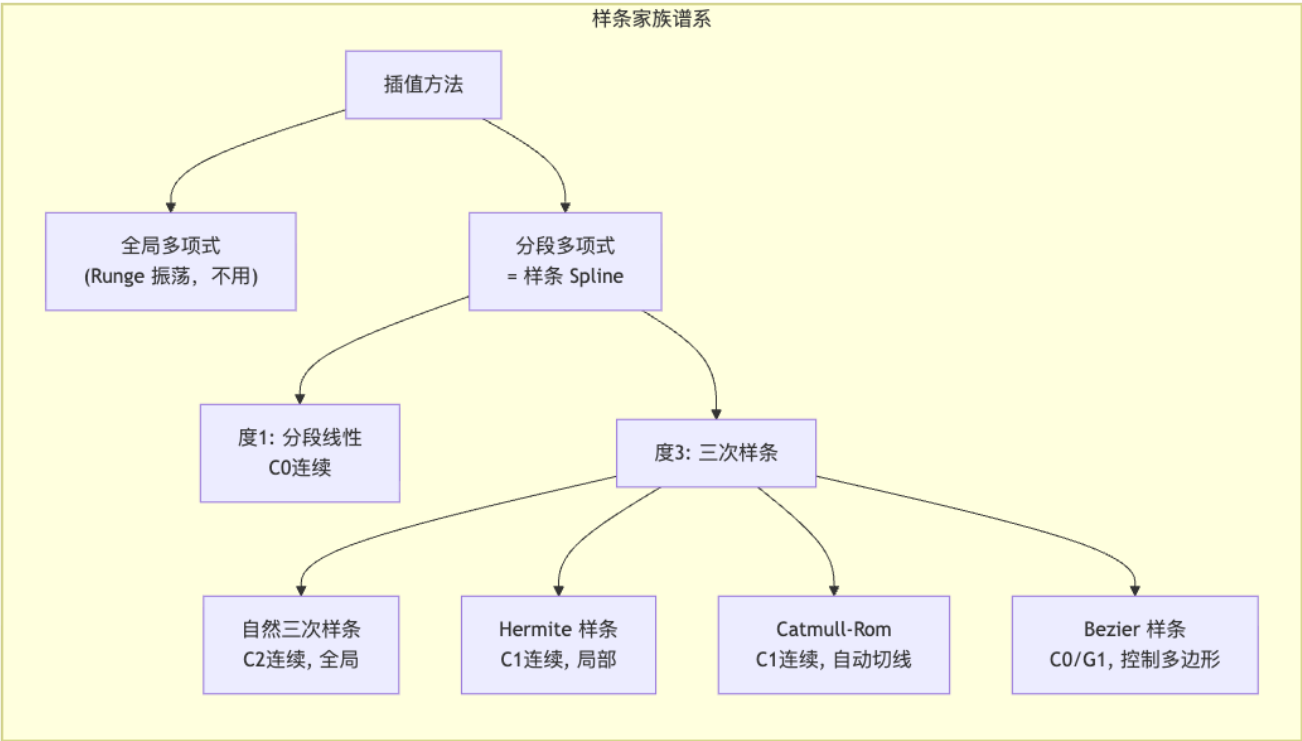
4. 样条插值 (Spline Interpolation): 核心工具

4.1 分段多项式的思想

将 $N + 1$ 个数据点之间的 N 段, 各用一个低阶多项式 $S_i(x)$ 拟合:

$$f(x) = S_i(x), \quad x \in [x_i, x_{i+1}], \quad i = 0, 1, \dots, N - 1 \quad (4.1)$$

不同阶数对应不同的“样条”名称。



4.2 分段线性样条

$$S_i(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i) \quad (4.2)$$

整体 C^0 ，与多段 LERP 完全等价。

5. 三次样条 (Cubic Spline)

5.1 形式与自由度

每段用三次多项式：

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i \quad (5.1)$$

N 段共有 $4N$ 个待定系数。加入以下约束恰好唯一确定系统：

约束类型	表达式	数量
插值条件（每段两端通过数据点）	$S_i(x_i) = y_i, S_i(x_{i+1}) = y_{i+1}$	$2N$
C^1 连续（内节点速度一致）	$S'_{i-1}(x_i) = S'_i(x_i),$ $i = 1, \dots, N-1$	$N-1$
C^2 连续（内节点加速度一致）	$S''_{i-1}(x_i) = S''_i(x_i),$ $i = 1, \dots, N-1$	$N-1$

约束类型	表达式	数量
边界条件（两端）	如 $S'_0(x_0), S'_{N-1}(x_N)$ 或 $S''_0(x_0), S''_{N-1}(x_N)$	2

合计 $4N$ 个方程，唯一确定 $4N$ 个参数，组成一个大型三对角线性方程组。

5.2 优缺点

[NOTE] 自然三次样条的优点

– 在所有 C^2 插值曲线中，自然三次样条最小化曲率平方积分 $\int (f'')^2 dx$ ——工程意义上最“光顺”。– 全局 C^2 ，视觉上极为平滑。

[WARN] 动画中的两大痛点

1. **无局部控制**：移动任何一个数据点，整条曲线都会改变形状。动画师无法独立调整某一段。2. **计算昂贵**： N 个关键帧需要解 $4N \times 4N$ 的线性系统；关键帧很多时难以实时处理。这正是动画软件更偏向 **Hermite** 和 **Catmull-Rom** 样条的原因。

6. 三次 Hermite 样条 (Cubic Hermite Spline)

6.1 核心思想：给出切线

Hermite 样条的关键改进：不仅指定数据点位置，还指定每个关键帧处的切线（一阶导数）。

$$D' = \{(x_i, m_i) \mid i = 0, \dots, N\}, \quad m_i = S'_i(x_i) \quad (6.1)$$

这样，每段 S_i 只依赖本段两端的 (y_i, m_i) 和 (y_{i+1}, m_{i+1}) ，天然实现**局部控制**。

6.2 单段推导

问题：给定 (x_1, y_1, m_1) 和 (x_2, y_2, m_2) ，求三次曲线 $S(x) = ax^3 + bx^2 + cx + d$ 满足

$$S(x_1) = y_1, \quad S(x_2) = y_2, \quad S'(x_1) = m_1, \quad S'(x_2) = m_2 \quad (6.2)$$

归一化：令 $t = (x - x_1)/(x_2 - x_1) \in [0, 1]$ ，将问题转化为求 $S(t) = at^3 + bt^2 + ct + d$ 满足

$$S(0) = y_1, \quad S(1) = y_2, \quad S'(0) = m_1, \quad S'(1) = m_2 \quad (6.3)$$

注意 $S'(t)$ 对 t 求导（还需乘以链式法则因子 $1/(x_2 - x_1)$ ，但习惯上吸收进 m_i 的定义中）。

代入求值：

$$\begin{cases} S(0) = d = y_1 \\ S(1) = a + b + c + d = y_2 \\ S'(0) = c = m_1 \\ S'(1) = 3a + 2b + c = m_2 \end{cases} \quad (6.4)$$

写成矩阵方程：

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix} \quad (6.5)$$

求逆得到 **Hermite 基矩阵** M_H ：

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \underbrace{\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{M_H} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix} \quad (6.6)$$

6.3 Hermite 基函数

将 $S(t) = [t^3 \ t^2 \ t \ 1] M_H [y_1 \ y_2 \ m_1 \ m_2]^T$ 展开，得到四个 **Hermite 基函数**：

$$S(t) = H_0(t) y_1 + H_1(t) y_2 + H_2(t) m_1 + H_3(t) m_2 \quad (6.7)$$

$$\begin{aligned} H_0(t) &= 2t^3 - 3t^2 + 1 && \text{(端点 } x_1 \text{ 处的位置基)} \\ H_1(t) &= -2t^3 + 3t^2 && \text{(端点 } x_2 \text{ 处的位置基)} \\ H_2(t) &= t^3 - 2t^2 + t && \text{(端点 } x_1 \text{ 处的切线基)} \\ H_3(t) &= t^3 - t^2 && \text{(端点 } x_2 \text{ 处的切线基)} \end{aligned} \quad (6.8)$$

[NOTE] 基函数的“单位性”验证

可直接代入验证：

	$H_0(t)$	$H_1(t)$	$H_2(t)$	$H_3(t)$
$t = 0$ 处值	1	0	0	0
$t = 1$ 处值	0	1	0	0
$t = 0$ 处导数	0	0	1	0
$t = 1$ 处导数	0	0	0	1

因此 H_0 和 H_1 分别在两端点处“激活”位置贡献， H_2 和 H_3 分别在两端点处“激活”切线贡献。这组基函数类似于有限元中的**形函数 (shape functions)**。

6.4 高维推广

动画中的姿态通常是向量（三维位置、关节角向量等）。向量情形直接推广：

$$S(t) = H_0(t) y_1 + H_1(t) y_2 + H_2(t) m_1 + H_3(t) m_2 \quad (6.9)$$

每个分量独立应用同一组 Hermite 基函数。

[EX] 应用：PowerPoint 的”曲线”工具

PowerPoint / Keynote 的贝塞尔/曲线绘制工具背后就是 Hermite（或等价的 Bezier）样条——拖动”切线控制柄”就是在调 m_i 。

7. Catmull–Rom 样条

7.1 动机：自动估算切线

Hermite 需要用户（或算法）提供切线 m_i 。但在关键帧动画中，动画师通常只给位置关键帧，不想手动指定切线。Catmull–Rom 样条用最简单的方式自动估算切线：

$$m_i = \frac{1}{2} \cdot \frac{p_{i+1} - p_{i-1}}{x_{i+1} - x_{i-1}} \quad (7.1)$$

几何直觉： m_i 是连接前后两个邻居数据点的弦的中点斜率。节点 i 处的切线”看向”前后两侧，方向沿 $p_{i-1} \rightarrow p_{i+1}$ 的方向。

7.2 代入 Hermite 得到 Catmull–Rom 段

将 (7.1) 代入 Hermite 公式 (6.7)，对第 $[p_1, p_2]$ 段（使用 p_0, p_1, p_2, p_3 四点）：

$$S(t) = H_0(t) p_1 + H_1(t) p_2 + H_2(t) \cdot \frac{p_2 - p_0}{2} + H_3(t) \cdot \frac{p_3 - p_1}{2} \quad (7.2)$$

可以写成矩阵形式（Catmull–Rom 基矩阵 M_{CR} ）：

$$S(t) = \frac{1}{2} [t^3 \ t^2 \ t \ 1] \begin{bmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix} \quad (7.3)$$

7.3 特性小结

特性	Catmull–Rom
连续性	C^1 （切线连续但二阶导不保证连续）
局部控制	是（每段只用相邻四点，移动 p_i 只影响前后两段）
是否过控制点	是（直接通过 $p_1, p_2, \dots$ ）
需要用户切线	否（自动由邻居估算）

[NOTE] Catmull–Rom 命名来源

Edwin Catmull（皮克斯与迪士尼动画前总裁）与 Raphael Rom（1974）共同提出。Edwin Catmull 与 Pat Hanrahan 因对 3D 计算机图形的奠基性贡献，共同获得 2019 年 ACM 图灵奖。

8. Bezier 样条 (简介)

虽然课件重点在 Hermite/Catmull-Rom, Bezier 样条是 CG 中同样广泛使用的工具, 值得简介。

8.1 de Casteljau 递归构造

三次 Bezier 曲线由四个控制点 b_0, b_1, b_2, b_3 定义, 通过 de Casteljau 算法递归计算:

$$b_i^{(r)}(t) = (1-t)b_i^{(r-1)}(t) + tb_{i+1}^{(r-1)}(t) \quad (8.1)$$

其中上标 (r) 表示第 r 轮细分, $b_i^{(0)} = b_i$ 。最终 $b_0^{(3)}(t)$ 即曲线上的点。

展开得到伯恩斯坦多项式 (Bernstein Polynomials) 形式:

$$P(t) = \sum_{k=0}^3 \binom{3}{k} (1-t)^{3-k} t^k b_k = (1-t)^3 b_0 + 3(1-t)^2 t b_1 + 3(1-t)t^2 b_2 + t^3 b_3 \quad (8.2)$$

8.2 与 Hermite 的关系

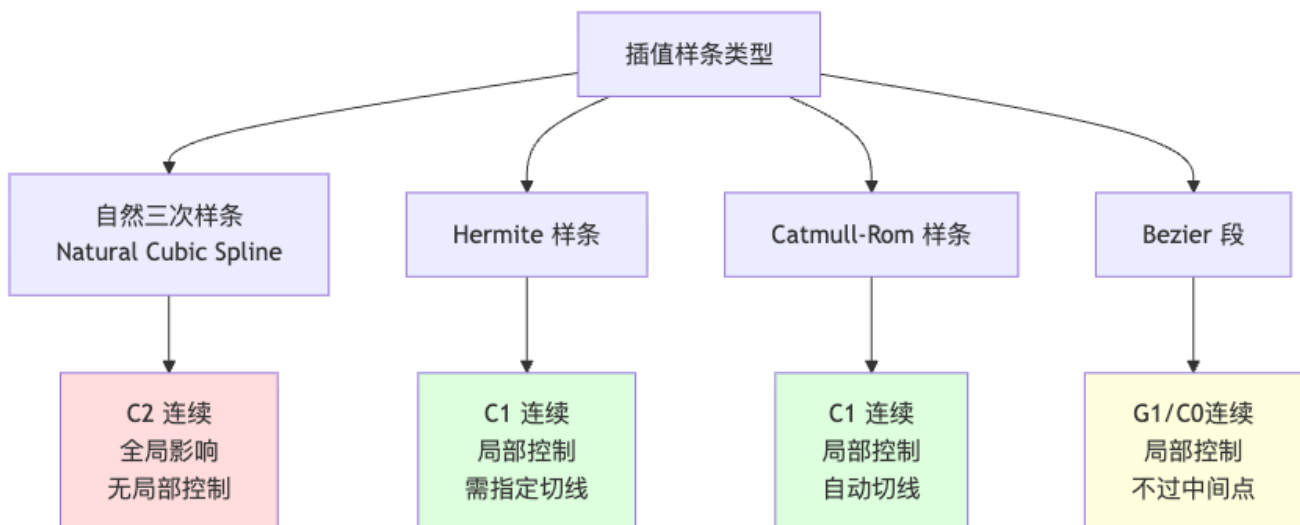
三次 Bezier 的端点切线为 $P'(0) = 3(b_1 - b_0)$ 和 $P'(1) = 3(b_3 - b_2)$ 。因此 Bezier 与 Hermite 是等价的: 给定 Hermite 参数 (y_1, y_2, m_1, m_2) , 对应 Bezier 控制点为

$$b_0 = y_1, \quad b_1 = y_1 + m_1/3, \quad b_2 = y_2 - m_2/3, \quad b_3 = y_2 \quad (8.3)$$

[WARN] Bezier 一般不过中间控制点

b_1 和 b_2 只是“切线控制柄”, 曲线不一定通过它们。Catmull-Rom 的优势正是直接过所有输入点。

9. 三类样条综合对比



样条类型	连续性	局部控制	是否过控制点	需切线输入	动画适用性
自然三次样条	C^2	否	是	否	不常用（无局部控制）
Cubic Hermite	C^1	是	是	是	常用（动画师手动切线）
Catmull-Rom	C^1	是	是	否（自动）	最常用 （游戏/影视）
Bezier（逐段）	G^1 / C^0	是	否（中间点）	否（控制柄隐含）	常用（路径/特效）

10. 旋转的插值 (Interpolation of Rotations)

旋转是角色动画中最核心、也最棘手的插值对象。

10.1 旋转的表示方式回顾

表示方式	形式	参数数量	主要问题
旋转矩阵	$R \in SO(3),$ $RR^T = I,$ $\det R = 1$	9（有 6 个正交约束）	过度参数化，插值结果可能离开 $SO(3)$
欧拉角	$R_x(\alpha)R_y(\beta)R_z(\gamma)$		万向锁 (Gimbal Lock) ，插值路径不直观
轴角	(u, θ) 或 $\theta = \theta u$	3（约束 $\ u\ = 1$ ）	轴角插值不保证单位轴
单位四元数	$q = (w, v),$ $\ q\ = 1$	4（在 S^3 上）	最优，但有“双重覆盖”（ q 和 $-q$ 表示同一旋转）

10.2 为什么不能直接线性插值旋转？

[WARN] 朴素线性插值旋转的三大问题

问题 1：欧拉角插值路径不是最短路径 欧拉角三个分量各自线性插值，组合后对应的旋转序列通常不沿球面测地线（最短路径）运动，会产生奇怪的“绕弯”旋转。

问题 2：四元数 LERP 速度不均匀 直接线性插值： $q(t) = (1-t)q_0 + tq_1$ （然后归一化）。结果在球面上不是匀速运动——接近端点处速度慢，中间快——导致动作“先慢后快”或“先快后慢”。

问题 3：旋转速度不恒定 对任意表示做逐分量线性插值，旋转的角速度通常随时间变化，视觉上显得不自然。

下图示意四元数球面 S^3 上的 NLERP（归一化线性插值）与 SLERP 的路径差异：



10.3 SLERP (球面线性插值) 的推导

目标：在单位四元数球面 S^3 上找一条匀角速度的测地线（大圆弧）连接 q_0 和 q_1 。

第 1 步：两个单位四元数之间的夹角 Ω ：

$$\cos \Omega = q_0 \cdot q_1 \quad (10.1)$$

(这里的点积是 $\mathbb{R}^4$ 中的普通内积： $q_0^w q_1^w + q_0^x q_1^x + q_0^y q_1^y + q_0^z q_1^z$ 。)

第 2 步：类比二维圆上的角度插值。在圆上从角度 0 到角度 Ω 做匀角速插值， t 时刻的角度为 $t\Omega$ 。圆上对应的 2D 点可以用两个基向量 e_1, e_2 表示：

$$p(t) = \cos(t\Omega) e_1 + \sin(t\Omega) e_2$$

其中 $e_1 = q_0$, $e_2 = (q_1 - \cos \Omega q_0) / \sin \Omega$ (q_1 在 q_0 方向上的垂直分量归一化)。

第 3 步：代入整理，消去 e_2 ，得到 SLERP 公式：

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1 \quad (10.2)$$

其中 Ω 由 (10.1) 确定， $t \in [0, 1]$ 。

[NOTE] SLERP 的几何意义

– 轨迹是 S^3 上的大圆弧（测地线），是球面上两点间的“最短路径”。– 旋转的角速度恒定：每单位时间旋转角度相同。– 权重系数 $\frac{\sin((1-t)\Omega)}{\sin \Omega}$ 和 $\frac{\sin(t\Omega)}{\sin \Omega}$ 是球面上的“正弦权重”，可类比平面上的线性权重 $(1-t)$ 和 t 。

[WARN] SLERP 的注意事项

1. 双重覆盖问题： q 和 $-q$ 代表同一旋转。若 $q_0 \cdot q_1 < 0$ ，应先取 $q_1 \leftarrow -q_1$ ，以选择短弧（夹角 $< 180^\circ$ 的那条路径）。
2. $\Omega \rightarrow 0$ 的数值稳定性：当两个旋转几乎相同时， $\sin \Omega \approx 0$ ，直接算公式会出现除以零。此时退化为 NLERP（线性插值后归一化）。
3. 只是“线性”：SLERP 在多关键帧之间拼接时，在节点处角速度会突变（只有 C^0 ，不是 C^1 ）。要做平滑的旋转样条需要更高阶方法。

10.4 四元数样条：SQUAD

SLERP 只解决两点之间的插值。多关键帧之间的平滑旋转曲线需要 **SQUAD（球面与四边形插值）**，由 Ken Shoemake 1985 年在 SIGGRAPH 提出：

$$\text{Squad}(q_i, q_{i+1}, s_i, s_{i+1}, t) = \text{Slerp}(\text{Slerp}(q_i, q_{i+1}, t), \text{Slerp}(s_i, s_{i+1}, t), 2t(1-t)) \quad (10.3)$$

其中辅助点 s_i 由相邻关键帧四元数确定（类似 Catmull-Rom 的切线估算）：

$$s_i = q_i \exp\left(-\frac{\log(q_i^{-1}q_{i+1}) + \log(q_i^{-1}q_{i-1})}{4}\right) \quad (10.4)$$

SQUAD 实现了四元数路径的 C^1 连续（切线连续），是动画软件中旋转插值的标准选择。

另有 **Kochanek-Bartels (TCB) 样条**，提供 Tension（张力）、Continuity（连续性）、Bias（偏置）三个手感旋钮，让动画师精细调节曲线形状。

11. 多维与散点插值

11.1 双线性插值 (Bilinear Interpolation)

在规则网格四角点 $p_{11}, p_{12}, p_{21}, p_{22}$ 上，参数 $(s, t) \in [0, 1]^2$ ：

第 1 步：沿 s 方向在两条边上插值：

$$m_2 = (1-s)p_{11} + sp_{21}, \quad m_4 = (1-s)p_{12} + sp_{22} \quad (11.1)$$

第 2 步：沿 t 方向在 m_2, m_4 之间插值：

$$x(s, t) = (1-t)m_2 + tm_4 = (1-t)(1-s)p_{11} + (1-t)sp_{21} + t(1-s)p_{12} + tsp_{22} \quad (11.2)$$

[NOTE] 双三次插值 (Bicubic Interpolation)

把两个方向都换成三次 Hermite/Catmull-Rom，需要 $4 \times 4 = 16$ 个控制点。结果 C^1 甚至 C^2 连续，用于纹理放大、地形平滑等。

11.2 重心插值 (Barycentric Interpolation)

三角形情形：三角形 ABC 内任意点 P 用重心坐标 (Barycentric Coordinates) 表示：

$$P = w_a A + w_b B + w_c C, \quad w_a + w_b + w_c = 1 \quad (11.3)$$

三个权重由面积比给出：

$$w_a = \frac{\Delta_{PBC}}{\Delta_{ABC}}, \quad w_b = \frac{\Delta_{PCA}}{\Delta_{ABC}}, \quad w_c = \frac{\Delta_{PAB}}{\Delta_{ABC}} \quad (11.4)$$

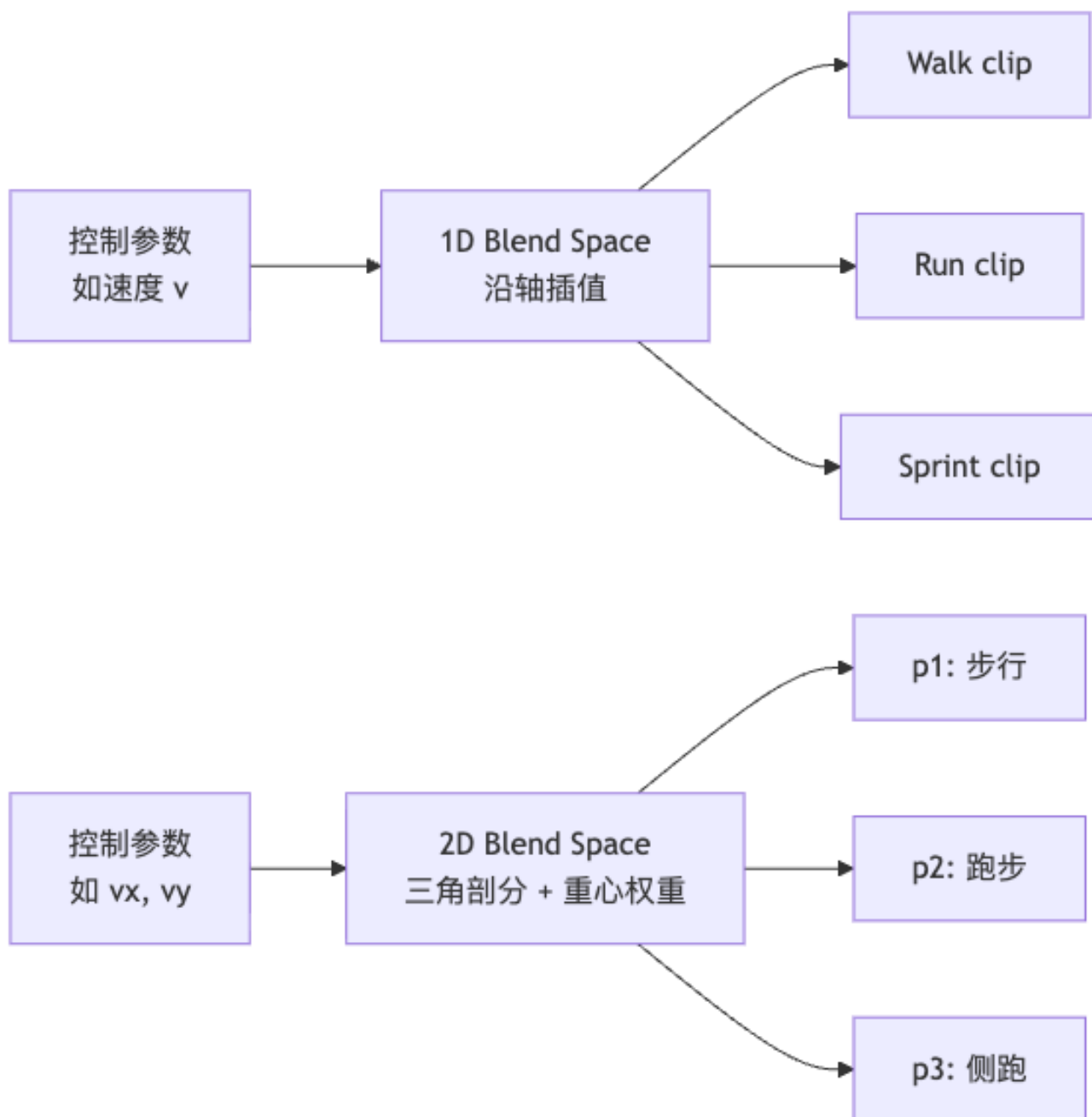
四面体情形（推广到 3D）：

$$P = w_a A + w_b B + w_c C + w_d D, \quad w_a + w_b + w_c + w_d = 1 \quad (11.5)$$

权重为体积比： $w_a = V_{PBCD}/V_{ABCD}$ ，以此类推。

11.3 Blend Space（混合空间）

游戏引擎（如 Unreal Engine、Unity）中多动画混合的核心技术：



2D Blend Space: 将若干动作 clip 摆在 2D 参数空间（如水平速度 $\times$ 垂直速度），对参数空间三角剖分，实时根据角色状态求重心权重：

$$x = w_a p_1 + w_b p_2 + w_c p_3 \quad (11.6)$$

其中 x 是混合后的骨骼姿态, w_a, w_b, w_c 由当前状态点在三角形内的重心坐标决定。

11.4 散点数据插值 (Scattered Data Interpolation)

当控制点不规则分布时 (真实动作捕捉库常见), 常用方法:

方法	特点
线性/最小二乘	简单快速, 精度有限
样条	平滑, 需规则结构
反距离加权 (IDW)	近邻影响大, 参数少
高斯过程 (GP)	提供不确定性估计, 计算量大
径向基函数 (RBF)	灵活、精确、广泛使用

11.5 径向基函数 (Radial Basis Function, RBF) 插值

[NOTE] RBF 的核心思想

每个数据点 x_i 在其附近”放置”一个径向对称的基函数, 权重叠加后重构整个函数。

$$f(x) = \sum_{i=1}^K w_i \varphi(\|x - x_i\|) \quad (11.7)$$

求权重: 令 $f(x_i) = y_i$, 得到线性方程组:

$$\underbrace{\begin{bmatrix} R_{1,1} & R_{1,2} & \cdots & R_{1,K} \\ R_{2,1} & R_{2,2} & & \vdots \\ \vdots & & \ddots & \vdots \\ R_{K,1} & \cdots & \cdots & R_{K,K} \end{bmatrix}}_{R, R_{ij}=\varphi(\|x_i-x_j\|)} \begin{bmatrix} w_1 \\ \vdots \\ w_K \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_K \end{bmatrix} \quad (11.8)$$

求解此 $K \times K$ 线性系统得到所有权重 w_i 。

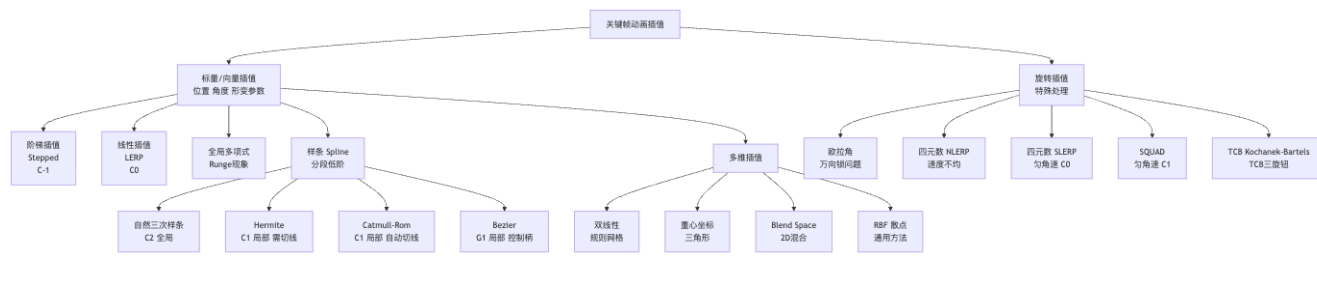
常用核函数:

名称	$\varphi(r)$	特点
Gaussian	e^{-r^2/c^2}	平滑, 需选参数 c
Inverse multiquadric	$1/\sqrt{r^2 + c^2}$	全局支撑, 平滑
Thin plate spline	$r^2 \log r$	最小弯曲能量
Polyharmonic	r^k (k 奇) 或 $r^k \log r$ (k 偶)	通用族

[EX] 空间关键帧 (Spatial Keyframing)

Igarashi et al. (SCA 2005) 用 RBF 实现”空间关键帧”: 将控制器拖到三维空间中不同位置作为空间关键帧, 每个位置对应一种角色姿态 (如手臂抬起 / 踢腿)。实时根据控制器当前位置用 RBF 加权混合姿态 $\rightarrow$ 即兴表演驱动动画。

12. 插值方法综合谱系图



13. Worked Examples (例题)

例题 1: 手算 Hermite 段

题目：给定关键帧 $t = 0$ 处 $y_1 = 0, m_1 = 1$; $t = 1$ 处 $y_2 = 2, m_2 = 0$ 。求 $t = 0.5$ 处的 Hermite 插值。

解：

代入 Hermite 基函数：

$$H_0(0.5) = 2(0.5)^3 - 3(0.5)^2 + 1 = 0.25 - 0.75 + 1 = 0.5$$

$$H_1(0.5) = -2(0.5)^3 + 3(0.5)^2 = -0.25 + 0.75 = 0.5$$

$$H_2(0.5) = (0.5)^3 - 2(0.5)^2 + 0.5 = 0.125 - 0.5 + 0.5 = 0.125$$

$$H_3(0.5) = (0.5)^3 - (0.5)^2 = 0.125 - 0.25 = -0.125$$

代入插值公式：

$$S(0.5) = 0.5 \times 0 + 0.5 \times 2 + 0.125 \times 1 + (-0.125) \times 0 = 0 + 1 + 0.125 + 0 = \boxed{1.125}$$

验证： $S(0) = H_0(0) \cdot 0 + H_1(0) \cdot 2 + H_2(0) \cdot 1 + H_3(0) \cdot 0 = 1 \cdot 0 = 0$ ； $S(1) = 0 + 1 \cdot 2 + 0 + 0 = 2$ 。

例题 2: Catmull-Rom 切线计算

题目：四个控制点在时间轴等间距（步长 $\Delta x = 1$ ）： $p_0 = 0, p_1 = 1, p_2 = 3, p_3 = 2$ 。求 p_1 处的 Catmull-Rom 切线 m_1 ，以及在 $[p_1, p_2]$ 段中 $t = 0.5$ 处的插值。

解：

切线：

$$m_1 = \frac{1}{2} \cdot \frac{p_2 - p_0}{x_2 - x_0} = \frac{1}{2} \cdot \frac{3 - 0}{2} = \frac{3}{4} = 0.75$$

$$m_2 = \frac{1}{2} \cdot \frac{p_3 - p_1}{x_3 - x_1} = \frac{1}{2} \cdot \frac{2 - 1}{2} = \frac{1}{4} = 0.25$$

$t = 0.5$ 插值 (代入 Hermite, $y_1 = p_1 = 1, y_2 = p_2 = 3$):

$$S(0.5) = 0.5 \times 1 + 0.5 \times 3 + 0.125 \times 0.75 + (-0.125) \times 0.25$$

$$= 0.5 + 1.5 + 0.09375 - 0.03125 = \boxed{2.0625}$$

例题 3: SLERP 计算

题目: 两个单位四元数 $q_0 = (1, 0, 0, 0)$ (恒等旋转) 和 $q_1 = (0, 1, 0, 0)$ (绕 x 轴 180° 旋转)。计算 $t = 0.5$ 处的 SLERP 结果。

解:

第 1 步: $\cos \Omega = q_0 \cdot q_1 = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 = 0$, 因此 $\Omega = \pi/2$ (90°)。

第 2 步: 代入 SLERP 公式:

$$\begin{aligned} q(0.5) &= \frac{\sin(0.5 \cdot \pi/2)}{\sin(\pi/2)} q_0 + \frac{\sin(0.5 \cdot \pi/2)}{\sin(\pi/2)} q_1 \\ &= \frac{\sin(\pi/4)}{1} q_0 + \frac{\sin(\pi/4)}{1} q_1 = \frac{\sqrt{2}}{2} (1, 0, 0, 0) + \frac{\sqrt{2}}{2} (0, 1, 0, 0) \\ &= \left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}, 0, 0 \right) \approx (0.707, 0.707, 0, 0) \end{aligned}$$

验证: $\|(0.707, 0.707, 0, 0)\| = \sqrt{0.5 + 0.5} = 1$ 。这个结果对应绕 x 轴旋转 90° , 是 0° 到 180° 的恰好中间。

例题 4: Bezier 控制点求曲线中点

题目: 三次 Bezier 曲线控制点 $b_0 = (0, 0)$, $b_1 = (1, 2)$, $b_2 = (3, 2)$, $b_3 = (4, 0)$ 。用 de Casteljau 算法求 $t = 0.5$ 处的曲线点。

解:

第 1 轮 (线性插值控制点对):

$$b_0^{(1)} = 0.5(0, 0) + 0.5(1, 2) = (0.5, 1)$$

$$b_1^{(1)} = 0.5(1, 2) + 0.5(3, 2) = (2, 2)$$

$$b_2^{(1)} = 0.5(3, 2) + 0.5(4, 0) = (3.5, 1)$$

第 2 轮:

$$b_0^{(2)} = 0.5(0.5, 1) + 0.5(2, 2) = (1.25, 1.5)$$

$$b_1^{(2)} = 0.5(2, 2) + 0.5(3.5, 1) = (2.75, 1.5)$$

第 3 轮:

$$b_0^{(3)} = 0.5(1.25, 1.5) + 0.5(2.75, 1.5) = \boxed{(2, 1.5)}$$

例题 5: RBF 插值的权重求解

题目: 1D 情形, 两个控制点 $x_1 = 0, y_1 = 1$ 和 $x_2 = 2, y_2 = 0$, 核函数为高斯 $\varphi(r) = e^{-r^2}$ 。求权重 w_1, w_2 。

解:

构建矩阵:

$$R_{ij} = \varphi(|x_i - x_j|) \Rightarrow R_{11} = e^0 = 1, R_{12} = e^{-4}, R_{21} = e^{-4}, R_{22} = 1$$

线性方程组:

$$\begin{bmatrix} 1 & e^{-4} \\ e^{-4} & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

求解 ($e^{-4} \approx 0.01832$):

$$w_1 = \frac{1}{1 - e^{-8}} \approx \frac{1}{1 - 0.000335} \approx 1.000336$$

$$w_2 = \frac{-e^{-4}}{1 - e^{-8}} \approx -0.01832 \times 1.000336 \approx -0.01832$$

验证: $f(0) = w_1 \cdot 1 + w_2 \cdot e^{-4} \approx 1.000336 - 0.01832 \times 0.01832 \approx 1$ 。

14. Anki 记忆卡片

#	正面 (Question)	背面 (Answer)
1	关键帧动画的核心数学问题是什么?	在时间轴上的离散关键帧之间做插值, 找到平滑函数 $f(x_i) = y_i$
2	线性插值 LERP 的公式是什么? 连续性等级?	$f(t) = (1-t)y_1 + ty_2$; 整体仅 C^0 (速度在关键帧处突变)
3	$C^0/C^1/C^2$ 各代表什么连续?	C^0 = 位置连续, C^1 = 速度连续, C^2 = 加速度连续
4	Runge 现象是什么? 如何避免?	高阶全局多项式在区间端点振荡; 避免方法: 分段低阶多项式 (样条)
5	自然三次样条的连续性? 最大缺点?	C^2 ; 无局部控制 (动一个点, 整条曲线都变); 计算昂贵
6	Hermite 样条的输入是什么? 基矩阵是什么?	输入 (y_1, y_2, m_1, m_2) ; 基矩阵 $M_H = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$
7	四个 Hermite 基函数是什么?	$H_0 = 2t^3 - 3t^2 + 1$, $H_1 = -2t^3 + 3t^2$, $H_2 = t^3 - 2t^2 + t$, $H_3 = t^3 - t^2$
8	Catmull-Rom 的切线公式? 与 Hermite 的关系?	$m_i = \frac{p_{i+1} - p_{i-1}}{2(x_{i+1} - x_{i-1})}$; 代入 Hermite 基即得 Catmull-Rom 段
9	为什么旋转不能直接线性插值欧拉角?	欧拉角插值不沿最短路径; 存在万向锁; 旋转速度不均匀
10	SLERP 公式是什么? Ω 如何求?	$q(t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1$; $\cos \Omega = q_0 \cdot q_1$
11	SLERP 的连续性? 要实现旋转 C^1 用什么?	仅 C^0 ; 需要 SQUAD 或 TCB 样条实现 C^1 旋转插值
12	双线性插值的公式是什么?	$x = (1-t)(1-s)p_{11} + (1-t)sp_{21} + t(1-s)p_{12} + tsp_{22}$
13	重心坐标的三个权重 w_a, w_b, w_c 如何计算?	面积比: $w_a = \Delta_{PBC}/\Delta_{ABC}$, $w_b = \Delta_{PCA}/\Delta_{ABC}$, $w_c = \Delta_{PAB}/\Delta_{ABC}$
14	RBF 插值的表达式? 如何求权重?	$f(x) = \sum_{i=1}^K w_i \varphi(\ x - x_i\)$; 令插值条件 $f(x_i) = y_i$ 建立 $K \times K$ 线性系统求解
15	Hermite 与 Bezier 的等价关系?	Bezier 控制点: $b_0 = y_1, b_1 = y_1 + m_1/3, b_2 = y_2 - m_2/3, b_3 = y_2$

15. 中英术语速查表

中文术语	English	关键说明
关键帧	Keyframe	动画师指定的关键时刻姿态
中间帧 / 补间帧	In-between / Tween	计算机插值自动生成的帧

中文术语	English	关键说明
插值	Interpolation	$f(x_i) = y_i$ 严格过点
回归 / 拟合	Regression / Fitting	最小化误差, 不必过点
外推	Extrapolation	在数据范围外求值
阶梯插值	Stepped Interpolation	区间内取常数, 不连续
线性插值	Linear Interpolation (LERP)	$(1-t)y_1 + ty_2$
样条	Spline	分段低阶多项式插值
三次样条	Cubic Spline	每段三次, 整体 C^2
Hermite 样条	Hermite Spline	需指定切线, 局部 C^1
Hermite 基函数	Hermite Basis Functions	H_0, H_1, H_2, H_3
Hermite 基矩阵	Hermite Basis Matrix	M_H , 由约束方程求逆得到
Catmull-Rom 样条	Catmull-Rom Spline	自动切线, 局部 C^1
Bezier 曲线	Bezier Curve	控制多边形, 一般不过中间点
de Casteljau 算法	de Casteljau Algorithm	递归二分构造 Bezier 曲线
伯恩斯坦多项式	Bernstein Polynomials	Bezier 的展开形式
万向锁	Gimbal Lock	欧拉角的奇异性问题
四元数	Quaternion	$q = (w, v)$
单位四元数	Unit Quaternion	$\ q\ = 1$, 在 S^3 上
球面线性插值	Spherical Linear Interpolation (SLERP)	大圆弧插值, 匀角速
SQUAD	Spherical & QUADrangle interpolation	四元数 C^1 样条
TCB 样条	Tension-Continuity-Bias Spline	三旋钮控制曲线形状
双线性插值	Bilinear Interpolation	规则网格两次 LERP
双三次插值	Bicubic Interpolation	4×4 控制点, C^1
重心坐标	Barycentric Coordinates	(w_a, w_b, w_c) , 面积比
混合空间	Blend Space	游戏引擎多动画混合框架
散点插值	Scattered Data Interpolation	控制点不规则分布时的插值
径向基函数	Radial Basis Function (RBF)	$\varphi(\ x - x_i\)$
Runge 现象	Runge's Phenomenon	高阶多项式端点振荡
C^k 连续	C^k Continuity	前 k 阶导数连续
G^1 连续	Geometric Continuity G^1	切线方向连续, 大小可不等
空间关键帧	Spatial Keyframing	用控制器空间位置驱动姿态插值

16. 复习要点

[TIP] 期末/期中复习要点

必会推导：1. 从四个约束方程 (6.4) 推出 Hermite 基矩阵 (6.6)，展开写出基函数 (6.8)。2. 从“大圆弧匀速运动”几何直觉推导 SLERP 公式 (10.2)。

必会分析：3. 给定任意插值场景，判断用哪种样条（连续性要求？局部控制？切线可用？数据规则？）。4. 解释为什么欧拉角和四元数 NLERP 不适合旋转插值，SLERP 如何解决。

必会计算：5. 手算 Hermite 基函数值 ($H_k(t)$ 代入具体 t)。6. 手算 Catmull-Rom 切线（给四个等间距点，求中间切线）。7. 手算 SLERP（给 q_0, q_1 ，求 $t = 0.5$ 处结果）。

核心结论速记：– 分段低阶 > 全局高阶 (Runge 的教训) – 局部控制 + 自动切线 → Catmull-Rom 是动画软件最常见选择 – 旋转插值必须在 S^3 上做 (SLERP)，不能简单逐分量线性插值 – Blend Space = 重心坐标 + 三角剖分，是游戏引擎动画混合的标准架构 – RBF 是散点插值的通用武器，用于姿态库混合 (Spatial Keyframing)

[NOTE] 参考文献与延伸阅读

– Edwin Catmull & Raphael Rom (1974): Catmull-Rom 样条原始论文。– Ken Shoemake (1985): Animating Rotation with Quaternion Curves, SIGGRAPH 1985。SLERP 与 SQUAD 的经典来源。– Kochanek & Bartels (1984): Interpolating Splines with Local Tension, Continuity and Bias Control, SIGGRAPH 1984。TCB 样条原始论文。– Igarashi et al. (2005): Spatial Keyframing for Performance-driven Animation, SCA 2005。RBF 空间关键帧应用。– 课程主页：MOCCA 2026, Libin Liu, 北京大学智能学院。

Lecture 05 —数据驱动动画 (Data-driven Animation)

[TIP] 学习指南

本节核心：用真实采集的动作数据驱动虚拟角色运动的完整工程链——从**动作捕捉 (MoCap)** 采集数据，到**动作重定向、过渡、混合**让数据可用，再到**动作图 (Motion Graph)** 与**动作匹配 (Motion Matching)** 实现在线交互合成。传统关键帧动画的极限是艺术家手工设计每一帧；数据驱动方法的核心是”以数据为原料、以检索/拼接/混合为工具，在线响应用户输入”。

前置知识： – Lec 03 一刚体运动学（旋转表示：四元数、欧拉角、旋转矩阵） – Lec 04 一骨骼与蒙皮（关节层级、局部 vs 全局旋转）

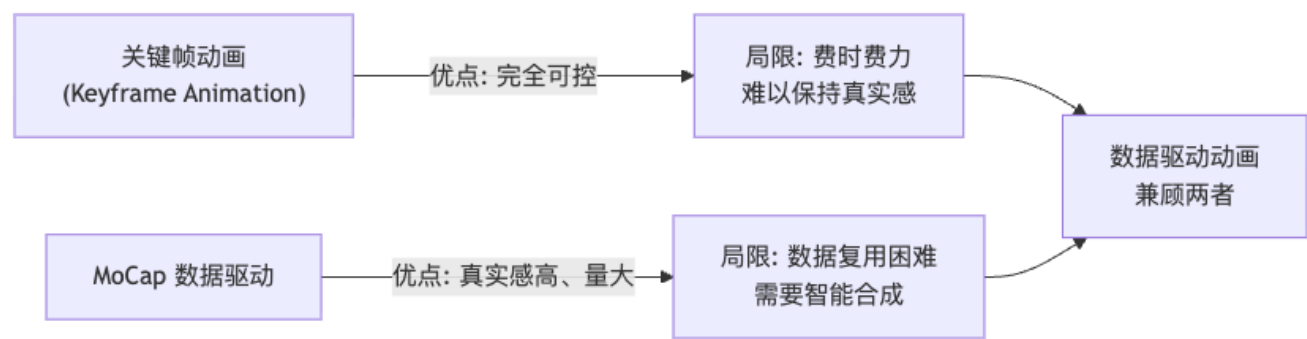
重点掌握： 1. MoCap 系统五大类的原理与优劣比较 2. 动作数据的姿态格式： $p_t = (t_0, R_0, R_1, \dots)$ 与 facing frame 分解 3. 动作过渡的线性混合公式（含旋转插值注意事项）与 root 对齐变换 4. Motion Graph：图论构造（节点 =clip, 边 =pose 距离阈值），图上搜索 (A*) 5. Motion Matching：特征向量设计、最近邻检索、inertialized blending 平滑

0. 名词预热

在正文前先把全篇反复出现的核心名词一次讲清，方便零基础读者顺畅阅读。

名词	一句话解释
姿态 (Pose)	角色某一时刻所有关节的旋转 + 根位置的完整描述，是动画的”一帧”
Clip / 动作片段	一段连续的 pose 序列，例如”走路 2 秒”或”挥拳 0.5 秒”
Root 关节	骨骼层次中的最顶层节点（通常是骨盆/髋），决定角色在世界中的位置与整体朝向
动作捕捉 (MoCap)	把真实演员的运动数字化，输出随时间变化的关节旋转序列
数据驱动动画	以 MoCap 数据库为原料，通过检索/拼接/混合在线生成角色运动，对立面是纯手工关键帧
最近邻 (NN)	在数据集中找与当前状态距离最小的帧；Motion Matching 的核心操作
Slerp	Spherical Linear Interpolation，四元数上的球面线性插值，是旋转混合的标准手段

1. 从关键帧到数据驱动：为什么需要 MoCap?

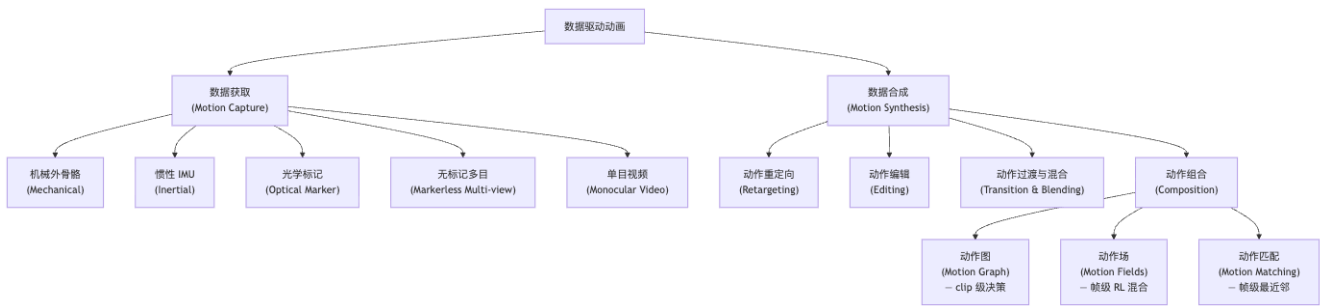


关键帧动画 (keyframe animation) 依靠艺术家逐帧手工定义 pose，经样条曲线插值产生连续运动——精度可控但费时费力，且人类对“真实运动”的记忆有限，产出往往不够自然。动作捕捉从根本上改变了采集方式：让真实演员完成动作，系统自动记录。但 MoCap 数据是固定的片段集合，若要响应多变的用户输入，还需要一套“如何组合这些片段”的合成方法，这正是本讲的主体。

[NOTE] 数据驱动方法的本质假设

“真实人类的运动在某个高维空间里形成一个低维流形。MoCap 数据是这个流形上的采样点。合理的新运动应当是这些采样点的插值或邻近点。”这一假设贯穿 motion blending、motion graph、motion matching 三大范式。

2. 数据驱动方法谱系



3. Motion Capture 动作捕捉

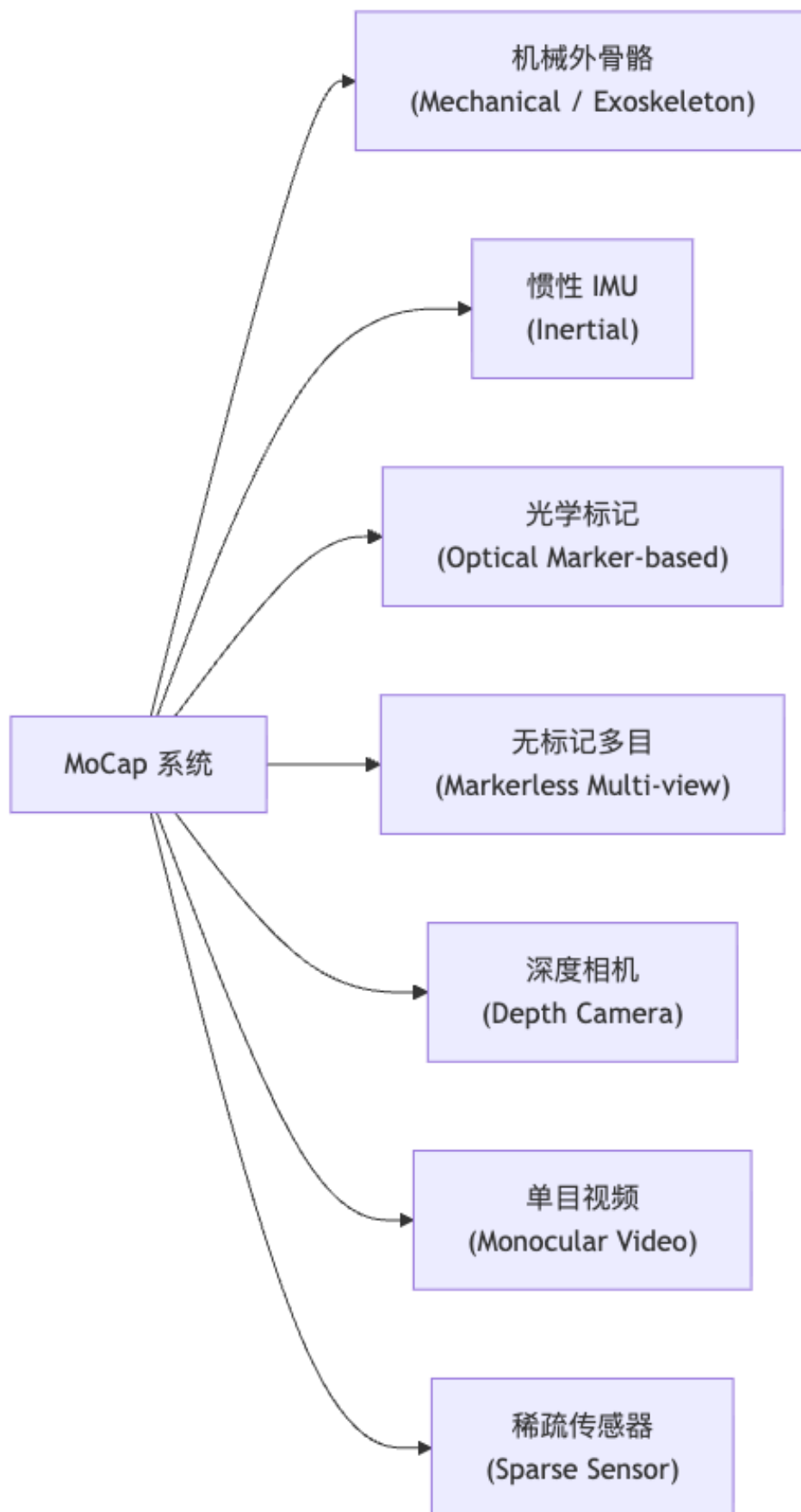
3.1 历史脉络

时期	技术 / 人物	核心理想
1878 ~1914	Eadweard Muybridge — 《奔马》摄影序列 Max Fleischer — Rotoscoping (转描, US patent 1242674)	用连拍照片分解运动 在真人影片上逐帧描线
1930s—50s	Disney 《白雪公主》《爱丽丝梦游仙境》	Rotoscoping 工业化
1941	万氏兄弟 《铁扇公主》	中国最早应用 rotoscoping
现代	光学、惯性、无标记、稀疏传感器……	全 3D 数字采集

[NOTE] Rotoscoping 的精神

Rotoscoping 是“用真实运动驱动虚拟绘制”的最早形式——在已拍好的视频每帧上描出轮廓/关节，直接复用人类运动的真实性。这正是 MoCap 的概念祖先。3D Rotoscoping 则把同一思想推广到三维虚拟角色。

3.2 MoCap 系统分类与原理



机械外骨骼 (Mechanical / Exoskeleton): 演员穿戴关节角度传感器, 直接读出各关节旋转, 无需相机。优点: 不怕遮挡、精度稳定; 缺点: 笨重、限制动作幅度、穿戴耗时。

惯性 MoCap (Inertial / IMU): 每个身体段贴一个 IMU (Inertial Measurement Unit), 内含 3 轴加速度计 + 3 轴陀螺仪 (6DoF), 可选磁力计。对加速度做积分得速度, 再积分得位移, 融合陀螺仪得姿态。优点: 穿戴轻便、无需采集场地; 缺点: 积分漂移 (drift), 长时间后位置误差累积, 需周期性修正。

光学标记 MoCap (Optical Marker-based): 在演员身体关键点贴上反光 marker 或主动发光 LED, 多台红外/可见光相机同时拍摄, 用多视图几何 (三角化) 重建 3D marker 位置, 再通过骨骼求解 (solving) 得到关节旋转。Holden (2018) 用去噪深度网络提升 solving 的鲁棒性。这是工业级标准 (Vicon、OptiTrack)。

[WARN] Occlusion 遮挡问题

演员背对相机或自遮挡时某些 marker 不可见, 需要多相机冗余覆盖 + 后处理补缺。通常需要 6—16 台相机环绕布置。

无标记多目 (Markerless Multi-view): 无需 marker, 用多台普通相机拍摄, 先做 2D 姿态估计, 再多视图三角化得 3D 姿态 (代表: Captury、Theia Markerless)。更舒适, 但精度略低于光学 marker。

深度相机 (Depth Camera): 单台深度相机 (如 Kinect) 直接获得每像素深度, 结合 RGB 做人体姿态估计。便宜, 但范围和精度有限, 适合室内近距离应用。

单目视频 (Monocular Video): 从单台 RGB 相机的视频估计 3D 姿态。2D 关键点检测 (OpenPose 等) → 3D lifting (基于学习)。最便利, 但深度歧义大, 精度最低, 适合粗粒度应用。

稀疏传感器 (Sparse Sensor): Yi et al. (2021) TransPose 用 6 个 IMU 配合神经网络估计全身 3D 动作与位置。兼顾便携性与精度, 是移动 MoCap 的前沿方向。

[NOTE] 一次典型的 MoCap Session 流程

场地布置与相机标定 → 演员穿戴 (贴 marker 或穿 IMU 套件) → T-pose / Range-of-Motion 校准 → 演员按脚本表演 → 后台实时求解骨骼 → 后处理 (去噪、补缺、平滑、retargeting) → 导出 BVH / FBX 文件。

4. 动作数据表示 (Motion Data Representation)

4.1 Pose 格式

一段动作 (motion clip) 是按时间排列的 N 帧 pose 序列:

$$\{p_t\}_{t=1,\dots,N} \quad (4.1)$$

每帧 pose 由根关节 (root) 的世界状态 + 所有子关节的局部旋转组成:

$$p_t = (t_0, R_0, R_1, R_2, \dots) \quad (4.2)$$

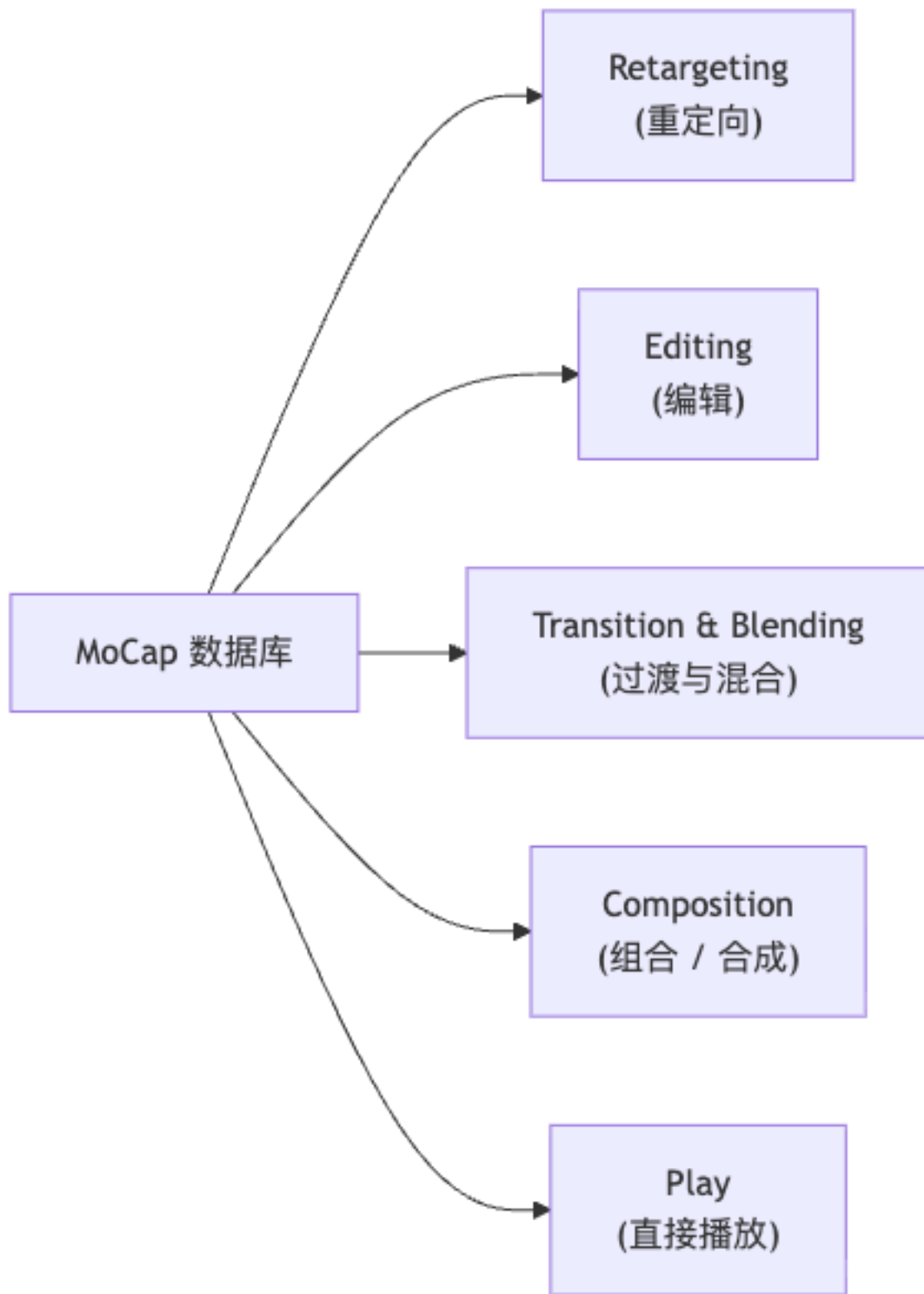
- $t_0 \in \mathbb{R}^3$: root 的世界空间位置 (通常是骨盆/髌中点)
- $R_0 \in SO(3)$: root 的世界朝向
- $R_i \in SO(3)$, $i \geq 1$: 第 i 关节相对父关节的**局部旋转 (local rotation)**

旋转的常见存储格式: 欧拉角 (易读, 有万向节锁)、四元数 (无奇点, 插值好)、6D 旋转表示 (网络输出首选, 无奇点)。

[NOTE] 为什么用局部旋转而非全局旋转?

局部旋转与骨骼拓扑无关——子关节如何旋转由父关节定义的局部坐标系决定, 换一个身高不同的角色, 局部旋转可以直接复用 (retargeting 的基础)。全局旋转则与角色在世界中的绝对位置耦合, 不利于迁移。

4.2 数据用途概览



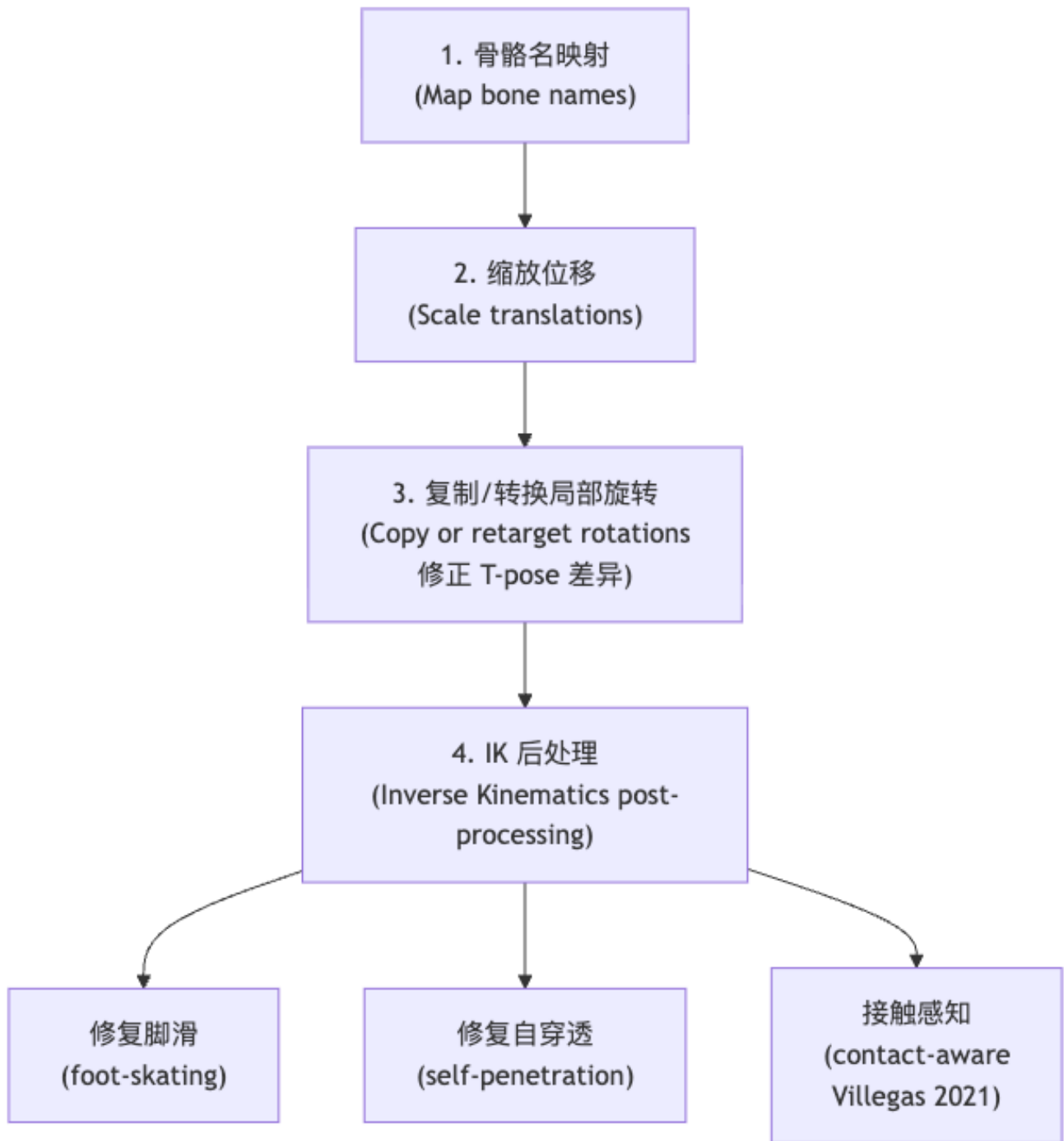
5. Motion Retargeting 动作重定向

5.1 问题定义

把角色 A 的 MoCap 动作迁移到骨架不同的角色 B。常见差异：

差异类型	例子
骨骼数量	角色有/没有尾巴、锁骨
骨骼命名	Bip01_L_Hand vs LeftHand
参考姿态 (T-pose)	A 掌心朝下, B 掌心朝前
骨长比例	身高 1.6m vs 3m (巨人角色)
骨架拓扑	人形 vs 四足

5.2 一种可行的 Retargeting Pipeline



[NOTE] 工业实现参考

– Autodesk Maya 内置 **HumanIK**: 定义骨骼语义 (哪根骨头是“左手腕”), 自动映射。– Unreal Engine 5 **IK Rig + IK Retargeter**: 在 UE5 内可视化配置 retargeting, 支持实时预览。– Ubisoft Toronto **IK Rig 原型**: 用一套“中性 control rig + 骨架语义描述符”实现跨角色重定向, 支持非人形怪物。

6. Motion Editing 动作编辑

6.1 Motion Displacement Mapping

Lee & Shin (SIGGRAPH 1999) 提出的层次化编辑方法：在原始动作 p_t 上叠加平滑 displacement 曲线 d_t ，使得在指定约束帧（如脚踏目标点、手摸目标物体）满足约束，其余时刻平滑过渡：

$$\tilde{p}_t = p_t \oplus d_t \quad (6.1)$$

其中 $\oplus$ 对旋转分量是旋转组合，对位置分量是向量相加。

6.2 几何方法 (Geometric Motion Editing)

把动作视作骨骼”曲面”，借用几何处理工具：

Laplacian 算子 (Choi et al. 2011, 使角色能钻进狭窄空间)：

$$L(x_i) = x_i - \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} x_j \quad (6.2)$$

其中 $\mathcal{N}_i$ 是 x_i 的 1-邻居。保持局部形状的同时对整体变形，可将动作”压缩”以适应环境约束。

空间关系保持编辑 (Ho et al. 2010)：在形变后保持角色与障碍物/其他角色的原有相对空间关系（不穿墙、不穿对方），适合多角色密集场景。

7. Motion Transition 动作过渡与混合

7.1 朴素拼接的问题

直接将片段 A 最后一帧与片段 B 第一帧首尾相连会产生：– Pose 不连续 (discontinuity)：手脚瞬间跳变 – Root 位置/朝向不连续：角色”瞬移”或朝向突变

[WARN] 不能简单跳切

即使选了两个”看起来相似”的接头帧，若不做混合，视觉上仍会有明显的一帧”闪跳”。过渡混合 (transition blending) 是数据驱动动画能够流畅运行的基础。

7.2 线性混合 (Linear Blend Transition)

在两段间选一段过渡窗口 (transition window)，窗口内第 t 帧的混合 pose：

$$p_t = (1 - t) p_0^{(i)} + t p_1^{(i)}, \quad t \in [0, 1] \quad (7.1)$$

更平滑的版本用一条单调过渡曲线 $\phi(t)$ 替换线性参数 t (典型选择：smoothstep $\phi(t) = 3t^2 - 2t^3$ 或 cosine ease $\phi(t) = \frac{1 - \cos(\pi t)}{2}$)：

$$p_t = (1 - \phi(t)) p_0^{(i)} + \phi(t) p_1^{(i)} \quad (7.2)$$

[WARN] 旋转混合不能用线性插值

关节旋转 R_i 必须用 **Slerp** (球面线性插值, 在四元数上操作) 或 **log-space** 混合, 而非直接对矩阵/欧拉角做线性插值。原因: 旋转矩阵的线性插值不保证正交性; 四元数线性插值 (LERP) 不保持单位长度且角速度不均匀。只有位置分量 t_0 才用普通线性插值。

7.3 Facing Frame 朝向坐标系

[NOTE] 什么是 Facing Frame?

一个随角色水平移动、其中一轴始终指向角色”朝向方向”的局部坐标系。它的旋转分量只含绕垂直轴 y 的旋转, 平移分量只含水平分量:

$$R_{\text{face}} = \theta e_y, \quad t_{\text{face}} = (t_x, 0, t_z) \quad (7.3)$$

剔除了上下平移 t_y 和”翻倒/俯仰”(pitch/roll) 旋转, 仅保留水平移动和转向。

为什么需要 Facing Frame? 做动作过渡时, 若直接把 root 旋转整体插值, 可能会产生”角色在空中旋转翻滚”的错误。Facing Frame 把 root 旋转拆分, 过渡时只对齐”水平朝向”, pitch/roll 和高度差分别处理, 结果更合理。

朝向 R 的几种常见定义: - **肩-髋平均法**: R 使世界 x 轴对齐左肩 $\rightarrow$ 右肩与左髋 $\rightarrow$ 右髋向量的平均, 可处理大幅上身扭转时的朝向歧义。- **旋转分解法**: 把 root 旋转分解为 $R_0 = R_y R_{xz}$, 取 R_y 作为朝向旋转 (见下节)。

7.4 旋转分解 $R_0 = R_y R_{xz}$

把任意 3D 旋转拆为”水平转动 (yaw) R_y + 翻倒 (tilt) R_{xz} ”。

$$R_0 = R_y R_{xz}, \quad R_y = \theta_y e_y, \quad R_{xz} = \theta u_{xz}, \quad u_{xz} = (u_x, 0, u_z) \quad (7.4)$$

构造步骤:

1. 找旋转 R' , 使局部 y 轴对齐到全局 y 轴 (即”消除翻倒”): 旋转轴 u_{xz} 在 xz 平面内。
2. 则 $R_y = R' R$ (在 R_0 的基础上先”消翻倒”, 剩下的就是纯 yaw)。
3. 反推 $R_{xz} = R_y^T R_0$ 。

7.5 Root 对齐: 拼接时的全局坐标变换

两段 motion 的 root 状态: 片段 0 末帧 (R_0, t_0) , 片段 1 首帧 (R_1, t_1) 。把片段 1 所有帧对齐到片段 0 末帧的公式:

$$R^{(i)} = R_0 R_1^T R_1^{(i)}, \quad t^{(i)} = R_0 R_1^T (t_1^{(i)} - t_1) + t_0 \quad (7.5)$$

[NOTE] 直觉解释

片段 1 的所有帧先”平移回原点” (减去 t_1), 再旋转 $R_0 R_1^T$ (把”片段 1 的朝向”转到”片段 0 的朝向”), 最后平移到 t_0 。实践中只在 facing frame 下做对齐 (只旋转 yaw 分量), 避免引入不必要的 pitch/roll 误差。

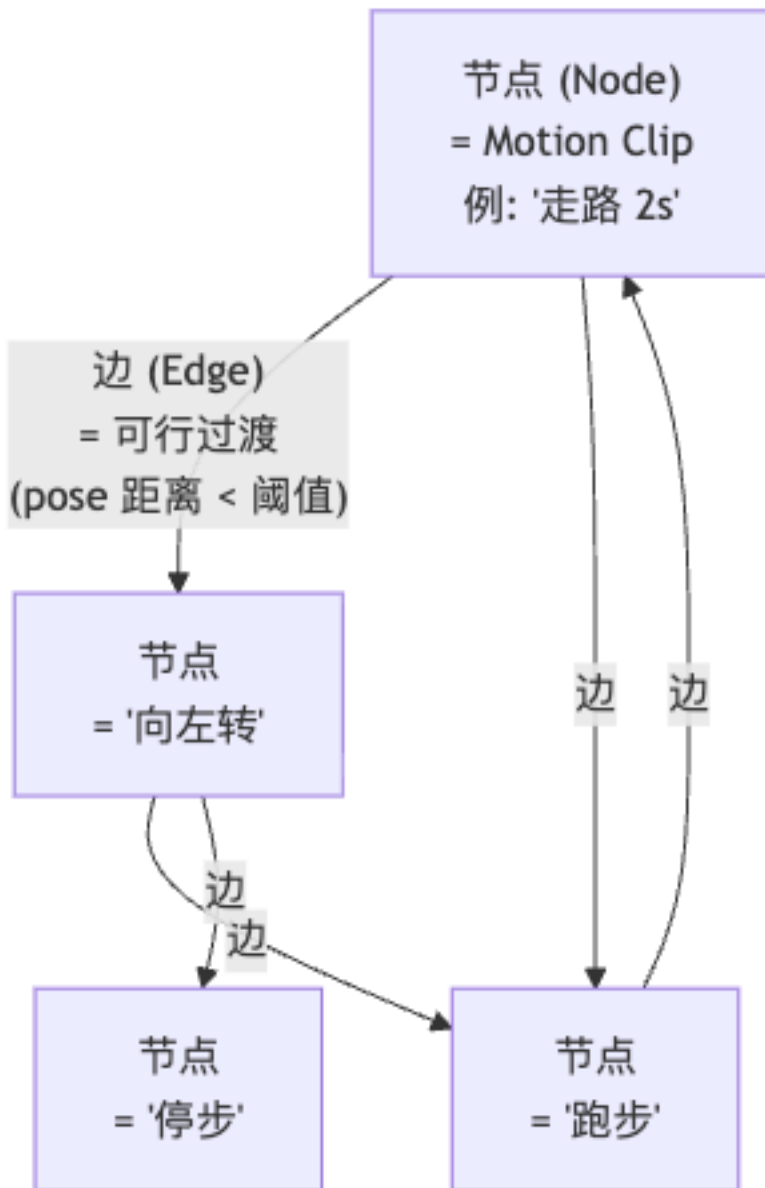
7.6 无 Root 平移 / 无 Root 旋转的数据

许多数据集会移除 root 的世界平移 (in-place 动画) 甚至同时移除 root 旋转: - 优点: 数据紧凑、容易做循环 (looping)、跨片段对齐简单; - 使用时: 根据用户控制输入或物理仿真将其补回, 例如把”in-place 跑步动画”通过路径积分还原成”在场景中沿路径奔跑”。

8. Motion Graph 动作图

8.1 核心思想

把动作数据库组织成一张有向图 (directed graph), 图上搜索得到响应用户输入的 clip 序列。



- 节点: 每个 motion clip (“走路”、“跑步”、“跳跃”、“挥拳”……)

- **边**：两个 clip 之间存在**可行过渡点**——即 clip A 的某帧与 clip B 的某帧 pose 足够接近，可无缝（经混合后）衔接。

8.2 自动构建：找可行过渡边

对所有帧对 (A_i, B_j) 计算 pose 距离（通常是关节位置 + 速度 + 旋转的加权 L2 距离）：

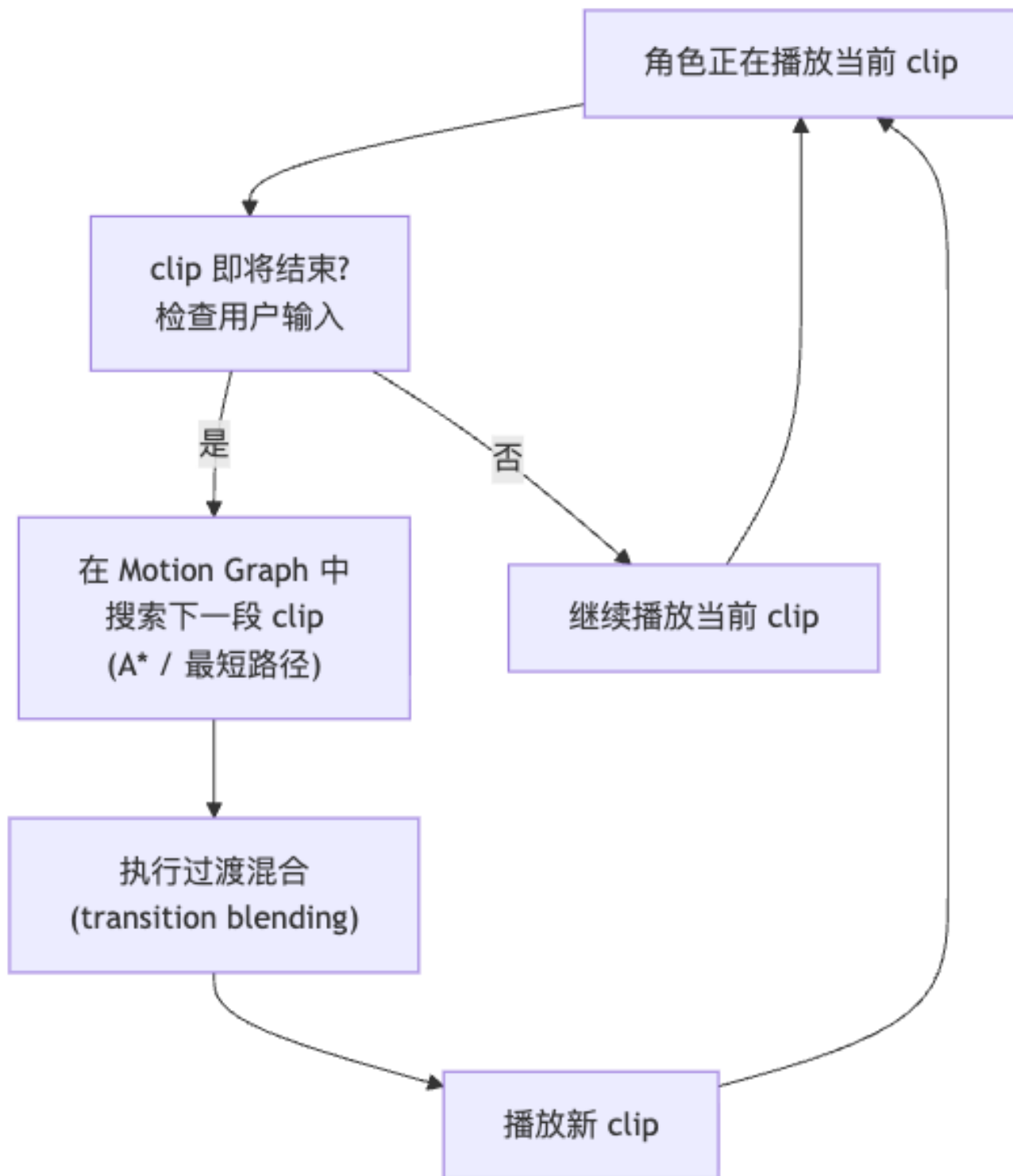
$$D(i, j) = \sum_k w_k \|\text{pose}_k^A(i) - \text{pose}_k^B(j)\|^2 \quad (8.1)$$

若 $D(i, j) < \varepsilon$ （距离阈值），则在图中添加一条从 A_i 到 B_j 的过渡边，过渡时用 §7.2 的混合公式平滑。

[NOTE] 自动建边 vs 手工设计状态机

传统游戏引擎（如 Unity Animator、UE4 的动画蓝图）需要动画师手工指定哪些 clip 之间可以过渡，耗时且容易遗漏。Motion Graph 通过 pose 距离自动发现过渡点，只需设定阈值 ε 。

8.3 Motion Graph 的运行流程



[WARN] Motion Graph 的局限

1. **响应延迟**: 等当前 clip 走完才切换, 响应延迟约等于 clip 长度 (可能 1—3 秒)。2. **覆盖受限**: 图中只有数据库采过的 clip; 未采集的过渡动作 (如边转身边跳跃) 无法生成。3. **图搜索开销**: 复杂场景下需要前瞻多步 (A* 搜索) 才能找到满足约束的路径, 计算量随图规模增长。

8.4 参数化动作图 (Parametric Motion Graphs)

Heck & Gleicher (2007) 进一步把“同类动作族”参数化：例如“往各方向行走”不再是多个离散节点，而是一个以行走方向为参数的连续 clip 簇，通过插值生成任意方向的步伐。控制更连续、数据更紧凑。

9. 动作组合范式比较

范式	决策频率	响应延迟	数据组织	代表工作
Motion Graph / State Machine	Clip 末尾	高（秒级）	有向图	Kovar 2002, Heck 2007
Parametric Motion Graphs	Clip 末尾	中	参数化 clip 簇	Heck 2007
Motion Fields	每帧	低（帧级）	状态空间 + RL	Lee 2010
Motion Matching	每帧（隔几帧搜）	低	KD-tree 特征空间	Clavet 2016, Holden 2020

10. Motion Fields 动作场

Lee et al. (SIGGRAPH 2010) 把决策粒度从“clip 末尾”推进到“每帧”：

算法（每帧执行）：

1. 读取用户输入（期望速度 / 转向）；
2. 在数据库中找到当前状态的 N 个最近邻 (nearest neighbors)；
3. 按用户输入对这 N 个邻居加权混合，得到下一帧 pose；
4. 更新角色并渲染。

关键问题：混合权重如何确定？Motion Fields 用强化学习 (Reinforcement Learning)——以“当前状态 + 用户输入 → 选邻居 + 设权重”为策略，回报由轨迹是否符合用户意图、运动是否平滑决定。

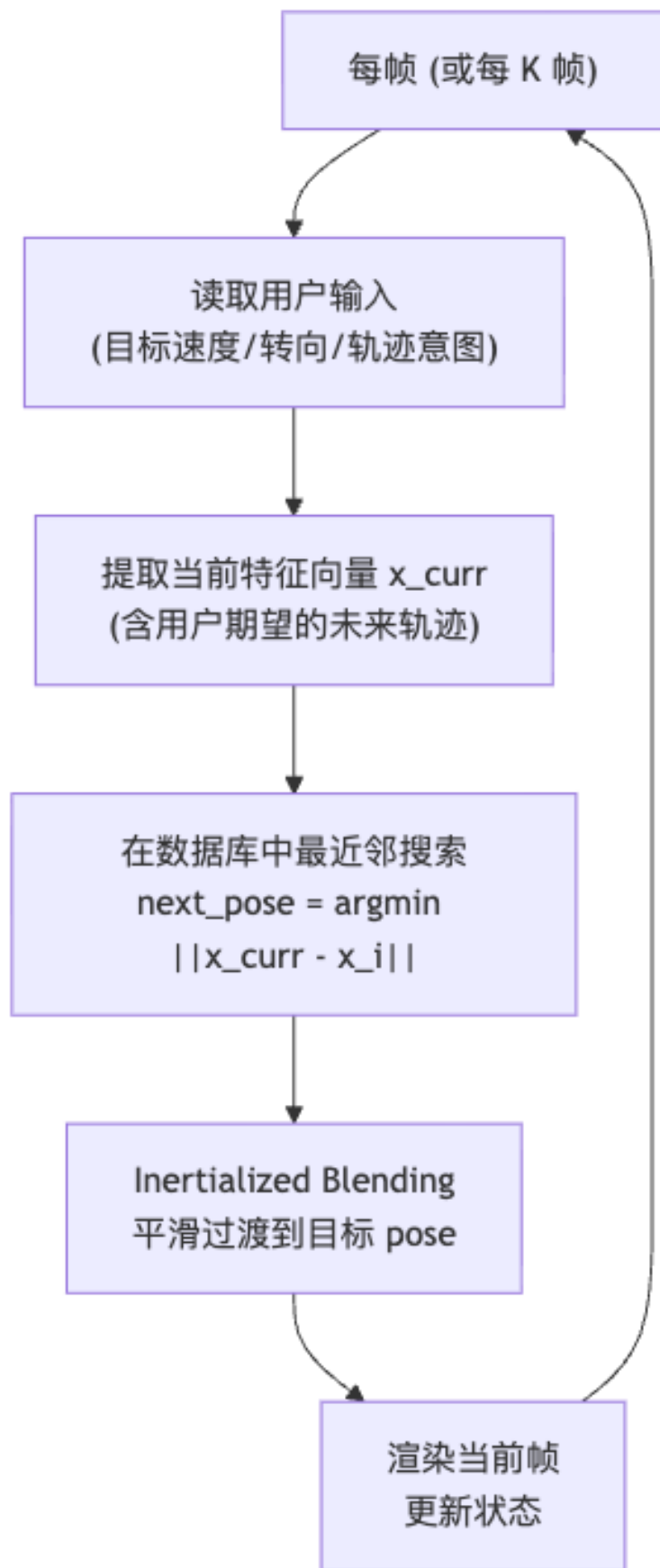
[NOTE] Motion Fields 的工程挑战

RL 训练复杂，且运行时需要实时做策略推断（神经网络前向传播）。优点是生成的运动极其流畅且响应即时。动作场是学术研究里向“每帧决策”迈出的关键一步，为 Motion Matching 的工程化实践铺路。

11. Motion Matching 动作匹配

Motion Matching 是工业界当前主流的每帧决策方案，由 Simon Clavet (Ubisoft, 《荣耀战魂》) 在 2016 年 GDC 演讲中普及，随后被 EA、Rockstar 等广泛采用。

11.1 核心思想



最近邻搜索的数学形式：

$$\text{next_pose} = \arg \min_{i \in \text{Dataset}} \|x_{\text{curr}} - x_i\| \quad (11.1)$$

其中 x 是特征向量 (feature vector)，而非原始的全身 pose（后者维度过高且含大量冗余）。

11.2 特征向量设计

特征向量 x 需要捕捉”当前状态”和”用户意图”两部分信息。常用特征组合：

特征类别	具体内容	作用
末端执行器速度	手脚相对 root 的线速度 + 角速度	描述当前肢体动态状态
Root 速度	root 的线速度 + 角速度	描述整体移动状态
未来朝向/位置	0.5 s、1.0 s、1.5 s 后 root 的预期位置和朝向	编码用户期望的轨迹意图
脚接触标签	每只脚是否接触地面 (0/1)	防止在腾空帧匹配到落地帧
末端关节位置	手脚相对 root 的位置	约束肢体形态

[TIP] 特征加权的工程意义

对每个特征分量乘以权重 w_k ，距离变为 $\sum_k w_k (x_{\text{curr},k} - x_{i,k})^2$ 。调高”未来朝向”权重 → 系统更强调沿用户指定轨迹走；调高”末端速度”权重 → 系统更强调动作风格的连贯性。这是 Motion Matching 主要的调参旋钮。

11.3 Inertialized Blending 惯性化混合

普通 LERP 切换时若两帧 pose 差距大，会有明显的”拉扯感”。Inertialized Blending (Daniel Holden, “Spring Roll Call”) 用临界阻尼弹簧追踪目标姿态：

设当前 pose 为 p_{curr} ，目标 pose 为 p_{target} ，offset 为 $\delta = p_{\text{curr}} - p_{\text{target}}$ ，弹簧方程：

$$\ddot{\delta} + 2\omega_0 \dot{\delta} + \omega_0^2 \delta = 0 \quad (11.2)$$

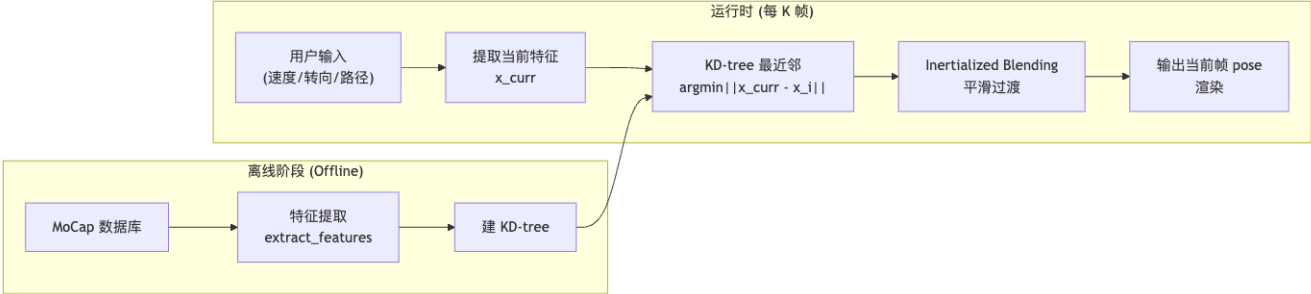
临界阻尼 ($\zeta = 1$) 的解析解可以在极短时间常数 (如 0.1 s) 内平滑衰减到零，不振荡。优点：只需调时间常数一个参数，效果远优于普通 LERP。

11.4 工程优化

[TIP] 让 Motion Matching 在游戏中跑得起来

- 不要每帧都搜：每隔 K (如 5–10) 帧搜索一次，中间帧用 inertialized blending 平滑过渡，这样每帧搜索开销仅 $1/K$ 。
- KD-tree 加速：对特征向量建 KD-tree，最近邻查询从 $O(N)$ 降至 $O(\log N)$ (N 为数据库帧数)。
- Dance Card (跳舞卡)：对数据库做剪枝，去掉冗余帧 (相邻帧差别极小的可以降采样)，保留覆盖均衡、品质高的”精华帧”，减小 N 。

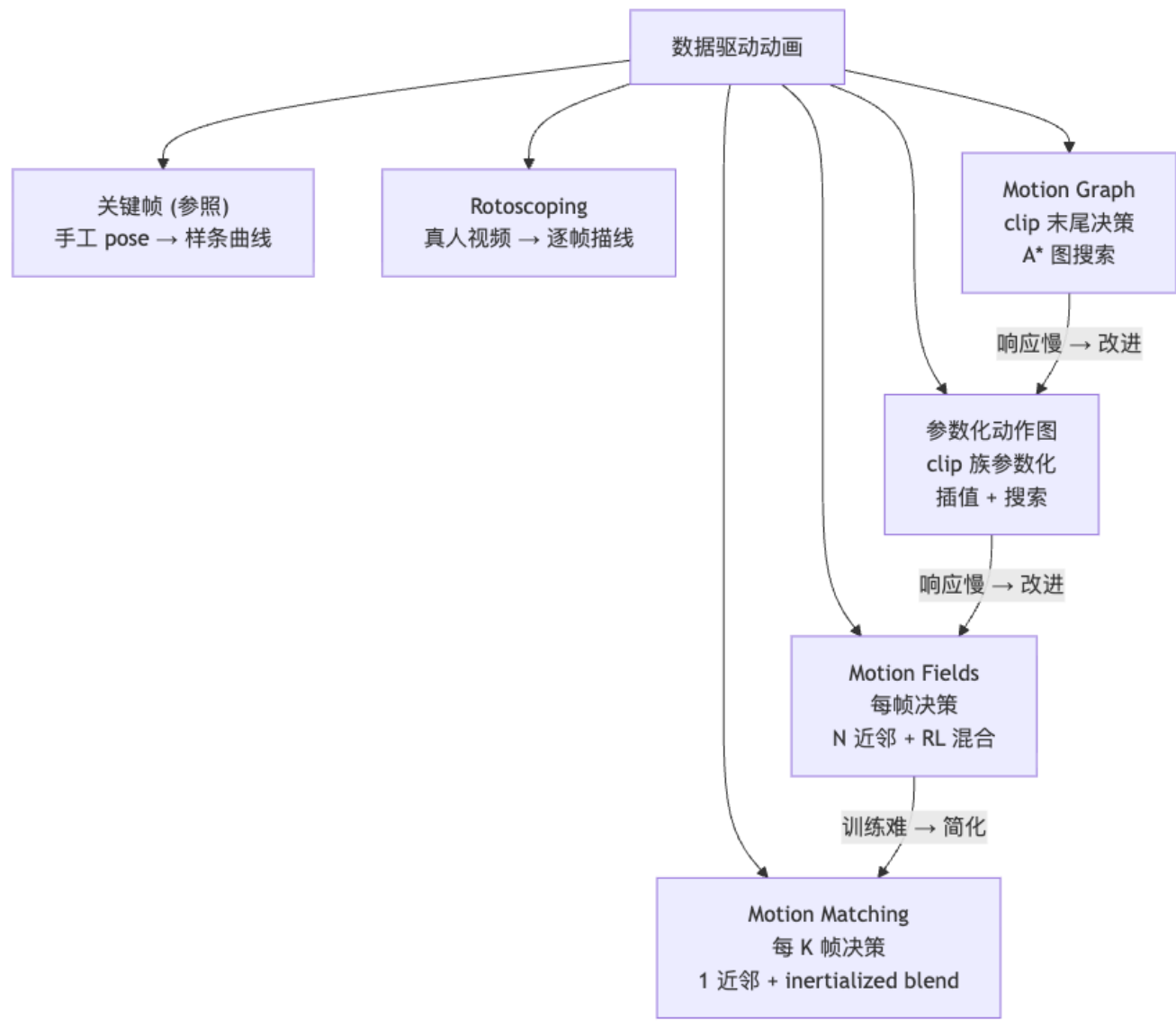
11.5 Motion Matching 完整流程图



11.6 Motion Fields vs Motion Matching 对比

维度	Motion Fields	Motion Matching
决策粒度	每帧	每 K 帧 ($K \approx 5-10$)
邻居使用方式	N 个邻居加权混合	1 个最近邻 + inertialized blend
混合权重来源	RL 策略学习	设计特征权重 + L2 距离
工程难度	高 (需训练 RL)	低 (只调特征权重 + KD-tree)
实时性能	需 RL 推断开销	KD-tree 查询, $O(\log N)$
工业采用度	学术为主	Ubisoft、EA、Rockstar 等广泛采用

12. 横向方法总览



方法	决策粒度	数据需求	控制响应	工程复杂度
Keyframe	—	手工设计	实时	低 (DCC 工具)
Rotoscoping	帧 (离线)	真人视频	离线	低
Motion Graph	Clip 末尾	MoCap 数据库	慢 (秒级)	中
Parametric Motion Graph	Clip 末尾	参数化 clip 族	中	中-高
Motion Fields	每帧	同上 + RL 训练	快	高
Motion Matching	每 K 帧	MoCap 数据库	快	中

13. Worked Examples 例题

[EX] 例题 1: 选择合适的 MoCap 系统

场景: 你需要为一款移动游戏采集演员跑步、跳跃、格斗的动作, 预算有限, 无专业采集场地, 需要半天内采集 50 个动作片段。请比较光学标记 vs 惯性 IMU 两种方案并给出建议。

分析: – **光学标记:** 精度最高, 但需多相机标定 (搭建耗时 4–8 h), marker 标定对场地要求严格 (黑色无反光背景), 遮挡问题需额外相机, 总成本高, 50 个动作采集可行但裕量不足。– **惯性 IMU:** 穿戴约 15 min, 无场地限制, 室外/走廊均可, 50 个动作采集完全可行; 缺点是位置 drift, 对格斗类大幅肢体交叉动作可能有朝向误差。

建议: 预算有限 + 无专业场地 → **惯性 IMU**。若格斗动作对手部位置精度要求高, 可后处理 + IK 修正脚手接触约束。

[EX] 例题 2: Motion Graph 中的 pose 距离设计

问: 在构建 Motion Graph 时, 仅用关节旋转作为 pose 距离 $D(i, j) = \|R_A(i) - R_B(j)\|_F^2$ (Frobenius 范数) 是否足够? 请分析不足并提出改进。

分析: 1. **仅旋转不足以表达动态:** 两帧 pose 旋转相同但速度不同 (一帧静止、一帧在运动), 过渡后会有突然加速感。应加入关节角速度或末端位置速度项。2. **末端关节比根关节重要:** 手脚位置的视觉权重高, 应对末端关节赋予更高权重 w_k 。

改进公式:

$$D(i, j) = \sum_k w_k (\|\Delta R_k\|^2 + \lambda \|\dot{R}_k^A(i) - \dot{R}_k^B(j)\|^2) \quad (\text{E.1})$$

其中 $\dot{R}_k$ 为角速度, λ 控制速度匹配的权重。末端关节 w_k 取 2–4 倍于中间关节。

[EX] 例题 3: Facing Frame 的必要性

问: 两段动作分别录自“角色面向 +Z 方向”和“角色面向 +X 方向”, 若不做 facing frame 对齐直接混合 root 旋转, 会出现什么问题? 如何用公式 (7.5) 修正?

分析: 直接混合 root 旋转: $R_{\text{blend}} = \text{Slerp}(R_0, R_1, t)$, 此时 R_0 对应 +Z, R_1 对应 +X, Slerp 会经过 45° 方向, 角色在过渡帧内先转向东南后再转向……且 pitch/roll 不为零时还会产生额外的翻滚。

修正: 用 Facing Frame 先把 R_1 对齐到 R_0 的水平坐标系:

$$R^{(i)} = R_0^{(\text{face})} R_1^{(\text{face})T} R_1^{(i)}$$

只对 yaw 分量做对齐 ($R_0^{(\text{face})}, R_1^{(\text{face})}$ 均只含 y 轴旋转), 然后在对齐后的同一朝向系内做混合, 角色不会出现无意义的翻滚。

[EX] 例题 4: Motion Matching 特征权重调试

场景: 在一款第三人称动作游戏中, 测试人员反映角色“跑步时转弯反应迟钝, 但动作本身很流畅”。请从 Motion Matching 特征权重的角度分析原因并给出调参方向。

分析: “转弯反应迟钝”= 系统没有快速匹配到“将要转向”的 clip。可能原因: “未来朝向”特征的权重 $w_{\text{future heading}}$ 太低, 导致搜索结果更偏向匹配“当前末端速度”而非“期望轨迹方向”。

调参: 增大 $w_{\text{future heading}}$ (0.5 s、1.0 s、1.5 s 预测点的权重), 观察转向提前量是否改善; 同时适当降低末端执行器位置权重, 防止过分拘泥于当前肢体形态而错过转向时机。若转弯后动作变不流畅, 说明权重改过头, 需在转向响应和动作连贯性之间取平衡。

14. Anki 记忆卡片

#	Q (正面)	A (背面)
1	MoCap 的 5 大类系统是什么？各自最大的优缺点？	机械（精准但笨重）、惯性 IMU（轻便但 drift）、光学标记（最精准但遮挡敏感）、无标记多目（舒适但精度低）、单目视频（最便利但精度最低）
2	Pose 的数学表示 p_t 包含哪些分量？	$p_t = (t_0, R_0, R_1, R_2, \dots)$: root 世界位置、root 世界朝向、各子关节局部旋转
3	动作过渡中，为什么旋转分量不能用线性插值？应该用什么？	LERP 不保持四元数单位长度、角速度不均匀；应用 Slerp （球面线性插值）或 log-space 混合
4	Facing Frame 的定义是什么？旋转和平移各保留什么分量？	水平随角色移动、一轴指向朝向方向的坐标系：旋转 = θe_y （只有 yaw），平移 = $(t_x, 0, t_z)$ （无竖直分量）
5	root 旋转分解 $R_0 = R_y R_{xz}$ 中， R_y 和 R_{xz} 各是什么？	R_y : 绕全局 y 轴的纯 yaw 旋转； R_{xz} : 旋转轴在 xz 平面内的“翻倒/tilt”旋转
6	两段动作拼接时，将片段 1 对齐到片段 0 的变换公式是什么？	$R^{(i)} = R_0 R_1^T R_1^{(i)}$, $t^{(i)} = R_0 R_1^T (t_1^{(i)} - t_1) + t_0$
7	Motion Graph 的节点和边分别代表什么？边如何自动构建？	节点 = motion clip；边 = 可行过渡点（pose 距离 $D(i, j) < \varepsilon$ ）；自动计算所有帧对的 pose 距离，低于阈值则连边
8	Motion Graph 的两大主要局限是什么？	① 响应延迟（等 clip 结束才切换）；② 覆盖受限（未采集的过渡无法生成）
9	Motion Matching 的核心公式是什么？	$\text{next_pose} = \arg \min_{i \in \text{Dataset}} \ x_{\text{curr}} - x_i\ $, x 为特征向量
10	Motion Matching 常用的特征向量包含哪些内容？	末端关节线/角速度（相对 root）、root 速度、未来朝向/位置（0.5 s / 1.0 s / 1.5 s）、脚接触标签、末端关节位置
11	Inertialized Blending 的基本思想和优点是什么？	用临界阻尼弹簧追踪目标 pose offset，解析解平滑衰减无振荡；只需调时间常数，效果远优于 LERP
12	Motion Matching 运行时为什么不每帧都做最近邻搜索？用什么代替中间帧？	搜索有开销；每隔 K （5–10）帧搜索一次，中间帧用 inertialized blending 平滑过渡
13	“Dance Card”在 Motion Matching 中指什么？作用是什么？	对数据库剪枝、去冗余帧的精华数据集；减小 N ，加速 KD-tree 查询，同时保证覆盖均衡
14	Motion Fields 和 Motion Matching 最核心的区别是什么？	Motion Fields: N 近邻 + RL 学到的混合权重；Motion Matching: 1 近邻 + 设计特征权重 + inertialized blend，工程难度更低
15	Motion Retargeting 需要处理哪些骨架差异？	骨骼数量、骨骼命名、参考姿态（T-pose）、骨长比例、骨架拓扑结构
16	什么是 IMU 的 drift 问题？为什么会发生？	对加速度积分两次得位置，积分误差随时间累积（小误差不断叠加），导致位置估计漂离真实值

15. 中英术语速查表

中文术语	English	首现章节
数据驱动动画	Data-driven Animation	§1
动作捕捉	Motion Capture (MoCap)	§3
转描	Rotoscoping	§3.1
机械外骨骼	Mechanical / Exoskeleton MoCap	§3.2
惯性测量单元	Inertial Measurement Unit (IMU)	§3.2

中文术语	English	首现章节
积分漂移	Drift	§3.2
光学标记	Optical Marker-based	§3.2
标记求解	Marker Solving	§3.2
无标记多目	Markerless Multi-view	§3.2
深度相机	Depth Camera	§3.2
单目视频	Monocular Video	§3.2
稀疏传感器	Sparse Sensor	§3.2
姿态	Pose	§4.1
动作片段	Motion Clip	§4.1
Root 关节	Root Joint	§4.1
局部旋转	Local Rotation	§4.1
球面线性插值	Slerp (Spherical Linear Interpolation)	§7.2
动作重定向	Motion Retargeting	§5
反向运动学	Inverse Kinematics (IK)	§5.2
脚滑	Foot-skating	§5.2
自穿透	Self-penetration	§5.2
位移映射	Motion Displacement Mapping	§6.1
拉普拉斯算子	Laplacian Operator	§6.2
动作过渡	Motion Transition	§7
过渡窗口	Transition Window	§7.2
平滑步进曲线	Smoothstep	§7.2
朝向坐标系	Facing Frame	§7.3
偏转	Yaw	§7.4
翻倒/俯仰	Tilt / Pitch	§7.4
旋转分解	Rotation Decomposition	§7.4
路径拟合	Path Fitting	§7.6
动作图	Motion Graph	§8
参数化动作图	Parametric Motion Graph	§8.4
状态机	State Machine	§8
动作场	Motion Fields	§10
强化学习	Reinforcement Learning (RL)	§10
动作匹配	Motion Matching	§11
特征向量	Feature Vector	§11.2
末端执行器	End Effector	§11.2
惯性化混合	Inertialized Blending	§11.3
临界阻尼	Critical Damping	§11.3
KD 树	KD-tree	§11.4
舞池数据集	Dance Card	§11.4
最近邻	Nearest Neighbor (NN)	§11.1

16. 复习要点

[TIP] 期末复习要点 (本讲必记)

1. **MoCap 五大类**: 机械 (无遮挡, 笨重) $\rightarrow$ IMU (轻便, drift) $\rightarrow$ 光学标记 (工业最准, 遮挡敏感) $\rightarrow$ 无标记多目 (舒适) $\rightarrow$ 单目视频 (最便利最低精度)。2. **Pose 格式**: $p_t = (t_0, R_0, R_1, \dots)$, 局部旋转 + root 世界状态, 旋转混合必须用 Slerp。3. **Facing Frame + 旋转分解**: $R_0 = R_y R_{xz}$, 过渡时只在水平朝向系对齐, 避免 pitch/roll 混乱。4. **Root 对齐变换**: $R^{(i)} = R_0 R_1^T R_1^{(i)}$, $t^{(i)} = R_0 R_1^T (t_1^{(i)} - t_1) + t_0$ 。5. **Motion Graph**: 节点 = clip, 边 = pose 距离 $D(i, j) < \varepsilon$ 自动构建, 运行时 A* 图搜索; 局限: 响应慢 (等 clip 结束), 覆盖受数据集限。6. **Motion Matching**: $\arg \min_i \|x_{\text{curr}} - x_i\|$, 特征向量含末端速度 + root 速度 + 未来朝向 + 脚接触; 每 K 帧搜一次 + KD-tree + inertialized blending + dance card 剪枝。7. **三大范式对比**: Motion Graph (clip 级, 响应慢) $\rightarrow$ Motion Fields (帧级 RL, 训练难) $\rightarrow$ Motion Matching (帧级最近邻, 工程简洁, 工业主流)。

Lecture 06 —基于学习的动画 (Learning-based Animation)

[TIP] 学习指南

本节核心：从“人类运动是高维空间中的低维流形”这一物理直觉出发，系统串联 **统计模型** → **自回归模型** → **生成模型** 三大类数据驱动角色动画方法。最终目标是：给定一批 mocap 示例，自动学会“什么是自然的运动”，并能在实时控制或离线生成中产出新运动。

前置知识（用到时均有零基础解释）：线性代数（特征值分解、SVD）、概率论（高斯分布、KL 散度、条件概率链式法则）、深度学习基础（MLP、SGD、反向传播）。

重点掌握：1. PCA 的几何推导 + PCA = 高斯先验的等价性 2. 自回归模型的歧义问题与 Phase 变量的解法 3. PFNN：相位驱动网络权重（Catmull-Rom 插值 + Weight-blended MoE） 4. MANN：可学习 Gating Network 替代手工 phase 5. VAE ELBO 完整推导、Reparameterization Trick 6. VQ-VAE：离散化潜变量 + “运动即外语”统一框架 7. 扩散模型：前向加噪 SDE、Score function、DDPM 训练算法 8. DeepMimic / AMP / ControlVAE：学习式运动控制核心思想

0. 名词预热：5 个必须先懂的词

名词	一句话解释
姿态 (pose)	角色在某一时刻的完整状态：根节点位置 t_0 ，各关节旋转 $R_0, R_1, \dots$ ，记作 $x_t \in \mathbb{R}^N$ ， $N \approx 60-300$
运动序列 (motion)	时间上连续的姿态序列 $X = (x_1, x_2, \dots, x_T)$
Mocap	动作捕捉数据，提供“真实人类运动”的数据集 $\{X_i\}$
Motion prior	一个评判姿态/运动“自然程度”的概率模型 $p(x)$ ，高 $p(x)$ = 更自然
隐变量 (latent variable)	低维压缩表示 z ，捕捉运动的本质结构；解码器从 z 重构 x

[NOTE] 为什么要“学习”？

手工 keyframing 耗时，motion graph (Lec 04) 只能拼接已有片段、无法泛化。Learning-based 方法把数据集蒸馏成模型，做到：(1) 评判自然程度 (motion prior)；(2) 自回归生成下一帧；(3) 从噪声/文本采样全新运动。

1. 人体运动的低维流形

1.1 核心观察

一个姿态 $x \in \mathbb{R}^N$ (N 约 60–300), 理论上可取遍 $\mathbb{R}^N$ 的任意点。但由于协调的四肢运动、肌肉骨骼结构限制、物理约束, 所有”自然”姿态只占据一个远小于 N 维的流形 $\mathcal{M}$ 。

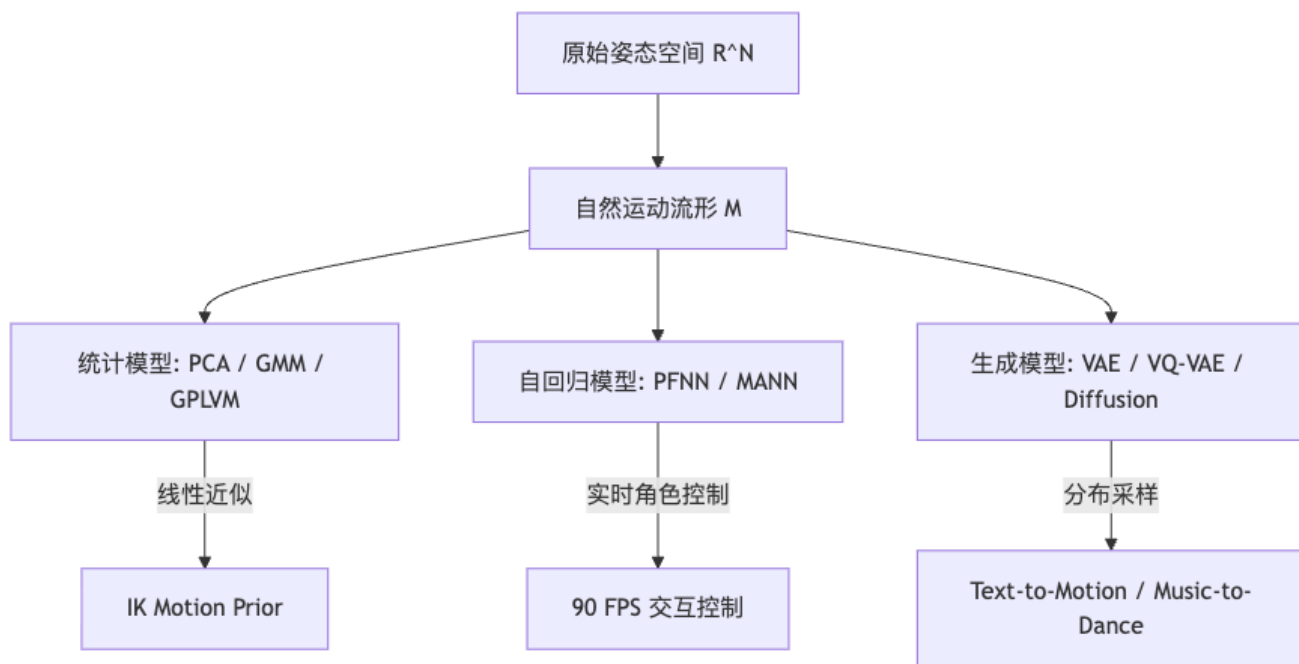
$$\mathcal{M} \subset \mathbb{R}^N, \quad \dim(\mathcal{M}) \ll N \quad (1.1)$$

[NOTE] 直觉理解

想象 $\mathbb{R}^N$ 是一个巨大的房间, 自然运动流形 $\mathcal{M}$ 是房间里一条细薄的曲面。随机采样一个高维点, 几乎不可能落在这条曲面上——所以”随机姿态”看起来总像鬼片。

所有 learning-based 方法的本质, 都是把这个隐含的流形 $\mathcal{M}$ 显式或隐式地参数化, 以便:

- 评判一个 x 是否在流形上 (motion prior)
- 沿流形移动 (运动生成与控制)



2. 主成分分析 (PCA)

2.1 神经网络之前: 为什么先讲 PCA?

PCA 是理解整条技术链路的基石: 它既是最简单的降维工具, 又与高斯先验等价, 还是 Autoencoder (进而 VAE) 的线性特例。

2.2 PCA 的几何推导

问题: 在数据集 $\{x_i\} \subset \mathbb{R}^N$ 中, 找一个单位方向 u ($\|u\| = 1$), 使各数据点在 u 上的投影 $w_i = x_i \cdot u$ 的方差最大:

$$\max_{u: \|u\|=1} \frac{1}{N} \sum_i (w_i - \bar{w})^2 \quad (2.1)$$

把数据中心化后堆成矩阵

$$X = \begin{bmatrix} (x_0 - \bar{x})^\top \\ (x_1 - \bar{x})^\top \\ \vdots \\ (x_{N-1} - \bar{x})^\top \end{bmatrix} \in \mathbb{R}^{N \times d} \quad (2.2)$$

可以证明，最优 u 恰好是协方差矩阵 $\Sigma = X^\top X$ 最大特征值对应的特征向量。继续取第 2 大、第 3 大.....特征向量，得到一组正交基 $\{u_k\}_{k=1}^n$ ：

$$\Sigma = X^\top X = U \begin{pmatrix} \sigma_1^2 & & \\ & \ddots & \\ & & \sigma_N^2 \end{pmatrix} U^\top, \quad U = [u_1, u_2, \dots, u_N] \quad (2.3)$$

其中 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_N \geq 0$ 是各方向的标准差（解释方差）。

2.3 PCA 的重建与截断

任意数据点的精确重建：

$$x_i = \bar{x} + \sum_{k=1}^N w_{i,k} u_k, \quad w_{i,k} = (x_i - \bar{x}) \cdot u_k \quad (2.4)$$

取前 n 个主成分（如选 n 使得解释方差 $\geq 95\%$ ）得截断近似：

$$\tilde{x}_i = \bar{x} + \sum_{k=1}^n w_{i,k} u_k \quad (2.5)$$

[NOTE] 走路的 PCA 实验

对走路 mocap 做 PCA，前几个主成分恰好对应步态周期（gait cycle）——双腿交替的节律被直接编码在最大方差方向上。在 σ_1, σ_2 张成的二维空间里，数据点形成一个环形轨道，对应一个完整步态周期。

2.4 PCA 的自然程度判别

在 u_k 方向上，一个姿态 x 的分量 $w_k = (x - \bar{x}) \cdot u_k$ 越小于 σ_k ，说明该姿态越“靠近流形中心”。定义自然程度分数：

$$\text{naturalness}(x) \propto \exp\left(-\frac{1}{2} \sum_k \frac{w_k^2}{\sigma_k^2}\right) \quad (2.6)$$

w_k/σ_k 越大（分量超出正常范围） $\rightarrow$ 姿态越奇怪。

2.5 带 PCA 先验的角色 IK

朴素 IK：给定末端目标 $\tilde{x}_i$ ，求使末端逼近目标的关节角 θ ：

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|_2^2 + \frac{\lambda}{2} \|\theta\|_2^2 \quad (2.7)$$

第二项 $\|\theta\|^2$ 使解偏向零（T 型站立），不自然。改用参考姿态 θ_0 ：

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|_2^2 + \frac{\lambda}{2} \|\theta - \theta_0\|_2^2 \quad (2.8)$$

加入 PCA motion prior：

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|_2^2 + \frac{w}{2} \sum_k \frac{((\theta - \bar{\theta}) \cdot u_k)^2}{\sigma_k^2} \quad (2.9)$$

[!important] PCA = 高斯先验 注意第二项可改写为 $-w \log p(\theta)$ ，其中

$$p(\theta) = \prod_k \mathcal{N}((\theta - \bar{\theta}) \cdot u_k; 0, \sigma_k^2) \quad (2.10)$$

IK + PCA motion prior 等价于 MAP 估计：数据项是负对数似然，PCA 项是负对数先验。代表工作：Grochow et al. SIGGRAPH 2004 (Style-Based IK)。

3. 高斯模型族

3.1 单高斯分布

将整个 pose 集合建模为多元高斯：

$$p(x) = \mathcal{N}(\bar{x}, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} \exp\left(-\frac{1}{2}(x - \bar{x})^\top \Sigma^{-1}(x - \bar{x})\right) \quad (3.1)$$

MLE 给出 $\bar{x} = \frac{1}{N} \sum_i x_i$ ， $\Sigma = \frac{1}{N} X^\top X$ ，代入 PCA 分解即得 §2.4 的自然程度公式：

$$p(x) = \prod_k \frac{1}{\sigma_k \sqrt{2\pi}} \exp\left(-\frac{1}{2} \frac{w_k^2}{\sigma_k^2}\right), \quad w_k = (x - \bar{x}) \cdot u_k \quad (3.2)$$

3.2 高斯混合模型 (GMM)

单高斯是单峰的，无法表达”走路 vs. 跑步 vs. 跳跃”等多模态运动。GMM 将多个高斯加权叠加：

$$p(x) = \sum_i \phi_i \mathcal{N}(\mu_i, \Sigma_i), \quad \sum_i \phi_i = 1, \quad \phi_i \geq 0 \quad (3.3)$$

用 EM 算法学习参数 $\{\phi_i, \mu_i, \Sigma_i\}$ 。代表工作：Min et al. SIGGRAPH 2009，用 GMM 同时容纳走、跑、跳等多种风格。

3.3 高斯过程潜变量模型 (GPLVM)

GMM 依然假设各分量是解析高斯。GPLVM 更进一步：把每个高维姿态 x 映射到低维潜变量 z ，假设 x 是 z 的高斯过程函数，学到连续可插值的风格流形。代表工作：Levine et al. SIGGRAPH 2012。

[WARN] 统计方法的上限

PCA / GMM / GPLVM 都依赖解析的概率假设（高斯或 GP），对真实大规模多风格数据集灵活性不足。神经网络方法（PFNN 起）不再假设具体概率形式，直接用非线性函数拟合分布。

4. 运动序列的两种建模视角

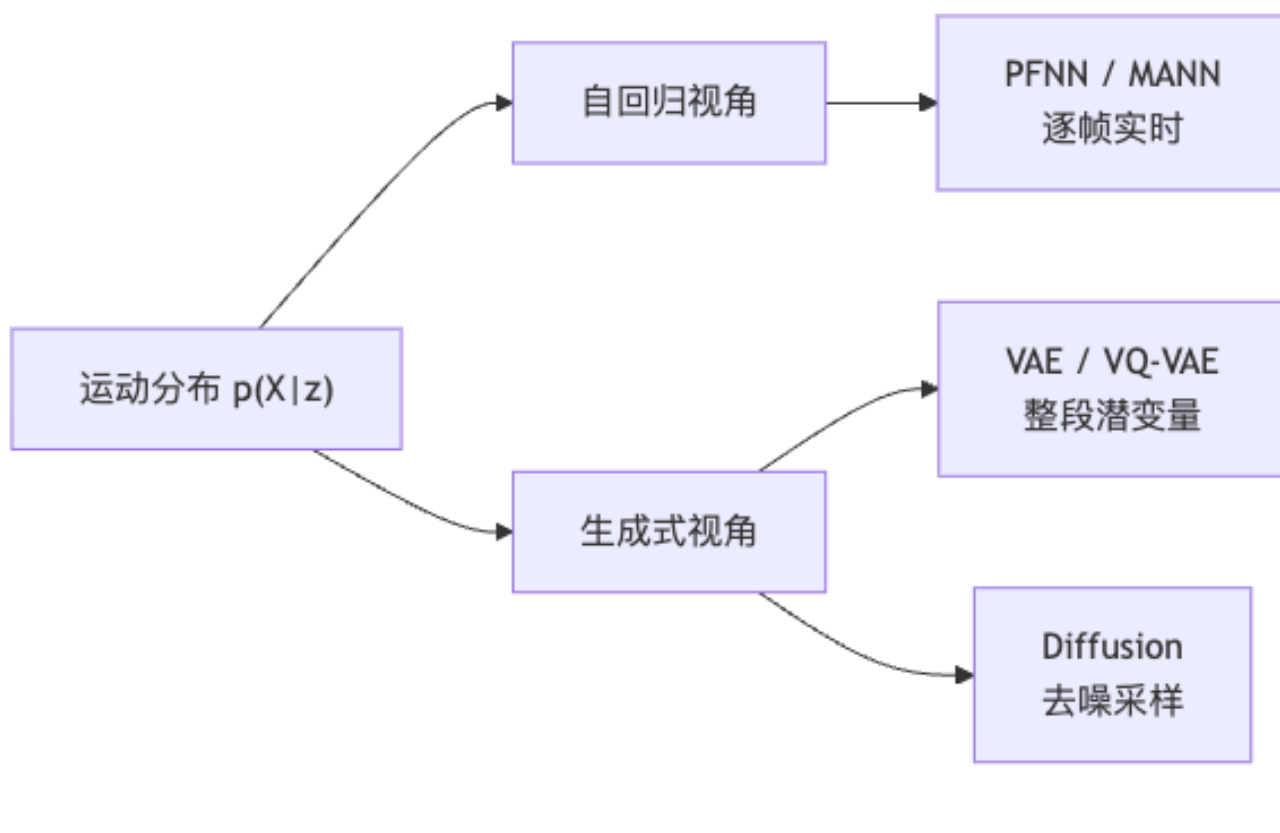
把每帧姿态记 $x_t = (t_0, R_0, R_1, \dots)$ ，整段运动 $X = (x_1, \dots, x_T)$ ，控制/隐变量为 z ，目标是建模条件分布 $p(X|z)$ 。

由条件概率链式法则（Chain Rule）：

$$p(X|z) = p(x_1) \prod_{t=2}^T p(x_t | x_{t-1}, \dots, x_1; z) \quad (4.1)$$

施加一阶 Markov 假设 $p(x_t | x_{t-1}, \dots; z) \approx p(x_t | x_{t-1}; z)$ ，得到两种等价视角：

视角	描述	适合场景
自回归 (Autoregressive)	$x_t = f(x_{t-1}; z)$ ，逐帧 roll out	实时交互控制 (90 FPS)
生成式 (Generative)	$X = f(z)$ ，整段一次性采样	text-to-motion, music-to-dance



5. 自回归模型与歧义问题

5.1 朴素自回归

最简单情形: $x_t = f(x_{t-1}; \theta)$ 。给定 transitions $\{(x_{t-1}, x_t)\}$, 训练 MLP:

$$F(\theta) = \sum_{(x_{t-1}, x_t)} \|f(x_{t-1}; \theta) - x_t\|^2 \quad (5.1)$$

SGD 更新:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} F(\theta), \quad \nabla_{\theta} F \approx \sum_i \nabla_{\theta} \|f(x_{t-1}^i; \theta) - x_t^i\|^2 \quad (5.2)$$

5.2 歧义问题 (Ambiguity)

[WARN] 一对多的歧义

在走路周期中, 同一个 x_{t-1} (比如双脚对称站立的中间相位) 可能对应两种合理的 x_t : 左脚先迈 vs. 右脚先迈。回归损失 $\|f - x_t\|^2$ 会把所有候选平均, 输出介于两者之间的”滑步”或”抽搐”姿态——这是 L2 回归面对多模态分布时的根本缺陷。

5.3 解决方案：隐变量 / 相位变量

在输入中注入额外信息 z_t 消除歧义：

$$x_t = f(x_{t-1}; z_t) \quad (5.3)$$

最经典的选择是步态相位 (gait phase) $\phi \in [0, 1)$ ——在走路周期中的位置。 ϕ 明确指出“现在是步态的哪个阶段”，从而区分“左脚迈出”和“右脚迈出”。

6. PFNN：相位函数神经网络

[NOTE] 参考文献

Holden et al., Phase-Functioned Neural Networks for Character Control, SIGGRAPH 2017.

6.1 输入设计

$$z_t = (c_t, \phi_t) \quad (6.1)$$

- c_t : 控制参数 (期望轨迹、地形高度图、目标速度、用户输入)
- $\phi_t \in [0, 1)$: 当前步态相位, 走路一步 = ϕ 从 0 走到 1

6.2 核心创新：权重随 Phase 连续变化

普通做法把 ϕ 作为额外输入特征，但网络容易忽略它。PFNN 的创新是让整套网络权重 θ_t 本身是 ϕ_t 的函数：

$$x_t = f(x_{t-1}; c_t, \theta_t), \quad \theta_t = \theta(\phi_t) \quad (6.2)$$

具体地, 在相位圆 ($[0, 1)$) 上预存 4 个控制点 $\theta_1, \theta_2, \theta_3, \theta_4$ (对应 $\phi = 0, 0.25, 0.5, 0.75$), 用 Catmull-Rom 三次样条在任意 ϕ_t 处插值：

$$\theta(\phi) = \theta_2 + \phi \cdot \frac{1}{2}(\theta_3 - \theta_1) + \phi^2 \left(\theta_1 - \frac{5}{2}\theta_2 + 2\theta_3 - \frac{1}{2}\theta_4 \right) + \phi^3 \left(-\frac{1}{2}\theta_1 + \frac{3}{2}\theta_2 - \frac{3}{2}\theta_3 + \frac{1}{2}\theta_4 \right) \quad (6.3)$$

[NOTE] 为什么用样条而不是直接输入 ϕ ?

把 ϕ 当输入特征，网络的所有权重在任何 ϕ 下都相同——网络要“用一套权重同时处理所有相位”，容量受限。样条插值让每个 ϕ 值对应一套略微不同的权重，极大增强表达力，而只增加 4 倍参数量。

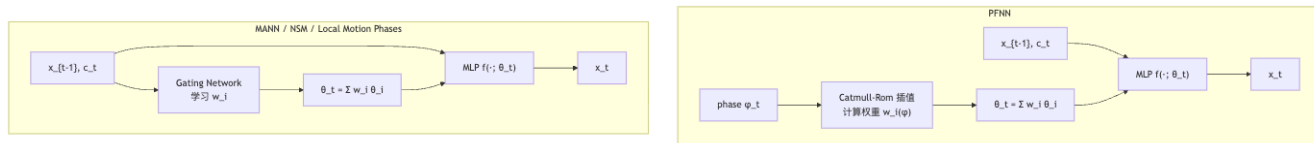
6.3 Mixture of Experts 视角

PFNN 本质是 Weight-blended Mixture of Experts (MoE)：

$$\text{Output-blended MoE: } y = \sum_i w_i f(x; \theta_i) \quad (6.4)$$

$$\text{Weight-blended MoE (PFNN): } y = f\left(x; \sum_i w_i \theta_i\right) \quad (6.5)$$

PFNN 的门控权重 $w_i(\phi_t)$ 来自 Catmull-Rom 插值系数，**不另外训练**——由手工提取的 ϕ 决定。



6.4 PFNN 的演进谱系

工作	时间	核心改进
PFNN	SIGGRAPH 2017	全局单 phase, Catmull-Rom 权重插值
MANN	SIGGRAPH 2018	用可学习 Gating Network 替代手工 phase, 可处理四足动物
NSM	SIGGRAPH 2019	加入环境编码 & 动作标签, 支持复杂场景
Local Motion Phases	SIGGRAPH 2020	每个肢体单独一个局部 phase, 处理篮球、格斗等异步多接触
Deep Phase	SIGGRAPH 2022	深度相位自编码器从数据自动发现相位流形, 无需手工标注

[NOTE] MANN 的关键突破

MANN (Motion-Aware Neural Networks) 把“相位”的概念从周期性步态推广到任意运动：Gating Network 从输入 x_{t-1} 自动学到门控权重 w_i ，四足动物（马、猫）的非规则步态也能正确处理，无需手工定义 phase。

7. 从 PCA 到 Autoencoder

7.1 PCA = 线性 Autoencoder

PCA 的编解码可以写成矩阵乘法形式：

$$z = U^\top(x - \bar{x}), \quad x' = \bar{x} + Uz \quad (7.1)$$

这正是一个**线性 Autoencoder**： $\mathcal{E}_{\text{linear}}(x) = U^\top(x - \bar{x})$, $\mathcal{D}_{\text{linear}}(z) = \bar{x} + Uz$ 。

7.2 非线性 Autoencoder

将线性编解码器换成深度神经网络（MLP 或 CNN）：

$$z = \mathcal{E}(x; \theta_E), \quad x' = \mathcal{D}(z; \theta_D), \quad \min_{\theta_E, \theta_D} \|\mathcal{D}(\mathcal{E}(x)) - x\|^2 \quad (7.2)$$

代表工作：**Holden et al. 2015**, Learning Motion Manifolds with Convolutional Autoencoders——第一次用卷积 AE 把 mocap 嵌入低维流形，在潜空间插值再解码出平滑过渡。

7.3 AE 的致命缺陷

AE 的潜空间 z 没有任何正则化。训练数据附近的 z 解码出合理姿态，但随机采样的 z 解码出怪异结果——潜空间有“空洞”，不能随机采样。解决方案：让 z 服从已知的先验分布。

8. 变分自编码器 (VAE)

[NOTE] 背景：什么是变分推断？

变分推断用一个简单分布 $q(z|x)$ 来近似难以计算的后验 $p(z|x)$ ，通过最大化下界（ELBO）同时达到“重建准确”和“ z 分布规则”两个目标。

8.1 目标：让 z 服从标准正态

强制 $z \sim \mathcal{N}(0, I)$ ，从而任意标准正态采样都能解码出合理运动：

$$p(x) = \int_z p(x|z) p(z) dz, \quad p(z) = \mathcal{N}(0, I) \quad (8.1)$$

8.2 ELBO 完整推导

最大化数据 log-likelihood: $\log p(x) = \log \int_z p(x|z) p(z) dz$ 。

引入变分后验 $q(z|x)$ ，乘以 $1 = q(z|x)/q(z|x)$ 并利用 Jensen 不等式 ($\log \mathbb{E} \geq \mathbb{E} \log$)：

$$\log p(x) = \log \int_z q(z|x) \frac{p(x|z)p(z)}{q(z|x)} dz \geq \int_z q(z|x) \log \frac{p(x|z)p(z)}{q(z|x)} dz \quad (8.2)$$

整理右侧：

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{重建项 (Reconstruction)}} - \underbrace{D_{\text{KL}}(q(z|x) \| p(z))}_{\text{正则项 (KL Penalty)}} \quad (8.3)$$

[!important] ELBO 的双重作用 – **重建项**：鼓励 encoder-decoder 把 x 压缩再还原，保留信息 – **KL 项**：惩罚后验 $q(z|x)$ 与先验 $\mathcal{N}(0, I)$ 的偏差，迫使潜空间整齐规则 – 两项的权衡由超参数 β (β -VAE) 控制： $\beta > 1$ 更强调潜空间规则性

KL 散度的含义： $D_{\text{KL}}(Q \| P) = \int q \log(q/p) dz \geq 0$ ，等号当且仅当 $Q = P$ 。

8.3 高斯参数化与损失函数

设：

$$p(x|z) \sim \mathcal{N}(\mathcal{D}(z), \sigma_D^2 I), \quad q(z|x) \sim \mathcal{N}(\mu_z = \mathcal{E}(x), \sigma_z^2 I) \quad (8.4)$$

用 Monte Carlo 估计重建项，最终训练损失为：

$$\mathcal{L}_{\text{VAE}} = \|\mathcal{D}(z) - x\|^2 + D_{\text{KL}}(\mathcal{N}(\mu_z, \sigma_z^2) \| \mathcal{N}(0, I)) \quad (8.5)$$

其中 KL 项有解析形式：

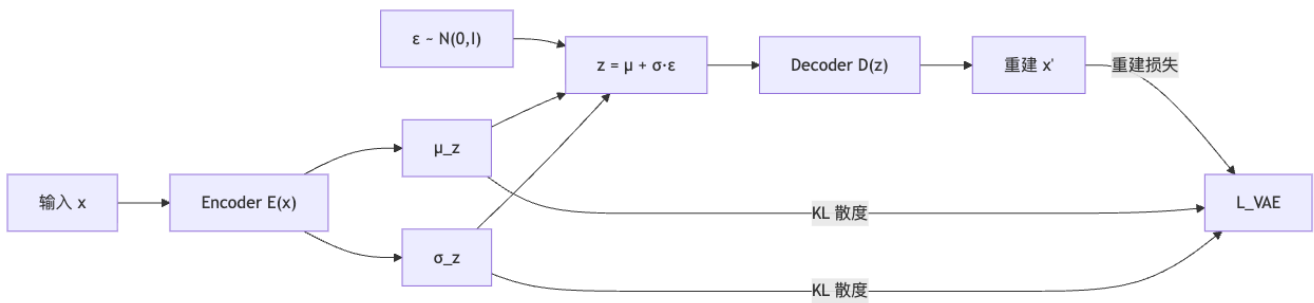
$$D_{\text{KL}}(\mathcal{N}(\mu_z, \sigma_z^2) \parallel \mathcal{N}(0, I)) = \frac{1}{2} \sum_j (\mu_{z,j}^2 + \sigma_{z,j}^2 - \log \sigma_{z,j}^2 - 1) \quad (8.6)$$

8.4 Reparameterization Trick

直接对 $z \sim \mathcal{N}(\mu_z, \sigma_z^2)$ 采样，随机性会切断梯度流。**重参数化技巧**将随机性剥离到固定噪声变量 ε ：

$$z = \mu_z + \sigma_z \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (8.7)$$

梯度可以从 $\mathcal{L}$ 反传到 μ_z 和 σ_z (即 encoder 的输出)，实现端到端训练。



8.5 Motion VAE 与 ControlVAE

工作	贡献
Motion VAE (Ling et al. 2021)	将每帧 x_t 编入 latent z_t ；训练 RL 策略网络 $\pi(z_t \mid x_{t-1}, \text{goal})$ 在 latent 空间输出动作。高维运动控制问题降到 32–64 维，RL 训练显著更稳定
ControlVAE (Yao et al. 2022)	把 Motion VAE 接入物理仿真器，统一运动学生成与动力学控制。策略网络在 VAE latent 上操作，解码出的动作直接驱动物理角色

9. VQ-VAE：向量量化变分自编码器

[NOTE] 背景：离散 vs 连续潜变量

VAE 的潜变量是连续的高斯向量，难以与语言模型（GPT 等）直接对接。VQ-VAE 引入**离散码本**，把运动编码为整数序列，开辟了“运动即语言”的统一框架。

9.1 核心思想：Codebook 量化

建立码本 (Codebook) $\mathcal{C} = \{z_j\}_{j=1}^K$ (如 $K = 512$ 个 d 维码字)。编码器输出的连续向量 $z = \mathcal{E}(x)$ 被量化到最近码字：

$$z' = z_j, \quad j = \arg \min_i \|z - z_i\|_2 \quad (9.1)$$

解码器用量化后的 z' 而非连续 z 重建: $x' = \mathcal{D}(z')$ 。

9.2 损失函数与直通估计

argmin 不可微, 用 **Straight-through estimator** + commit loss:

$$\mathcal{L}_{\text{VQ-VAE}} = \|\mathcal{D}(z') - x\|^2 + \|\text{sg}[z] - z'\|^2 + \beta \|z - \text{sg}[z']\|^2 \quad (9.2)$$

- 第一项: 重建损失
- 第二项: 更新码本, 让 z' 向 z 移动 ($\text{sg}[\cdot] = \text{stop-gradient}$)
- 第三项 ($\beta \approx 0.25$): commit loss, 防止 encoder 输出漂移过快

9.3 运动 Token 序列

对一段运动序列, 对每帧 (或每几帧) 的 embedding 做量化, 得到整数序列:

$$X \mapsto (j_1, j_2, j_3, \dots, j_T), \quad j_t \in \{0, 1, \dots, K-1\} \quad (9.3)$$

[!important] “运动即外语”(Motion as a Foreign Language) 把英语-西班牙语翻译的框架搬来: **英语句子** → **运动 token 序列**。把运动看作一种特殊的“语言”, 与文本语言共同训练 GPT-style Transformer, 实现双向翻译: 文本生成运动、运动生成文本描述、运动风格迁移。

9.4 代表工作

工作	贡献
MotionGPT (Jiang et al. 2023)	统一 text → motion 的 GPT, 在 VQ token 上同时训练文本与运动的生成
MoConVQ (Yao et al. 2024)	Codebook + 物理解码器, codebook token 可直接驱动物理仿真角色
LLM 抽象任务	用 LLM 将“走方形轨迹”解析为“前走 → 右转 90° → 前走 →“再映射到 token 序列

10. 扩散模型 (Diffusion Models)

[NOTE] 背景: 为什么需要扩散模型?

VAE 的潜空间是连续高斯, 表达力受限 (生成图像/运动质量不如 GAN)。GAN 训练不稳定。扩散模型用迭代去噪过程学到极高质量的分布, 但采样步数多 (100–1000 步), 是当前图像/运动生成的 SOTA。

10.1 前向加噪过程

将清洁数据 x_0 在 T 步内逐步加入高斯噪声, 直至 $x_T \sim \mathcal{N}(0, I)$:

$$q(x_t|x_0) = \mathcal{N}(\alpha_t x_0, \sigma_t^2 I), \quad \sigma_t^2 = 1 - \alpha_t^2 \quad (10.1)$$

其中 $\alpha_0 = 1, \alpha_T = 0$ (噪声调度)。等价地, 可直接采样:

$$x_t = \alpha_t x_0 + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (10.2)$$

10.2 反向去噪过程

反向过程 $p_\theta(x_{t-1}|x_t)$ 逐步从 x_T 恢复 x_0 ——这正是我们要学的。直接写出 $p_\theta(x_{0:T})$ 不可解析, 需要神经网络近似。

10.3 Score Function 视角 (SDE 框架)

将扩散写成随机微分方程 (SDE):

$$\text{Forward SDE: } dx = f(x, t) dt + g(t) dW \quad (10.3)$$

$$\text{Reverse SDE: } dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{W} \quad (10.4)$$

反向 SDE 的唯一未知量是 **score function** $\nabla_x \log p_t(x)$, 即数据分布在各时刻的对数梯度。学一个网络 $s_\theta(x, t) \approx \nabla_x \log p_t(x)$ 即可模拟反向过程。

10.4 训练目标: 去噪 Score Matching

直接最小化 $\mathbb{E} \|s_\theta(x, t) - \nabla_x \log p_t(x)\|^2$ 不可计算 (p_t 不 tractable)。等价的可计算目标:

$$\mathcal{L} = \mathbb{E}_{t, x_0, x_t} \left[\|s_\theta(x_t, t) - \nabla_{x_t} \log p_t(x_t|x_0)\|^2 \right] \quad (10.5)$$

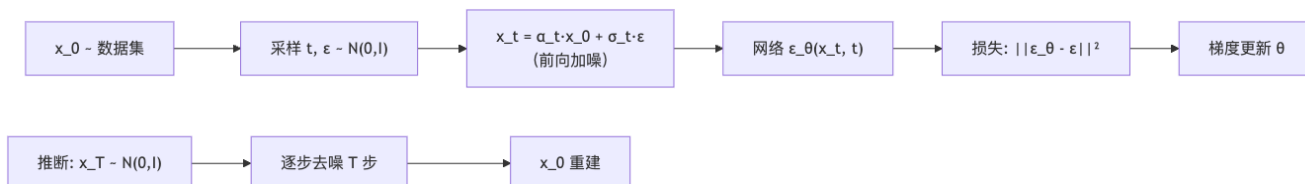
由于 $p_t(x_t|x_0) \sim \mathcal{N}(\alpha_t x_0, \sigma_t^2 I)$, 其 score 为:

$$\nabla_{x_t} \log p_t(x_t|x_0) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t} \quad (10.6)$$

其中 ε 是前向加噪时使用的噪声。这给出 DDPM 形式的**噪声预测损失**:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, x_0, \varepsilon} \left[\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2 \right] \quad (10.7)$$

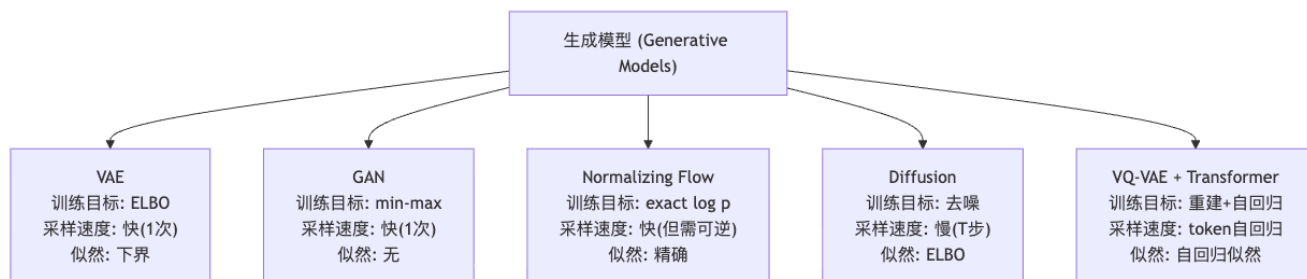
[!important] DDPM 训练算法 (一句话) 采一个数据 x_0 , 采一个时间步 t , 采一个噪声 ε , 构造 $x_t = \alpha_t x_0 + \sigma_t \varepsilon$, 让网络从 x_t 预测 ε 。



10.5 运动扩散代表工作

工作	任务	特色
EDGE (Tseng et al. 2022)	音乐驱动舞蹈生成	可编辑（锁定特定帧），基于 diffusion
GestureDiffuCLIP (Ao et al. 2023)	语音驱动手势	引入 CLIP latent 作为语义条件
Kimodo (NVIDIA)	物理角色控制	diffusion 与物理仿真结合

11. 生成模型大比较



模型	训练目标	采样速度	似然估计	运动应用
VAE	ELBO（重建 + KL）	一次前向，快	下界	Motion VAE, ControlVAE
GAN	$\min_G \max_D$ 对抗	一次前向，快	无	AMP（AMP 用判别器作先验）
Normalizing Flow	$\log p(x)$ via Jacobian	一次前向（需可逆结构）	精确	MoGlow (Henter 2020)
Diffusion	去噪 score matching	多步（100—1000 步），慢	ELBO	EDGE, Kimodo
VQ-VAE + GPT	重建 + 自回归 token	token 自回归	自回归似然	MotionGPT, MoConVQ

12. 学习式运动控制

[NOTE] 与前述生成模型的关系

上述方法学到”什么是自然运动”（运动先验）；本节把这个先验接入**强化学习 + 物理仿真**，得到既自然又能与环境真实交互的控制策略。

12.1 DeepMimic：基于模仿的运动控制

核心思想：给定一段参考 mocap 动作 $\hat{x}_{0:T}$ ，用强化学习（RL）训练物理角色模仿复现它。

奖励设计：每步奖励分为模仿奖励 r^I + 目标奖励 r^G ：

$$r_t = w^I r_t^I + w^G r_t^G \quad (12.1)$$

$$r_t^I = \exp \left[-w_p \sum_j \|\hat{q}_j - q_j\|^2 - w_v \sum_j \|\hat{\dot{q}}_j - \dot{q}_j\|^2 - w_e \sum_e \|\hat{p}_e - p_e\|^2 - w_c \|v_{\text{com}} - \hat{v}_{\text{com}}\|^2 \right] \quad (12.2)$$

其中 q_j 是关节角、 p_e 是末端执行器位置、 v_{com} 是质心速度。

[WARN] DeepMimic 的局限

每次只能模仿一段参考动作；切换到不同动作需要重新训练策略。真实应用中需要成百上千种动作，不可能为每个动作训练一个策略。

12.2 AMP：对抗动作先验

核心思想 (Peng et al. 2021)：用 GAN 风格的判别器 D_ϕ 作为运动自然程度的通用先验，替代手工设计的模仿奖励。

$$r_t^{AMP} = -\log(1 - D_\phi(s_t, a_t, s_{t+1})) \quad (12.3)$$

D_ϕ 区分“真实 mocap 转移”vs “策略产生的转移”——策略越自然， D_ϕ 越判为真实，奖励越高。这样一个判别器可对应整个 mocap 数据库，策略自然性由数据驱动而非手工。

总奖励：

$$r_t = w^G r_t^G + w^{AMP} r_t^{AMP} \quad (12.4)$$

ASE (Peng et al. 2022) 在 AMP 基础上加入技能潜变量 z ，训练条件策略 $\pi(a | s, z)$ ，从 z 连续采样可产生多样风格。

12.3 ControlVAE：统一 Kinematic 与 Dynamic

核心思想 (Yao et al. 2022)：

1. 先用 Motion VAE 在 mocap 数据上学到运动 latent 空间
2. 把 latent 空间作为策略的动作空间，训练 RL 策略 $\pi(z_t | s_t, \text{goal})$
3. VAE 解码器把 z_t 转成 PD 目标角，驱动物理仿真角色

优点：策略只需在低维 latent 空间探索（32–64 维 vs 原始关节角 ~ 200 维），RL 样本效率大幅提升；VAE 先验保证生成的运动自然。

13. 其他应用

应用	代表工作	核心方法
运动重定向 (Motion Retargeting)	Aberman et al. 2020 Skeleton-aware networks	骨架感知网络把动作迁移到不同骨架比例的角色
风格迁移 (Style Transfer)	Aberman et al. 2020 Unpaired Motion Style Transfer	从视频抽取风格嵌入，把“愤怒”风格叠加到中性走路
运动补帧 (Motion In-betweening)	Qin et al. 2022 Two-Stage Transformer	给定首尾关键帧，用两阶段 Transformer 补全中间帧

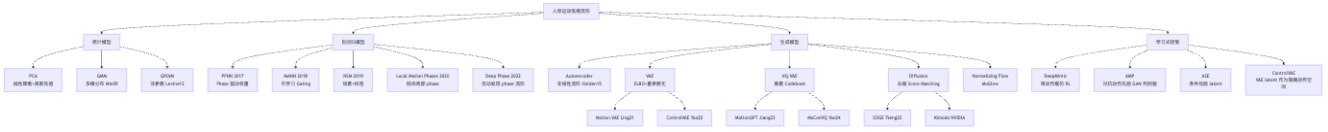
应用	代表工作	核心方法
Learned Motion Matching	Holden et al. 2020	把 motion matching 数据库压缩为神经网络，查询速度更快
MoGlow	Henter et al. 2020	基于 Normalizing Flow 的概率可控运动合成
多目标控制	Lee et al. 2019	同时满足”走到目标点 + 注视相机 + 抓取物体”等多目标

14. 总结

[TIP] 三句话总结全讲

1. 低维流形是一切 learning-based animation 的物理基础；PCA 是线性近似，神经网络是非线性扩展。
2. PFNN 系列用 phase/gating 变量解决同一历史对应多种未来的歧义，实现 90 FPS 实时角色控制。
3. 生成模型三巨头：VAE 提供平滑连续 latent (Motion VAE、ControlVAE)；VQ-VAE 把运动离散化为可被 GPT 处理的 token (MotionGPT)；Diffusion 提供当前最高生成质量 (EDGE、Kimodo)。

方法谱系全图：



15. Worked Examples (例题)

[EX] 例题 1: PCA 先验与 MAP 估计

问：设 mocap 数据的 PCA 分解给出主轴 u_1, u_2 ，方差 $\sigma_1^2 = 4, \sigma_2^2 = 0.25$ 。有两个候选姿态（在主轴坐标中）： $A = (w_1 = 1, w_2 = 0)$ 和 $B = (w_1 = 0, w_2 = 1)$ 。哪个更”自然”？

解：由公式 (2.6)，计算自然程度分数：

$$\text{naturalness}(A) \propto \exp\left(-\frac{1^2}{2 \cdot 4} - \frac{0^2}{2 \cdot 0.25}\right) = \exp(-0.125) \approx 0.882$$

$$\text{naturalness}(B) \propto \exp\left(-\frac{0^2}{2 \cdot 4} - \frac{1^2}{2 \cdot 0.25}\right) = \exp(-2) \approx 0.135$$

A 更自然。直觉： $\sigma_2 = 0.5$ 远小于 $\sigma_1 = 2$ ，在 u_2 方向偏离 1 个单位的惩罚远大于在 u_1 方向偏离 1 个单位。 u_2 是”小方差方向”，即数据在这个方向几乎不变化——随意偏离就不自然。

[EX] 例题 2: PFNN 的权重插值

问: PFNN 有 4 个控制点 $\theta_1, \theta_2, \theta_3, \theta_4$, 当 $\phi = 0$ (步态起点) 时, 各控制点权重是多少?

解: 由公式 (6.3), 代入 $\phi = 0$:

$$\theta(0) = \theta_2 + 0 \cdot (\dots) + 0^2(\dots) + 0^3(\dots) = \theta_2$$

当 $\phi = 0$ 时, 网络权重完全等于 θ_2 (第 2 个控制点)。这是 Catmull-Rom 样条的标准性质: $\phi = 0$ 对应样条段起点 $= \theta_2$, $\phi = 1$ 对应终点 $= \theta_3$; θ_1, θ_4 只影响端点处的切线方向 (保证曲率连续)。

[EX] 例题 3: VAE 的 KL 散度计算

问: VAE encoder 对某样本输出 $\mu_z = (1, 0)^\top$, $\sigma_z^2 = (2, 0.5)^\top$ (二维独立高斯)。计算 $D_{\text{KL}}(q||p)$, 其中 $p = \mathcal{N}(0, I)$ 。

解: 由公式 (8.6):

$$D_{\text{KL}} = \frac{1}{2} \sum_{j=1}^2 (\mu_{z,j}^2 + \sigma_{z,j}^2 - \log \sigma_{z,j}^2 - 1)$$

第 1 维: $\frac{1}{2}(1 + 2 - \log 2 - 1) = \frac{1}{2}(2 - 0.693) = 0.654$

第 2 维: $\frac{1}{2}(0 + 0.5 - \log 0.5 - 1) = \frac{1}{2}(0.5 + 0.693 - 1) = 0.097$

$D_{\text{KL}} = 0.654 + 0.097 = 0.751$

解读: 第 1 维的 KL 贡献更大, 因为 $\mu_{z,1} = 1$ 偏离 0, 且方差 $\sigma^2 = 2$ 也偏离 1 (先验方差)。训练过程中这个惩罚会把 μ_z 推向 0、 σ_z^2 推向 1。

[EX] 例题 4: 扩散模型前向加噪

问: 在 DDPM 中, $\alpha_t = 0.8$ (对应某中间时间步)。若原始数据 $x_0 = 5$ (标量), 采样噪声 $\varepsilon = 1$, 求 x_t , 并说明 σ_t^2 。

解: 由公式 (10.2):

$$\sigma_t^2 = 1 - \alpha_t^2 = 1 - 0.64 = 0.36, \quad \sigma_t = 0.6$$

$$x_t = \alpha_t x_0 + \sigma_t \varepsilon = 0.8 \times 5 + 0.6 \times 1 = 4.0 + 0.6 = 4.6$$

网络的任务: 给定 $x_t = 4.6$ 和时间步 t , 预测噪声 $\varepsilon = 1$, 从而能还原 $x_0 = (x_t - \sigma_t \varepsilon) / \alpha_t = (4.6 - 0.6) / 0.8 = 5$ 。训练时直接最小化 $\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2 = \|\varepsilon_\theta(4.6, t) - 1\|^2$ 。

16. Anki 记忆卡片

#	Q (正面)	A (背面)
1	人体运动低维性的三个物理原因?	协调的四肢运动、肌肉骨骼结构约束、物理定律 (重力、平衡)
2	PCA 的优化目标是什么?	找单位方向 u , 使数据投影 $w_i = x_i \cdot u$ 的方差 $\frac{1}{N} \sum (w_i - \bar{w})^2$ 最大化
3	PCA 的协方差矩阵分解公式?	$\Sigma = X^\top X = U \text{diag}(\sigma_1^2, \dots, \sigma_N^2) U^\top$

#	Q (正面)	A (背面)
4	PCA 等价于哪种先验?	多元高斯先验: $p(\theta) = \prod_k \mathcal{N}(w_k; 0, \sigma_k^2)$; IK + PCA = MAP 估计
5	自回归模型的歧义问题是什么? 如何解决?	同一 x_{t-1} 对应多种 x_t , L2 回归输出平均; 解决: 加相位/隐变量 z_t , $x_t = f(x_{t-1}; z_t)$
6	PFNN 的核心创新是什么?	网络权重 θ_t 是 gait phase ϕ_t 的函数, 用 Catmull-Rom 样条在 4 个控制点间插值
7	Weight-blended MoE 公式? 与 Output-blended MoE 的区别?	Weight-blended: $y = f(x; \sum_i w_i \theta_i)$; Output-blended: $y = \sum_i w_i f(x; \theta_i)$. PFNN 用前者
8	MANN 相比 PFNN 的改进?	用可学习 Gating Network 替代手工 phase, 自动学到门控权重, 可处理四足等非周期运动
9	VAE ELBO 公式? 各项含义?	$\mathcal{L} = \mathbb{E}_q[\log p(x z)] - D_{\text{KL}}(q(z x) \ p(z))$; 重建项 + KL 正则项
10	Reparameterization Trick 的公式与目的?	$z = \mu_z + \sigma_z \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$; 把随机性剥离, 允许梯度反传到 μ_z, σ_z
11	VQ-VAE 的量化公式? 损失函数?	$z' = z_j = \arg \min_i \ z - z_i\ $; $\mathcal{L} = \ D(z') - x\ ^2 + \ \text{sg}[z] - z'\ ^2 + \beta \ z - \text{sg}[z']\ ^2$
12	“运动即外语”的含义? 代表工作?	VQ-VAE 把运动编码为整数 token 序列, 与文本 token 共享 GPT 框架; MotionGPT (Jiang 2023)
13	DDPM 训练的三步?	采数据 x_0 、采时间 t 和噪声 ε 、构造 $x_t = \alpha_t x_0 + \sigma_t \varepsilon$, 学网络预测 ε
14	扩散模型前向/反向过程的 SDE 形式?	前向: $dx = f dt + g dW$; 反向: $dx = [f - g^2 \nabla_x \log p_t] dt + g d\bar{W}$
15	AMP 的核心思想与奖励公式?	用 GAN 判别器 D_ϕ 作为运动先验; 奖励 $r^{\text{AMP}} = -\log(1 - D_\phi(s, a, s'))$
16	ControlVAE 如何统一运动学与动力学?	Motion VAE 学 latent 空间 $\rightarrow$ RL 策略在 latent 上操作 $\rightarrow$ VAE 解码为关节角 $\rightarrow$ 物理仿真
17	五大生成模型的采样速度和似然估计对比?	VAE: 快/下界; GAN: 快/无; Flow: 快/精确; Diffusion: 慢/ELBO; VQ+GPT: token 自回归/自回归似然
18	DeepMimic 的模仿奖励包含哪些项?	关节角误差 w_p 、关节速度误差 w_v 、末端位置误差 w_e 、质心速度误差 w_c , 指数形式综合

17. 中英术语速查表

中文	English	首次出现
基于学习的动画	Learning-based Animation	标题
姿态	Pose	§0
运动序列	Motion Sequence	§0
动作捕捉	Motion Capture (Mocap)	§0

中文	English	首次出现
运动先验	Motion Prior	§0
低维流形	Low-dimensional Manifold	§1
主成分分析	Principal Component Analysis (PCA)	§2
主成分	Principal Component	§2
解释方差	Explained Variance	§2
马哈拉诺比斯距离	Mahalanobis Distance	§2.4
逆运动学	Inverse Kinematics (IK)	§2.5
最大后验估计	Maximum A Posteriori (MAP)	§2.5
高斯混合模型	Gaussian Mixture Model (GMM)	§3.2
高斯过程潜变量模型	Gaussian Process Latent Variable Model (GPLVM)	§3.3
链式法则	Chain Rule	§4
马尔可夫假设	Markov Assumption	§4
自回归模型	Autoregressive Model	§5
歧义问题	Ambiguity Issue	§5.2
步态相位	Gait Phase	§5.3
相位函数神经网络	Phase-Functioned Neural Networks (PFNN)	§6
混合专家	Mixture of Experts (MoE)	§6.3
门控网络	Gating Network	§6.3
Catmull-Rom 样条	Catmull-Rom Spline	§6.2
自编码器	Autoencoder (AE)	§7.2
变分自编码器	Variational Autoencoder (VAE)	§8
证据下界	Evidence Lower BOund (ELBO)	§8.2
KL 散度	Kullback-Leibler Divergence	§8.2
重参数化技巧	Reparameterization Trick	§8.4
向量量化变分自编码器	Vector Quantized-VAE (VQ-VAE)	§9
码本	Codebook	§9.1
直通估计器	Straight-through Estimator	§9.2
扩散模型	Diffusion Model	§10
得分函数	Score Function	§10.3
去噪得分匹配	Denoising Score Matching	§10.4
随机微分方程	Stochastic Differential Equation (SDE)	§10.3
对抗动作先验	Adversarial Motion Prior (AMP)	§12.2
运动重定向	Motion Retargeting	§13
运动风格迁移	Motion Style Transfer	§13
运动补帧	Motion In-betweening	§13

18. 复习要点

[TIP] 期末复习要点 (本讲必记)

1. 低维流形是出发点：所有方法都在做“把 $\mathbb{R}^N$ 中的自然运动流形参数化”这件事。PCA = 线性近似；神经网络 = 非线性近似。
2. 三大方法类别：
 - 统计：PCA (等价高斯先验/MAP) → GMM (多峰) → GPLVM (非参数)
 - 自回归：朴素 MLP → PFNN (phase 驱动权重) → MANN (学习 gating) → Local Motion Phases
 - 生成：AE → VAE (ELBO + KL) → VQ-VAE (离散 codebook) → Diffusion (去噪 score matching)
3. PFNN 关键点：Phase ϕ 不是普通输入特征，而是驱动权重插值 (Catmull-Rom in weight space) = Weight-blended MoE。
4. VAE ELBO = 重建项 - KL 项；KL 解析公式；Reparameterization Trick 让梯度可传。
5. VQ-VAE 把运动编成整数 token → 可接 GPT (MotionGPT)；commit loss 三项各有其用。
6. Diffusion 的等价训练目标：预测噪声 ε ；score function $\nabla_x \log p_t(x)$ 出现在反向 SDE 中。
7. 学习式控制：DeepMimic 单动作模仿 (指数奖励) → AMP 判别器先验 (整个 mocap 库) → ControlVAE (VAE latent 作为 RL 动作空间)。

[NOTE] 数据来源

本笔记整理自 MOCCA 2026 (角色动画与运动仿真) 课件 Lecture 06: Learning-based Character Animation (Libin Liu, SIST @ PKU)，结合以下原始论文：Grochow et al. SIGGRAPH 2004；Min et al. SIGGRAPH 2009；Levine et al. SIGGRAPH 2012；Holden et al. SIGGRAPH 2015/2017/2020；Starke et al. SIGGRAPH 2018/2020/2022；Ling et al. 2021；Yao et al. 2022/2024；Jiang et al. 2023；Tseng et al. 2022；Ao et al. 2023；Peng et al. (DeepMimic) 2018, (AMP) 2021, (ASE) 2022；Aberman et al. 2020；Qin et al. 2022；Henter et al. 2020；Ho et al. (DDPM) 2020；Song et al. (Score SDE) 2021。

Lecture 07 —蒙皮与表情动画 (Skinning & Facial Animation)

[TIP] 学习指南

本节核心：把“骨骼姿态序列”翻译成“网格顶点位移”——这是角色动画最底层也最常被工程师直接使用的技术。主线分两条：– 蒙皮 (Skinning)：LBS 如何用加权刚体变换驱动顶点；为什么会产生糖果包裹伪影；DQS 如何用对偶四元数在 $SE(3)$ 上做内蕴混合修复它；PSD 如何用示例 + RBF 插值处理肌肉细节。– 表情动画 (Facial Animation)：Blendshapes 的线性叠加原理；面部身份与表情的分解；FACS 与 ARKit 标准；表演捕捉的优化建模；语音驱动。

前置知识：刚体变换 ($SO(3)$, $SE(3)$)、四元数 (SLERP)、PCA、RBF 插值。

重点掌握：1. LBS 公式推导与等价矩阵化形式 2. Candy-wrapper 伪影的几何根因（旋转矩阵线性混合不闭合）3. 对偶四元数 = 实部四元数 + ε 部四元数，单位化条件，与 $SE(3)$ 的对应 4. DQS 的 DLB 步骤：加权 → 归一化 → 变换 5. Blendshape 线性叠加；Morphable Face 的 ID + Exp 分解 6. SMPL = LBS (SSD) + Pose Blend Shapes (EBS)

0. 角色动画流水线总览



输入：静止网格 $\{v_i\}$ (Stanford Bunny 式的三角面片集合) + 骨架 (每根骨骼在绑定姿态下的位置/朝向) + 蒙皮权重 $\{w_{ij}\}$ 。输出：每帧每个顶点的新世界坐标 x'_i 。

[NOTE] 为什么需要蒙皮权重

骨骼是离散刚体，网格是连续曲面。关节处的顶点“骑在”两根骨骼之间——权重 w_{ij} 量化顶点 i 跟随骨骼 j 运动的程度，使过渡自然而非断裂。

1. Skinning Deformation 几何推导

1.1 单骨骼—刚体变换

骨骼 j 在 bind pose (绑定姿态/静止姿态) 下由 (o_j, Q_j) 描述： o_j 是骨骼原点 (关节位置)， $Q_j \in SO(3)$ 是骨骼朝向矩阵 (本讲用正交矩阵记号；也可用四元数)。当前帧运动后：

$$Q'_j = R Q_j, \quad o'_j = o_j + t \quad (1.1)$$

刚体上任一点 x （绑定姿态世界系）经此运动后的新位置：

$$x' = Q'_j Q_j^T (x - o_j) + o'_j \quad (1.2)$$

1.2 骨骼局部坐标 r_{ij}

定义 **bone-local 残差**（常量，bind pose 下算一次即固定）：

$$r_{ij} = Q_j^T (x_i - o_j) \quad (1.3)$$

则单骨骼变换化简为：

$$x' = Q'_j r_{ij} + o'_j \quad (1.4)$$

[NOTE] 直觉理解

r_{ij} 是顶点 i 在骨骼 j 局部坐标系内的偏移量——它在整个动画过程中不变。每帧只需把”骨骼带着它一起转”，即用新朝向 Q'_j 将 r_{ij} 投回世界系，再加新原点 o'_j 。

1.3 多骨骼加权—LBS 直接推导

每根骨骼对顶点 i 各给出一个”候选位置” $x'_{ij} = Q'_j r_{ij} + o'_j$ 。蒙皮取它们的加权平均：

$$x'_i = \sum_{j=1}^m w_{ij} (Q'_j r_{ij} + o'_j) \quad (1.5)$$

这就是 **Linear Blend Skinning (LBS)**，又称 **Skeleton Subspace Deformation (SSD)**。

2. Linear Blend Skinning (LBS / 线性混合蒙皮)

2.1 公式等价化简

将 $r_{ij} = Q_j^T (x_i - o_j)$ 代入 (1.5)，展开：

$$x'_i = \sum_{j=1}^m w_{ij} [Q'_j Q_j^T x_i + (o'_j - Q'_j Q_j^T o_j)] \quad (2.1)$$

令 $R_j = Q'_j Q_j^T \in SO(3)$, $t_j = o'_j - R_j o_j$, 得到相对 bind pose 的刚体变换 $T_j = (R_j, t_j) \in SE(3)$, 于是：

$$x'_i = \sum_{j=1}^m w_{ij}(R_j x_i + t_j) = \underbrace{\left(\sum_{j=1}^m w_{ij} R_j\right)}_{\text{混合旋转}} x_i + \underbrace{\sum_{j=1}^m w_{ij} t_j}_{\text{混合平移}} \quad (2.2)$$

[TIP] LBS 的三要素

1. Bind pose / rest pose (绑定姿态): $\{o_j, Q_j\}$ 与 $\{x_i\}$ 在制作时确定, 之后固定。2. Skinning weights (蒙皮权重): $w_{ij} \geq 0$, 通常约束 $\sum_j w_{ij} = 1$ (归一化), 描述顶点受哪些骨骼影响及程度。3. Skinning transformation (蒙皮变换): 每帧更新的 $\{Q'_j, o'_j\}$, 等价于 $\{T_j = (R_j, t_j)\}$ 。

2.2 LBS 的工程地位

- 广泛工业应用: 实时游戏引擎 (Unity, Unreal, Godot) 的默认蒙皮方案。
- GPU 友好: 矩阵/向量运算可直接在 vertex shader 中执行, 每顶点只需 4–8 根骨骼就够。
- 高效: $O(m)$ per vertex, m 通常为 4。

2.3 蒙皮权重设计

手动刷权重: 美术在 DCC 工具 (Maya, Blender) 中逐顶点涂色, 费时费力。

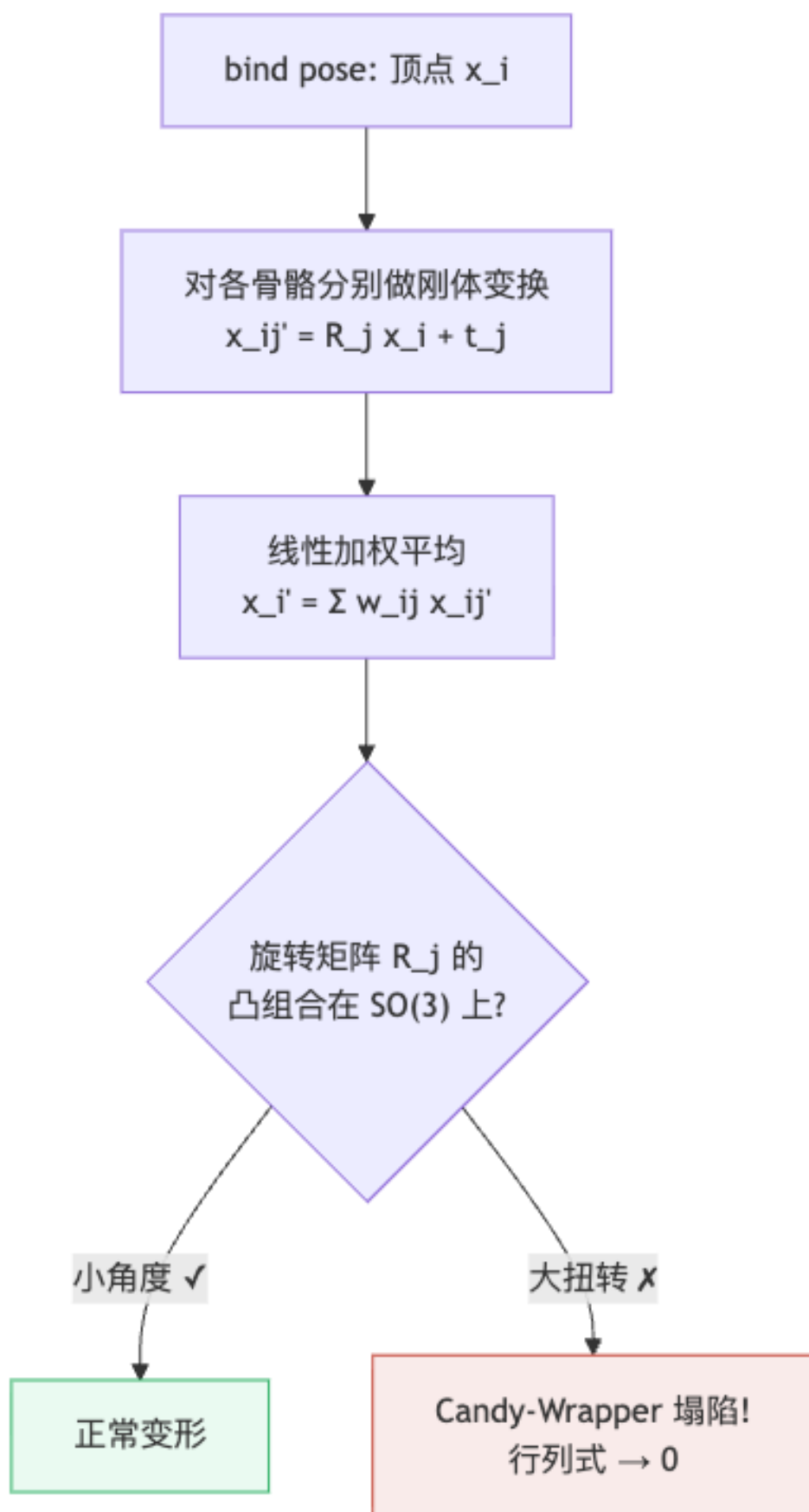
自动权重: – Pinocchio [Baran et al., 2007]: 自动骨架嵌入到 mesh, 并估算权重。– Bounded Biharmonic Weights [Jacobson et al., 2011]: 以双调和方程约束的平滑非负权重, 质量优于 LBS 手动权重。– 近年学习方法 (SIGGRAPH 2019, 2021): 用神经网络或隐式场预测权重。

2.4 Candy-Wrapper Artifact (糖果包裹 / 糖纸伪影)

[WARN] LBS 的根本缺陷—Candy-Wrapper

当关节扭转角度大 (例如前臂旋转 180°) 时, $\sum_j w_{ij} R_j$ 不再是旋转矩阵 (旋转矩阵的线性组合一般不在 $SO(3)$ 上), 其行列式可能趋近于 0, 导致顶点被“压扁”, 网格塌陷成类似扭过的糖果包装纸形状。

几何根因: $SO(3)$ 是非线性流形; 在 $\mathbb{R}^{3 \times 3}$ 的线性插值会偏离流形, 导致旋转矩阵丢失正交性 (体积不保持)。



3. Dual Quaternion Skinning (DQS / 对偶四元数蒙皮)

3.1 思路：在 $SE(3)$ 上做内蕴混合

LBS 在欧氏空间 $\mathbb{R}^{3 \times 4}$ 中线性组合变换——等价于把 $SE(3)$ 流形展开后插值，会引入不在流形上的点。理想方案是在 $SE(3)$ 上做 **intrinsic blending (内蕴均值)**：沿流形上的测地线插值。

对 $SO(3)$ ，单位四元数 + SLERP 已是优秀近似。**DQS 的核心思想**：把 $SE(3)$ 的元素（同时含旋转和平移）紧凑地表示为**单位对偶四元数**，再做类 SLERP 的线性归一化混合。

3.2 对偶数 (Dual Numbers)

$$x = a + b\varepsilon, \quad \varepsilon^2 = 0 \quad (3.1)$$

与复数 $x = a + bi$ ($i^2 = -1$) 类比： ε 的“零幂”性质使对偶数能编码一阶微分信息。

基本运算：- 共轭： $\bar{x} = a - b\varepsilon$ - 乘法： $(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon$ (ε^2 项消失)

3.3 对偶四元数 (Dual Quaternion)

将对偶数的两个分量换成四元数：

$$\hat{q} = q_0 + \varepsilon q_\varepsilon, \quad \varepsilon^2 = 0 \quad (3.2)$$

其中 q_0 (实部) 和 q_ε (对偶部) 都是普通四元数。

代数运算：

$$s\hat{q} = sq_0 + \varepsilon sq_\varepsilon \quad (3.3)$$

$$\hat{q}_1 + \hat{q}_2 = (q_{r1} + q_{r2}) + \varepsilon(q_{\varepsilon1} + q_{\varepsilon2}) \quad (3.4)$$

$$\hat{q}_1 \hat{q}_2 = q_{r1} q_{r2} + \varepsilon(q_{r1} q_{\varepsilon2} + q_{\varepsilon1} q_{r2}) \quad (3.5)$$

三种共轭：

$$\hat{q}^* = q_0^* + \varepsilon q_\varepsilon^* \quad (\text{四元数共轭}) \quad (3.6a)$$

$$\hat{q}^\circ = q_0 - \varepsilon q_\varepsilon \quad (\text{对偶共轭}) \quad (3.6b)$$

$$\hat{q}^\star = q_0^* - \varepsilon q_\varepsilon^* = (\hat{q}^*)^\circ = (\hat{q}^\circ)^* \quad (\text{组合共轭}) \quad (3.6c)$$

模：

$$\|\hat{q}\| = \sqrt{\hat{q}^* \hat{q}} = \|q_0\| + \varepsilon \frac{q_0 \cdot q_\varepsilon}{\|q_0\|} \quad (3.7)$$

3.4 单位对偶四元数

$$\|\hat{q}\| = 1 \iff \|q_0\| = 1 \text{ 且 } q_0 \cdot q_\varepsilon = 0 \quad (3.8)$$

单位对偶四元数构成集合 $\hat{Q}$ ，与 $SE(3)$ 一一对应（差一个双重覆盖）。

3.5 单位对偶四元数 $\leftrightarrow$ 刚体变换

任意刚体变换 $T = (R \mid t) \in SE(3)$ 转换为单位对偶四元数：

$$q_0 = r \quad (\text{旋转 } R \text{ 对应的单位四元数}) \quad (3.9a)$$

$$q_\varepsilon = \frac{1}{2} t r \quad (t = (0, t_x, t_y, t_z) \text{ 为平移的纯四元数}) \quad (3.9b)$$

向量变换：

$$\hat{v}' = \hat{q} \hat{v} \hat{q}^*, \quad \hat{v} = 1 + \varepsilon(0, v) = (1, 0) + \varepsilon(0, v_x, v_y, v_z) \quad (3.10)$$

[WARN] 双重覆盖 (Double Cover)

$\hat{q}$ 和 $-\hat{q}$ 表示同一刚体变换， $\hat{Q}$ 是 $SE(3)$ 的二重覆盖（类似单位四元数对 $SO(3)$ ）。做线性混合时若两个对偶四元数处于“不同半球”，直接加权会跨越覆盖引发大错误（等价于 SLERP 的短弧/长弧问题）。**修正**：混合前确保 $\hat{q}_j \cdot \hat{q}_{\text{ref}} \geq 0$ ，否则取反。

3.6 DLB 与 DQS

Dual-Quaternion Linear Blending (DLB)——两变换插值：

$$\hat{q}_t = \frac{(1-t)\hat{q}_0 + t\hat{q}_1}{\|(1-t)\hat{q}_0 + t\hat{q}_1\|} \quad (3.11)$$

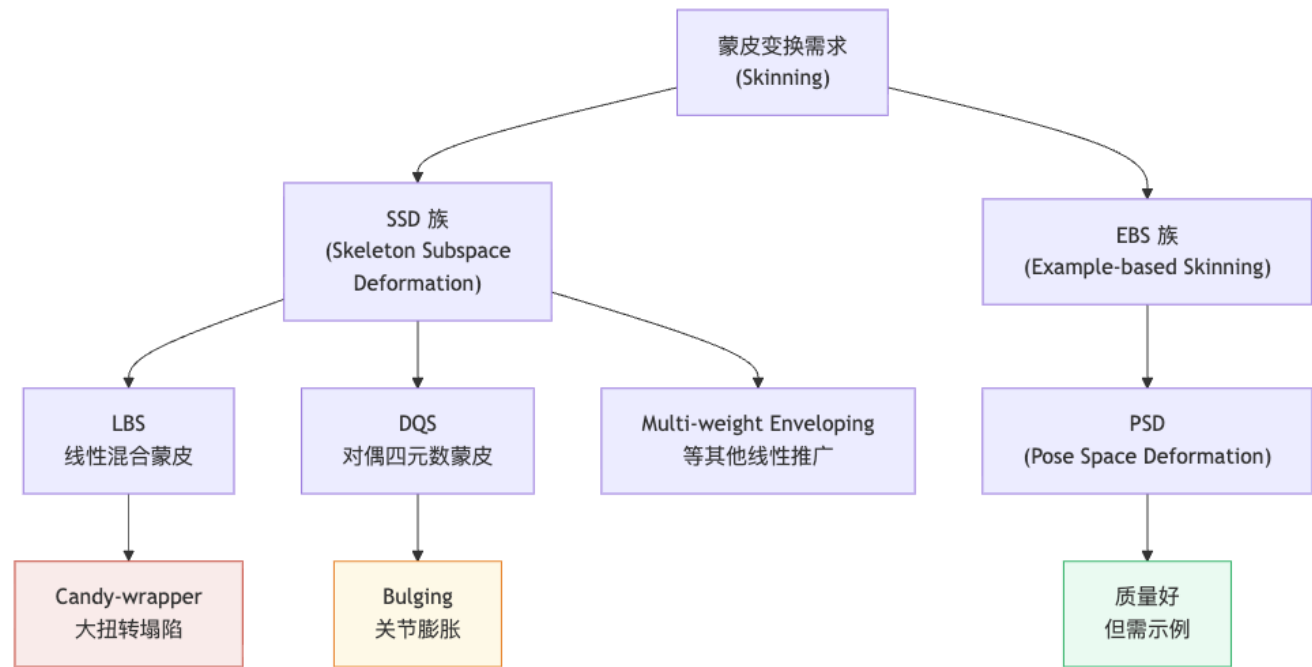
推广到多骨骼得到 DQS：

$$\hat{q}_i = \frac{\sum_{j=1}^m w_{ij} \hat{q}_j}{\left\| \sum_{j=1}^m w_{ij} \hat{q}_j \right\|}, \quad x'_i \text{ 从 } \hat{v}' = \hat{q}_i \hat{v}_i \hat{q}_i^* \text{ 中提取} \quad (3.12)$$

[NOTE] DQS 性质

– 修复 **candy-wrapper**：对偶四元数混合在 $\hat{Q}$ 的“线性近似”上操作，然后归一化回流形，保持体积。– 仍是近似：DLB 是 $SE(3)$ 内蕴均值的良好近似（Kavan et al. 2008），但不精确。– **Bulging artifact**（鼓包伪影）：骨骼弯曲时关节处轻微膨胀——比 candy-wrapper 视觉上可接受，但仍是缺陷。

4. 蒙皮方法对比



方法	变换空间	核心公式	优点	主要缺陷
LBS	$\mathbb{R}^{3 \times 4}$	$\sum_j w_{ij}(R_j x_i + t_j)$	快、GPU 友好、易实现	Candy-wrapper (大扭转塌陷)
DQS	$\hat{Q}$ (双重覆盖 $SE(3)$)	加权 $\hat{q}$ 后归一化再变换	保体积、修复扭转	Bulging artifact; 计算略贵
PSD	LBS + 姿态偏移	$SKIN(x) + \delta x(\theta)$	肌肉细节、高质量	需示例、额外存储、插值调参

5. Pose Space Deformation (PSD)

5.1 动机：修正 LBS 的系统误差

PSD [Lewis et al., SIGGRAPH 2000] 在 LBS 基础上加入**姿态相关形变修正**：

$$x'_i = \sum_{j=1}^m w_{ij} T_j x_i + \delta x_i(\theta) \quad (5.1)$$

其中 θ 是当前关节配置（角度向量）， $\delta x_i(\theta)$ 是待插值的“修正位移”，捕捉肌肉鼓胀、皮肤皱褶等姿态相关细节。

5.2 RBF 插值——求 δx

给定 K 个示例姿态下手工/扫描得到的修正形状 $\{(\theta_k, \delta x_k)\}_{k=1}^K$ ，用 Radial Basis Function (RBF) 插值：

$$\delta x(\theta) = \sum_{k=1}^K c_k \varphi(\|\theta - \theta_k\|) \quad (5.2)$$

权重 c 由插值条件 $\delta x(\theta_k) = \delta x_k$ 确定，即解线性方程组：

$$\underbrace{\begin{pmatrix} R_{11} & \cdots & R_{1K} \\ \vdots & \ddots & \vdots \\ R_{K1} & \cdots & R_{KK} \end{pmatrix}}_{R, R_{ij}=\varphi(\|\theta_i-\theta_j\|)} \begin{pmatrix} c_1 \\ \vdots \\ c_K \end{pmatrix} = \begin{pmatrix} \delta x_1 \\ \vdots \\ \delta x_K \end{pmatrix} \quad (5.3)$$

常用基函数 φ ：

名称	公式
高斯	$\varphi(r) = e^{-r^2/c^2}$
反多重二次	$\varphi(r) = 1/\sqrt{r^2 + c^2}$
薄板样条	$\varphi(r) = r^2 \log r$
多调和样条	$\varphi(r) = r^k \ (k \text{ 奇}), r^k \log r \ (k \text{ 偶})$

5.3 PSD 设计抉择

[WARN] PSD 三大设计维度

1. **粒度**：整体形状插值 (per-shape) vs 逐顶点插值 (per-vertex)? 2. **局部 vs 全局**：顶点是否受所有关节影响? (局部更可控、全局更平滑) 3. **插值算法**：RBF 是否够用? (神经网络等非线性方法可代替)

5.4 EBS vs SSD 的综合对比

类别	代表方法	优点	缺点
EBS (示例驱动)	PSD	易控制；质量好；可捕捉肌肉/皱褶等姿态细节	创建示例耗时；额外存储；插值需调参
SSD (骨骼子空间)	LBS, DQS	易实现；快；GPU 友好	各种伪影；权重难调；无法自动产生姿态细节

6. SMPL 人体模型

6.1 概述

SMPL = Skinned Multi-Person Linear Model，是机器学习与计算机视觉领域最广泛使用的人体参数化模型。核心贡献：把 SSD (LBS) 与 EBS (PSD 思想) 统一，通过大规模扫描数据学习形状与姿态基。

6.2 PCA 形状基

对扫描到的人体网格数据集 $\{x_i\} \subset \mathbb{R}^N$ ，PCA 给出：

$$x_i = \bar{x} + \sum_{k=1}^n w_{i,k} u_k \quad (6.1)$$

其中 u_k 是协方差矩阵 $\Sigma = X^T X$ 的第 k 大特征向量（主成分）， $w_{i,k} = (x_i - \bar{x}) \cdot u_k$ 是投影系数。

SMPL 的形状模板（体型参数 β ）：

$$T(\beta) = \bar{T} + \sum_{m=1}^{|\beta|} \beta_m S_m \quad (6.2)$$

β 的每个维度对应一个 PCA 主成分（矮/高/胖/瘦/肌肉发达等体型变化）。

6.3 Pose Blend Shapes（姿态混合形状）

LBS 的 candy-wrapper 在 SMPL 中用姿态相关线性修正弥补：

$$T(\beta, \theta) = \bar{T} + \sum_m \beta_m S_m + \sum_n \theta_n P_n \quad (6.3)$$

θ 是关节角向量， P_n 是通过对大量动作数据优化得到的“姿态主成分”（类 PSD，但不用 RBF，而是线性的 θ_n 系数）。

6.4 最终变形

$$x = \text{SKIN}(T(\beta, \theta), \theta, \mathcal{W}) \quad (6.4)$$

$\mathcal{W}$ 是学习到的蒙皮权重，SKIN 通常取 LBS 或 DQS。

[TIP] SMPL 的工程精髓

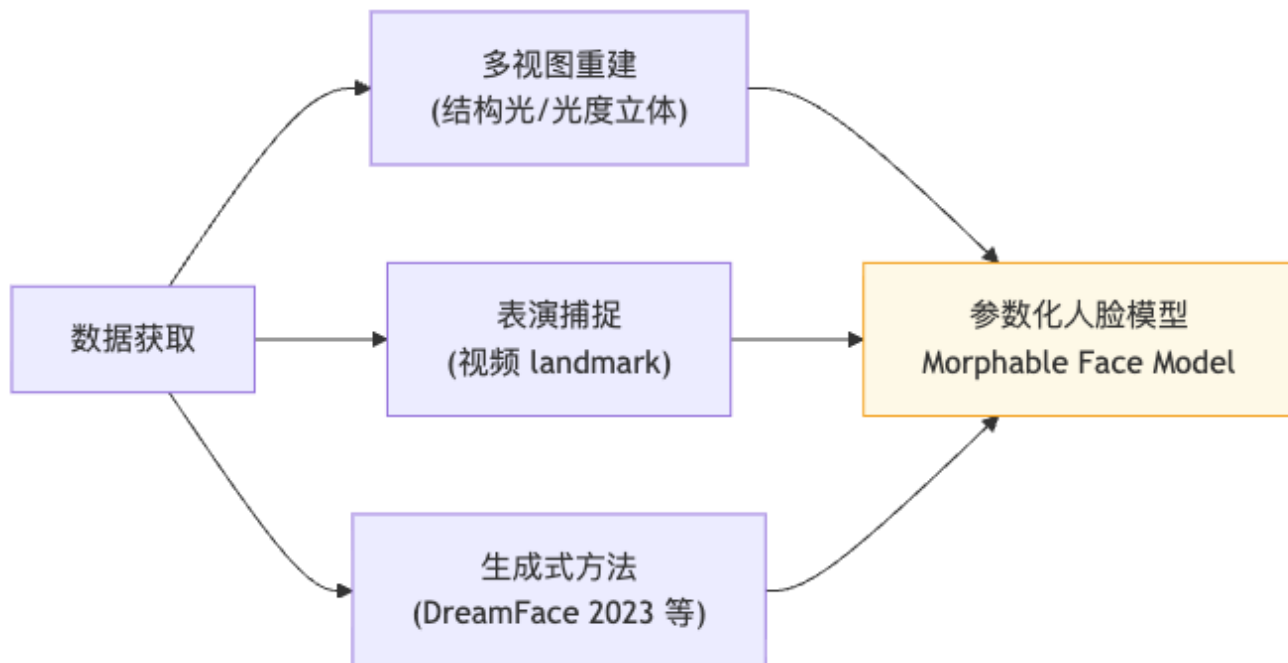
- 体型 β 压缩在 10–300 维 PCA 空间：几十个数描述任意体型。
- 姿态修正线性于 θ ：与 LBS 一起可微，适合梯度优化。
- 兼容现有引擎：最终走 LBS，能直接挂进游戏/动画软件。

7. Facial Animation（面部动画）

7.1 人脸的特殊性

面部动画比身体动画难的根本原因：面部没有清晰刚体骨骼支撑（皮肤直接附着在肌肉上），传统 LBS 骨骼驱动不够精细；同时面部表情语义丰富，需要系统化编码。

7.2 人脸模型获取



代表性数据采集：**Light Stage** (USC ICT) 多视角多光照同步拍摄；近年有 HACK [SIGGRAPH 2023]——头颈一体参数化模型。

7.3 ID + Expression 分解

[NOTE] Morphable Face Model 核心公式

人脸网格 X 由平均脸加上身份基和表情基的线性叠加表示：

$$X = X_0 + \underbrace{\sum_i \beta_i B_i^{\text{ID}}}_{\text{Identity (脸型定制)}} + \underbrace{\sum_j \theta_j B_j^{\text{Exp}}}_{\text{Expression (表情)}} \quad (7.1)$$

- X_0 : 平均脸 (average face), 所有人脸的均值网格。
- $\{B_i^{\text{ID}}\}$: 身份基 (Identity Blendshapes), 由 PCA 从大量人脸扫描学得, 控制五官形状、脸型宽窄等。
- $\{B_j^{\text{Exp}}\}$: 表情基 (Expression Blendshapes), 控制眉毛、嘴唇、眼睑等肌肉动作。

7.4 Blendshapes (混合形状)

Blendshape 把”特定动作姿态下的顶点位移网格”作为基, 任意表情通过线性叠加得到:

$$X = \sum_i w_i B_i \quad (7.2)$$

每个 B_i 是一个”形状” (存储绝对顶点位置, 或相对 rest pose 的位移); $w_i \in [0, 1]$ 是混合权重。

[NOTE] Blendshapes 与 Example-based Skinning 的关系

两者本质相同：都是“在若干示例形状上做插值”。区别在于：– Blendshape 通常不挂骨骼，直接对网格做线性叠加（适合面部小幅形变）。– PSD 在 LBS 骨骼驱动之上额外加修正量（适合身体关节处）。

7.5 ARKit 的 52 个标准 Blendshapes

Apple ARKit 定义了 52 个语义 blendshape 通道，覆盖人脸全部运动单元，是业界事实标准（iPhone TrueDepth 相机实时输出）：

眼部 (14 个): eyeBlinkLeft/Right、eyeLookDown/In/Out/Up Left/Right、eyeSquint/Wide Left/Right

下颌 (4 个): jawForward、jawLeft、jawRight、jawOpen

嘴部 (24 个): mouthClose、mouthFunnel、mouthPucker、mouthRight/Left、mouthSmile/Frown Left/Right、mouthDimple Left/Right、mouthStretch Left/Right、mouthRollLower/Upper、mouthShrugLower/Upper、mouthPress Left/Right、mouthLowerDown Left/Right、mouthUpperUp Left/Right

眉部 (5 个): browDownLeft/Right、browInnerUp、browOuterUpLeft/Right

其他 (5 个): cheekPuff、cheekSquintLeft/Right、noseSneerLeft/Right、tongueOut

7.6 FACS 与 Blendshapes 的关系

FACS (Facial Action Coding System, 面部动作编码系统) 是心理学家 Paul Ekman 提出的标准化人脸肌肉动作描述体系，定义 46 个 Action Units (AU)，如 AU1 (内眉上抬)、AU12 (嘴角拉起/微笑) 等。ARKit 的 52 个 blendshape 可看作 FACS 的工程实现与扩展——把抽象肌肉动作映射成具体几何位移基。

[TIP] FACS 在面部动画中的地位

表演者可用 FACS AU 来标注情绪状态；动画系统把 AU 转换为 blendshape 权重；反向（捕捉图像 → 估计 AU/权重）即为表演捕捉。FACS 提供了跨模型、跨引擎的统一语义层。

7.7 Morphable Face Model 的工业落地

- 3DMM [Egger et al. 2020 综述]: 基于大规模人脸扫描的统计形状模型，是现代参数化人脸的奠基工作。
- MetaHuman (Unreal Engine): 高保真可驱动人脸，内置 ID + Exp blendshape 系统，配合面部捕捉实现真人级数字分身。

7.8 表演捕捉 (Facial Performance Capture)

输入：视频帧中的人脸 landmark / 多视图点云 Y 。

优化目标——求最优身份/表情参数 (β_i, θ_j) :

$$\min_{\beta_i, \theta_j} \left\| X_0 + \sum_i \beta_i B_i^{\text{ID}} + \sum_j \theta_j B_j^{\text{Exp}} - Y \right\|^2 + E(\beta_i, \theta_j) \quad (7.3)$$

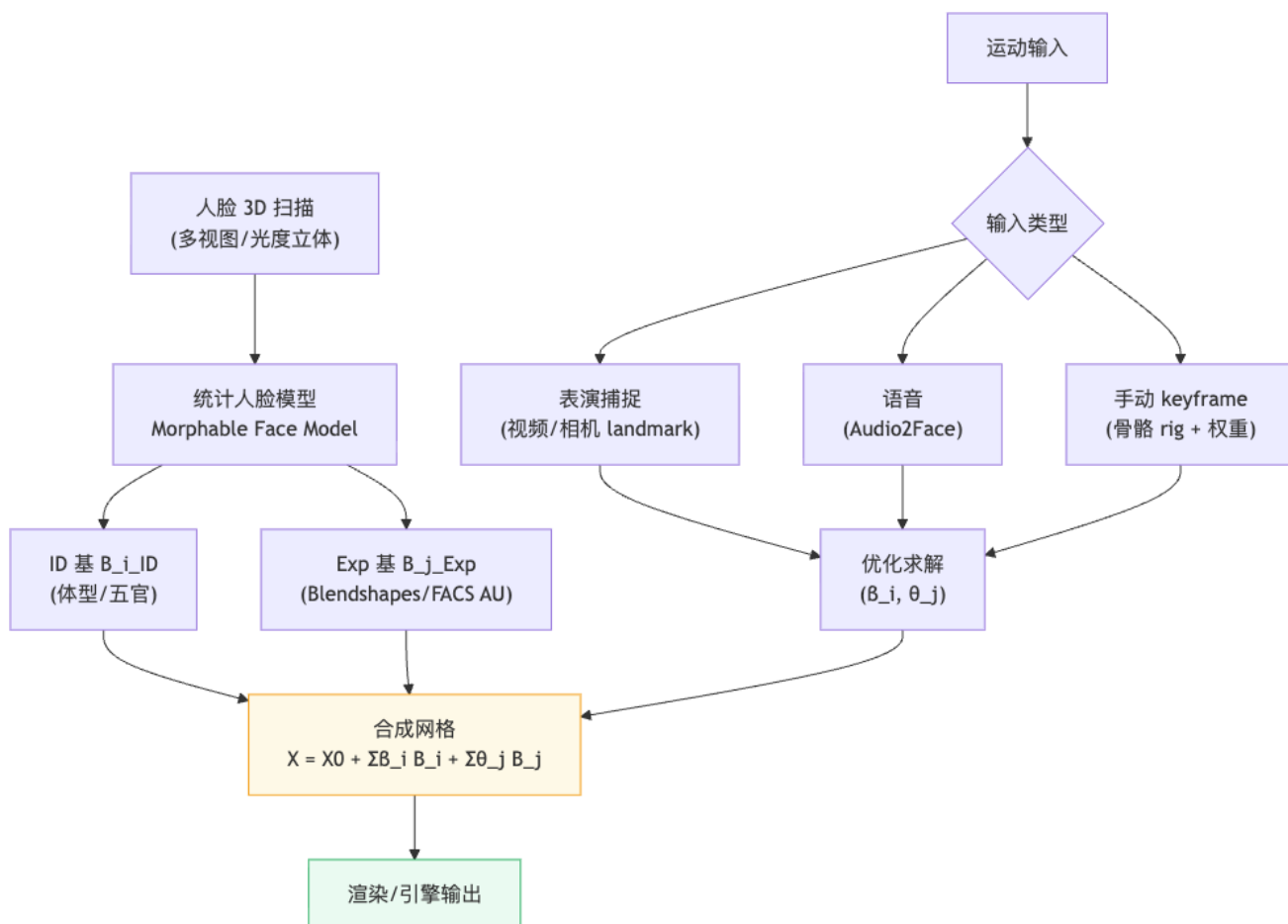
正则项 E 可包含: β 接近先验 (正态分布)、 θ 时间平滑、物理约束等。

工业级实现：– Google ML Kit: 手机端实时面部 landmark 检测 (468 点 3D landmark)。– Apple ARKit: 实时输出 52 个 blendshape 权重。– Industrial Light & Magic (ILM): 电影级多相机绑定全脸捕捉系统。

7.9 语音驱动面部动画 (Speech-driven Facial Animation)

输入音频/语音信号，直接预测 blendshape 权重时间序列（无需手工 keyframe）。代表系统：**NVIDIA Omniverse Audio2Face**——实时 LSTM/神经网络将语音编码特征映射到口型与情绪 blendshape，同时支持情绪控制。

8. 面部动画完整流水线



9. 总结：核心公式速查

[TIP] 期末复习要点

| 方法 | 核心公式 | 关键点 | |——|——|——| | **LBS** | $x'_i = (\sum_j w_{ij} R_j) x_i + \sum_j w_{ij} t_j$ | 旋转线性混合 → Candy-wrapper | | **DQS** | $\hat{q}_i = \frac{\sum w_{ij} \hat{q}_j}{\|\sum w_{ij} \hat{q}_j\|}$, x' 由 $\hat{q}_i \hat{v} \hat{q}_i^*$ 提取 | 单位对偶四元数; 修复扭转但有鼓包 | | **对偶四元数转换** | $q_0 = r$, $q_\varepsilon = \frac{1}{2} tr$ | t 为纯四元数 | | **PSD** | $x' = \text{SKIN}(x) + \delta x(\theta)$, δ 用 RBF | 示例 + 插值; 姿态细节 | | **SMPL** | $T(\beta, \theta) = \bar{T} + \sum \beta_m S_m + \sum \theta_n P_n$ + LBS | SSD + EBS 统一 | | **Morphable Face** | $X = X_0 + \sum \beta_i B_i^{\text{ID}} + \sum \theta_j B_j^{\text{Exp}}$ | ID + Exp 分解 | | **表演捕捉** | $\min \|X(\beta, \theta) - Y\|^2 + E(\beta, \theta)$ | 逆向求参数 |

10. Worked Examples (例题)

[EX] 例题 1: 推导 LBS 矩阵形式

问: 设有两根骨骼, 顶点 i 在 bind pose 下坐标为 $x_i = (1, 0, 0)^T$, 蒙皮权重 $w_{i1} = 0.6$, $w_{i2} = 0.4$. 骨骼 1 相对 bind pose 绕 z 轴转 90° 、无平移; 骨骼 2 无旋转、平移 $(0, 1, 0)^T$. 求 x'_i .
解:

$$R_1 = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad t_1 = 0; \quad R_2 = I, \quad t_2 = (0, 1, 0)^T$$

$$x'_i = 0.6(R_1 x_i + t_1) + 0.4(R_2 x_i + t_2)$$

$$= 0.6 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + 0.4 \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.4 \\ 1.0 \\ 0 \end{pmatrix}$$

注意: 混合后“旋转部分” $0.6R_1 + 0.4R_2$ 不是正交矩阵 (行列式 $\neq 1$), 这正是 LBS 的问题根源。

[EX] 例题 2：单位对偶四元数的验证

问：设旋转 R 为绕 z 轴转 90° ，对应单位四元数 $r = (\cos 45^\circ, 0, 0, \sin 45^\circ) = \frac{\sqrt{2}}{2}(1, 0, 0, 1)$ ；平移 $t = (2, 0, 0)$ （纯四元数 $(0, 2, 0, 0)$ ）。

求对偶四元数 $\hat{q}$ 并验证单位性。

解：

$$q_0 = r = \frac{\sqrt{2}}{2}(1, 0, 0, 1)$$

$$q_\epsilon = \frac{1}{2}tr = \frac{1}{2}(0, 2, 0, 0) \cdot \frac{\sqrt{2}}{2}(1, 0, 0, 1)$$

四元数乘法（纯四元数 $(0, a) \times$ 单位四元数 (w, b) ）：

$$= \frac{\sqrt{2}}{2}(-a \cdot b, wa + a \times b) = \frac{\sqrt{2}}{2}(0, 0, 2, 0) \cdot \frac{1}{2} = \frac{\sqrt{2}}{4}(0, 0, 2, 0) = \frac{\sqrt{2}}{2}(0, 0, 1, 0)$$

$$\text{验证：} \|q_0\| = 1 \quad ; \quad q_0 \cdot q_\epsilon = \frac{\sqrt{2}}{2} \cdot \frac{\sqrt{2}}{2}(1 \cdot 0 + 0 \cdot 0 + 0 \cdot 1 + 1 \cdot 0) = 0 \quad .$$

[EX] 例题 3：Candy-Wrapper 的几何理解

问：为什么 LBS 在关节扭转 180° 时会让网格体积趋近于零？给出数学解释。

解：设两根骨骼， $w_{i1} = w_{i2} = 0.5$ ，骨骼 1 旋转 $R_1 = I$ ，骨骼 2 旋转 R_2 （绕某轴转 180° ）。

混合矩阵 $M = 0.5I + 0.5R_2$ 。当旋转轴为 z ：

$$R_2 = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad M = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$\det(M) = 0$ ！顶点被投影到一维（ z 轴）， xy 平面上所有信息丢失——这就是“塌陷”的代数表现。DQS 通过先在 $\hat{Q}$ 上混合、再归一化回 $SE(3)$ ，避免了这一退化。

[EX] 例题 4：表演捕捉优化

问：设人脸 Morphable Model 有 10 个 ID 基和 52 个 Exp 基，检测到 68 个 2D landmark 点 $Y \in \mathbb{R}^{68 \times 2}$ 。写出捕捉优化问题的完整形式。

解：设 Π 为相机投影函数（已知内参），正则项采用 L2：

$$\min_{\{\beta_i\}_{i=1}^{10}, \{\theta_j\}_{j=1}^{52}} \sum_{k=1}^{68} \left\| \Pi \left(X_0 + \sum_i \beta_i B_i^{\text{ID}} + \sum_j \theta_j B_j^{\text{Exp}} \right)_k - Y_k \right\|^2 + \lambda_\beta \|\beta\|^2 + \lambda_\theta \|\theta\|^2$$

共 62 个未知数， $68 \times 2 = 136$ 个方程约束（超定），正则项防止过拟合（物理上： β, θ 不要离先验太远）。通常用 Gauss-Newton 或 L-BFGS 迭代求解，每步需对 $X(\beta, \theta)$ 关于参数求雅可比。

11. Anki 记忆卡片

#	Q (正面)	A (背面)
1	LBS 的完整公式 (用 R_j, t_j)?	$x'_i = (\sum_j w_{ij} R_j) x_i + \sum_j w_{ij} t_j$, 其中 $R_j = Q'_j Q_j^T$, $t_j = o'_j - R_j o_j$
2	bind pose (绑定姿态) 中 bone-local 残差 r_{ij} 如何定义?	$r_{ij} = Q_j^T (x_i - o_j)$, 即顶点在骨骼局部坐标系中的偏移, 动画时固定不变
3	Candy-Wrapper 伪影的几何根因?	旋转矩阵的线性混合 $\sum w_j R_j$ 不在 $SO(3)$ 上, 行列式可趋近 0, 导致网格塌缩
4	对偶数的定义与关键性质?	$x = a + b\varepsilon$, $\varepsilon^2 = 0$; 乘法: $(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon$
5	对偶四元数表示刚体变换的公式?	$\hat{q} = q_0 + \varepsilon q_\varepsilon$, 其中 $q_0 = r$ (旋转四元数), $q_\varepsilon = \frac{1}{2} tr$ (t 为纯四元数平移)
6	单位对偶四元数的两个条件?	$\ q_0\ = 1$ 且 $q_0 \cdot q_\varepsilon = 0$
7	DQS 的计算步骤 (三步)?	① 对各骨骼对偶四元数 $\hat{q}_j$ 做加权求和; ② 归一化 (除以模) 得 $\hat{q}_i$; ③ 用 $\hat{v}' = \hat{q}_i \hat{v} \hat{q}_i^*$ 变换顶点
8	DQS 相比 LBS 解决了什么, 引入了什么新问题?	解决: Candy-wrapper (大扭转塌陷); 引入: Bulging artifact (关节处轻微鼓包)
9	PSD 的公式及 δx 如何求?	$x' = \sum w_{ij} T_j x_i + \delta x(\theta)$; δx 用 RBF 对已知示例插值: $\delta x(\theta) = \sum_k c_k \varphi(\ \theta - \theta_k\)$
10	SMPL 模型的两类形状参数及各自作用?	β (体型): PCA 形状基 S_m , 控制高矮胖瘦; θ (姿态): Pose Blend Shapes P_n , 弥补 LBS 的姿态伪影
11	Morphable Face Model 的三项组成?	$X = X_0$ (平均脸) + $\sum \beta_i B_i^D$ (身份定制) + $\sum \theta_j B_j^{Exp}$ (表情 blendshape)
12	Blendshape 与 FACS 的关系?	FACS (面部动作编码系统) 定义 46 个肌肉动作单元 (AU); Blendshape 是 FACS 的几何实现, 把抽象 AU 映射为具体网格位移基; ARKit 52 个 blendshape 是业界标准实现
13	表演捕捉的优化形式?	$\min_{\beta, \theta} \ X(\beta, \theta) - Y\ ^2 + E(\beta, \theta)$, Y 为观测 landmark, E 为正则项
14	EBS 与 SSD 的主要权衡?	EBS (PSD): 质量好/细节丰富, 但需示例、存储大; SSD (LBS/DQS): 快/GPU 友好, 但有伪影/难做姿态细节
15	双重覆盖对 DQS 的影响及修正方法?	$\hat{q}$ 与 $-\hat{q}$ 代表同一变换; 混合前若 $\hat{q}_j \cdot \hat{q}_{ref} < 0$ 则取反 $\hat{q}_j \leftarrow -\hat{q}_j$, 保证都在同一半球

12. 中英术语速查表

中文	English	简要说明
蒙皮 绑定 / 骨骼绑定	Skinning Rigging	把骨骼运动驱动到网格顶点的技术 为角色网格创建骨架控制结构

中文	English	简要说明
绑定姿态 / 静止姿态	Bind pose / Rest pose	制作时确定的参考姿态，权重在此姿态下计算
蒙皮权重	Skinning weights	w_{ij} ，顶点受各骨骼影响程度
线性混合蒙皮	Linear Blend Skinning (LBS)	最常用的实时蒙皮方法
骨骼子空间变形	Skeleton Subspace Deformation (SSD)	LBS/DQS 的统称
糖果包裹伪影	Candy-wrapper artifact	LBS 大扭转时的体积塌缩缺陷
对偶数	Dual number	$a + b\varepsilon, \varepsilon^2 = 0$
对偶四元数	Dual quaternion	编码 $SE(3)$ 刚体变换的紧凑表示
对偶四元数蒙皮	Dual Quaternion Skinning (DQS)	修复 candy-wrapper 的非线性蒙皮
对偶四元数线性混合	Dual Quaternion Linear Blending (DLB)	DQS 中对对偶四元数做加权归一化
鼓包伪影	Bulging artifact	DQS 在关节弯曲处的轻微膨胀缺陷
姿态空间变形	Pose Space Deformation (PSD)	LBS + 示例插值修正，处理肌肉细节
示例驱动蒙皮	Example-based Skinning (EBS)	基于示例形状插值的蒙皮方法
径向基函数	Radial Basis Function (RBF)	PSD 中常用的散点插值方法
混合形状	Blendshape	线性叠加的形状位移基，用于面部动画
混合空间	Blend space	Blendshape 权重参数空间
形状混合空间	Blend space / Shape space	示例形状的权重参数空间
面部动作编码系统	FACS (Facial Action Coding System)	心理学肌肉动作单元标准，46 个 AU
动作单元	Action Unit (AU)	FACS 定义的基本面部肌肉动作
可变形面部模型	Morphable Face Model (3DMM)	基于 PCA 的参数化人脸统计模型
身份基	Identity blendshapes	描述脸型/五官形状的 PCA 基
表情基	Expression blendshapes	描述面部表情动作的基向量
表演捕捉	Facial performance capture	从视频/图像恢复面部动画参数
语音驱动	Speech-driven animation	从音频直接生成面部动画
主成分分析	Principal Component Analysis (PCA)	提取数据主成分/降维方法
内蕴混合	Intrinsic blending	在流形内（沿测地线）做插值
双重覆盖	Double cover	对偶四元数与 $SE(3)$ 的二重覆盖关系

13. 复习要点

[TIP] 核心结论 (5 条必记)

1. **LBS 推导**：从“单骨骼刚体”→“bone-local 残差”→“多骨骼加权”→“矩阵化”，最终 $x'_i = (\sum_j w_{ij} R_j) x_i + \sum_j w_{ij} t_j$ 。推导是本讲最重要的手推技能。2. **Candy-wrapper 根因**：旋转矩阵是非线性流形 $SO(3)$ 上的元素，线性混合 $\sum w_j R_j$ 偏离流形（行列式可趋 0）→ 体积塌缩。DQS 用在 $\hat{Q}$ 上线性混合 + 归一化回避了这一问题。3. **对偶四元数 = 旋转 + 平移的紧凑编码**： $q_0 = r$ （旋转）， $q_\varepsilon = \frac{1}{2}tr$ （平移编码进对偶部）。单位条件： $\|q_0\| = 1, q_0 \cdot q_\varepsilon = 0$ 。4. **Blendshape 线性叠加**： $X = X_0 + \sum \beta_i B_i^{\text{ID}} + \sum \theta_j B_j^{\text{Exp}}$ ，ID 控制体型，Exp 控制表情（FACS AU 的几何实现）。5. **SMPL = SSD + EBS**：形状用 PCA (β) 压缩，姿态伪影用 Pose Blend Shapes (θ) 修正，最终接 LBS——这是“合理性”与“兼容性”的最优折衷。

Lecture 08 —基于物理的仿真 (Simulation)

[TIP] 学习指南

本节核心：从牛顿第二定律出发，搭建一条完整的物理仿真管线——数值积分 → 单刚体动力学 → 铰接刚体约束 → 接触与摩擦建模。六个模块环环相扣，最终落脚于线性互补问题（LCP）和投影高斯-赛德尔（PGS）求解器。

前置知识：线性代数（旋转矩阵、四元数、叉积、特征分解）、常微分方程基础、Lec 03–07 刚体运动学。

重点掌握：1. 三种 Euler 积分的更新公式、稳定性分析（弹簧矩阵谱半径）2. $\dot{R} = \omega^\times R$ 的推导；四元数对应的 $\dot{q} = \frac{1}{2}\bar{\omega}q$ 3. Newton–Euler 方程 $I\dot{\omega} + \omega \times I\omega = \tau$ 的来源 4. 惯量张量 $I = \sum_i (-m_i r_i^{\times 2})$, 旋转变换 $I = R I_0 R^\top$ 5. 约束力 $f_c = J^\top \lambda$, Lagrange 乘子方程的推导与求解 6. 接触 LCP 的互补条件；摩擦锥线性化；PGS 迭代格式

0. Motivation：为什么需要物理仿真？

运动学方法（Lec 03–07）只能指定动作，无法回答“角色与环境真实交互后会发生什么”。一旦涉及跌倒、踢球、站在滑动平面上，纯关键帧或 IK 就会产生穿模、漂浮脚等失真。物理仿真把动作的合法性交给牛顿定律：给定外力、关节约束、接触条件，下一帧状态由动力学方程唯一决定。这条路线催生了基于物理的角色动画（Physics-based Character Animation），如 ControlVAE 等方法。

本节回答三个核心问题：

1. 给定 $f(x, v, t)$ ，如何稳定、高效地从 (x_n, v_n) 推进到 (x_{n+1}, v_{n+1}) ？
2. 刚体姿态 R 如何参与积分？ ω 与 $\dot{R}$ 是什么关系？
3. 多个刚体被关节连接、脚踩地面时，方程如何一次性求解？

1. 质点动力学与时间离散

1.1 连续问题

对质量 m 、位置 $x(t)$ 、速度 $v(t)$ 的质点，牛顿第二定律给出：

$$f = ma, \quad a = \dot{v}, \quad v = \dot{x} \implies a = \frac{f(x, v, t)}{m} \quad (1)$$

积分得速度和位置：

$$v(t) = v_0 + \int_{t_0}^t a \, dt, \quad x(t) = x_0 + \int_{t_0}^t v \, dt \quad (2)$$

只有当 f 恒为常量（如纯重力）时才有解析解 $x_t = x_0 + v_0 t + \frac{1}{2} a t^2$ ；一般情形（ f 依赖 x, v, t ）必须数值积分。

1.2 时间离散

设仿真步长为 h ，第 n 步时刻 $t_n = nh$ ，则

$$v_{n+1} = v_n + \int_{t_n}^{t_{n+1}} a \, dt, \quad x_{n+1} = x_n + \int_{t_n}^{t_{n+1}} v \, dt \quad (3)$$

用矩形近似替代积分，有三种取样时刻的选择，对应三种 Euler 格式。

2. 数值积分方法

2.1 三种 Euler 格式

显式 (Explicit/Forward) Euler:

$$v_{n+1} = v_n + a_n h, \quad x_{n+1} = x_n + v_n h \quad (4)$$

隐式 (Implicit/Backward) Euler:

$$v_{n+1} = v_n + a_{n+1} h, \quad x_{n+1} = x_n + v_{n+1} h \quad (5)$$

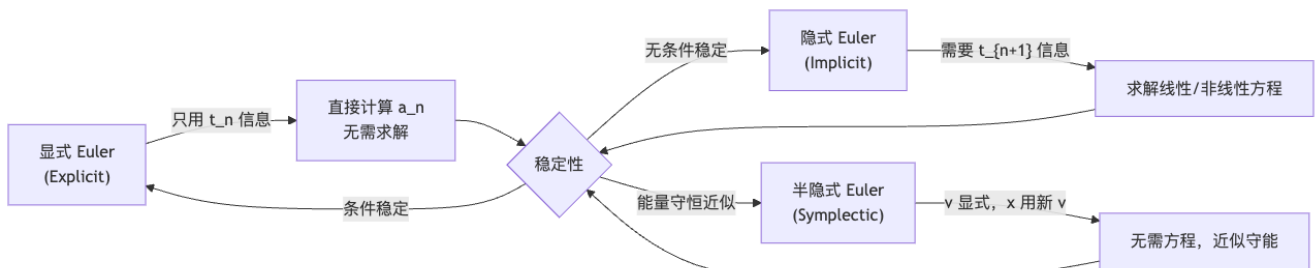
半隐式 / Symplectic Euler:

$$v_{n+1} = v_n + a_n h, \quad x_{n+1} = x_n + v_{n+1} h \quad (6)$$

式 (5) 的 $a_{n+1} = f(x_{n+1}, v_{n+1})/m$ 依赖未知量，展开得：

$$v_{n+1} = v_n + \frac{f(x_{n+1}, v_{n+1})}{m} h \quad (7)$$

这是关于 v_{n+1} 的（通常非线性的）方程，需要迭代或线性化求解。



2.2 弹簧-质点系统的稳定性分析

以 $f = -kx$ (弹簧恢复力), $\hat{k} = k/m$ 为例, 将三种格式写成状态转移矩阵形式:

$$\text{显式: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & -\hat{k}h \\ h & 1 \end{bmatrix}}_{A_{\text{exp}}} \begin{bmatrix} v_n \\ x_n \end{bmatrix} \quad (8)$$

$$\text{半隐式: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 - \hat{k}h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix} \quad (9)$$

$$\text{隐式: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \frac{1}{1 + \hat{k}h^2} \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix} \quad (10)$$

稳定性取决于转移矩阵谱半径 $\rho(A)$ (所有特征值的模的最大值):

格式	谱半径 ρ	稳定条件
显式 Euler	$\rho > 1$ (h 大时)	$h < 2/\sqrt{\hat{k}}$ (条件稳定)
半隐式 Euler	$\rho \approx 1$	$h < 2/\sqrt{\hat{k}}$, 能量近似守恒
隐式 Euler	$\rho < 1$	任意 h (无条件稳定, 但能量会耗散)

[WARN] 显式 Euler 不稳定的直觉

显式 Euler 用当前加速度预测未来位置, 步长过大时, 弹簧已经过平衡点很远, 算出的“推力”反而比实际更大, 导致每步振荡幅度越来越大, 能量发散。物理上, 显式 Euler 相当于在相空间沿椭圆“向外螺旋”。

[WARN] 隐式 Euler 的精度代价

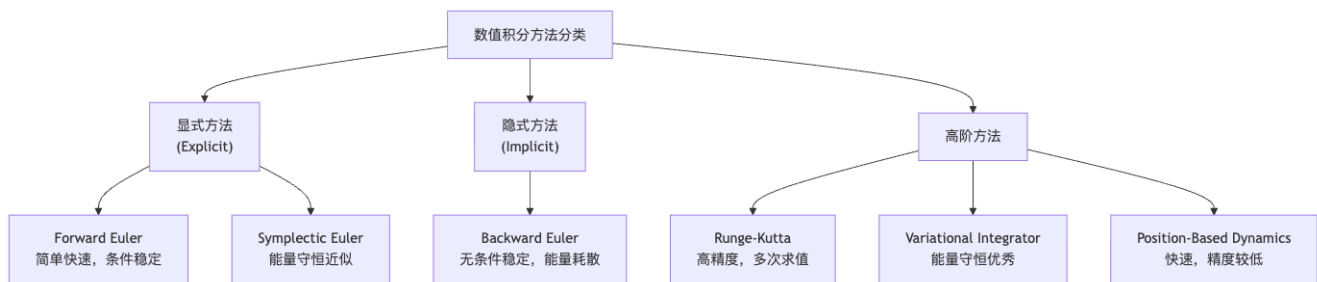
隐式格式在大步长下“过于平滑”——能量被耗散, 仿真看起来像在黏稠流体中运动。因此步长也不应无限制增大, 而是在“稳定但精度尚可”的范围内选取。

2.3 精度与步长的权衡

矩形近似的局部截断误差为 $O(h)$ (一阶精度)。减小 h 同时提升精度与稳定性, 但不能解决所有问题: 若速度很大、物体一步跨过薄墙, 再小的 h 也会漏检碰撞 (需要连续碰撞检测 CCD)。

2.4 更高阶积分方案

- **Runge-Kutta (RK2/RK4)**: 在 $[t_n, t_{n+1}]$ 上取 2 或 4 个采样点, 误差 $O(h^2)$ 或 $O(h^4)$, 计算量随之增加。
- **Variational Integrator**: 从离散最小作用量原理 (Discrete Least Action Principle) 推导, 长时间模拟能量守恒更好。
- **Position-Based Dynamics (PBD)**: 不对速度积分, 直接对位置做约束投影, 快速但物理精度较低。



3. 刚体运动学：位姿、速度与积分

3.1 位姿表示

刚体状态由质心位置 $x \in \mathbb{R}^3$ 和姿态矩阵 $R \in SO(3)$ 完全描述。刚体上任意点 x' （在物体坐标系中偏移为 r_0 ）的世界坐标为：

$$x' = x + Rr_0 = x + r \quad (11)$$

其中 $r = Rr_0$ 是在世界系下的瞬时偏移（随姿态旋转而变）。

3.2 角速度与 $\dot{R}$ 的关系推导

对刚体约束 $RR^\top = I$ 两边求时间导：

$$\dot{R}R^\top + R\dot{R}^\top = 0 \implies \dot{R}R^\top + (\dot{R}R^\top)^\top = 0 \quad (12)$$

式 (12) 表明 $\dot{R}R^\top$ 是一个反对称矩阵 (Skew-Symmetric Matrix)，可以写成：

$$\dot{R}R^\top = \omega^\times \equiv \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (13)$$

其中向量 $\omega = (\omega_x, \omega_y, \omega_z)^\top$ 正是角速度。由此得：

$$\boxed{\dot{R} = \omega^\times R} \quad (14)$$

几何直觉： R 的三列 (e_x, e_y, e_z) 是物体坐标轴的单位向量，每个都满足 $\dot{e}_i = \omega \times e_i$ ，合并即得 (14)。

也可从 Rodrigues 旋转公式推导：绕单位轴 u 旋转微角 $\delta\theta$ 时，

$$\delta x = \sin(\delta\theta) u \times x + (1 - \cos \delta\theta) u \times (u \times x)$$

取 $\delta\theta \rightarrow 0$ 的极限得 $\dot{x} = \dot{\theta} u \times x = \omega \times x$ ，与 (14) 一致。

3.3 刚体上任意点的速度

$$\dot{x}' = \dot{x} + \dot{R}r_0 = v + \omega \times Rr_0 = v + \omega \times r \quad (15)$$

运动学方程汇总：

$$\dot{x} = v, \quad \dot{R} = \omega^\times R \quad (16)$$

3.4 位姿的数值积分

方案 A (旋转矩阵)：

$$x' = x + \delta t \cdot v, \quad R' = R + \delta t \cdot \omega^\times R \quad (17)$$

问题：累计误差会使 R' 偏离 $SO(3)$ ，需要定期正交化（如 SVD 投影）。

方案 B (四元数)： R 用单位四元数 q 表示，其时间导为：

$$\dot{q} = \frac{1}{2}\bar{\omega}q, \quad \bar{\omega} = (0, \omega) \quad (18)$$

更新公式：

$$x' = x + \delta t \cdot v, \quad q' = q + \delta t \cdot \dot{q} \quad (19)$$

更新后需对 q' 归一化，以确保 $|q'| = 1$ 。

[NOTE] 四元数 vs 旋转矩阵的积分对比

旋转矩阵积分需正交化 (3×3 SVD)，四元数只需归一化 (1 个除法)，计算更高效。实际物理引擎 (Bullet、PhysX、MuJoCo) 均用四元数表示姿态。

4. 刚体动力学：Newton—Euler 方程

4.1 角动量与惯量张量

将刚体视为 N 个质点的集合，质心处的角动量为：

$$L = \sum_i m_i r_i \times v_i \quad (20)$$

其中 $r_i = x_i - x_c$ 是第 i 个质点相对质心的偏移。将 $v_i = v_c + \omega \times r_i$ 代入，利用 $\sum_i m_i r_i = 0$ (质心定义)，化简得：

$$L = \sum_i m_i r_i \times (\omega \times r_i) = \sum_i (-m_i r_i^{\times 2}) \omega \quad (21)$$

定义惯量张量 (Moment of Inertia Tensor)：

$$I = \sum_i (-m_i r_i^{\times 2}) \in \mathbb{R}^{3 \times 3} \quad (22)$$

其中 $r_i^{\times}$ 表示 r_i 的反对称矩阵（叉积矩阵）。于是：

$$L = I\omega \quad (23)$$

[NOTE] 惯量张量的物理含义

I 是 3×3 对称正定矩阵，描述刚体“对旋转的阻力”在各方向的分布。对角元 I_{xx}, I_{yy}, I_{zz} 叫**转动惯量**（绕对应轴转动的惯性），非对角元叫**惯性积**（反映质量分布不对称性）。同等质量不同形状的物体有不同的 I ——盘状体绕其法轴的转动惯量最大，细杆绕纵轴最小。

4.2 惯量张量的旋转变换

物体坐标系下的惯量张量 I_0 （常量，与姿态无关）与世界系下 I 的关系：

$$I = R I_0 R^\top \quad (24)$$

推导：世界系偏移 $r = R r_0$ ，代入 $r^{\times} = R r_0^{\times} R^\top$ ，则 $r^{\times 2} = R r_0^{\times 2} R^\top$ ，求和得 (24)。

4.3 主轴分解

由于 I_0 对称正定，可做特征分解：

$$I_0 = V \text{diag}(I_1, I_2, I_3) V^\top \quad (25)$$

选物体坐标系与特征向量对齐后， $I_0 = \text{diag}(I_1, I_2, I_3)$ （对角阵）， I_1, I_2, I_3 称为**主转动惯量**。这极大简化了运算，物理引擎通常只存三个主转动惯量。

4.4 Newton—Euler 方程

线动量 $p = Mv$ 的方程：

$$\dot{p} = f \implies M\dot{v} = f \quad (26)$$

角动量 $L = I\omega$ 的方程（Euler's Second Law）：

$$\dot{L} = \tau \quad (27)$$

展开 $\dot{L} = \dot{I}\omega + I\dot{\omega}$ ，利用 $\dot{I} = \omega^{\times} I - I\omega^{\times}$ （对 (24) 求导），化简得：

$$\boxed{I\dot{\omega} + \omega \times I\omega = \tau} \quad (28)$$

合并为矩阵形式——Newton—Euler 方程：

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix} \quad (29)$$

[NOTE] 陀螺力矩 $\{\boldsymbol{\omega}\} \times \{\boldsymbol{\omega}\}$

式 (28) 右边多出了一项 $\boldsymbol{\omega} \times I\boldsymbol{\omega}$, 称为**陀螺力矩 (gyroscopic torque)**。它来自于惯量张量本身随旋转变化的, 即使外力矩 $\boldsymbol{\tau} = 0$, 高速旋转的非球对称刚体也会因此产生进动 (如陀螺)。

4.5 单刚体的半隐式时间积分

离散化 (29), 以半隐式方式 (速度用 $n+1$ 时刻, 陀螺项用 n 时刻) 得:

$$\frac{1}{h} \begin{bmatrix} mI_3 & 0 \\ 0 & I_n \end{bmatrix} \begin{bmatrix} v_{n+1} - v_n \\ \omega_{n+1} - \omega_n \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_n \times I_n \omega_n \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix} \quad (30)$$

完整仿真一步 (对单刚体):

1. 由当前 R_n , 计算 $I_n = R_n I_0 R_n^\top$
2. 由 (30) 求解 v_{n+1}, ω_{n+1}
3. 更新位置: $x_{n+1} = x_n + h v_{n+1}$
4. 更新姿态: $q_{n+1} = q_n + \frac{h}{2} \bar{\omega}_{n+1} q_n$, 归一化

5. 铰接刚体: 关节约束与 Lagrange 乘子

5.1 多刚体系统的运动方程

N 个刚体的广义速度 $v = (v_1, \omega_1, \dots, v_N, \omega_N)^\top \in \mathbb{R}^{6N}$, Newton-Euler 方程 (无约束) 写成矩阵块对角形式:

$$\underbrace{\begin{bmatrix} m_1 I_3 & & \\ & I_1 & \\ & & \ddots \end{bmatrix}}_M \dot{v} + C(x, v) = f \quad (31)$$

其中 C 收集所有陀螺项。

5.2 约束的几何表示

两刚体通过关节 (铰接点 x_J) 连接, 约束要求:

$$g(x) = x_1 + R_1 r_{01} - x_2 - R_2 r_{02} = 0 \quad (32)$$

(两体上的关节附着点在世界系中重合)。对时间求导得**速度层约束 (Jacobian 形式)**:

$$Jv = 0, \quad J = \frac{\partial g}{\partial x} \quad (33)$$

具体展开:

$$J = [I_3 \quad -r_1^\times \quad -I_3 \quad r_2^\times] \quad (34)$$

其中 $r_k = R_k r_{0k}$ (当前关节相对各刚体质心的偏移), $r_k^\times$ 为叉积矩阵。

5.3 约束力与 Lagrange 乘子

约束是“被动的”——不产生或消耗能量，即约束力 f_c 与约束方向垂直于运动方向：

$$f_c^\top v = 0 \quad (35)$$

结合 $Jv = 0$ ，可知约束力必须是 J 行空间中的向量，即：

$$f_c = J^\top \lambda \quad (36)$$

其中 $\lambda \in \mathbb{R}^c$ (c = 约束数) 是 **Lagrange 乘子**，代表关节内力（约束力大小）。

[NOTE] Lagrange 乘子的物理意义

λ 就是关节“传递”的内力。正是它使两个刚体保持连接而不穿透彼此。求解出 λ 后，将 $J^\top \lambda$ 分别施加到两个刚体上（等大反向），即满足牛顿第三定律。

5.4 带约束的运动方程与求解

$$M\dot{v} + C = f + J^\top \lambda \quad (37)$$

$$Jv_{n+1} = b_n \quad (38)$$

(b_n 通常为 0，引入误差修正项后为 $b_n = \frac{\alpha(C-g(x_n))}{h}$ ， α 称为 Error Reduction Parameter, ERP)。

离散化后，消去 v_{n+1} ，得到关于 λ 的线性方程组：

$$(JM^{-1}J^\top + \beta I)\lambda = c_n \quad (39)$$

其中 β (Constraint Force Mixing, CFM) 是正则化项，防止病态矩阵，也可物理理解为给关节加一点弹性。

[WARN] 约束漂移 (Constraint Drift)

数值积分中，小误差会逐渐积累，导致关节点不再完全重合。ERP (α) 通过在约束速度中加入修正量，将位置误差“拉回”约束流形，防止漂移。单纯的速度层约束 $Jv = 0$ 不能修正已有的位置误差。

5.5 不同类型的关节

关节类型通过改变 Jacobian J 的形式实现：

关节类型	约束数	自由度	J 的行
球铰 (Ball-and-socket)	3	3 (转动)	3 行，仅约束平动
旋转铰 (Revolute/Hinge)	5	1 (单轴转)	3 行平动 + 2 行角度
万向节 (Universal)	4	2	3 行平动 + 1 行角度
固连 (Fixed)	6	0	6 行全约束

5.6 大型铰接系统

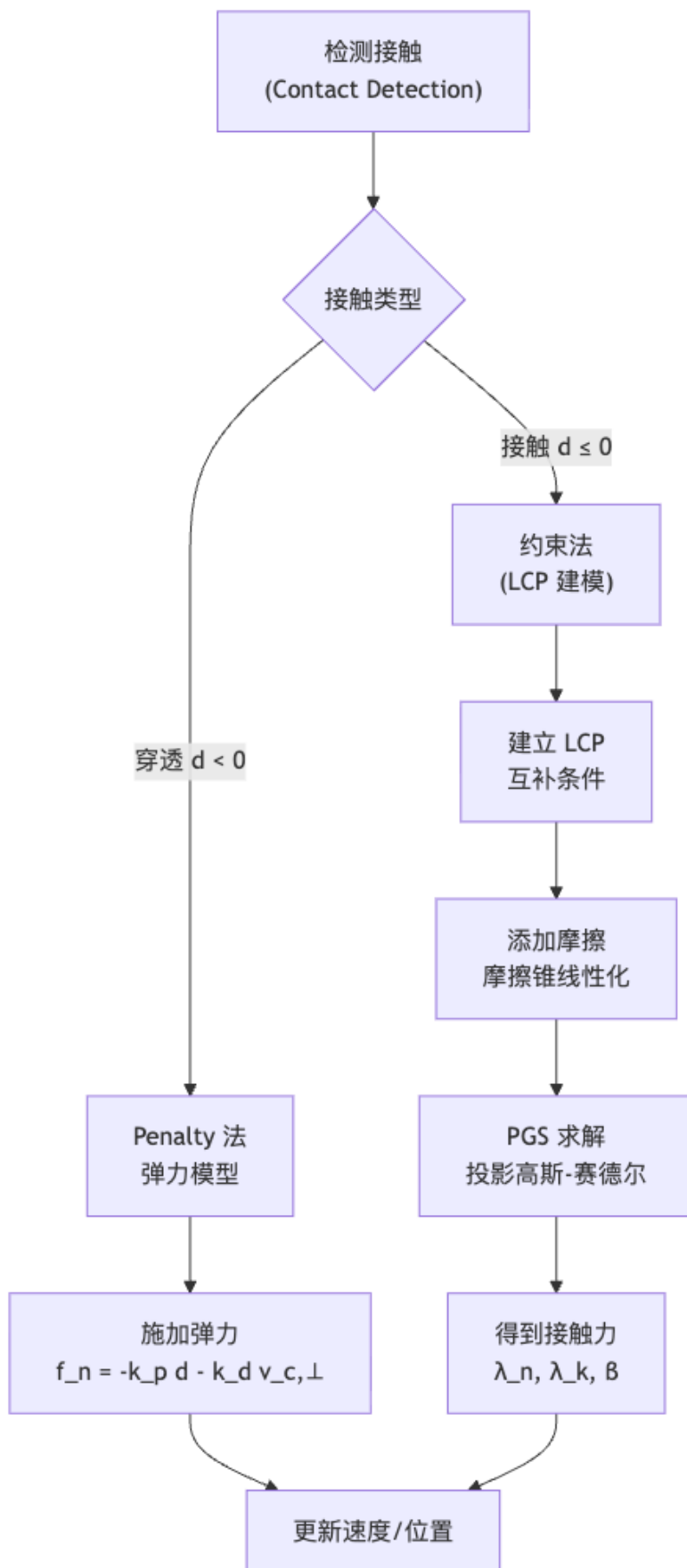
对 N 个刚体、 K 个关节，系统方程：

$$M\dot{v} + C(x, v) = f + J^\top \lambda \quad (40)$$

$$Jv = 0 \quad (41)$$

M 为 $6N \times 6N$ 块对角矩阵， J 为 $3K \times 6N$ 稀疏矩阵，方程 (39) 的规模为 $3K \times 3K$ 。对人体骨骼系统（约 20 刚体、19 关节），这是一个可实时求解的中等规模线性系统。

6. 接触建模



6.1 接触检测

设接触点 $x_c = x + r_c$ (r_c 为质心到接触点的偏移), 接触点速度:

$$v_c = v + \omega \times r_c = J_c \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (42)$$

分解为法向 ($\perp$) 和切向 ($\parallel$) 分量: $v_{c,\perp}$ (正值 = 远离地面), $v_{c,\parallel}$ (滑动速度)。

6.2 Penalty 法 (弹力接触模型)

当物体穿入地面深度 $d < 0$ 时, 施加法向弹力:

$$f_n = -k_p d - k_d v_{c,\perp} \quad (43)$$

其中 k_p 是弹簧刚度 (位置恢复), k_d 是阻尼 (速度耗散)。

动摩擦 (物体相对接触面滑动时):

$$f_t = -\mu f_n \frac{v_{c,\parallel}}{|v_{c,\parallel}|} \quad (44)$$

静摩擦 (物体趋于静止, 需防止滑动): 引入参考锚点 p_0 (上一帧接触点位置),

$$\bar{f}_t = k_p^f (p_0 - p) - k_d^f v_{c,\parallel} \quad (45)$$

若 $|\bar{f}_t| \geq \mu f_n$, 切换为动摩擦 (44); 否则用 (45) 作为静摩擦。

[WARN] Penalty 法的缺陷

刚度系数 k_p 越大, 接触越真实, 但方程组的条件数越差, 数值积分不稳定。通常 k_p 只能取到使系统“刚好不穿透”的临界值, 导致仍有少量穿透 (视觉上可接受, 但物理不准确)。

6.3 约束法: 接触作为单边约束

接触比关节更复杂: 关节是**双边约束** ($g(x) = 0$), 接触是**单边约束** (法向速度 ≥ 0 , 法向力 ≥ 0), 两者不能同时为正 (分离时不需要接触力):

$$v_{c,\perp} \geq 0, \quad \lambda_n \geq 0, \quad v_{c,\perp} \cdot \lambda_n = 0 \quad (46)$$

运动方程:

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I \omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix} + J_{c,\perp}^\top \lambda_n \quad (47)$$

6.4 线性互补问题 (Linear Complementarity Problem, LCP)

联立 (46)(47), 消去速度后, 接触问题化为标准 LCP:

$$Ax + b \geq 0, \quad x \geq 0, \quad x^\top (Ax + b) = 0 \quad (48)$$

用符号 $0 \leq u \perp x \geq 0$ 表示互补条件。

互补条件的物理含义 (以单接触为例):

- $v_{c,\perp} > 0$ (物体远离地面) $\implies \lambda_n = 0$ (无接触力)
- $\lambda_n > 0$ (有接触力) $\implies v_{c,\perp} = 0$ (物体不分离)
- 两者不能同时 > 0 (不存在“物体既在空中又有地面压力”)

[NOTE] LCP 的几何直觉

把 $(\lambda_n, v_{c,\perp})$ 看成二维平面上的点, 互补条件要求它落在坐标轴 (非负轴) 上——要么在 λ 轴 (有力无速度), 要么在 v 轴 (有速度无力), 要么在原点 (静止接触)。允许的解集是两段坐标轴的并集, 这就是“互补”名字的来源。

6.5 摩擦锥 (Friction Cone) 与线性化

库仑摩擦定律的三维形式: 切向力 f_t 的模不超过法向力的 μ 倍:

$$|f_t| \leq \mu f_n \quad (49)$$

这在三维空间中定义了以接触法线为轴、半角 $\arctan \mu$ 的**摩擦锥 (Friction Cone)**: 接触力的方向必须在锥内才满足库仑定律。

为纳入 LCP 框架, 将摩擦锥**线性化**为多面体锥 (取 K 条边 $b_1, \dots, b_K$):

$$f_t = \sum_{k=1}^K \lambda_k b_k, \quad \lambda_k \geq 0 \quad (50)$$

约束 $|f_t| \leq \mu f_n$ 线性化为:

$$\mu \lambda_n - \sum_k \lambda_k \geq 0 \quad (51)$$

滑动条件: 引入滑动标量 $\beta \geq 0$, 将投影接触速度写成向量 $\bar{v}_c$ (各方向分量 $v_k = v_{c,\parallel} \cdot b_k$), 则:

$$\beta e + \bar{v}_c \geq 0, \quad \beta \geq 0 \quad (52)$$

$\beta > 0$ 当且仅当接触点在滑动 ($|v_{c,\parallel}| > 0$)。

6.6 完整的 LCP 公式

将法向、切向、滑动三组互补条件合并, 完整的带摩擦接触 LCP (变量 $x = (\lambda_n, \lambda_k, \beta)$) 为:

$$0 \leq Ax + b \perp x \geq 0 \quad (53)$$

具体三组互补条件：

$$0 \leq v_{c,\perp} \perp \lambda_n \geq 0 \quad (54a)$$

$$0 \leq \beta e + \bar{v}_c \perp \lambda_k \geq 0 \quad (54b)$$

$$0 \leq \mu \lambda_n - \sum_k \lambda_k \perp \beta \geq 0 \quad (54c)$$

7. 投影高斯-赛德尔 (PGS) 求解器

7.1 标准高斯-赛德尔 (Gauss-Seidel)

对线性方程 $Ax = b$ ，分解 $A = L^* + U$ (L^* 含下三角和对角， U 含严格上三角)，迭代格式：

$$L^* x^{k+1} = b - Ux^k \quad (55)$$

逐分量写出，第 i 个变量的更新（只用当前步骤内已更新的 x_j^{k+1} ）：

$$x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{k+1} - \sum_{j>i} a_{ij} x_j^k \right) \quad (56)$$

收敛条件： A 对称正定，或严格对角占优 $a_{ii} \geq \sum_{j \neq i} |a_{ij}|$ 。

7.2 投影高斯-赛德尔 (Projected Gauss-Seidel, PGS)

对 LCP $0 \leq Ax + b \perp x \geq 0$ ，在 Gauss-Seidel 基础上加**投影步**（截断到非负域）：

$$z_i^k = \frac{1}{a_{ii}} \left(-b_i - \sum_{j<i} a_{ij} x_j^{k+1} - \sum_{j>i} a_{ij} x_j^k \right) \quad (57)$$

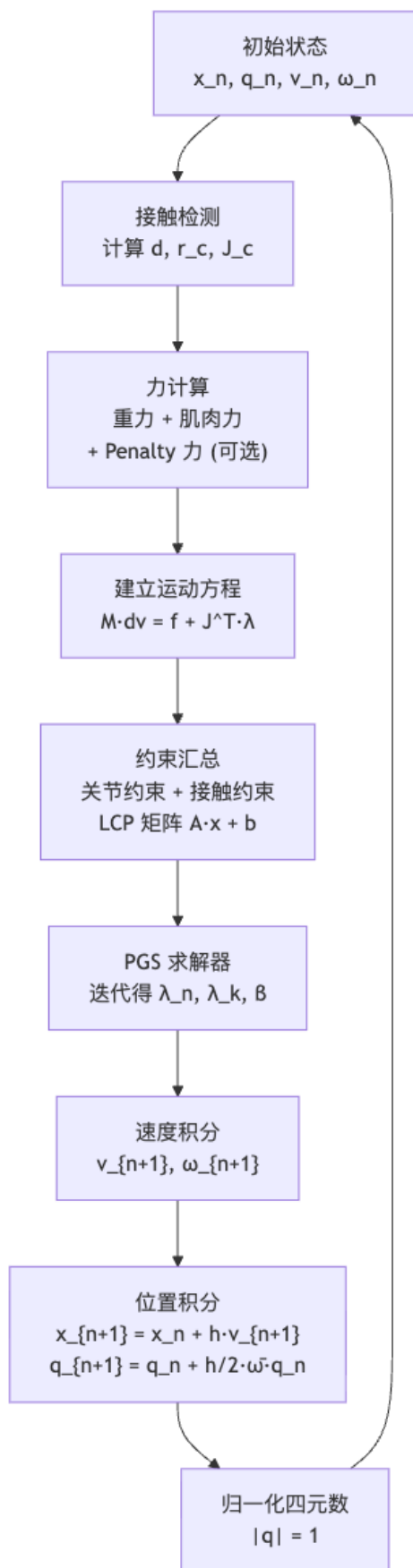
$$x_i^{k+1} = \max(0, z_i^k) \quad (58)$$

从初始猜测 $x^0 \geq 0$ 出发，不断迭代 (57)(58) 直至收敛。

[NOTE] PGS 在物理引擎中的地位

PGS 是 ODE (Open Dynamics Engine)、Bullet、MuJoCo 等开源物理引擎的核心求解器。它不需要组装全局矩阵，逐约束迭代，天然适合稀疏多接触系统。每次迭代计算量 $O(Nc)$ (N 为刚体数， c 为约束数)，收敛速度在实际场景中往往优于理论最坏情况。

8. 完整仿真管线



9. Worked Examples (例题)

例题 1: 显隐式 Euler 稳定性分析

问题: 质量 $m = 1$, 弹簧 $k = 100$ ($\hat{k} = 100$), 步长 $h = 0.05$ 。分析显式和隐式 Euler 的稳定性。

解:

稳定条件 $h < 2/\sqrt{\hat{k}} = 2/10 = 0.2$ 。 $h = 0.05 < 0.2$, 显式 Euler 满足稳定条件, 不发散。

若取 $h = 0.3 > 0.2$, 显式转移矩阵:

$$A = \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 \end{bmatrix} = \begin{bmatrix} 1 & -30 \\ 0.3 & 1 \end{bmatrix}$$

特征值: $\lambda = 1 \pm i\sqrt{\hat{k}h} \cdot \dots$, 可算出 $|\lambda|^2 = 1 + \hat{k}h^2 - 2 = 1 + 100 \cdot 0.09 - 2 \cdot 1 > 1$, 谱半径超过 1, 发散。

隐式 Euler 转移矩阵有前置系数 $\frac{1}{1+\hat{k}h^2}$, 谱半径 < 1 , 任何 h 均收缩, 无条件稳定。

例题 2: 单刚体的 Newton-Euler 积分

问题: 均匀长方体, $m = 2 \text{ kg}$, $I_0 = \text{diag}(1, 4, 4) \text{ kg} \cdot \text{m}^2$, 当前 $R = I$, $\omega = (0, 1, 0)^\top \text{ rad/s}$, $\tau = 0$, $h = 0.01$ 。求一步后的 ω_{n+1} 。

解:

$$I_n = RI_0R^\top = I_0 \text{ (因 } R = I\text{)}。$$

$$\omega \times I\omega = (0, 1, 0) \times (0, 4, 0) = (0, 0, 0)^\top \text{ (同向, 叉积为零)}。$$

$$\text{代入 (30): } I_0(\omega_{n+1} - \omega_n)/h = \tau - \omega \times I\omega = 0。$$

$$\text{故 } \omega_{n+1} = \omega_n = (0, 1, 0)^\top \text{ (匀速旋转, 无陀螺效应)}。$$

若 $\omega = (1, 1, 0)^\top$, 则 $I\omega = (1, 4, 0)^\top$, $\omega \times I\omega = (1, 1, 0) \times (1, 4, 0) = (0, 0, 3)^\top$, 陀螺力矩非零, 旋转轴将偏转。

例题 3: 单点接触的冲量计算

问题: 质量 m 、惯量 I 的刚体, 接触点在 r_c , 接触前法向速度 $v_n^- < 0$ (正在接近)。设恢复系数 e ($e = 0$ 完全非弹性, $e = 1$ 完全弹性), 求法向冲量 J_n 。

解:

接触冲量 J_n 通过 $J_c^\top$ 方向作用, 线速度变化 $\Delta v = J_n n/m$, 角速度变化 $\Delta \omega = I^{-1}(r_c \times J_n n)$ 。

接触点法向速度变化:

$$\Delta v_n = J_n \left(\frac{1}{m} + n^\top r_c^\times I^{-1} r_c^\times n \right) \equiv J_n / m_{\text{eff}}$$

弹性条件 $v_n^+ = -ev_n^-$, 则 $\Delta v_n = -(1+e)v_n^-$, 解得:

$$J_n = \frac{-(1+e)v_n^-}{1/m + n^\top r_c^\times I^{-1} r_c^\times n} \quad (59)$$

$J_n > 0$ 是推离地面的正法向冲量。对 $e = 0$ (完全塑性碰撞), 冲量最小 (接触后速度为零); $e = 1$ 冲量最大 (完全反弹)。

例题 4: PGS 求解简单 LCP

问题: 二维 LCP, $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $b = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, 从 $x^0 = (0, 0)^\top$ 出发迭代 2 步。

解:

迭代公式 (对角元 $a_{11} = a_{22} = 2$):

$$x_1^{k+1} = \max\left(0, \frac{1 - x_2^k}{2}\right), \quad x_2^{k+1} = \max\left(0, \frac{1 - x_1^k}{2}\right)$$

第 1 步: $x_1^1 = \max(0, 1/2) = 0.5$, $x_2^1 = \max(0, (1 - 0.5)/2) = 0.25$ 。

第 2 步: $x_1^2 = \max(0, (1 - 0.25)/2) = 0.375$, $x_2^2 = \max(0, (1 - 0.375)/2) = 0.3125$ 。

理论解 $x^* = (1/3, 1/3)^\top$, PGS 收敛于此 (A 对称正定)。

例题 5: 摩擦锥线性化与约束数

问题: 用 $K = 4$ 条边线性化摩擦锥 (正方形截面), 摩擦系数 $\mu = 0.4$, 法向力 $\lambda_n = 10\text{ N}$ 。求切向力限制及总 LCP 变量数。

解:

线性摩擦锥边界: $\sum_k \lambda_k \leq \mu \lambda_n = 4\text{ N}$ 。实际允许合力方向限制在四条边张成的多面体锥内, 近似满足 $|f_t| \leq 4\text{ N}$ (线性化误差约 $1 - 1/\sqrt{2} \approx 29\%$, 可用 $K = 8$ 改善)。

LCP 变量: λ_n (1 个) + λ_k ($K = 4$ 个) + β (1 个) = 6 个变量, A 为 6×6 。

10. 中英术语速查表

中文	English	简释
数值积分	Numerical Integration	用离散步长近似连续积分
显式 Euler	Explicit/Forward Euler	用当前步信息预测下一步
隐式 Euler	Implicit/Backward Euler	用下一步未知量列方程
半隐式 Euler	Symplectic/Semi-implicit Euler	速度显式, 位置隐式
Verlet 积分	Verlet Integration	二阶精度, 保能量
Runge-Kutta	Runge-Kutta Methods	多采样高阶精度
刚体	Rigid Body	形变为零的理想体
角速度	Angular Velocity	ω , 旋转快慢和方向
角动量	Angular Momentum	$L = I\omega$

中文	English	简释
惯量张量	Moment of Inertia Tensor	I , 旋转阻力的矩阵描述
主转动惯量	Principal Moments of Inertia	I_1, I_2, I_3 , 特征分解得到
陀螺力矩	Gyroscopic Torque	$\omega \times I\omega$
Newton—Euler 方程	Newton—Euler Equations	刚体的动力学方程 (平动 + 转动)
铰接刚体	Articulated Rigid Body	多刚体通过关节连接
约束	Constraint	限制系统自由度的条件
Jacobian 矩阵	Jacobian Matrix	$J = \partial g / \partial x$, 约束的速度形式
Lagrange 乘子	Lagrange Multiplier	λ , 约束力的大小
ERP	Error Reduction Parameter	数值误差修正参数 α
CFM	Constraint Force Mixing	正则化参数 β
接触力	Contact Force	地面对物体的支持力
Penalty 法	Penalty Method	用弹簧力近似接触约束
线性互补问题	Linear Complementarity Problem	LCP, 互补条件的线性约束系统
互补条件	Complementarity Condition	$u \geq 0, x \geq 0, u^\top x = 0$
库仑摩擦	Coulomb Friction	$ f_t \leq \mu f_n$
摩擦锥	Friction Cone	满足库仑定律的接触力方向锥
摩擦锥线性化	Linearized Friction Cone	用多面体锥近似圆锥
投影高斯—赛德尔	Projected Gauss—Seidel (PGS)	LCP 迭代求解器, 带投影步
静摩擦	Static Friction	物体无相对滑动时的摩擦力
动摩擦	Kinetic/Dynamic Friction	物体相对滑动时的摩擦力
谱半径	Spectral Radius	矩阵最大特征值的模 $\rho(A)$

11. Anki 复习卡片

卡片 1 Q: 显式 Euler、隐式 Euler、半隐式 Euler 的速度和位置更新公式各是什么? A: - 显式: $v_{n+1} = v_n + a_n h$; $x_{n+1} = x_n + v_n h$ - 隐式: $v_{n+1} = v_n + a_{n+1} h$; $x_{n+1} = x_n + v_{n+1} h$ - 半隐式: $v_{n+1} = v_n + a_n h$; $x_{n+1} = x_n + v_{n+1} h$ (先更新 v , 再用新 v 更新 x)

卡片 2 Q: 弹簧—质点系统中, 显式 Euler 的稳定条件是什么? A: $h < 2/\sqrt{k/m}$ 。步长超过此值则谱半径 $\rho > 1$, 能量发散。

卡片 3 Q: 半隐式 Euler 为什么是物理动画的默认选择? A: 与显式同样快 (无需求解方程), 但位置用更新后的速度, 能量近似守恒, 不会像显式那样发散; 比隐式更真实 (不过度耗散能量)。

卡片 4 Q: $\dot{R} = \omega^\times R$ 是怎么推出来的? A: 由旋转矩阵约束 $RR^\top = I$ 求导得 $\dot{R}R^\top + (\dot{R}R^\top)^\top = 0$, 故 $\dot{R}R^\top$ 反对称, 其三个独立分量定义角速度向量 ω , 即 $\dot{R}R^\top = \omega^\times$, 两边右乘 R 得结论。

卡片 5 Q: 四元数角速度方程 $\dot{q} = ?$ A: $\dot{q} = \frac{1}{2}\bar{\omega}q$, 其中 $\bar{\omega} = (0, \omega)$ 是纯四元数。积分后需对 q 归一化。

卡片 6 Q: 惯量张量的定义公式是什么? 旋转后如何变换? A: $I = \sum_i (-m_i r_i^{\times 2})$; 旋转后 $I = R I_0 R^\top$, 其中 I_0 是物体坐标系下 (常量) 的惯量张量。

卡片 7 Q: Newton—Euler 方程的完整形式 (含陀螺项) 是什么? A:

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I \omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

陀螺项 $\omega \times I \omega$ 来自惯量张量随旋转变换。

卡片 8 Q: 约束力为什么等于 $J^\top \lambda$? A: 约束不产生能量, 即 $f_c^\top v = 0$ 。同时 $Jv = 0$ (约束速度为零)。因此 f_c 必须在 J 的行空间中, 故 $f_c = J^\top \lambda$, λ 为 Lagrange 乘子 (关节内力)。

卡片 9 Q: ERP 和 CFM 各自解决什么问题? A: ERP (α) 解决约束漂移——通过在速度层约束中加入位置误差修正量, 将已偏离的约束“拉回”; CFM (β) 解决病态矩阵——在 $JM^{-1}J^\top$ 对角加正则化, 同时赋予约束少量“弹性”。

卡片 10 Q: 接触 LCP 的互补条件 (以法向为例) 是什么? 物理含义? A: $v_{c,\perp} \geq 0, \lambda_n \geq 0, v_{c,\perp} \cdot \lambda_n = 0$ 。含义: 物体远离地面时无接触力; 有接触力时物体不分离; 两者不能同时非零。

卡片 11 Q: Penalty 法与约束 (LCP) 法的核心区别是什么? A: Penalty 法用弹力模拟接触, 有穿透, 刚度越大越不稳定; LCP 法用互补条件严格禁止穿透, 物理更准确, 但需要求解约束方程 (PGS 等)。

卡片 12 Q: 摩擦锥线性化的思路和变量设置是什么? A: 将圆锥用 K 个半空间 (多面体锥) 近似, 切向力 $f_t = \sum_k \lambda_k b_k$ ($\lambda_k \geq 0$), 约束 $\mu \lambda_n - \sum_k \lambda_k \geq 0$; 引入滑动变量 $\beta \geq 0$, 整个带摩擦接触化为标准 LCP, 用 PGS 求解。

卡片 13 Q: PGS (投影高斯—赛德尔) 的迭代公式? 投影步起什么作用? A: 每步先做 Gauss—Seidel 更新得 z_i^k , 再做投影 $x_i^{k+1} = \max(0, z_i^k)$ 。投影步强制解非负, 将标准线性求解器推广到 LCP (互补约束)。

卡片 14 Q: 仿真主循环的 5 个步骤是什么? A: ① 接触检测 (计算穿透深度 d 、接触点 r_c 、Jacobian J_c) → ② 力计算 (重力、控制力、Penalty 力) → ③ 建立运动方程 + 约束 (组装 LCP) → ④ PGS 求解 (得 λ) → ⑤ 位姿积分 (更新 v, ω, x, q)。

12. 复习要点

[TIP] 期末复习要点

1. **积分器选型**: Explicit Euler (快但条件稳定) $\rightarrow$ Symplectic Euler (近似守能, 动画默认) $\rightarrow$ Implicit Euler (无条件稳定, 能量耗散)。弹簧系统稳定界 $h < 2/\sqrt{k/m}$ 。2. $\dot{R} = \omega^\times R$ 是刚体旋转运动学的核心, 直接导出四元数方程 $\dot{q} = \frac{1}{2}\bar{\omega}q$ 。3. **Newton-Euler 方程**: $I\dot{\omega} + \omega \times I\omega = \tau$, 陀螺项不能遗漏; 惯量张量须随旋转变换 $I = RI_0R^\top$ 。4. **约束统一框架**: 关节 $\rightarrow$ 双边约束 ($Jv = 0$); 接触 $\rightarrow$ 单边约束 (互补条件); 约束力统一表示为 $J^\top \lambda$, λ 由 $(JM^{-1}J^\top + \beta I)\lambda = c$ 求解。5. **接触的 LCP 结构**: 法向互补 + 摩擦锥线性化 (K 边) + 滑动变量 β , 组成标准 LCP, 用 PGS 逐变量迭代求解 (每步投影截断)。

Lecture 09 —驱动角色 (Actuating Characters)

[TIP] 学习指南

本节核心：在 Lec 08 已建立的被动仿真框架 (ragdoll) 之上，给角色装上”肌肉”——以关节力矩 τ 为唯一执行器，由控制器 π 根据当前状态实时计算并注入仿真器。动作生成问题变成：何时、对哪个关节、施加多大力矩。

前置知识：– Lec 08 —Newton—Euler 方程 $M\dot{v} + C(x, v) = f + J^T\lambda$ ；半隐式 Euler 积分；接触/约束求解 – Lec 03 —正运动学 (FK)；雅可比矩阵 $J = \partial x / \partial \theta$

重点掌握：1. 极大坐标 vs. 广义坐标的区别与各自代价 2. 关节力矩的物理本质：“一对反向力偶”——子刚体 + τ 、父刚体 - τ 3. 正动力学 (FD) 与逆动力学 (ID) 的二元性；RNEA 两遍递归 4. 显式 PD 的稳定性危机 ($|\lambda| > 1$ 发散) 及 Stable PD 的隐式解法 5. Jacobian Transpose：从功率等价出发推导 $\tau = J^T f$ 6. 欠驱动 vs. 全驱动的含义；残差力/力矩的作用与代价 7. 静态平衡：CoM、支撑多边形、踝/髌策略、动量控制

0. 核心问题 / Motivation

Lec 08 给出的仿真器把角色当”布娃娃”——只有重力、接触约束，没有主动控制。要让角色”自己动”，必须引入控制器 π ：

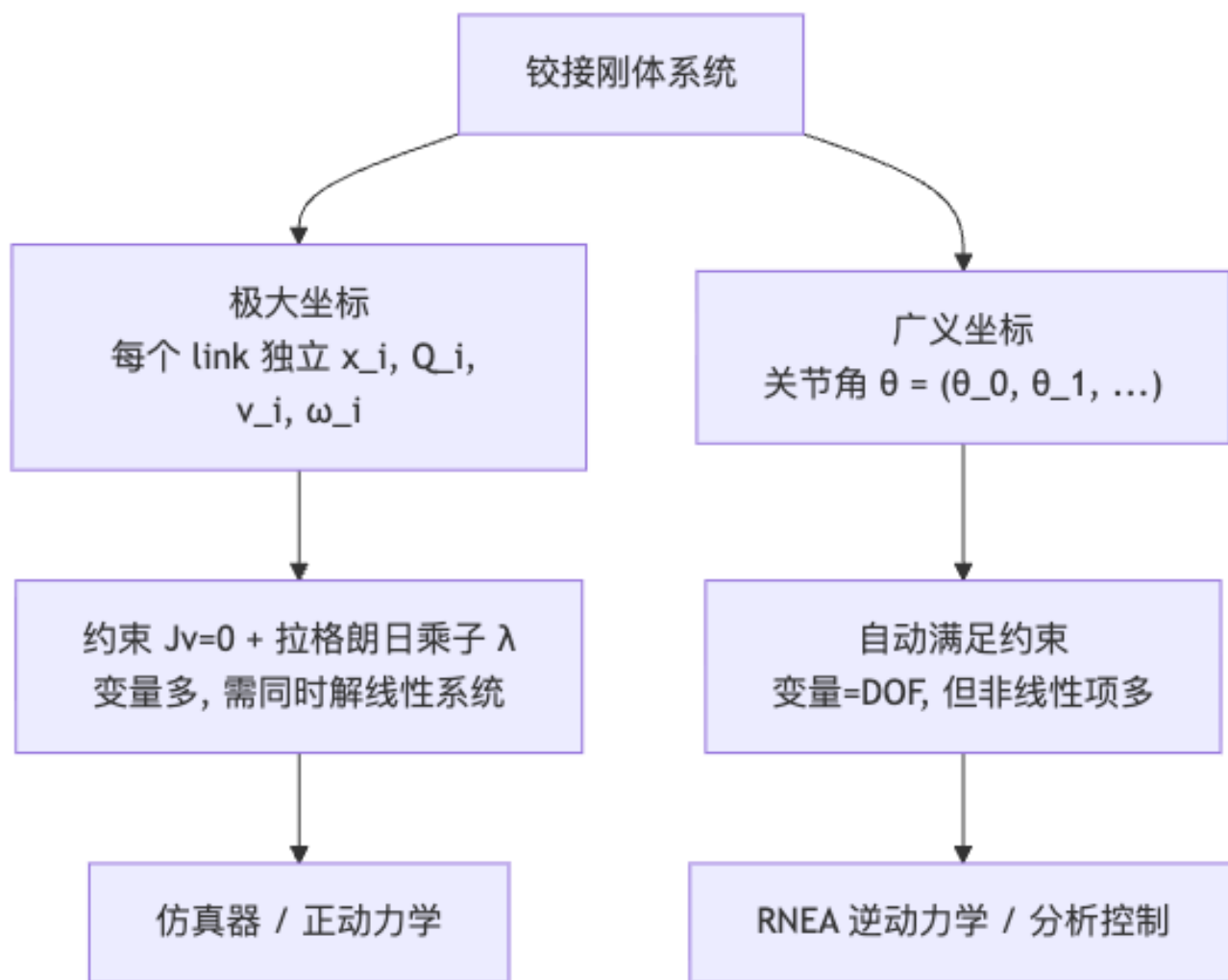
$$\pi : (s_t, t) \mapsto (f, \tau) \quad (0.1)$$

其中 $s_t = (x_t, v_t, R_t, \omega_t)$ 是当前状态， τ 是关节力矩向量（每个关节一个/三个分量）， f 是可能的外部辅助力（通常只在调试或残差力场景下使用）。

本节回答七个核心问题：

1. 为什么用关节力矩而非直接施力？
2. 关节力矩如何插入 Newton—Euler 方程？
3. 极大坐标与广义坐标的取舍是什么？
4. 正动力学（力 $\rightarrow$ 加速度）与逆动力学（加速度 $\rightarrow$ 力）如何互为镜像？
5. 最简反馈律 PD 控制在大刚度时为何数值爆炸？Stable PD 如何解决？
6. 如何用关节力矩”虚拟”一个作用在末端的力（Jacobian Transpose）？
7. 静止站立时为何要把 CoM 投影保持在支撑多边形内，用什么关节力矩策略做到？

1. 两种坐标体系



1.1 单刚体动力学回顾

线性动量: $\dot{p} = f$, $p = mv$; 角动量: $\dot{L} = \tau$, $L = I\omega$ 。合写为 Newton-Euler 方程:

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix} \quad (1.1)$$

铰接刚体系统 (来自 Lec 08):

$$M\dot{v} + C(x, v) = f + J^T\lambda, \quad Jv = 0 \quad (1.2)$$

1.2 极大坐标 (Maximized Coordinates)

极大坐标: 对 N 个 link, 每个 link 独立存储其笛卡尔位姿 (x_i, Q_i) 和速度 (v_i, ω_i) , 共 $13N$ 个标量变量 (位置 3 + 四元数 4 + 线速度 3 + 角速度 3 = 13/link)。

- 关节以约束方程 $Jv = 0$ 的形式体现; 用拉格朗日乘子 λ 处理。
- 优点: 运动方程结构简单 (各 link 独立, 矩阵稀疏); 方便接触/碰撞检测。

- 缺点：变量数远大于自由度，每步需解含约束的线性系统。

1.3 广义坐标 (Generalized Coordinates)

广义坐标：直接以关节角参数化整棵树。对人形角色：

$$\theta = (t_0, \theta_0, \theta_1, \theta_2, \dots), \quad \dot{\theta} = (v_0, \omega_0, \dot{\theta}_1, \dots), \quad \ddot{\theta} = (\dot{v}_0, \dot{\omega}_0, \ddot{\theta}_1, \dots) \quad (1.3)$$

其中 t_0 是根关节平移 (3D), θ_0 是根关节旋转 (球关节, 3D), θ_k 是各铰关节角 (1D 或 3D)。

- 关节约束自动满足——广义坐标本就只描述可行位形。
- 优点：变量数等于 DoF；**逆动力学 (ID)** 在广义坐标下形式优雅：

$$M(\theta)\ddot{\theta} + C(\theta, \dot{\theta}) + G(\theta) = \tau \quad (1.4)$$

- 缺点：正向运动学 (FK) 涉及累乘旋转矩阵，加速度递推含大量二次速度项 (科氏/离心力)；与接触仿真器对接复杂。

[NOTE] 两种坐标系的直觉对比

想象一条 3-link 手臂：极大坐标相当于“分别用 GPS 记录每节骨头的绝对位置，再加一堆‘骨骼不能分开’的约束”；广义坐标相当于“只记录肩角、肘角、腕角，骨骼结构自动满足”。前者用于**物理仿真器**（数值稳定、易接触）；后者用于**控制器设计**（变量少、方程干净）。

1.4 正运动学：广义坐标 → 极大坐标

对一条串行链，设 link i 的局部转轴 u_i 、关节角 θ_i 、关节到质心偏移 l_i 、质心到下一关节偏移 d_i ：

姿态（沿链累乘旋转）：

$$Q_i = Q_{i-1}R(\theta_i u_i), \quad Q_0 = R(\theta_0 u_0) \quad (1.5)$$

位置：

$$x_i = x_{i-1} + Q_{i-1}d_{i-1} + Q_i l_i \quad (1.6)$$

角速度（递推）：

$$\omega_i = \omega_{i-1} + Q_{i-1}\dot{\theta}_i u_i \quad (1.7)$$

线速度（递推）：

$$v_i = v_{i-1} + \omega_{i-1} \times Q_{i-1}d_{i-1} + \omega_i \times Q_i l_i \quad (1.8)$$

角加速度（再对时间求导）：

$$\dot{\omega}_i = \dot{\omega}_{i-1} + \omega_{i-1} \times Q_{i-1}\dot{\theta}_i u_i + Q_{i-1}\ddot{\theta}_i u_i \quad (1.9)$$

线加速度：

$$\dot{v}_i = \dot{v}_{i-1} + \dot{\omega}_{i-1} \times Q_{i-1}d_{i-1} + \omega_{i-1} \times (\omega_{i-1} \times Q_{i-1}d_{i-1}) + \dot{\omega}_i \times Q_i l_i + \omega_i \times (\omega_i \times Q_i l_i) \quad (1.10)$$

矩阵化（雅可比形式）：速度和加速度可写成对 $\dot{\theta}$ 、 $\ddot{\theta}$ 的线性 + 二次型：

$$\omega_i = J_i^\omega(\theta)\dot{\theta}, \quad v_i = J_i^v(\theta)\dot{\theta} \quad (1.11)$$

$$\dot{\omega}_i = J_i^\omega\ddot{\theta} + \dot{\theta}^T H_i^\omega \dot{\theta}, \quad \dot{v}_i = J_i^v\ddot{\theta} + \dot{\theta}^T H_i^v \dot{\theta} \quad (1.12)$$

其中 H_i 是二阶 Hessian 项，对应科氏力与离心力。

2. 关节力矩：物理本质与施加方式

2.1 自由刚体：力与等效力矩

外力 f 作用在刚体上距质心 r_f 处，Newton—Euler 方程右端变为：

$$\begin{bmatrix} f \\ \tau + r_f \times f \end{bmatrix} \quad (2.1)$$

[EX] 力偶 (Force Couple)

两个等大反向、作用在不同点的力： f 在 r_1 、 $-f$ 在 r_2 。合力 = 0，合力矩 = $(r_1 - r_2) \times f$ 。此力矩对任意参考点都相同——纯力偶完全由力矩描述，与施力位置无关。这正是“力矩”概念的核心价值。

2.2 关节内力的净效果：一对反向力偶

考虑两个刚体（link 1、link 2）通过关节 x_J 相连。关节本身无质量，因此关节内部的所有内力集合 $\{f_i\}$ 必须满足：

$$\sum_i f_i = 0 \quad (2.2)$$

即关节对外的净力为零（不能凭空产生线冲量）。但力矩不为零：

设各力 f_i 相对关节点的力臂为 r_i （沿关节内分布），则 link 1 受到的合力矩为：

$$\tau_1 = \sum_i (r_1 + r_i) \times f_i = r_1 \times \underbrace{\sum_i f_i}_{=0} + \sum_i r_i \times f_i = \sum_i r_i \times f_i \quad (2.3)$$

link 2（受到反作用力 $-f_i$ ）：

$$\tau_2 = \sum_i r_i \times (-f_i) = -\tau_1 \quad (2.4)$$

[NOTE] 结论：关节力矩的施加规则

$$\tau_1 = \tau, \quad \tau_2 = -\tau \quad (2.5)$$

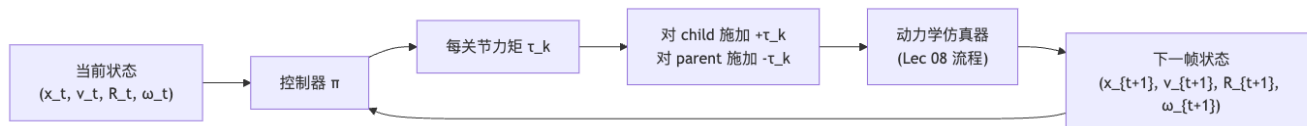
实现一个”关节力矩 τ “只需要：对子刚体 (child) 加 $+\tau$ ，对父刚体 (parent) 加 $-\tau$ 。整个角色所有关节力矩的总和为零——内力不改变系统总动量，角色不能靠内部力矩”飞起来”。

2.3 带关节力矩的运动方程

两刚体系统，关节力矩 τ 作用在关节 j (child = link 2)：

$$M \begin{bmatrix} \dot{v}_1 \\ \dot{\omega}_1 \\ \dot{v}_2 \\ \dot{\omega}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \times I_1 \omega_1 \\ 0 \\ \omega_2 \times I_2 \omega_2 \end{bmatrix} = \begin{bmatrix} 0 \\ -\tau \\ 0 \\ +\tau \end{bmatrix} + J^T \lambda, \quad Jv = 0 \quad (2.6)$$

2.4 控制回路总览



3. 全驱动与欠驱动

$$\text{执行器数} \begin{cases} \geq \text{DoF} & \text{全驱动 (Fully-Actuated)} \\ < \text{DoF} & \text{欠驱动 (Underactuated)} \end{cases} \quad (3.1)$$

类型	示例	特点
全驱动	工业机械臂（固定基座）	对任意 $(x, v, \dot{v})$ 都存在 f 实现； ID 始终有唯一解
欠驱动	人形角色（根关节自由漂浮）	许多 $(x, v, \dot{v})$ 无任何 f 可实现； 必须借助接触力间接控制

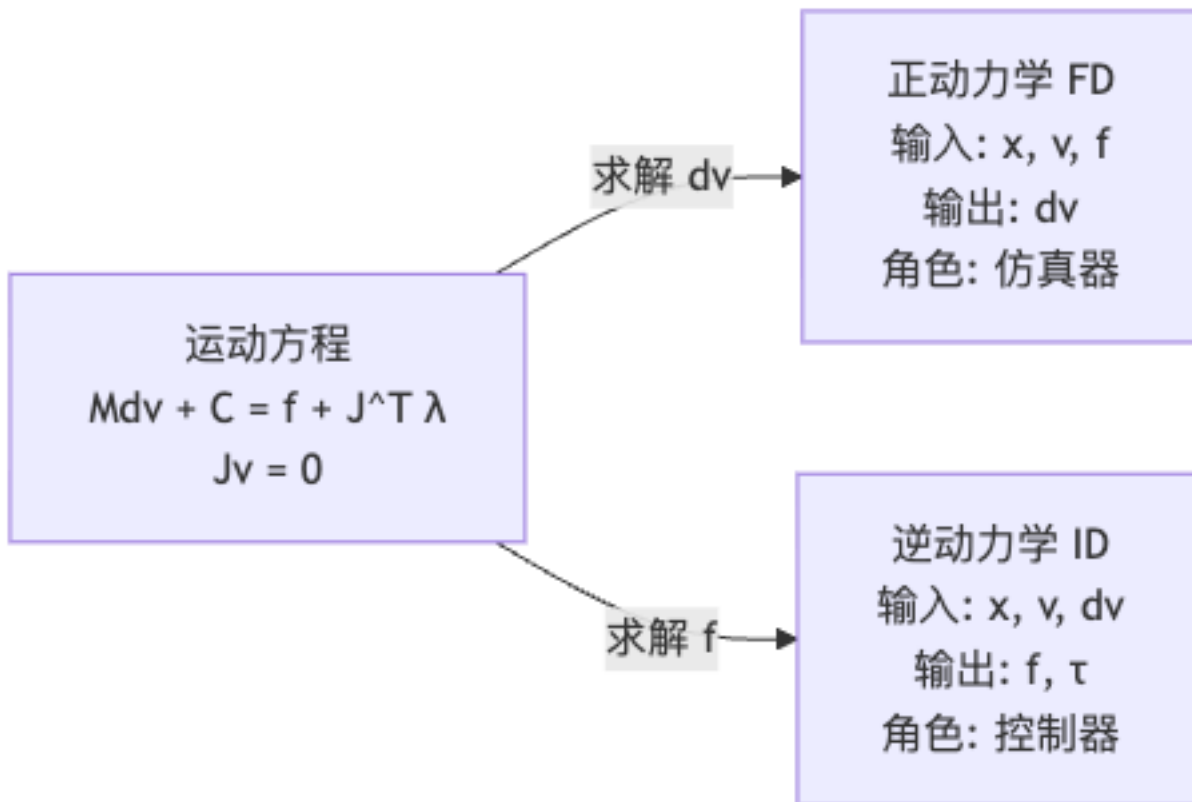
[WARN] 人形角色的根关节

人形角色的骨盆 (root joint) 是一个 6-DoF 自由刚体，没有对应的电机。这 6 个 DoF 只能通过腿—脚—地面接触力间接控制。这就是欠驱动的根本，也是角色平衡控制困难的本质原因。

4. 正动力学与逆动力学

4.1 二元关系

运动方程 $M\dot{v} + C = f + J^T \lambda$ 在 $(x, v, f, \dot{v})$ 四组量之间建立联系：



	已知	求解	在角色动画中的角色
正动力学 (FD)	x, v, f	$\dot{v}$	物理仿真器 (Lec 08)
逆动力学 (ID)	$x, v, \dot{v}$	f, τ	控制器 (本节核心)

逆动力学的显式形式 (在广义坐标下):

$$\tau = M(\theta)\ddot{\theta} + C(\theta, \dot{\theta}) + G(\theta) \quad (4.1)$$

给定期望加速度 $\ddot{\theta}$, 反推所需关节力矩。这也是为什么逆动力学是控制器设计的工具: 告诉我你想怎么动, 我算出需要什么力。

4.2 递归 Newton-Euler 算法 (RNEA)

RNEA 是铰接链逆动力学的经典 $O(N)$ 算法, 由两个“遍历”组成:

[EX] RNEA 两遍递归详解

第一遍：前向遍历 (root → tip) ——运动学传播

从根关节出发，沿树向叶节点传播位置/速度/加速度（用 §1.4 公式逐 link 累乘）：

$$Q_i, x_i, \omega_i, v_i, \dot{\omega}_i, \dot{v}_i$$

直觉：先用广义坐标 $(\theta, \dot{\theta}, \ddot{\theta})$ 算出每个 link 的实际加速度（极大坐标）。

第二遍：后向遍历 (tip → root) ——力矩反推

从叶节点往根节点“回收”力和力矩。对 link i （子节点集合 c_i ）：

线性方程（Newton 第二定律）：

$$m_i \dot{v}_i = f_i - \sum_{k \in c_i} f_k + \sum_j f_j^{\text{ext}} \quad (4.2)$$

角方程（Euler 方程）：

$$I_i \dot{\omega}_i + \omega_i \times I_i \omega_i = \tau_i + r_i \times f_i - \sum_{k \in c_i} (\tau_k + d_{ik} \times f_k) + \sum_j \tau_j^{\text{ext}} \quad (4.3)$$

其中： f_i, τ_i 是 link i 通过关节作用在其父节点上的力/力矩； f_k, τ_k 是子节点 k 通过关节作用在 i 上的反作用（牛顿第三定律 → 符号取负）。

直觉：从末梢开始“往上捞”力——每个 link 自己的惯性力加上所有子链传来的力，就等于关节力矩加上父节点通过关节作用在它上面的力。逐 link 解方程，每一步都是显式的，因此整个算法是 $O(N)$ 。

4.3 示例：重力补偿 (Gravity Compensation)

[EX] 三段串行链，全身静止目标： $\dot{\mathbf{v}}_i = \mathbf{0}, \dot{\boldsymbol{\omega}}_i = \mathbf{0}$

外力只有重力 g （向下），每个 link 受到 $f_i^{\text{ext}} = m_i g$ 。

RNEA 第二遍从叶节点 (link 3) 开始：

Link 3（叶节点，无子节点）：

$$0 = f_3 + m_3 g \Rightarrow f_3 = -m_3 g$$

$$0 = \tau_3 + r_3 \times f_3 + r_3 \times m_3 g \Rightarrow \tau_3 = r_3 \times m_3 g$$

Link 2（一个子节点 link 3）：

$$0 = f_2 - f_3 + m_2 g \Rightarrow f_2 = -m_2 g - m_3 g$$

$$\tau_2 = r_2 \times m_2 g + (r_2 - d_{23} + r_3) \times m_3 g$$

Link 1（链根）：撑住全部上方质量， $f_1 = -(m_1 + m_2 + m_3)g$ 。

Jacobian Transpose 视角 (§5 会推导) 提供了更简洁的公式：

$$\tau_k = \sum_{i=k}^N (x_i - p_k) \times (-m_i g) \quad (4.4)$$

其中 p_k 是关节 k 的位置——每段力矩就是“这个关节到该 link 质心的力臂”又乘“重力”的累加。

5. PD / PID 控制：最简的反馈律

5.1 前馈 vs. 反馈

控制策略 π 分为两类：

- **前馈（开环）控制**： $(f, \tau) = \pi(t)$ ，与当前状态无关。按预设信号播放，任何扰动都会累积误差，不自我修正。
- **反馈（闭环）控制**： $(f, \tau) = \pi(s_t, t)$ ，根据当前状态修正信号，能抵抗一定扰动。

[NOTE] PD 既是前馈也是反馈

PD 控制律 $\tau = k_p(\bar{q} - q) - k_d\dot{q}$ 中，目标姿态 $\bar{q}(t)$ 是“前馈指令”（由轨迹规划器给出），而 $-k_p q - k_d\dot{q}$ 是“反馈修正”（依赖当前状态）。所以全身 PD Tracking Controller 同时具有两者的性质。

5.2 比例控制（P Control）

一维质点，目标位置 $\bar{x}$ ，当前位置 x ，误差 $e = \bar{x} - x$ ：

$$f = k_p e = k_p(\bar{x} - x) \quad (5.1)$$

k_p 为**比例增益（刚度）**。纯 P 控制在重力 mg 作用下存在**稳态误差** e_0 ：

$$k_p e_0 = mg \Rightarrow e_0 = \frac{mg}{k_p} \quad (5.2)$$

增大 k_p 可减小 e_0 ，但会导致数值不稳定（下面详述）。

5.3 PD 控制

加入阻尼（微分）项：

$$f = k_p e + k_d \dot{e} = k_p(\bar{x} - x) + k_d(\dot{\bar{x}} - \dot{x}) \quad (5.3)$$

对角色场景，目标速度 $\dot{\bar{q}}$ 通常取 0，每个关节的力矩：

$$\tau = k_p(\bar{q} - q) - k_d\dot{q} \quad (5.4)$$

- k_p ：**刚度（比例增益）**，越大跟踪越精准，但易不稳定
- k_d ：**阻尼（微分增益）**，越大振荡越小，但过大使动作“黏稠”

[NOTE] 典型经验参数（50 kg 人形角色）

$k_p = 200$, $k_d = 20$, 仿真步长 $h = 0.5 \sim 1$ ms（即 1000 ~ 2000 Hz）。轻质 link（如手指）需更小 gain；动态运动（跳跃）需更大 gain；高 gain 必须配合更小步长。

5.4 PID 控制

加入积分项专门消除稳态误差：

$$f = k_p e + k_d \dot{e} + k_i \int_0^t e dt' \quad (5.5)$$

[WARN] I 项在角色动画中很少使用

积分项会累积历史误差，引入“记忆效应”和过冲，调参困难。角色动画的稳态误差通常直接用逆动力学重力补偿处理，而非 I 项。

5.5 PD 的稳定性分析

考虑一维无重力系统， $m = 1$ ， $f = -k_p x - k_d v$ （即目标为原点）。使用半隐式 Euler 积分（先更新速度，再更新位置）：

$$v_{n+1} = v_n + h(-k_p x_n - k_d v_n), \quad x_{n+1} = x_n + h v_{n+1} \quad (5.6)$$

整理为矩阵形式：

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}}_A \begin{bmatrix} v_n \\ x_n \end{bmatrix} \quad (5.7)$$

记 $s_n = (v_n, x_n)^T$ ，则 $s_{n+1} = A s_n$ ，反复迭代得 $s_n = A^{n-1} s_1$ 。

对 A 特征分解： $A = P \begin{pmatrix} \lambda_1 & \\ & \lambda_2 \end{pmatrix} P^{-1}$ ，令 $z = P^{-1} s$ ：

$$z_{n+1} = \begin{pmatrix} \lambda_1 & \\ & \lambda_2 \end{pmatrix} z_n \Rightarrow z_i^{(n)} = \lambda_i^{n-1} z_i^{(1)} \quad (5.8)$$

$$\boxed{\text{稳定条件: } |\lambda_i| \leq 1, \quad \forall i} \quad (5.9)$$

任一 $|\lambda_i| > 1 \rightarrow$ 指数发散，系统不稳定。 k_p 越大、步长 h 越大 $\rightarrow$ 特征值更容易超出单位圆 $\rightarrow$ 更容易发散。

[WARN] 显式 PD 高刚度发散（最常见的实现 bug）

对 50 kg 角色用 $k_p = 2000$ （刚度提升 10 倍）、步长 $h = 1$ ms，矩阵 A 的特征值可能超过 1，导致仿真在几帧内爆炸。**症状**：角色突然飞到无穷远或四肢剧烈抖动。**根因**：显式 PD 对 v_n 是显式的（力用当前帧速度算），当 $k_p h^2 / m > 1$ 时积分不稳定。**解决**：降低步长，或改用 Stable PD（下节）。

5.6 Stable PD 控制（隐式阻尼）

核心思想：对阻尼项使用隐式速度（下一帧 v_{n+1} ）而非显式速度（当前帧 v_n ）：

$$v_{n+1} = v_n + h(-k_p x_n - k_d v_{n+1}) \quad (5.10)$$

解出 v_{n+1} (显式解):

$$v_{n+1} = \frac{v_n - hk_p x_n}{1 + hk_d} \quad (5.11)$$

位置更新不变: $x_{n+1} = x_n + hv_{n+1}$ 。

整体矩阵形式 ($m = 1$):

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \frac{1}{1 + hk_d} \begin{bmatrix} 1 & -k_p h \\ h & 1 + k_d h - k_p h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix} \quad (5.12)$$

[NOTE] Stable PD 的收益

对阻尼项隐式化使系统对 k_d **无条件稳定** (分母 $1 + hk_d > 1$, 特征值幅度整体缩小), 对 k_p 的容忍度也大幅提升。实践效果: Stable PD 允许将步长从 $h = 0.5 \text{ ms}$ 放宽到 $h \approx 1/120 \sim 1/60 \text{ s}$ (与渲染帧率同步), 同时保持高增益跟踪。这具有巨大工程价值——不再需要以 2000 Hz 运行仿真。

多 DoF 真实版本 (Tan et al. 2011): 在每步对整套铰接动力学局部线性化, 求解 $O(N)$ 线性方程组, 将隐式阻尼推广到全身所有关节。

两种 PD 的对比:

属性	显式 PD	Stable PD
阻尼项使用的速度	v_n (当前帧, 显式)	v_{n+1} (下帧, 隐式)
典型步长	$0.5 \sim 1 \text{ ms}$	$1/120 \sim 1/60 \text{ s}$
高增益稳定性	易发散	大幅改善
实现复杂度	极简 (一行公式)	需解线性方程组

6. Jacobian Transpose 控制 (虚拟力控制)

6.1 问题: 末端力 $\rightarrow$ 关节力矩

设末端点 x (如手端、CoM), 希望对其施加虚拟力 f (这个力实际上不存在), 但想通过关节力矩 τ 重现相同的即时效果。

关键约束: 两套驱动方式的功率相等 (能量等价原则):

$$P = f^T \dot{x} = \tau^T \dot{\theta} \quad (6.1)$$

6.2 推导: 功率匹配 $\rightarrow$ 雅可比转置

从正运动学 $x = g(\theta)$ 对时间求导:

$$\dot{x} = \frac{\partial g}{\partial \theta} \dot{\theta} = J \dot{\theta}, \quad J = \frac{\partial g}{\partial \theta} \in \mathbb{R}^{3 \times n} \quad (6.2)$$

代入功率等式:

$$f^T(J\dot{\theta}) = \tau^T\dot{\theta} \Rightarrow (f^T J)\dot{\theta} = \tau^T\dot{\theta}, \quad \forall \dot{\theta} \quad (6.3)$$

因此：

$$\boxed{\tau = J^T f} \quad (6.4)$$

这就是 Jacobian Transpose Control，也称虚拟模型控制 (Virtual Model Control)。

6.3 几何解读：每个铰链关节的贡献

对铰链 (hinge) 关节 i ，转轴方向 a_i ，从关节位置到末端的向量 $r_i = x - p_i$ ：

$$\frac{\partial g}{\partial \theta_i} = a_i \times r_i \quad (6.5)$$

代入 $\tau = J^T f$ ：

$$\tau_i = (a_i \times r_i) \cdot f = a_i \cdot (r_i \times f) \quad (6.6)$$

对球关节 (可绕任意轴转动)，力矩向量为：

$$\boxed{\tau_i = (x - p_i) \times f} \quad (6.7)$$

直觉：关节力矩 = “末端到关节的力臂” × “虚拟力”，即虚拟力对该关节产生的力矩。

[EX] Jacobian Transpose 的三个典型应用

1. **重力补偿**：对每个 link 的质心施加虚拟力 $f_i = -m_i g$ ，再对链上所有关节累加贡献：

$$\tau_k = \sum_{i \geq k} (x_i - p_k) \times (-m_i g)$$

等价于 RNEA 在零加速度条件下的输出，但这个形式更直观。

2. **末端跟踪 (任务空间控制)**：手要跟踪目标 $\bar{x}$ ，先算末端 PD 力：

$$f = k_p(\bar{x} - x) - k_d\dot{x}$$

再翻译成关节力矩 $\tau = J^T f$ 。好处：用笛卡尔坐标设目标，避免繁琐的 IK。

3. **平衡辅助**：对 CoM 施加水平校正力，再通过腿部关节的 J^T 翻译为踝/膝/腕力矩 (见 §8)。

7. 跟踪控制与残差力

7.1 全身 Tracking Controller

每个关节 j 独立执行 PD：

$$\tau_j = k_p(\bar{q}_j - q_j) - k_d\dot{q}_j \quad (7.1)$$

目标姿态 $\bar{q}(t)$ 来源:

- mocap 数据序列 (按时间索引)
- 手工关键帧 / 状态机驱动的姿态
- 抽象模型 (如逆向摆模型 + IK)

[EX] 经典工程案例

– Hodgins & Wooten 1995: “Animating Human Athletics”——手编关节轨迹 + PD 跟踪完成跳水、跑步等高动态动作。– NaturalMotion Endorphin: 把关键帧动画交给物理跟踪, 做”会受力反应”的动画角色。– SAMCON (Liu et al. 2010): 将 mocap 剪切片段, 用 PD 跟踪 + MCTS 采样拼接成稳定的全身运动。

7.2 纯关节力矩的局限

用纯关节力矩跟踪 mocap 时:

- 稳态误差: 在重力下, PD 永远差一个 $e_0 = mg/k_p$ 偏差, 手臂永远到不了目标角度。
- 相位滞后: 仿真轨迹永远落后参考轨迹一个相位 (PD 是”被动追赶”。
- 刚度困境: 大 k_p 减小误差但不稳定; 小 k_p 稳定但跟不上。

7.3 残差力/力矩 (Residual Force / Torque)

[WARN] 残差力的含义与代价

在正常关节力矩 τ_j 之外, 额外对根节点 (如骨盆) 施加一个任意力 f_0 和力矩 τ_0 , 称为残差力/残差力矩:

$$f_0, \tau_0 \text{ 直接作用在 root link 上 (不满足”净内力为零”)} \quad (7.2)$$

优点: 有了”来自根节点的任意外力”, 跟踪误差几乎可以消除, mocap 跟踪极其稳健。

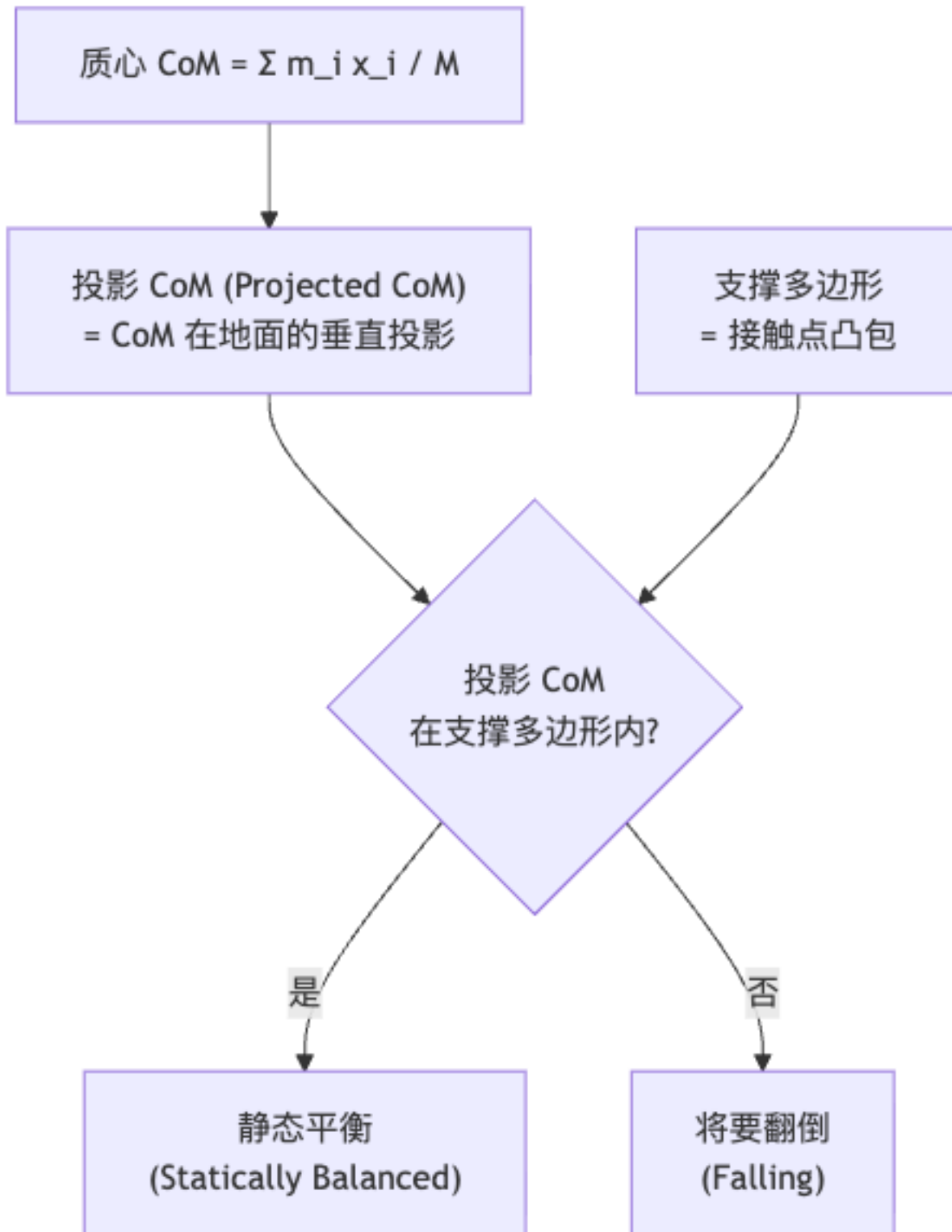
代价: 违背动量守恒——系统的总外力净不为零, 就像有”隐形的”手在扶着角色。这在物理上不可能存在。

使用场景: – 电影级物理动画 (追求视觉效果, 接受物理作弊) – RL 训练初期辅助 (加速收敛, 训练完去掉) – 论文 demo (Zordan et al. 2005”Dynamic Response for Mocap Animation”)

残差力的大小可以衡量”控制器有多依赖作弊”——力越小, 控制器越真实。

8. 静态平衡 (Static Balance)

8.1 平衡的几何定义



- 质心 (Center of Mass, CoM): $c = \sum_i m_i x_i / M$, 其中 $M = \sum_i m_i$ 为总质量。

- 支撑多边形 (Support Polygon): 所有与地面接触点 (双脚脚底、手等) 的凸包 (convex hull)。
- 静态平衡判据: CoM 的地面投影 $c_{\perp}$ 落在支撑多边形内部。

[NOTE] 物理直觉

当 $c_{\perp}$ 在支撑多边形外时, 重力产生的力矩超过接触力所能提供的反力矩, 角色必然翻转。就像倾斜一个杯子: 当重心的竖直线越过底面边缘, 杯子就倒。

8.2 踝策略与髋策略

策略	适用场合	机制
踝策略 (Ankle Strategy)	小扰动	只调节踝关节力矩, 整身后后摆动如”倒立摆”
髋策略 (Hip Strategy)	较大扰动	髋关节屈伸重新分配上下半身角动量
迈步策略 (Stepping)	大扰动	迈步重置支撑多边形 (动态平衡, Lec 10 内容)

8.3 简单 PD 平衡控制器

目标: 保持 CoM 投影 $c_{\perp}$ 接近支撑多边形中心 $\bar{c}$ 。

方法 A: 直接对踝/髋关节施加 PD 力矩

$$\tau_{\text{balance}} = k_p(\bar{c} - c) - k_d\dot{c} \quad (8.1)$$

将此力矩叠加到踝关节 (踝策略) 或髋关节 (髋策略) 的 PD 输出上。

方法 B: 虚拟力 + Jacobian Transpose (推荐)

1. 计算 CoM 处所需的水平校正虚拟力:

$$f_{\text{CoM}} = k_p(\bar{c} - c) - k_d\dot{c} \quad (8.2)$$

2. 用腿部各关节的 Jacobian Transpose 翻译:

$$\tau_i = (c - p_i) \times f_{\text{CoM}} \quad (8.3)$$

方法 B 的优点: 力矩自动按”力臂”分配给各关节, 不需要手动指定哪个关节承担多少; 自然地同时作用在踝、膝、髋。

[WARN] PD 平衡控制的局限

k_p, k_d 需要手工调节; 只能处理准静态扰动; 对动态扰动 (如被推一把) 无法快速响应。这引出了更高级的动量控制方法。

8.4 动量控制 (Momentum Control)

Macchietto et al. 2009 “Momentum Control for Balance”:

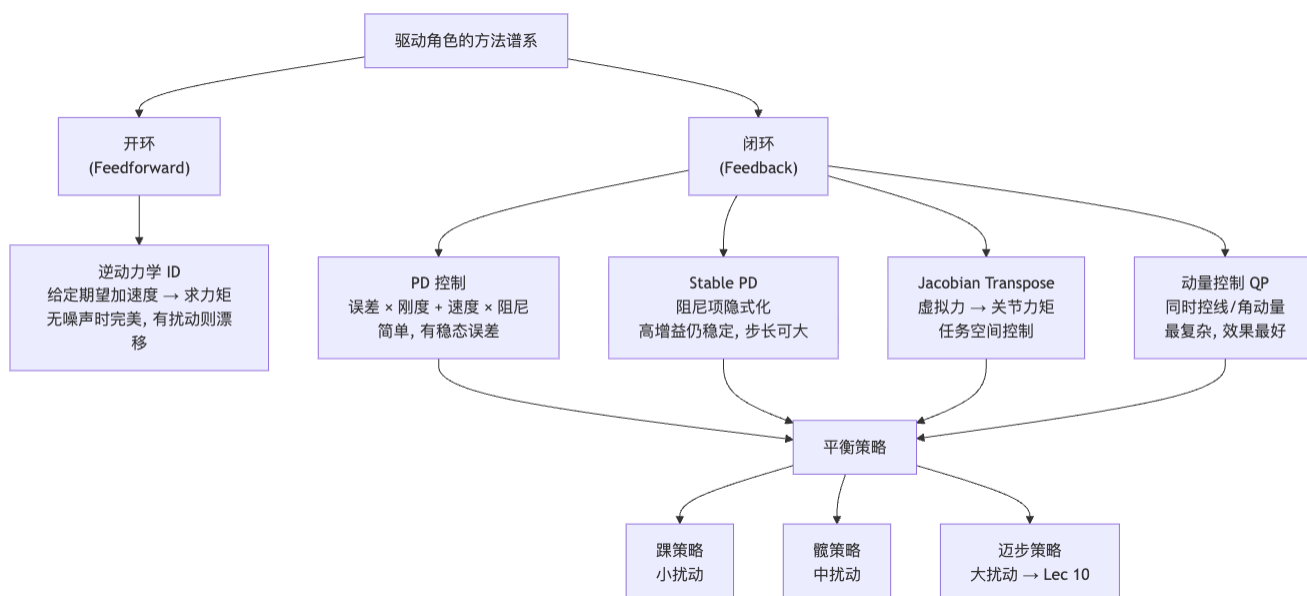
[EX] 单步二次规划 (QP)

同时控制线动量 (CoM 速度) 和角动量:

$$\min_{\tau, \lambda} \|M_c \dot{v}_{\text{des}} - (M_c \dot{v} + C_c - \tau_c \circ e - J_c^T \lambda)\|^2 + w \|\tau\|^2 \quad (8.4)$$

约束: 接触力在摩擦锥内; 关节力矩不超过饱和值; mocap 跟踪作为目标函数项。
每仿真步求解。相比纯 PD, 可以”主动调度”上下肢的角动量贡献——跌倒前能自动伸臂、屈膝来恢复平衡。

9. 控制方法谱系与平衡策略总览



10. Worked Examples (例题)

[EX] 例题 1: 关节力矩施加

问: 一个两刚体系统, link 1 (父) 和 link 2 (子) 通过关节相连。想对关节施加 $\tau = [0, 0, 5]^T$ N·m (绕 z 轴) 的力矩。应如何修改两个刚体的运动方程?

答: 对 link 2 (child) 在角动量方程右端加 $+\tau = [0, 0, 5]^T$; 对 link 1 (parent) 在角动量方程右端加 $-\tau = [0, 0, -5]^T$ 。线动量方程不受影响 (合力为零)。整个系统的净外力矩贡献为零, 动量守恒。

[EX] 例题 2: PD 稳定性判断

问: 参数 $k_p = 400$, $k_d = 20$, $m = 1$ kg, 步长 $h = 0.01$ s。矩阵 A 为:

$$A = \begin{bmatrix} 1 - 20 \times 0.01 & -400 \times 0.01 \\ 0.01 \times (1 - 20 \times 0.01) & 1 - 400 \times 0.01^2 \end{bmatrix} = \begin{bmatrix} 0.8 & -4 \\ 0.008 & 0.96 \end{bmatrix}$$

计算 $\text{trace}(A) = 1.76$, $\det(A) = 0.8 \times 0.96 - (-4) \times 0.008 = 0.768 + 0.032 = 0.8$ 。特征多项式 $\lambda^2 - 1.76\lambda + 0.8 = 0$, 解得 $\lambda_{1,2} = (1.76 \pm \sqrt{3.0976 - 3.2})/2$ 。判别式 < 0 , 特征值为复数, $|\lambda| = \sqrt{\det(A)} = \sqrt{0.8} \approx 0.894 < 1$, 系统稳定。

如果改为 $h = 0.1$ s: $A = \begin{bmatrix} -1 & -40 \\ -0.1 & -3 \end{bmatrix}$, $\det(A) = 3 - 4 = -1$, $|\det(A)| = 1$, $|\lambda| > 1$, 系统不稳定 (发散)。

[EX] 例题 3: Jacobian Transpose 重力补偿

问: 一个两段平面连杆, link 1 质量 $m_1 = 2$ kg (质心在关节 1 到关节 2 中点), link 2 质量 $m_2 = 1$ kg (质心在关节 2 到末端中点)。关节 1 在原点, 关节 2 在 $(0, 0.5)$ m, 末端在 $(0, 1.0)$ m (竖直向上)。求关节 1 和关节 2 需要的重力补偿力矩 (取 $g = 10$ m/s²)。

答: 用公式 $\tau_k = \sum_{i \geq k} (x_i - p_k) \times (-m_i g)$, 重力 $g = [0, -10]^T$ m/s²。

link 1 质心 $x_1 = [0, 0.25]^T$, link 2 质心 $x_2 = [0, 0.75]^T$, 关节 1 位置 $p_1 = [0, 0]^T$, 关节 2 位置 $p_2 = [0, 0.5]^T$ 。

关节 2 力矩: $\tau_2 = (x_2 - p_2) \times (-m_2 g) = [0, 0.25] \times [0, 10] = 0$ N·m (链段与重力平行, 力矩为零)。

关节 1 力矩: $\tau_1 = (x_1 - p_1) \times (-m_1 g) + (x_2 - p_1) \times (-m_2 g) = [0, 0.25] \times [0, 20] + [0, 0.75] \times [0, 10] = 0 + 0 = 0$ N·m (竖直链, 重力补偿力矩也为零——因为系统已在重力方向对齐, 无需力矩维持姿态)。

[EX] 例题 4: Stable PD 的优势

问: 同样的弹簧-阻尼系统 $k_p = 400$, $k_d = 20$, $m = 1$, 改用 Stable PD (阻尼隐式)。写出一步更新公式, 并说明为何比显式 PD 更稳定。

答: Stable PD 更新:

$$v_{n+1} = \frac{v_n - h k_p x_n}{1 + h k_d} = \frac{v_n - 4x_n}{1.2}$$

$$x_{n+1} = x_n + h v_{n+1}$$

矩阵 $A_{\text{stable}} = \frac{1}{1.2} \begin{bmatrix} 1 & -4 \\ 0.01 & 1.2 - 0.04 \end{bmatrix}$ 。分母 $1 + h k_d = 1.2 > 1$ 使矩阵整体“缩小”—— $|\det(A_{\text{stable}})| = |\det(A_{\text{explicit}})| / (1 + h k_d)$, 特征值幅度显著减小。物理解释: 对阻尼项隐式化等价于“预测下一时刻的阻尼效果”, 使阻尼力永远不会“过冲”, 从而在较大步长下仍保持稳定。

[EX] 例题 5：静态平衡判断与策略选择

问：一个人形角色双脚站立，左脚接触点 $(-0.1, 0)$ m，右脚接触点 $(0.1, 0)$ m（仅考虑 2D）。CoM 当前位置 $(0.15, 0.9)$ m，速度 $\dot{c} = (0.02, 0)$ m/s。支撑多边形为区间 $[-0.1, 0.1]$ 。判断平衡状态并建议控制策略。

答：CoM 投影 $c_{\perp} = 0.15$ m，超出支撑多边形右边界 0.1 m，角色正在失衡。

建议策略：用 PD 计算校正虚拟力 $f_x = k_p(0 - 0.15) - k_d(0.02)$ （目标为支撑多边形中心 $x = 0$ ），由于失衡较小，可采用**踝策略**——将虚拟力通过 Jacobian Transpose 转为踝关节力矩，略微背屈（向左）来将 CoM 拉回。如果扰动持续增大，切换**髌策略**或**迈步策略**。

11. Anki 卡片

#	Q (正面)	A (背面)
1	极大坐标与广义坐标分别用什么描述铰接系统？各有什么代价？	极大坐标：每 link 独立 $(x_i, Q_i, v_i, \omega_i)$ ，需约束 $Jv = 0$ + 拉格朗日乘子；广义坐标：关节角 θ ，自动满足约束但含大量非线性项
2	关节力矩 τ 如何施加到两个相连刚体上？	对 child（子）刚体加 $+\tau$ ，对 parent（父）刚体加 $-\tau$ ；系统净外力矩为零，动量守恒
3	正动力学 (FD) 与逆动力学 (ID) 的输入输出分别是什么？	FD：已知 $x, v, f \rightarrow$ 求 $\dot{v}$ （仿真器）；ID：已知 $x, v, \dot{v} \rightarrow$ 求 f, τ （控制器）
4	RNEA 两遍递归各做什么？	前向：root $\rightarrow$ tip，传播运动学（位置/速度/加速度）；后向：tip $\rightarrow$ root，从叶节点“回收”力矩，逐 link 解 Newton-Euler 方程
5	广义坐标下的逆动力学方程（含重力项）是什么？	$\tau = M(\theta)\ddot{\theta} + C(\theta, \dot{\theta}) + G(\theta)$
6	PD 控制律（对角色关节）是什么公式？	$\tau = k_p(\bar{q} - q) - k_d\dot{q}$ ； k_p 为刚度， k_d 为阻尼
7	显式 PD 数值稳定条件是什么？为何高刚度易发散？	矩阵 A 的所有特征值 $ \lambda_i \leq 1$ ； k_p 越大、 h 越大，特征值越容易超出单位圆，指数发散
8	Stable PD 与显式 PD 的关键区别是什么？	Stable PD 对阻尼项使用下一帧速度 v_{n+1} （隐式），显式 PD 用当前帧 v_n ；Stable PD 在大增益下仍稳定，步长可放宽到 $1/60$ s
9	Stable PD 的一步速度更新公式是什么？	$v_{n+1} = (v_n - hk_p x_n)/(1 + hk_d)$
10	Jacobian Transpose 控制的推导出发点是什么？结论公式？	功率匹配： $f^T \dot{x} = \tau^T \dot{\theta}$ ，利用 $\dot{x} = J\dot{\theta}$ 推出 $\tau = J^T f$
11	对球关节 i ，虚拟力 f 作用在末端 x ，关节力矩是什么？	$\tau_i = (x - p_i) \times f$ ，其中 p_i 为关节位置
12	欠驱动角色与全驱动角色的区别？人形角色为何欠驱动？	欠驱动：执行器数 $<$ DoF，某些运动无法实现；人形角色根关节（骨盆）6 DoF 无电机，只能靠接触力间接控制
13	静态平衡的几何判据是什么？	CoM 的地面投影（Projected CoM）落在支撑多边形（接触点凸包）内部
14	踝策略与髌策略分别适用于什么扰动级别？	踝策略：小扰动（整身如倒立摆微摆）；髌策略：中等扰动（髌屈伸重新分配角动量）
15	残差力的优点和代价各是什么？	优点：消除跟踪误差，mocap 跟踪极稳健；代价：违背动量守恒（系统净外力不为零，物理不可实现）

#	Q (正面)	A (背面)
16	PID 中 I 项为何在角色动画中很少用?	积分项累积历史误差引入记忆效应和过冲, 调参困难; 稳态误差通常用逆动力学重力补偿处理

12. 中英术语速查表

中文	English	说明
极大坐标	Maximized Coordinates	每 link 独立存储笛卡尔位姿
广义坐标	Generalized Coordinates	关节角参数化, 维数 = DoF
关节力矩	Joint Torque	驱动关节转动的力矩 τ
正动力学	Forward Dynamics (FD)	力 $\rightarrow$ 加速度; 仿真器
逆动力学	Inverse Dynamics (ID)	加速度 $\rightarrow$ 力; 控制器
递归 Newton—Euler 算法	Recursive Newton—Euler Algorithm (RNEA)	$O(N)$ 的 ID 算法
比例-微分控制	Proportional-Derivative Control (PD)	$\tau = k_p e + k_d \dot{e}$
比例-积分-微分控制	Proportional-Integral-Derivative (PID)	含积分消除稳态误差
稳定 PD 控制	Stable PD Control	阻尼项隐式化的 PD
前馈控制	Feedforward / Open-loop Control	与状态无关的控制信号
反馈控制	Feedback / Closed-loop Control	依赖当前状态的控制
雅可比转置控制	Jacobian Transpose Control	$\tau = J^T f$
虚拟力控制	Virtual Model Control	虚拟力 + Jacobian Transpose
欠驱动	Underactuated	执行器数 < DoF
全驱动	Fully-Actuated	执行器数 $\geq$ DoF
残差力/力矩	Residual Force / Torque	施加在根节点的辅助外力 (作弊)
质心	Center of Mass (CoM)	$c = \sum m_i x_i / M$
支撑多边形	Support Polygon	接触点的凸包
踝策略	Ankle Strategy	仅调踝关节应对小扰动
髋策略	Hip Strategy	调髋关节应对中等扰动
动量控制	Momentum Control	同时控制线动量和角动量的 QP 方法
刚度/比例增益	Stiffness / Proportional Gain (k_p)	PD 的位置误差权重
阻尼/微分增益	Damping / Derivative Gain (k_d)	PD 的速度项权重
重力补偿	Gravity Compensation	用 ID 计算抵消重力所需力矩
全身跟踪控制器	Full-body Tracking Controller	每关节独立 PD 跟踪目标姿态

13. 复习要点

[TIP] 期末复习要点 (本讲必记)

1. 两种坐标系: 极大坐标 (每 link 独立, 需约束) vs. 广义坐标 (关节角参数化, 自动满足约束, DoF 等于变量数); 正运动学 (FK) 连接二者, RNEA 在广义坐标下实现 $O(N)$ 逆动力学。
2. 关节力矩的施加规则: $+\tau$ 给 child, $-\tau$ 给 parent。净内力为零 $\rightarrow$ 总动量守恒。这意味着角色不能靠内部力矩“飞起来”, 必须借助接触力 (地面反力) 才能改变系统总动量。
3. FD vs. ID: $M\ddot{\theta} + C + G = \tau$; FD 是仿真器 (已知 τ 求 $\ddot{\theta}$), ID 是控制器 (已知期望 $\ddot{\theta}$ 求 τ)。重力补偿是最简单的 ID 应用: 给 $\ddot{\theta} = 0$, 算出抵消 G 所需力矩。
4. PD 稳定性: 矩阵 A 特征值 $|\lambda| \leq 1$ 是稳定条件; k_p 或 h 过大会让特征值超出单位圆。Stable PD 对阻尼隐式化 (用 v_{n+1} 代替 v_n), 显著扩大稳定域, 允许大步长 (1/60 s) 高增益运行。
5. Jacobian Transpose: 从功率匹配 $f^T \dot{x} = \tau^T \dot{\theta}$ 推导 $\tau = J^T f$; 对球关节 $\tau_i = (x - p_i) \times f$; 用于虚拟力控制 (重力补偿、末端跟踪、平衡辅助)。
6. 欠驱动的含义: 人形角色根关节无电机, 必须靠接触力间接控制 CoM。这是平衡控制困难的本质。
7. 静态平衡: CoM 投影在支撑多边形内 = 平衡; 踝策略 (小扰动)、髌策略 (中扰动)、迈步 (大扰动); 动量控制 QP 是最完整的解法。
8. 残差力的双面性: 消除跟踪误差 (工程实用), 但违背物理守恒 (sim-to-real 危险)。

[NOTE] 本讲在课程中的位置

– 承接 Lec 08: 被动仿真框架 + Newton–Euler 方程。– 承接 Lec 03: FK / Jacobian, $\tau = J^T f$ 直接用到。– 向 Lec 10 输出: 踝/髌静态平衡 $\rightarrow$ 走路是“连续的失衡” $\rightarrow$ ZMP、倒立摆、SIMBICON 等动态平衡。– 向 RL 类课程输出: PD 控制是 RL 动作空间的标配——策略网络输出目标关节角 $\bar{q}$, PD 转成 τ (DeepMimic, AMP 均基于此范式)。

Lecture 10 —学习走路 (Learning to Walk)

[TIP] 学习指南

本节核心：走路 = 连续可控的失衡 + 步伐规划。Lec 09 的静态平衡（CoM 投影始终在支撑多边形内）过于保守——真正的人类步态允许 CoM 瞬时离开支撑区，靠”及时迈步”重置。本讲围绕四条主线展开：(1) 步态相位分析——单/双支撑相与飞行相的几何意义；(2) ZMP / CoP——脚不翻转的充要判据与力矩平衡推导；(3) 简化动力学模型——从 IPM 到 LIPM 再到弹簧负载 SLIP，以及轨道能量与步态切换的关系；(4) 走路控制器——Cart-Table / Preview Control 做轨迹生成，SIMBICON 做 FSM + 线性反馈。

前置知识：- lec08 多刚体动力学 $M\dot{v} + C = f + f_J$ ；接触力/约束 - lec09 CoM、支撑多边形、PD 控制、Jacobian Transpose、ankle/hip strategy

重点掌握：1. 步态相位图（单/双支撑相）在时间轴上的排布与几何含义 2. ZMP 的力矩平衡推导、CoP 公式、脚翻转判据（ZMP 出支撑多边形 → 脚不能平贴地）3. LIPM 完整推导（固定高度假设 → 线性化 → $\ddot{x} = \frac{g}{z_c}(x - p)$ ）与守恒轨道能量 4. Cart-Table 模型：从 LIPM 方程解出 ZMP，离散化得三对角系统，Preview Control 的必要性 5. SIMBICON 三步骤：FSM + world-frame 躯干控制 + CoM 线性反馈

0. 名词预热

名词	一句话解释
CoM	Center of Mass, 质心
GRF	Ground Reaction Force, 地面反作用力（向上）
CoP	Center of Pressure, 压力中心：法向 GRF 的作用点
ZMP	Zero-Moment Point, 零力矩点：使水平力矩为零的地面点
支撑多边形	Support Polygon: 所有接触点的凸包
IPM / LIPM	Inverted Pendulum Model / Linear IPM
SLIP	Spring-Loaded Inverted Pendulum, 弹簧负载倒立摆
FSM	Finite State Machine, 有限状态机

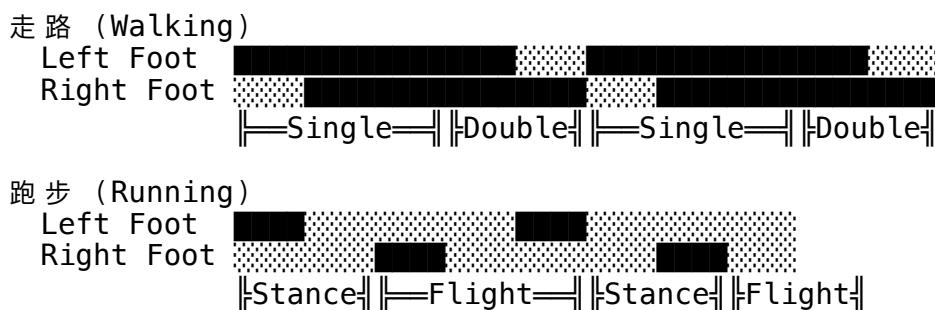
[NOTE] 直觉：走路的本质

走路不是”谨慎地让重心永远在脚下”，而是”有节律地向前失衡，再及时迈步接住自己”。每一步都是受控的跌倒。理解这一点，本讲的所有数学就有了物理直觉的根基。

1. 步态周期与相位分析

1.1 走路 vs. 跑步的相位定义

步态周期（Gait Cycle）是从一侧脚触地到同侧脚再次触地的完整循环。用时间轴描述左右脚的接地状态：



关键区别：

步态	相位	特征
走路 (Walking)	Single Stance (单支撑) + Double Stance (双支撑)	至少一脚始终接地，无腾空
跑步 (Running)	Stance (支撑) + Flight (飞行)	存在双脚离地的腾空相

[WARN] 走路的严格定义

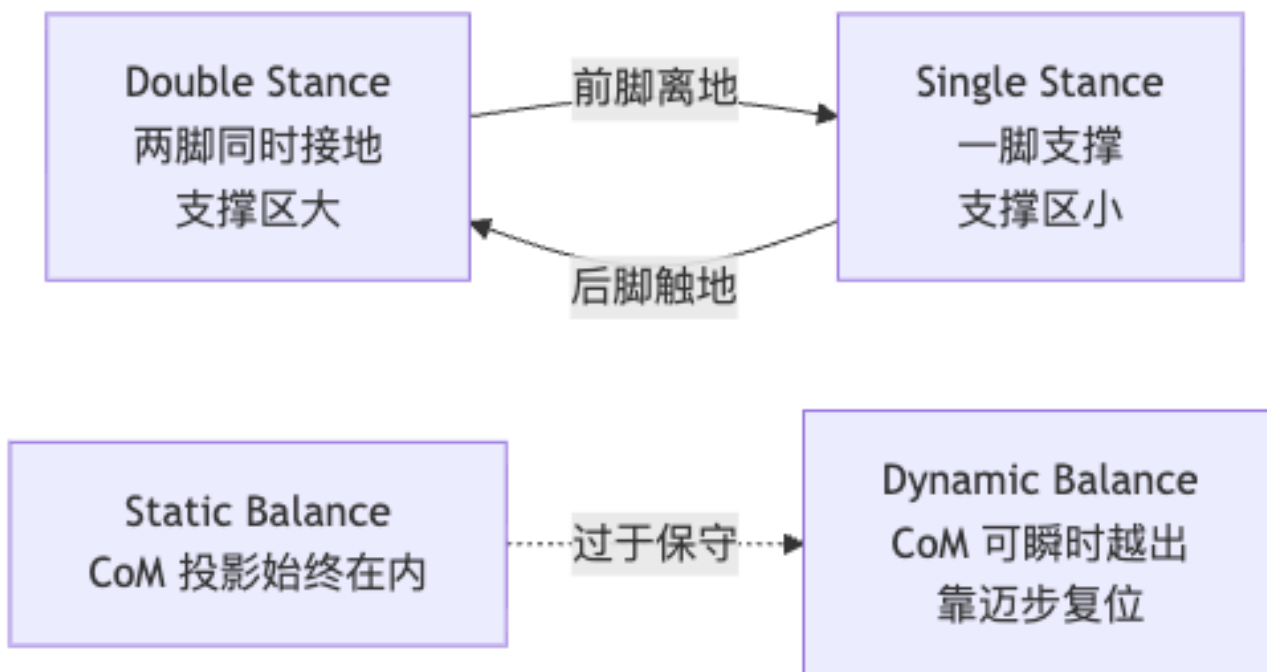
走路 = 没有飞行相 (flight phase)，即任意时刻至少一只脚与地面保持接触。这是行走与跑步的分水岭，也是 ZMP/LIPM 等地面约束成立的前提。

1.2 静态平衡走路：保守但僵硬

用静态平衡走路，则需要在每个瞬间保持 CoM 投影在支撑多边形内：

- **Double stance**：支撑多边形 = 两脚之间的凸包（较大）
- **Single stance**：支撑多边形 = 单脚底面（很小）

静态平衡要求 CoM 始终在当前支撑多边形内 → 步幅极小、行走僵硬（早期 ASIMO 的特征）。实际上，在 single stance 的大部分时间，人类的 CoM 投影在一只脚的支撑区外——这正是为什么需要动态平衡分析。



2. 零力矩点 ZMP

2.1 地面反作用力的分解

设地面接触区离散为 N 个接触点，第 i 点的位置为 p_i ，受到 GRF f_i 。地面平坦时，每个 f_i 可分为：

$$f_i = \underbrace{f_i^y}_{\text{法向 (竖直)}} + \underbrace{f_i^{xz}}_{\text{摩擦 (水平)}} \quad (2.1)$$

合力与对参考点 p 的总力矩：

$$f_{\text{GRF}} = \sum_i f_i \quad (2.2)$$

$$\tau_{\text{GRF}}(p) = \sum_i (p_i - p) \times f_i \quad (2.3)$$

2.2 水平 / 竖直力矩的拆分

地面平坦时， $(p_i - p)$ 始终在水平面内。叉积的水平分量只来自竖直力，竖直分量只来自水平力：

$$\tau_{\text{GRF}}^{xz} = \sum_i (p_i - p) \times f_i^y \quad (2.4a)$$

$$\tau_{\text{GRF}}^y = \sum_i (p_i - p) \times f_i^{xz} \quad (2.4b)$$

2.3 CoP 的推导

寻找 p 使得水平力矩分量 $\tau_{\text{GRF}}^{xz} = 0$ ：

$$\sum_i (p_i - p) \times f_i^y = \sum_i p_i \times f_i^y - p \times \sum_i f_i^y \hat{y} = 0$$

解出：

$$p_{\text{CoP}} = \frac{\sum_i p_i f_i^y}{\sum_i f_i^y} \quad (2.5)$$

这就是压力中心 (Center of Pressure, CoP)——法向 GRF 的加权质心，权重为各点法向力大小。

[NOTE] ZMP $\equiv$ CoP (平地条件下)

“零力矩点”的名称来自它让水平力矩分量为零。在平坦地面上，ZMP 就等于 CoP，两个说法完全等价。课件中常混用，指的是同一个点。

2.4 平衡方程与 ZMP 的力学意义

考察站立脚（stance phase）的静力学。假设角色整体处于准静态，脚踝从小腿接受内力 f_{ankle} 和力矩 τ_{ankle} ，系统受重力 mg ：

力平衡：

$$f_{\text{ankle}} + f_{\text{GRF}} + mg = 0 \quad (2.6)$$

对参考点 o 的力矩平衡（取水平分量 xz ）：

$$(u - o) \times f_{\text{ankle}}|_{xz} + (p - o) \times f_{\text{GRF}}|_{xz} + (x - o) \times mg + \tau_{\text{ankle}}|_{xz} = 0 \quad (2.7)$$

其中 u 是脚踝位置， x 是 CoM 位置。给定所有已知量，可以解出 p ，即 ZMP 的水平位置。

2.5 ZMP 判据：脚是否翻转

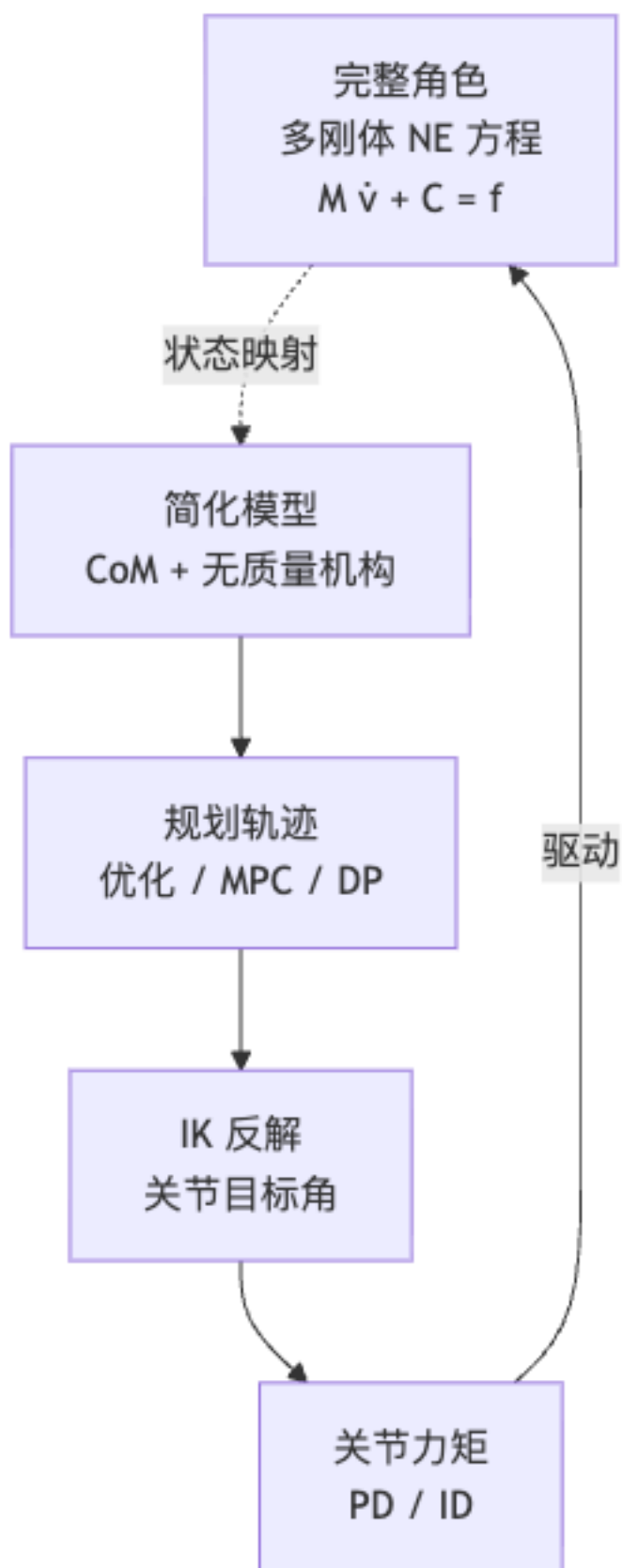
[WARN] 脚翻转的充要判据

解出 p 后：- $p \in \text{Support Polygon}$ ：ZMP 在脚底接触区内，GRF 可物理实现，脚不翻转。这是动态平衡存在的充要条件。- $p \notin \text{Support Polygon}$ ：所有 GRF 都来自多边形内，故真实 CoP p' 无法等于 p ，于是 $\tau_{\text{GRF}}^{xz} \neq 0$ ，脚会绕边缘滚动翻转。

控制目标：设计运动轨迹使得任意时刻，从方程 (2.7) 算出的 p 始终在支撑多边形内 → 保证脚贴地 → 维持动态平衡。

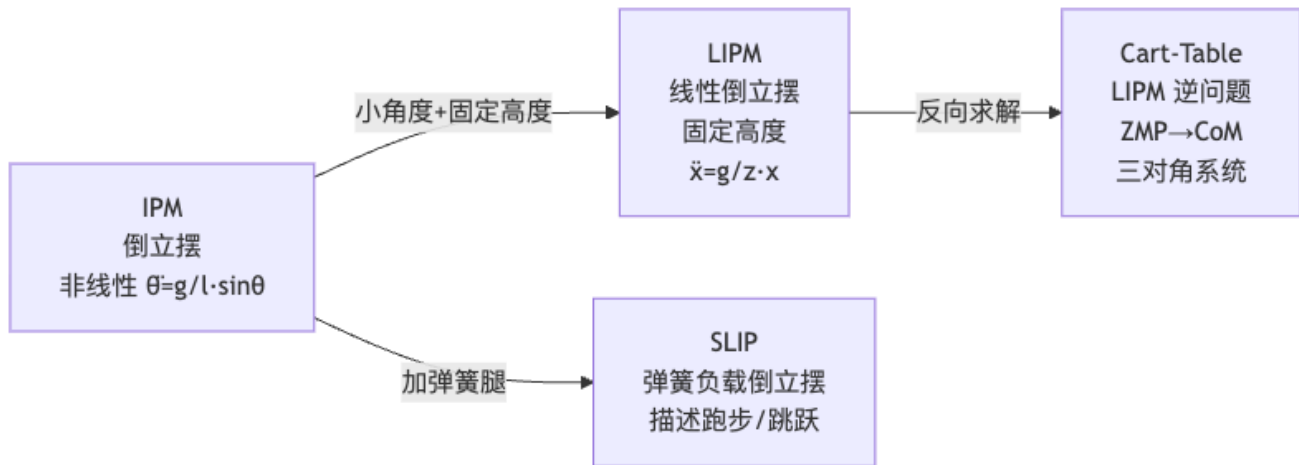
3. 简化模型总论

要直接控制完整人形机器人的关节力矩实现走路，自由度太多、计算太复杂。标准方法是简化层级建模：



核心思想：把机器人状态映射到低维简化模型，在低维空间规划控制，再通过 IK / ID 反射回全身。

3.1 模型谱系



4. 倒立摆模型 IPM

4.1 基本方程

质量 m 集中在顶端、杆长 l 、支点固定，仅受重力：

$$\ddot{\theta} = \frac{g}{l} \sin \theta \quad (4.1)$$

$\theta = 0$ (竖直向上) 是不稳定平衡——任意微小扰动都会指数发散。这个不稳定性正是走路“受控失衡”的物理基础。

[EX] 倒立摆在小车上

若把支点装在可以水平滑动的小车上，通过移动小车产生水平惯性力，可以稳定倒立摆——这是经典控制教材中的基准问题，也是 Cart-Table 模型的直观前身。参见 <https://www.youtube.com/watch?v=nOSTzpA0nGk>

4.2 弹簧负载倒立摆 SLIP

SLIP 在腿中加入弹簧，可以描述跑步和跳跃中的能量储存与释放 (Stance phase 压缩弹簧蓄能, Flight phase 释放)。Mordatch et al. 2010 “Robust Physics-Based Locomotion Using Low-Dimensional Planning”用 SLIP 在地形中规划低维轨迹，再用逆动力学转回全身力矩。

[NOTE] SLIP vs IPM

IPM 是纯刚杆 (腿长固定)，适合走路。SLIP 腿长可变，适合弹跳类运动。两者都把全身简化为单点质量 + 无质量机构。

5. 线性倒立摆模型 LIPM

5.1 线性化假设

LIPM 在 IPM 基础上增加一个约束：CoM 在固定高度 z_c 的水平面内运动 (即 $\dot{z} = 0$, $z = z_c = \text{const}$)。

物理意义：忽略走路时身体的上下浮动，将整个步态压缩到一个水平面内分析。这一假设使方程从非线性变为线性，是 LIPM 的核心创新。

5.2 完整推导

设 ZMP（支点）位置为 p （水平），CoM 位置为 (x, z_c) ，GRF 方向沿杆（从支点到 CoM）：

水平方向： $m\ddot{x} = f \sin \theta$

竖直方向： $f \cos \theta = mg$

其中 $\tan \theta = (x - p)/z_c$ （几何关系）。两式相除消去 f ：

$$m\ddot{x} = mg \cdot \frac{(x - p)/z_c}{1} = \frac{mg(x - p)}{z_c}$$

$$\ddot{x}_{\text{com}} = \frac{g}{z_c}(x_{\text{com}} - p_{\text{zmp}})$$

(5.1)

[NOTE] 方程的物理直觉

(5.1) 说：CoM 相对 ZMP 越偏右，向右的加速度越大——这是不稳定系统的典型特征（加速度与偏差同号）。ZMP 的位置（即“支点”位置）可以通过迈步来改变，从而影响 CoM 的运动轨迹。两个水平方向（前后 x 和左右 z ）的方程完全解耦，可以独立规划。

5.3 与 ZMP 的关系

将 (5.1) 反过来解出 ZMP：

$$p_{\text{zmp}} = x_{\text{com}} - \frac{z_c}{g} \ddot{x}_{\text{com}} \quad (5.2)$$

给定 CoM 轨迹 → 计算 ZMP 轨迹（正问题 / 步态分析方向）。

给定 ZMP 轨迹 → 计算 CoM 轨迹（逆问题 / 步态规划方向，见 §6 Cart-Table）。

5.4 轨道能量与步态切换

当 ZMP 固定在原点 ($p = 0$) 时，方程变为 $\ddot{x} = \frac{g}{z_c}x$ 。定义**轨道能量**（Orbital Energy）：

$$E = \frac{1}{2}\dot{x}^2 - \frac{g}{2z_c}x^2$$

(5.3)

[NOTE] 为什么叫“轨道能量”？

第一项是动能，第二项的负号反映不稳定势（势能“倒置”）。可以验证 $\dot{E} = \dot{x}\ddot{x} - \frac{g}{z_c}x\dot{x} = \dot{x}\left(\frac{g}{z_c}x - \frac{g}{z_c}x\right) = 0$ ，即 E 是运动常数——在 ZMP 固定时守恒。不同初始条件对应相平面上不同的轨道（双曲线族），故称“轨道能量”。

步态切换时的能量分析：设换脚时刻 CoM 到达位置 x_f ，速度 v_f ，迈步距离（步长）为 s ：

换脚前（以旧 ZMP 为参考）：

$$E_1 = \frac{1}{2}v_f^2 - \frac{g}{2z_c}x_f^2 \quad (5.4a)$$

换脚后（新 ZMP 在距旧 ZMP 距离 s 处，重置参考系）：

$$E_2 = \frac{1}{2}v_f^2 - \frac{g}{2z_c}(x_f - s)^2 \quad (5.4b)$$

速度控制规律：

换脚时机	x_f 的大小	E_2 与 E_1 的关系	效果
早换脚	x_f 小	$E_2 < E_1$	减速（轨道能量降低）
晚换脚	x_f 大	$E_2 > E_1$	加速（轨道能量升高）

[EX] 直觉：走路速度的”自然旋钮”

改变换脚时刻（即步频）可以调节行走速度，无需显式指定力矩。这正是**被动动态行走（Passive Dynamic Walking, McGeer 1990）**的核心发现：放在斜坡上的纯机械人，靠重力就能稳定行走——坡度补充能量，换脚时机决定速度。

5.5 IPM 步态规划（Coros et al. 2010）

将角色 CoM 和站立脚抽象为 IPM，规划下一步落脚位置：

[EX] Generalized Biped Walking Control 流程

1. 将角色 CoM + 站立脚映射为 IPM（记录当前 CoM 位置、速度和支点位置）2. 解析预测：要使下一步摆动完成后 CoM 恰好经过新支点正上方，落脚点应在何处？3. 根据落脚目标生成摆动腿轨迹（样条曲线或正弦轨迹）4. IK 求关关节目标角 $\bar{q}(t)$ 5. 关节 PD 跟踪，加 Jacobian Transpose 维持上半身姿态

6. Cart-Table 模型与 Preview Control

6.1 反问题设定

LIPM 的正问题（CoM $\rightarrow$ ZMP）用于步态分析和 IPM 步态规划。工业人形机器人（ASIMO、HRP 等）的设计方向不同：**先规划好脚步（ZMP 轨迹），再求腰部 CoM 轨迹。**这是逆问题：

给定 ZMP 轨迹 $\{p_1, p_2, \dots, p_{N-1}\} \rightarrow$ 求 CoM 轨迹 $\{x_0, x_1, \dots, x_N\}$

6.2 Cart-Table 物理直觉

将 LIPM 形象化为”无质量桌子 + 桌面上滑动的小车”：

- 桌腿：立于 ZMP 之上，代表无质量支撑结构
- 小车：质量 m ，在桌面上水平滑动，代表 CoM
- 桌子翻转：当小车位置不对，桌子会围绕底边翻倒

由 LIPM 方程 (5.1) 解出 ZMP:

$$p_{\text{zmp}} = x - \frac{z_c}{g} \ddot{x} \quad (6.1)$$

“桌子不翻”的条件等价于 p_{zmp} 落在支撑多边形内——和 §2.5 的 ZMP 判据完全一致。

[NOTE] 为什么要“控制小车运动”?

(6.1) 表明: 给定目标 ZMP (脚步规划), 必须让小车 (CoM) 以合适的加速度 $\ddot{x}$ 运动, 才能保证桌子不翻。这个加速度规划问题就是 Preview Control 要解决的。

6.3 离散化与三对角线性系统

将时间离散为步长 Δt , 用二阶中心差分近似加速度:

$$\ddot{x}_i \approx \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta t^2} \quad (6.2)$$

代入 (6.1):

$$p_i = x_i - \frac{z_c}{g} \cdot \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta t^2} = a_i x_{i-1} + b_i x_i + c_i x_{i+1} \quad (6.3)$$

其中系数:

$$a_i = -\frac{z_c}{g\Delta t^2}, \quad b_i = \frac{2z_c}{g\Delta t^2} + 1, \quad c_i = -\frac{z_c}{g\Delta t^2} \quad (6.4)$$

注意 $a_i = c_i$ (系统空间均匀时所有系数相同)。写成矩阵形式:

$$\begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{N-1} \end{bmatrix} = \begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{N-1} & b_{N-1} & \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} \quad (6.5)$$

这是三对角线性方程组, 给定边界条件 x_0, x_N , 可用 **Thomas 算法** 在 $O(N)$ 时间内求解。每个水平轴 (前后/左右) 独立求解。

[WARN] 边界条件很重要

方程 (6.5) 中 x_0 和 x_N 作为已知量 (边界条件), 必须预先给定。常见做法: 让角色在原地振荡几个周期“预热”, 取稳态作为起始条件; 终态同样需要设定 (如在目标位置静止)。

6.4 Preview Control (预览控制)

直接解三对角系统需要提前知道全部 ZMP 序列 (离线批处理), 无法处理实时变化的环境。Kajita et al. (2003) 提出预览控制 (Preview Control):

核心思想: 在每个时刻, “预览”未来 $T_{\text{preview}} \approx 1 \text{ s}$ 内的期望 ZMP 序列, 用 LQR (线性二次调节器) 框架求最优 CoM 加速度 $\ddot{x}^*$ 。

$$\min_{\ddot{x}} \sum_{k=0}^{T_{\text{preview}}} [w_p(p_k - p_k^*)^2 + w_v \dot{x}_k^2 + w_a \ddot{x}_k^2] \quad (6.6)$$

其中 p_k^* 是第 k 帧期望 ZMP，权重 w_p, w_v, w_a 调节 ZMP 跟踪精度与运动平滑性的权衡。

[NOTE] 为什么需要“预览”？

LIPM 是不稳定系统 ($\ddot{x}$ 与 $(x - p)$ 同号)。单纯负反馈无法稳定不稳定系统。将未来 ZMP 目标作为前馈输入 (lookahead)，在失衡发生前就提前调整 CoM，类似于人看到前方障碍物会提前调整步伐。这就是 ASIMO、HRP-2、Atlas 等工业机器人行走的核心算法。

[EX] 历史意义

Cart-Table + Preview Control 是 2000 年代人形机器人实现稳定行走的关键突破，对应本田 ASIMO 等机器人的商业成功。其后 2020 年代的深度强化学习方法虽然性能更强，但 ZMP/Preview Control 仍是理解“走路的数学”的最佳入门路径。

7. SIMBICON：有限状态机走路控制

[NOTE] SIMBICON = SIMple Blped LOcomotion CONtrol

Yin et al. 2007, SIGGRAPH。在 PD + FSM 框架上加一个简单线性反馈，即可使物理仿真双足角色稳定行走，并对外力扰动有较好鲁棒性。是基于物理的角色动画领域的里程碑。

7.1 Step 1: 周期 Base Motion (FSM)

用有限状态机定义走路的周期性姿态序列：



每个状态指定一组关节目标角 $\bar{q}$ (可来自 mocap 片段或手工设计)，关节 PD 控制器跟踪。状态切换条件：时间到 0.3 s，或摆动脚触地 (foot strike)。

7.2 Step 2: World-Frame 控制躯干与摆动腿

[WARN] 关节 PD 的问题

若对所有关节用“相对父 link 的角度”做 PD，则躯干会跟随支撑腿前倾——角色会越走越弯腰。必须改为在**世界坐标系 (World Frame)** 下控制躯干和摆动腿关节的绝对角度。

设 stance hip 关节为 A，躯干，swing hip 关节为 B：

- PD 控制躯干在世界系下的期望角度 $\rightarrow$ 得到力矩 τ_{torso}
- PD 控制 swing hip 在世界系下的期望角度 $\rightarrow$ 得到力矩 τ_B
- 由牛顿第三定律，stance hip A 的力矩由反力矩决定：

$$\tau_A = -\tau_{\text{torso}} - \tau_B \quad (7.1)$$

[NOTE] 为什么是这个公式？

stance leg 已撑地无法移动。躯干和摆动腿想往某个方向旋转，必然对 stance hip 施加等大反向的力矩 (Newton 第三定律)。stance hip 把这个反力矩经由腿传给地面，产生实际的支撑。这个关系完全由牛顿第三定律决定，不需要额外设计。

7.3 Step 3: CoM 线性反馈（核心创新）

仅靠 FSM + World-frame 控制，面对外部扰动（被推一把）仍会摔倒。SIMBICON 的关键创新是将 CoM 状态反馈到摆动腿的目标角度：

$$\bar{\theta}_d = \bar{\theta}_{d0} + c_d \cdot d + c_v \cdot v \quad (7.2)$$

其中： $-\bar{\theta}_{d0}$ ：FSM 给出的基础目标角度 - d ：CoM 相对 stance foot 的水平位移（前后方向） - v ：CoM 在 stance foot 参考系下的水平速度 - $c_d, c_v > 0$ ：反馈增益（经验值： $c_d \approx 0.5$, $c_v \approx 0.2$, 50 kg 人形）

[EX] 直觉：跨步策略 (Stepping Strategy)

- CoM 偏前 ($d > 0$) $\rightarrow \bar{\theta}_d$ 增大 $\rightarrow$ 摆动腿向前迈更大一步 $\rightarrow$ “接住”前倾的身体 - CoM 偏后 ($d < 0$) $\rightarrow$ 摆动腿后缩 $\rightarrow$ 重心被拉回前方 - 速度大 (v 大) $\rightarrow$ 需要更大步子才能稳住
这是 Lec 09 末尾 stepping strategy 的具体实现——通过改变摆动腿的落点位置来实时调整支撑多边形。

7.4 三层控制律总览

$$\underbrace{\text{FSM (base motion)}}_{\text{Layer 1: 周期骨架}} + \underbrace{\text{world-frame torso/swing}}_{\text{Layer 2: 姿态稳定}} + \underbrace{\text{linear CoM feedback}}_{\text{Layer 3: 扰动恢复}}$$

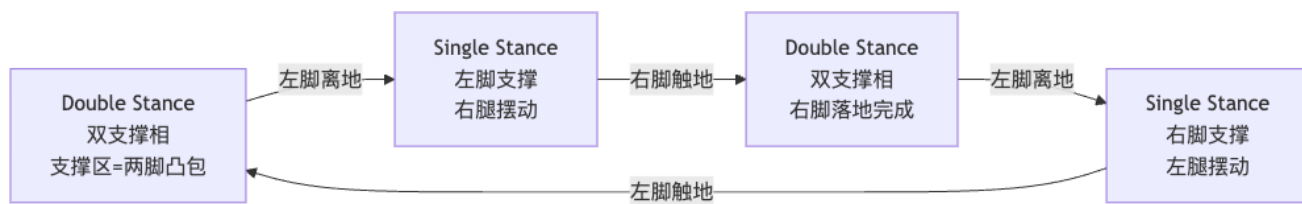
三层叠加，即可实现仿真角色的稳定行走、跑步、转向，以及对外力扰动的鲁棒恢复。这是 2007 年物理动画领域第一次在通用物理引擎（ODE）上用极简控制律实现 robust locomotion。

7.5 SIMBICON 的局限

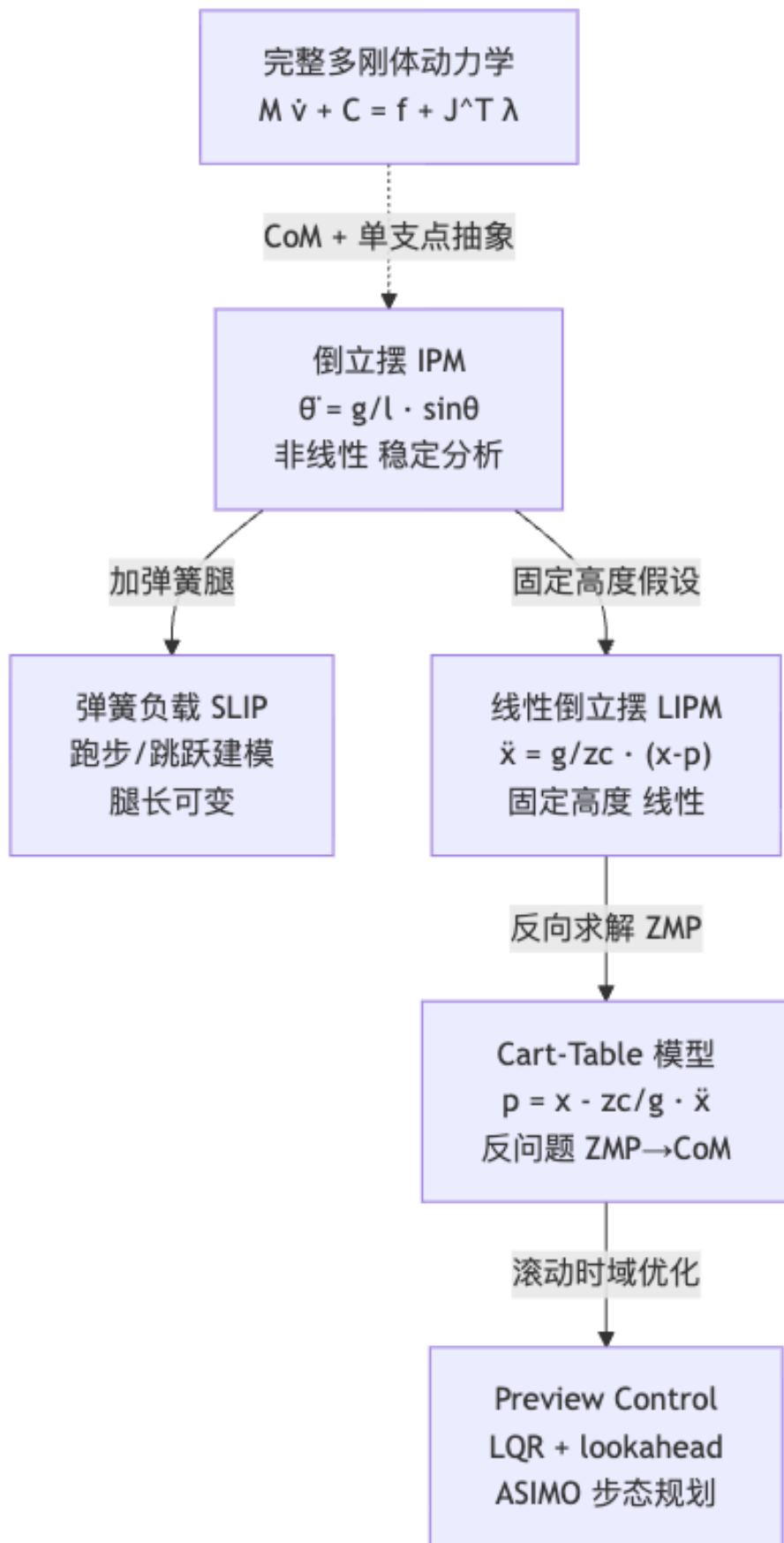
局限	描述	解决方向
周期性	只对周期步态好用，突发动作（跳、翻滚）需要更复杂 FSM	学习派 policy
手调参数	c_d, c_v 及 FSM 时长需手工调节	数据驱动 / RL
身形泛化	不同体型需重新设计 FSM 和目标角	通用化运动控制

8. 概念架构图

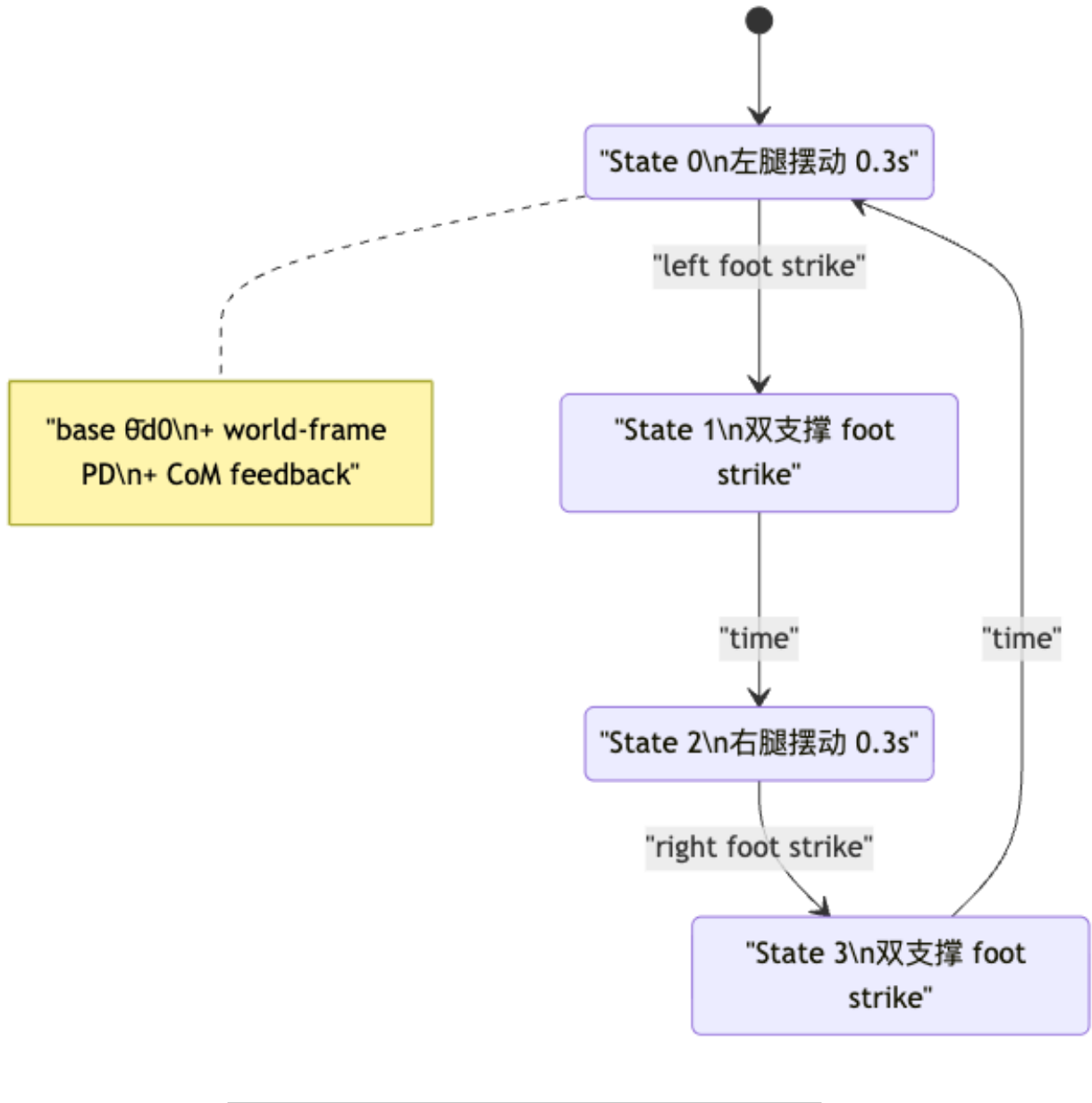
8.1 步态相位环



8.2 平衡模型谱系



8.3 SIMBICON 状态机完整图

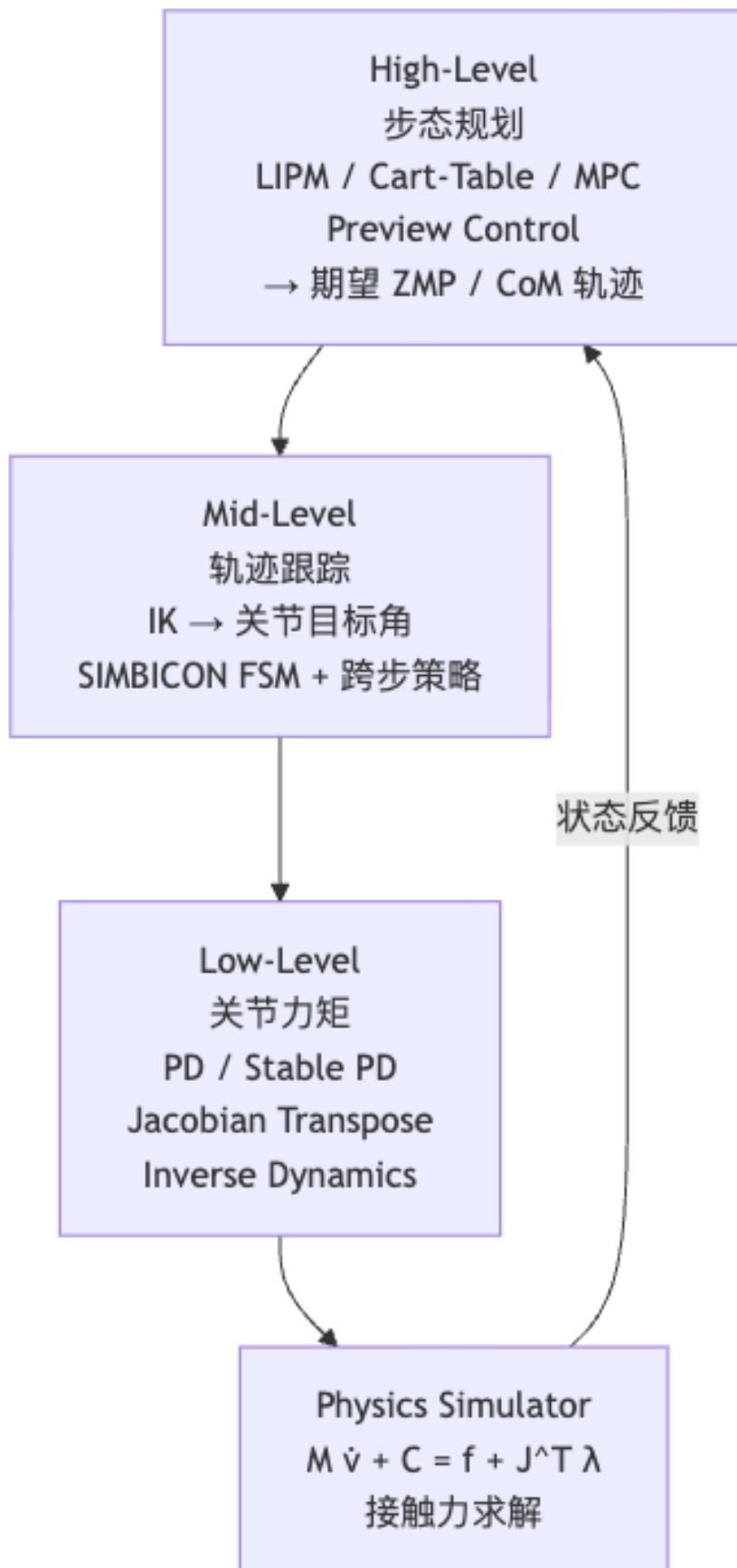


9. 三种简化模型对比

模型	关键假设	核心方程	适用场景	主要代价
IPM	单点质量 + 刚性杆	$\ddot{\theta} = \frac{g}{l} \sin \theta$	直觉建模、步长规划	非线性，难以直接规划
LIPM	IPM + CoM 固定高度 z_c	$\ddot{x} = \frac{g}{z_c} (x - p)$	步态规划、能量分析	z_c 固定，忽略垂直运动
Cart-Table	LIPM 反向	$p = x - \frac{z_c}{g} \ddot{x}$	ZMP→CoM 三对角系统	需要预知完整 ZMP 序列
SLIP	IPM + 弹簧腿	非线性弹簧力 + 飞行抛物线	跑步、跳跃	需数值积分，无解析解

模型	关键假设	核心方程	适用场景	主要代价
完整模型	多刚体 Newton–Euler	$M\dot{v} + C = f$	仿真 / 学习	高维，计算量大

10. 走路控制的层级结构



[EX] 例题 3: Cart-Table 三对角系数计算

问: Cart-Table 模型中, $z_c = 0.8 \text{ m}$, $g = 9.8 \text{ m/s}^2$, 时间步长 $\Delta t = 0.01 \text{ s}$ 。计算系数 a_i, b_i, c_i , 并说明 $b_i \gg |a_i|$ 的物理含义。

答: $\frac{z_c}{g\Delta t^2} = \frac{0.8}{9.8 \times 0.0001} \approx 816.3$

$a_i = c_i = -816.3$, $b_i = 2 \times 816.3 + 1 = 1633.6$

$b_i \gg |a_i|$ 说明 ZMP 主要由当前时刻的 CoM 位置 x_i 决定 (对角占主导), 相邻时刻 $x_{i\pm 1}$ 通过加速度修正量参与。这一数值刚度 (stiffness) 反映了“ZMP 对 CoM 加速度非常敏感”——小的加速度变化就能显著移动 ZMP, 故而控制要精确。

[EX] 例题 4: SIMBICON CoM 反馈分析

问: 一个 SIMBICON 控制的角色行走时, 被从后方推了一把, CoM 速度骤增 $v = 0.8 \text{ m/s}$, CoM 偏离量 $d = 0.15 \text{ m}$ (偏前)。已知 $c_d = 0.5$, $c_v = 0.2$, 基础目标角 $\theta_{d0} = 0.3 \text{ rad}$ 。

(a) 计算新的目标摆动角 $\bar{\theta}_d$; (b) 解释这个调整如何恢复平衡; (c) 若 $c_d = c_v = 0$, 会发生什么?

答: (a) $\bar{\theta}_d = 0.3 + 0.5 \times 0.15 + 0.2 \times 0.8 = 0.3 + 0.075 + 0.16 = 0.535 \text{ rad}$, 约增加 0.235 rad (约 13.5°)。

(b) 摆动腿目标角增大 $\rightarrow$ 落脚点向前移动 $\rightarrow$ 新支撑多边形前移 $\rightarrow$ CoM 相对新 ZMP 的偏离减小, 系统回到稳定轨道。这正是人在被推后本能地“迈大步”的物理实现。

(c) $c_d = c_v = 0$: 纯 FSM, 无反馈。被推后摆动腿仍按原计划落地, CoM 超出支撑区, 角色摔倒。这说明反馈项是 SIMBICON 鲁棒性的核心。

13. Anki 记忆卡片

#	Q (正面)	A (背面)
1	走路与跑步在相位上的根本区别?	走路无飞行相 (任意时刻至少一脚接地); 跑步有双脚离地的飞行相
2	静态平衡走路 vs. 动态平衡走路的区别?	静态: CoM 投影始终在支撑多边形内 (保守, 僵硬); 动态: CoM 可瞬时越出, 靠及时迈步复位 (自然步态)
3	CoP (压力中心) 的公式?	$p_{\text{CoP}} = \frac{\sum_i p_i f_i^y}{\sum_i f_i^y}$ (法向力加权中心)
4	ZMP 与 CoP 的关系 (平地)?	等价——ZMP 是使水平力矩分量为零的点, 在平地与 CoP 完全相同
5	ZMP 在支撑多边形外意味着什么?	脚不能平贴地, 会绕边缘翻转 (水平力矩 $\tau_{\text{GRF}}^{xz} \neq 0$)
6	LIPM 的核心假设与核心方程?	假设: CoM 运动在固定高度 z_c ; 方程: $\ddot{x} = \frac{g}{z_c}(x - p_{\text{zmp}})$
7	轨道能量 E 的公式, 并说明为什么守恒?	$E = \frac{1}{2}\dot{x}^2 - \frac{g}{2z_c}x^2$; ZMP 固定时 $\dot{E} = \dot{x}(\ddot{x} - \frac{g}{z_c}x) = 0$
8	早换脚 vs. 晚换脚对速度的影响?	早换脚 $E_2 < E_1 \rightarrow$ 减速; 晚换脚 $E_2 > E_1 \rightarrow$ 加速
9	Cart-Table 模型的 ZMP 方程?	$p_{\text{zmp}} = x - \frac{z_c}{g}\ddot{x}$
10	Cart-Table 三对角系数 a_i, b_i, c_i ?	$a_i = c_i = -\frac{z_c}{g\Delta t^2}$, $b_i = \frac{2z_c}{g\Delta t^2} + 1$
11	Preview Control 的必要性?	LIPM 不稳定 ($\ddot{x}$ 与偏差同号), 纯反馈无法稳定; 需预览未来 ZMP 作前馈

#	Q (正面)	A (背面)
12	SIMBICON 三步骤?	①FSM 周期 base motion; ②World-frame 躯干/摆动腿控制; ③CoM 线性反馈
13	SIMBICON 中 stance hip 力矩 τ_A 的公式与原因?	$\bar{\theta}_d = \bar{\theta}_{d0} + c_d d + c_v v$ $\tau_A = -\tau_{\text{torso}} - \tau_B$; 由 Newton 第三定律, stance leg 撑地不动, 躯干/摆动腿的反力矩由 stance hip 承担
14	SLIP 适用于什么场景? 与 LIPM 的区别?	跑步/跳跃 (有飞行相); LIPM 腿长固定, SLIP 腿长可变 (弹簧), 可描述能量储放
15	被动动态行走 (Passive Dynamic Walking) 的核心思想?	McGeer 1990: 纯机械人放在斜坡上靠重力稳定行走, 无需主动控制; 坡度补能, 换脚时机控速

14. 中英术语速查表

中文	English	首次出现
步态周期	Gait Cycle	§1
单支撑相	Single Stance Phase	§1
双支撑相	Double Stance Phase	§1
飞行相	Flight Phase	§1
静态平衡	Static Balance	§1
动态平衡	Dynamic Balance	§1
支撑多边形	Support Polygon	§1
质心	Center of Mass (CoM)	§1
地面反作用力	Ground Reaction Force (GRF)	§2
压力中心	Center of Pressure (CoP)	§2
零力矩点	Zero-Moment Point (ZMP)	§2
法向力	Normal Force	§2
摩擦力	Friction Force	§2
脚踝力矩	Ankle Torque	§2
倒立摆模型	Inverted Pendulum Model (IPM)	§4
弹簧负载倒立摆	Spring-Loaded Inverted Pendulum (SLIP)	§4
线性倒立摆模型	Linear Inverted Pendulum Model (LIPM)	§5
固定高度假设	Fixed Height Constraint	§5
轨道能量	Orbital Energy	§5
步长	Step Length	§5
被动动态行走	Passive Dynamic Walking	§5
桌-车模型	Cart-Table Model	§6
三对角线性系统	Tridiagonal Linear System	§6
Thomas 算法	Thomas Algorithm	§6
预览控制	Preview Control	§6
有限状态机	Finite State Machine (FSM)	§7
触地	Foot Strike	§7
世界系	World Frame	§7
跨步策略	Stepping Strategy	§7
线性反馈	Linear Feedback	§7
摆动腿	Swing Leg	§7
支撑腿	Stance Leg	§7

15. 复习要点

[TIP] 期末复习要点 (本讲必记)

1. **步态相位**: 走路 = 单/双支撑相交替, 无飞行相; 跑步有飞行相。静态平衡 vs. 动态平衡的几何刻画。
2. **ZMP 完整推导**: f_i 分解 (法向 + 摩擦) $\rightarrow$ 水平力矩 $\tau^{xz} \rightarrow$ CoP 公式 $\rightarrow$ 力矩平衡方程 $\rightarrow$ ZMP 在支撑多边形内 = 脚不翻转。
3. **LIPM 推导链**: IPM 方程 $\rightarrow$ “固定高度”线性化 $\rightarrow \ddot{x} = \frac{g}{z_c}(x - p) \rightarrow$ 轨道能量 $E = \frac{1}{2}\dot{x}^2 - \frac{g}{2z_c}x^2 \rightarrow$ 早/晚换脚 = 减/加速。
4. **Cart-Table 完整流程**: LIPM 反向求 ZMP $\rightarrow$ 离散化三对角系数 $a, b, c \rightarrow$ 矩阵方程 $\rightarrow$ Thomas 算法 $O(N) \rightarrow$ Preview Control 的必要性 (LIPM 不稳定, 需 lookahead)。
5. **SIMBICON 三步骤**: FSM 状态定义 $\rightarrow$ World-frame PD (躯干 + swing hip) $\rightarrow$ stance hip 由 Newton 第三定律决定 $\rightarrow$ CoM 线性反馈 $\bar{\theta}_d = \bar{\theta}_{d0} + c_d d + c_v v$ 。
6. **模型谱系**: IPM $\rightarrow$ LIPM (固定高度) $\rightarrow$ Cart-Table (反向); IPM $\rightarrow$ SLIP (加弹簧)。每层抹掉细节换简洁。
7. **控制层级**: High-Level (步态规划 / LIPM / Preview) $\rightarrow$ Mid-Level (IK / FSM / 跨步策略) $\rightarrow$ Low-Level (PD / ID) $\rightarrow$ 物理引擎。
8. **学习派衔接**: DeepMimic / AMP 保留 Low-Level PD, 用神经网络策略替换 High/Mid-Level 的 FSM + 线性反馈。

16. 关键文献

文献	贡献
Vukobratovic & Borovac (1972)	ZMP 概念提出
McGeer (1990)	Passive Dynamic Walking
Kajita et al. (2001)	LIPM 命名与论证
Kajita et al. (2003)	Cart-Table + ZMP Preview Control
Yin et al. (2007)	SIMBICON
Coros et al. (2010)	Generalized Biped Walking Control (IPM-based)
Mordatch et al. (2010)	Low-dim Planning with SLIP
Peng et al. (2018)	DeepMimic (学习派基线)
Peng et al. (2021)	AMP (GAN 风格运动控制)

Lecture 11 —最优控制与强化学习 (Optimal Control & Reinforcement Learning)

[TIP] 学习指南

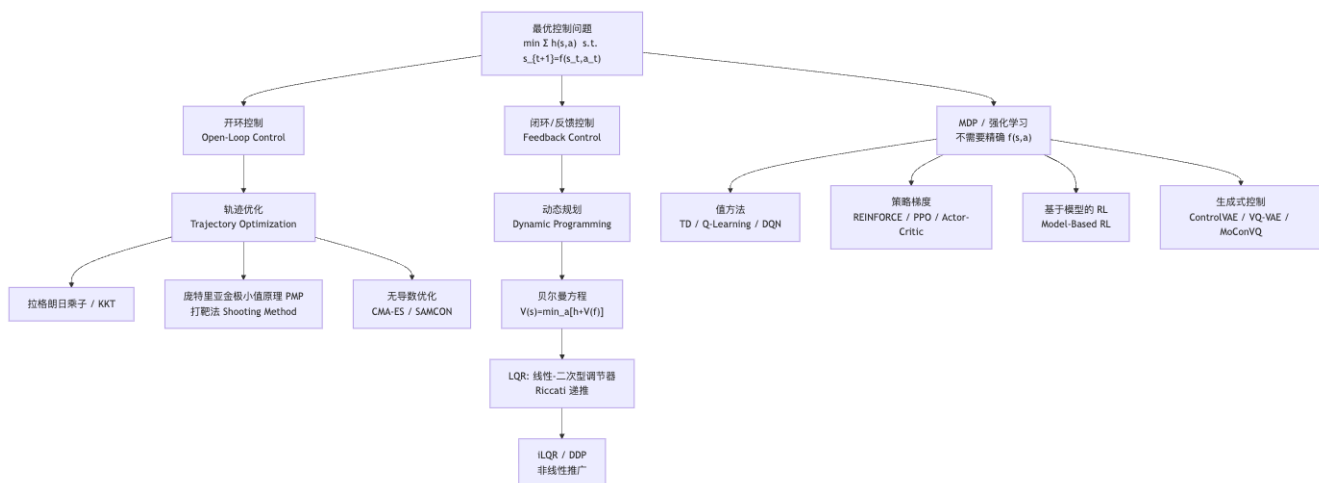
本讲是全课程压轴，把前 10 讲的一切（物理仿真、动捕跟踪、PD 控制、运动规划）融入一个统一的数学框架：**最优控制 (Optimal Control)**，并延伸到不需要精确动力学模型的**强化学习 (Reinforcement Learning)**。

核心主线（四层递进）：1. **轨迹优化 (Trajectory Optimization)**：给定初始状态，找最优动作序列（开环）。工具：拉格朗日乘子、KKT、庞特里亚金极小值原理 (PMP)、打靶法。2. **动态规划 (Dynamic Programming)**：学习值函数 $V(s)$ 后对任意状态都能给出最优动作（闭环）。工具：贝尔曼方程、LQR/iLQR/DDP。3. **马尔可夫决策过程 (MDP)**：放宽“已知精确动力学”假设，用概率转移模型刻画不确定性。工具：值方法 (TD、Q-Learning、DQN)、策略梯度 (REINFORCE、PPO、Actor-Critic)。4. **生成式控制 (Generative Control)**：用 VAE/VQ-VAE 学习紧凑动作表示，再结合高层策略实现多技能、文本驱动角色控制 (ControlVAE、MoConVQ)。

前置知识：微积分与线性代数（梯度、矩阵求逆）；lec04 PD 控制器与物理仿真基础；lec09–10 动捕数据跟踪控制。

重点掌握：1. 轨迹优化的拉格朗日形式化与 KKT 条件 2. 协态/伴随变量 λ_t 的物理意义与 PMP 打靶法 3. 贝尔曼方程的推导与值函数/Q 函数的关系 4. LQR 的 Riccati 递推及 iLQR/DDP 非线性推广 5. MDP 形式化、TD 学习与 Q-Learning 更新规则 6. DQN 的经验回放与目标网络两大技巧 7. 策略梯度定理与 PPO clip 目标 8. ControlVAE / VQ-VAE / MoConVQ 的设计思路

0. 全景鸟瞰



[NOTE] 开环 vs 闭环，一句话区分

开环 (Open-loop): 从 s_0 出发，预先计算好全部动作序列 $a_0, \dots, a_{T-1}$ ，再逐步执行——遇到扰动就翻车。
闭环/反馈 (Closed-loop): 每个时刻 t 观测当前状态 s_t ，由策略 $\pi(s_t, t)$ 实时计算动作——对扰动有鲁棒性。

1. 轨迹优化 (Trajectory Optimization)

1.1 问题形式化

时空约束 (Spacetime Constraints) (Witkin & Kass 1988) 首次将角色动画的控制问题写成数学优化：

$$\min_{\{s_t\}, \{a_t\}} \Phi(s_T) + \sum_{t=0}^{T-1} h(s_t, a_t) \quad (1)$$

$$\text{s.t. } s_{t+1} = f(s_t, a_t), \quad 0 \leq t < T \quad (2)$$

符号	含义
$s_t \in \mathbb{R}^n$	状态 (关节角度、速度等)
$a_t \in \mathbb{R}^m$	动作 (力矩、PD 目标姿态等)
$f(s_t, a_t)$	物理动力学 ($M\dot{v} + C(x, v) = f + J^T\lambda$, 含接触约束 $g(x, v) \geq 0$)
$\Phi(s_T)$	终端代价 (terminal cost)
$h(s_t, a_t)$	运行代价 (running cost, 如跟踪误差)

[EX] 正弦跟踪示例

弹簧-质量系统，控制力 $f = k_p(\bar{x} - x) - k_d v$ ，目标 $x_t = \sin t$ 。优化变量 $\bar{x}_t$ (PD 目标轨迹)：

$$\min_{\bar{x}_n} \sum_{n=0}^N (\sin t_n - x_n)^2 + \sum_{n=0}^N \bar{x}_n^2 \quad (3)$$

$$\text{s.t. } v_{n+1} = v_n + h[k_p(\bar{x}_n - x_n) - k_d v_n], \quad x_{n+1} = x_n + h v_{n+1}$$

第一项让仿真轨迹接近目标；第二项正则化控制幅度，防止目标姿态过激。

1.2 约束优化与拉格朗日乘子

软约束的问题： $\min_x f(x) + w \cdot g(x)$ 不能保证严格满足 $g(x) = 0$ (解可能漂移出约束曲面)。

正确做法： 构造拉格朗日函数 (Lagrangian)：

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T g(x) \quad (4)$$

KKT 一阶必要条件 (等式约束版)：

$$\frac{\partial \mathcal{L}}{\partial x} = f'(x) + \lambda^T g'(x) = 0 \quad (\text{梯度平行}) \quad (5)$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = g(x) = 0 \quad (\text{约束满足}) \quad (6)$$

几何直觉：最优点处，目标函数梯度 $f'(x)$ 与约束曲面法向量 $g'(x)$ 共线，即沿约束曲面的切向分量为零，无法在满足约束的前提下继续减小目标值。

1.3 轨迹优化的拉格朗日推导

对问题 (1)–(2) 引入协态变量（伴随变量） $\lambda_{t+1} \in \mathbb{R}^n$ （每个时刻动力学约束对应一个乘子向量）：

$$\mathcal{L}(s, a, \lambda) = \Phi(s_T) + \sum_{t=0}^{T-1} [h(s_t, a_t) + \lambda_{t+1}^T (f(s_t, a_t) - s_{t+1})] \quad (7)$$

对各变量求偏导令其为零，得到四组方程：

$$\frac{\partial \mathcal{L}}{\partial s_T} = 0 \Rightarrow \lambda_T = \Phi'(s_T) \quad (8)$$

$$\frac{\partial \mathcal{L}}{\partial s_t} = 0 \Rightarrow \lambda_t = h'_s(s_t, a_t) + f'_s(s_t, a_t)^T \lambda_{t+1} \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial a_t} = 0 \Rightarrow h'_a(s_t, a_t) + f'_a(s_t, a_t)^T \lambda_{t+1} = 0 \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_t} = 0 \Rightarrow s_{t+1} = f(s_t, a_t) \quad (11)$$

[NOTE] 协态变量的物理意义

λ_t 是“总代价对状态 s_t 的灵敏度（影子价格）”——沿时间反向传播，告诉我们此刻的状态偏差对总代价的贡献。这与神经网络反向传播梯度完全同构： λ 对应的正是“损失对中间激活的梯度”。

1.4 庞特里亚金极小值原理 (PMP) 与打靶法

将以上条件整理成离散 PMP 的标准形式：

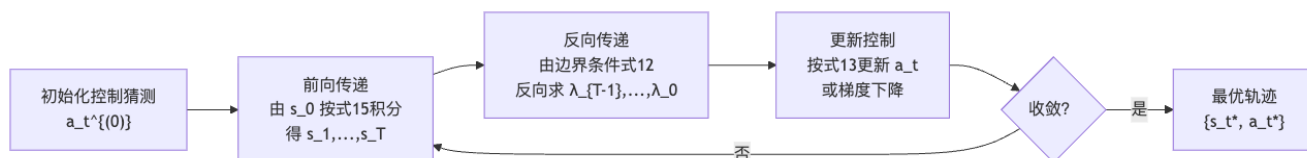
$$\boxed{\lambda_T = \Phi'(s_T)} \quad (12)$$

$$\boxed{a_t^* = \arg \min_a [h(s_t, a) + f(s_t, a)^T \lambda_{t+1}]} \quad (13)$$

$$\boxed{\lambda_t = h'_s(s_t, a_t) + f'_s(s_t, a_t)^T \lambda_{t+1}} \quad (14)$$

$$s_{t+1} = f(s_t, a_t) \quad (15)$$

打靶法 (Shooting Method): 给定初始控制猜测 $\{a_t^{(0)}\}$, 迭代执行三步:



[WARN] 打靶法的局限

对复杂角色运动 (多接触、高维状态空间), 优化景观高度非线性, 梯度不可靠或不可得 (黑盒仿真器)。此时需要无导数方法或结合多重打靶法 (multiple shooting)。

2. 无导数优化: CMA-ES 与 SAMCON

2.1 梯度法的挑战

即使可以用打靶法计算梯度, 人形角色的运动优化景观依然极度非凸 (如 Härmäläinen et al. 2020 可视化所示), 梯度极易陷入局部极小值或发散。此外, 许多仿真器是黑盒, 根本不提供梯度。

2.2 CMA-ES (协方差矩阵自适应进化策略)

CMA-ES (Hansen 2006): 维护一个多元高斯分布 $x \sim \mathcal{N}(\mu, \Sigma)$, 迭代:

1. 采样 N 个候选解: $x_i \sim \mathcal{N}(\mu, \Sigma)$ (每个 x_i 是一组控制参数 θ)
2. 对每个候选解运行物理仿真, 评估代价 $f_i = f(x_i)$
3. 按代价排序, 保留前 k 个精英 (elite) 样本
4. 用精英样本更新 μ (均值朝精英方向移动) 和 Σ (用累积演化路径 evolution path 自适应调整方向和步长)

优势: 无需梯度, 对非凸景观鲁棒, 支持黑盒仿真器。

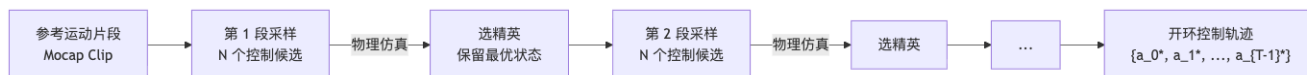
[EX] CMA-ES 在角色动画中的应用

– Wampler & Popovic 2009: 优化动物运动步态与体型 – Al Borno et al. 2013: 优化含复杂接触的全身运动轨迹 – Liu et al. 2012 Terrain Runner: 用 CMA 优化 108 维的线性反馈策略矩阵 (12 分钟 / 24 核)

2.3 SAMCON (基于采样的运动控制)

SAMCON (Liu et al. 2010, 2015) 受序贯蒙特卡罗 (SMC) 启发, 把长段运动分解为逐段采样:

将参考运动按帧切分 → 每段起点并行采样大量 PD 控制目标 → 仿真评估跟踪误差 → 选精英传递到下一段:



特点: 不需梯度; 可并行; 对多接触、翻滚等复杂运动效果优秀。

与 CMA-ES 区别: SAMCON 利用运动时间结构 (逐段进行), 而 CMA-ES 把整段控制参数作为整体优化。

3. 反馈控制与动态规划

3.1 从开环到闭环

开环轨迹优化得到的 $\{a_t^*\}$ 是针对 s_0 的特定控制序列，扰动一来就失效。

反馈控制：寻找策略 $\pi(s_t, t)$ ，使得对任意状态 s_t 都能实时计算最优动作：

$$a_t = \pi(s_t, t) \quad (16)$$

闭环问题与开环问题的目标函数相同（式 1-2），只是把“找动作序列”改为“找策略函数”。

3.2 贝尔曼最优性原理

[NOTE] 贝尔曼最优性原理（Bellman 1957）

一条最优策略的子策略，对子问题也必须是最优的（最优子结构）。

精确表述：若 $\{a_0^*, \dots, a_{T-1}^*\}$ 是从 s_0 出发的最优动作序列，则对任意 t ， $\{a_t^*, \dots, a_{T-1}^*\}$ 也是从 s_t 出发的最优动作序列。

值函数 (Value Function)：

$$V(s) \triangleq \min_{\pi} \sum_{t=0}^T h(s_t, a_t) \Big|_{s_0=s, a_t=\pi(s_t, t)} \quad (17)$$

$V(s)$ 是从状态 s 出发、遵循最优策略到达终点的最小总代价。

3.3 贝尔曼方程（最优性方程）

由最优子结构直接推导——对式 (17) 剥离第一步：

$$V(s_0) = \min_{a_0} \left[h(s_0, a_0) + \min_{\pi} \sum_{t=1}^T h(s_t, a_t) \right] = \min_{a_0} [h(s_0, a_0) + V(f(s_0, a_0))]$$

因此对任意状态 s ：

$$V(s) = \min_a [h(s, a) + V(f(s, a))] \quad (18)$$

Q 函数（状态-动作值函数，State-Action Value Function）：

$$Q(s, a) \triangleq h(s, a) + V(f(s, a)) \quad (19)$$

则最优策略为：

$$\pi^*(s) = \arg \min_a Q(s, a), \quad V(s) = \min_a Q(s, a) \quad (20)$$

[NOTE] 直觉：V vs Q

– $V(s)$: “从 s 出发，最优控制的最低总代价”——需要知道 f 才能求 – $Q(s, a)$: “在 s 采取 a ，再最优控制的总代价”——可以直接从数据估计
对离散动作空间， $\arg \min_a Q$ 只需枚举；对连续动作空间，则需策略梯度等方法。

奖励最大化版本 (RL 标准记法，令 $r = -h$ ，折扣因子 $\gamma \in (0, 1]$):

$$V(s) = \max_a [r(s, a) + \gamma V(f(s, a))] \quad (21)$$

折扣因子 γ 让远期奖励权重衰减，保证无限期求和收敛，同时体现“近期奖励更确定”的经济学直觉。

4. 线性二次型调节器 (LQR)

4.1 LQR 问题定义

LQR 是最优控制中有解析解的特殊情形，同时满足：

- 线性动力学： $s_{t+1} = A_t s_t + B_t a_t$
- 二次型代价： $\Phi(s_T) = s_T^T Q_T s_T$; $h(s_t, a_t) = s_t^T Q_t s_t + a_t^T R_t a_t$

$$\min s_T^T Q_T s_T + \sum_{t=0}^{T-1} (s_t^T Q_t s_t + a_t^T R_t a_t) \quad (22)$$

$$\text{s.t. } s_{t+1} = A_t s_t + B_t a_t \quad (23)$$

- $Q_t \succeq 0$: 状态代价权重 (半正定)
- $R_t \succ 0$: 控制代价权重 (正定，惩罚过大控制力)
- A_t, B_t : 线性动力学矩阵

4.2 Riccati 递推完整推导

步骤一：边界条件 (第 T 步):

$$V(s_T) = s_T^T Q_T s_T \Rightarrow P_T = Q_T \quad (24)$$

步骤二：利用贝尔曼方程计算 $Q(s_{T-1}, a_{T-1})$ (代入 $s_T = A_{T-1} s_{T-1} + B_{T-1} a_{T-1}$):

$$Q(s_{T-1}, a_{T-1}) = s_{T-1}^T Q_{T-1} s_{T-1} + a_{T-1}^T R_{T-1} a_{T-1} + (A_{T-1} s_{T-1} + B_{T-1} a_{T-1})^T Q_T (A_{T-1} s_{T-1} + B_{T-1} a_{T-1}) \quad (25)$$

整理后 Q 函数的形式为关于 (s_{T-1}, a_{T-1}) 的二次型，包含三类项： $s^T Q s$ 、 $a^T R a$ 、交叉项 $s^T A B a$ 。

步骤三：对 a_{T-1} 求导令其为零 ($\partial Q / \partial a_{T-1} = 0$)，得最优线性反馈律：

$$a_{T-1}^* = -K_{T-1} s_{T-1} \quad (26)$$

$$K_{T-1} = (R_{T-1} + B_{T-1}^T Q_T B_{T-1})^{-1} B_{T-1}^T Q_T A_{T-1} \quad (27)$$

步骤四：代回后值函数仍为二次型 $V(s_{T-1}) = s_{T-1}^T P_{T-1} s_{T-1}$ ，矩阵 P_{T-1} 由 Riccati 方程确定：

$$P_t = Q_t + A_t^T P_{t+1} A_t - A_t^T P_{t+1} B_t \underbrace{(R_t + B_t^T P_{t+1} B_t)^{-1}} B_t^T P_{t+1} A_t \quad (28)$$

完整算法：从 $P_T = Q_T$ 反向递推到 P_0 ，得到每时刻的增益矩阵 K_t ，在线执行只需矩阵-向量乘法 $a_t^* = -K_t s_t$ 。

[NOTE] LQR 的优雅性质

1. 值函数始终为二次型： $V(s) = s^T P s$ ，数学上自洽，可精确递推 2. 最优策略始终为线性： $a^* = -K s$ ，实时执行极简单 3. 只需一次离线 Riccati 递推，在线开销为 $O(mn)$ 矩阵乘法

4.3 iLQR 与 DDP (非线性推广)

对非线性问题，在当前参考轨迹 $(\bar{s}_t, \bar{a}_t)$ 附近做 Taylor 展开：

动力学近似：

$$f(s_t, a_t) \approx f(\bar{s}_t, \bar{a}_t) + \nabla_s f \cdot \delta s_t + \nabla_a f \cdot \delta a_t \quad (29)$$

(iLQR 只用一阶；DDP 还加入 $\nabla^2 f$ 的二阶项)

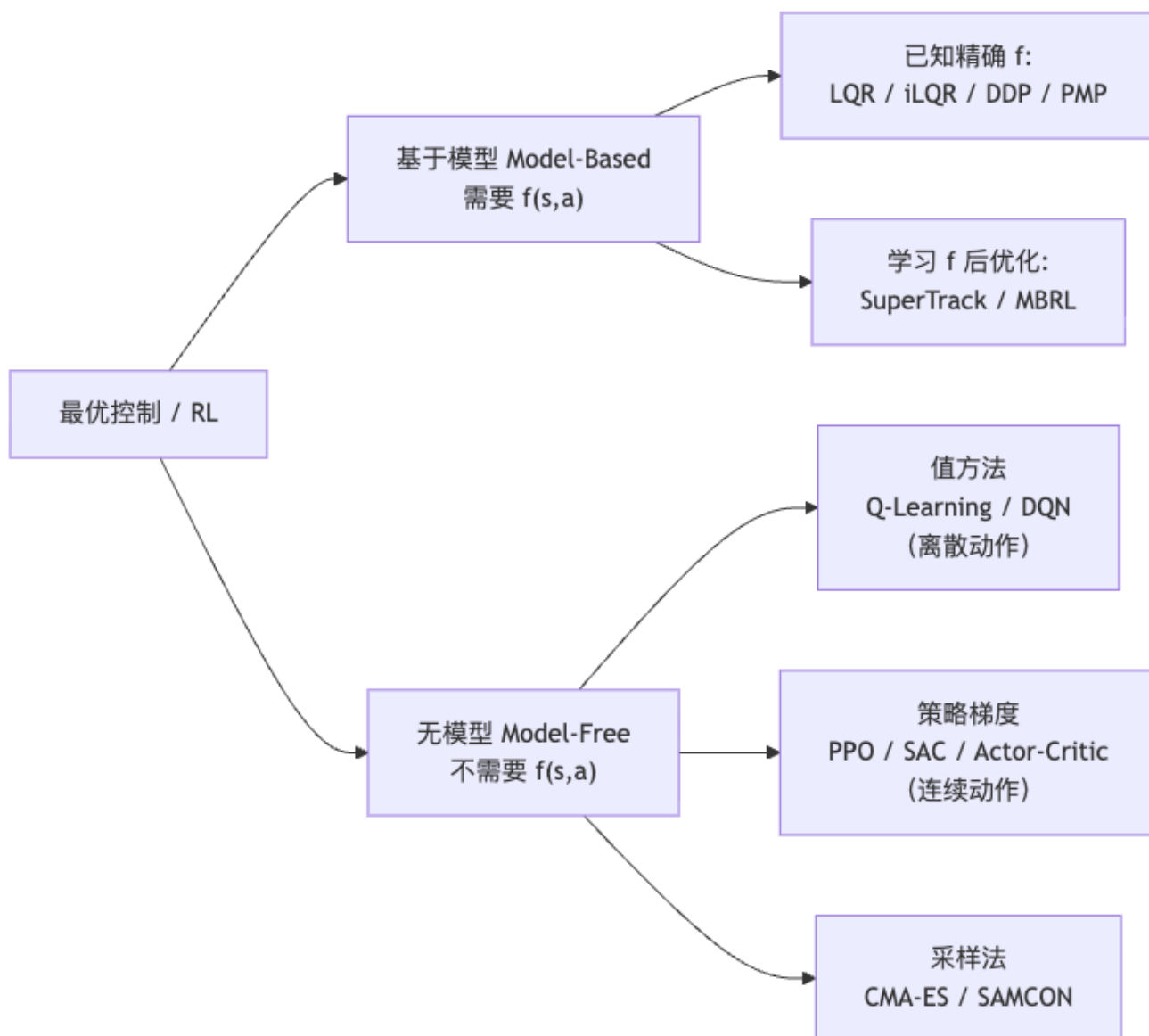
代价函数近似：

$$h(s_t, a_t) \approx h(\bar{s}_t, \bar{a}_t) + \nabla h \cdot \begin{pmatrix} \delta s \\ \delta a \end{pmatrix} + \frac{1}{2} \begin{pmatrix} \delta s \\ \delta a \end{pmatrix}^T \nabla^2 h \begin{pmatrix} \delta s \\ \delta a \end{pmatrix} \quad (30)$$

局部近似后问题变为 LQR，可精确求解；然后在新得到的轨迹处重新线性化，迭代直到收敛。

方法	动力学近似	代价函数近似	精度	计算量
LQR	精确线性	精确二次	精确 (线性系统)	低
iLQR	一阶 Taylor	二阶 Taylor	局部最优	中
DDP	二阶 Taylor	二阶 Taylor	局部最优 (更准)	高

5. 模型法 vs 无模型法



关键问题：若 $f(s, a)$ 未知、含噪声、高度非线性，或计算梯度代价极高——此时强化学习登场，通过与环境交互来学习控制策略。

6. 马尔可夫决策过程 (MDP)

6.1 形式化定义

MDP 是 RL 的标准数学框架：

$$\mathcal{M} = \{S, A, p, r, \gamma\} \quad (31)$$

元素	含义	角色动画实例
S	状态空间	关节角度、角速度、质心位置、相位
A	动作空间	关节力矩 / PD 目标姿态
$p(s' s, a)$	随机转移概率	含噪声的物理仿真
$r(s, a)$	即时奖励	— 跟踪误差 — 能量消耗
$\gamma \in (0, 1]$	折扣因子	控制远期奖励权重

与最优控制的关键区别： $p(s'|s, a)$ 是**随机**的且通常**未知**（需从交互中学习）。

随机策略 $\pi(a|s)$ ：在状态 s 采取动作 a 的概率分布（允许探索）。

轨迹回报（折扣总奖励）：

$$R = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \quad (32)$$

RL 目标：最大化期望回报：

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] = \mathbb{E}_{s \sim d^\pi} [V^\pi(s)] \quad (33)$$

其中 $d^\pi(s)$ 是策略 π 诱导的状态稳态分布。

6.2 MDP 中的贝尔曼方程

策略值函数（对特定策略 π 求，非最优）：

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi} [r(s, a) + \gamma V^\pi(s')] \quad (34)$$

策略 Q 函数：

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p} [V^\pi(s')] \quad (35)$$

最优贝尔曼方程（目标）：

$$V^*(s) = \max_a [r(s, a) + \gamma \mathbb{E}_{s' \sim p} [V^*(s')]] \quad (36)$$

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q^*(s', a') \right] \quad (37)$$

7. 值方法：TD 学习、Q-Learning 与 DQN

7.1 时序差分 (TD) 学习

直接从与环境的交互序列中在线更新值函数，无需知道 $p(s'|s, a)$ ：

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[r_t + \gamma V(s_{t+1}) - V(s_t)]}_{\delta_t, \text{TD 误差}} \quad (38)$$

直觉：当前估计 $V(s_t)$ 应等于“本步奖励 + 折扣后续价值 $\gamma V(s_{t+1})$ ”，TD 误差 δ_t 反映当前估计与该目标的偏差，用于修正。

7.2 Q-Learning

Q-Learning 直接学习最优 Q 函数，属于 **off-policy** 方法（可利用任意历史数据）：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (39)$$

目标值用的是 $\max_{a'} Q(s_{t+1}, a')$ （最优贪心），而非当前策略下的动作值——这使其可以利用历史经验 (experience replay)。

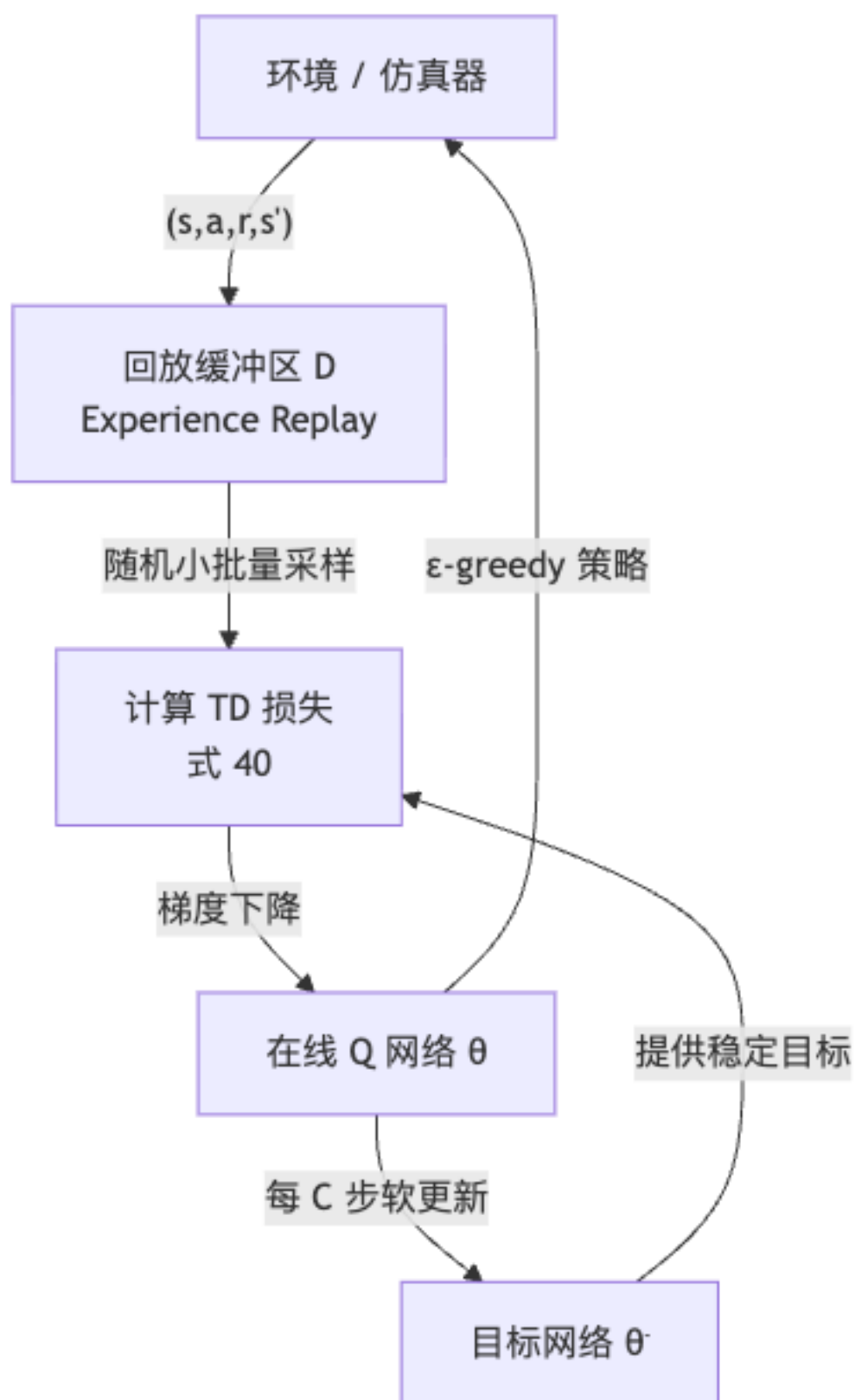
7.3 深度 Q 网络 (DQN)

DQN (Mnih et al. 2015) 用深度神经网络参数化 $Q(s, a; \theta)$ ，解决大规模连续状态空间问题：

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-) - Q(s, a; \theta))^2 \right] \quad (40)$$

两大稳定性技巧：

1. **经验回放 (Experience Replay)：**把所有交互历史 (s, a, r, s') 存入回放缓冲区 $\mathcal{D}$ ，每步随机采样小批量训练——打破序列相关性，提高数据利用率。
2. **目标网络 (Target Network)：**引入参数延迟更新的“目标网络” $Q_{\text{target}}(\cdot; \theta^-)$ ，定期软更新 $(\theta^- \leftarrow (1 - \beta)\theta^- + \beta\theta)$ ——让学习目标在一段时间内稳定，避免“追着移动靶射击”导致发散。



[WARN] DQN 的局限

DQN 需要枚举 $\max_{a'} Q(s', a')$ ，只对离散动作空间高效。角色控制的连续关节力矩空间无法直接使用，需改用策略梯度或 Actor-Critic（如 PPO、SAC）。

8. 策略梯度与 PPO

8.1 策略梯度定理

参数化随机策略 $\pi_\theta(a|s)$ （如高斯分布均值由神经网络给出），直接对 $J(\theta)$ 求梯度优化。

策略梯度定理（Sutton et al. 1999 NeurIPS）：

$$\nabla_\theta J(\theta) = \sum_s d^\pi(s) \sum_a Q^\pi(s, a) \nabla_\theta \pi_\theta(a|s) \quad (41)$$

$$\propto \mathbb{E}_{s \sim d^\pi, a \sim \pi} [Q^\pi(s, a) \nabla_\theta \log \pi_\theta(a|s)] \quad (42)$$

推导关键（log-derivative trick）： $\nabla_\theta \pi_\theta(a|s) = \pi_\theta(a|s) \cdot \nabla_\theta \log \pi_\theta(a|s)$ ，将其代入期望即得式（42）。

带优势函数的蒙特卡罗估计（REINFORCE with baseline）：

$$\nabla_\theta J \approx \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A^\pi(s_t, a_t) \right] \quad (43)$$

其中优势函数 $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 衡量该动作“比平均好多少”，用于减少梯度估计的方差。

8.2 Actor-Critic

同时维护两个网络，互相促进：

组件	网络	更新方式	作用
Actor（演员）	$\pi_\theta(a s)$	策略梯度（式 43）	生成动作
Critic（评论家）	$V_\phi(s)$	TD 学习（式 38）	评估状态价值，给 Actor 提供基线

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \approx A^\pi(s_t, a_t) \quad (44)$$

Critic 的 TD 误差 δ_t 是优势函数的近似估计，直接用于 Actor 的梯度更新。

8.3 PPO（近端策略优化，Proximal Policy Optimization）

动机：朴素策略梯度步长难以选择——太大策略剧变崩溃，太小收敛慢。TRPO 用 KL 散度约束解决此问题，但实现复杂；PPO 用简单的 clip 代替 KL 约束，效果相当。

重要性采样比（允许复用旧数据）：

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (45)$$

PPO clip 目标函数：

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (46)$$

clip 的直觉：– 当 $\hat{A}_t > 0$ （该动作好于平均）：允许把概率最多放大到 $1 + \epsilon$ 倍，超过则梯度截断 – 当 $\hat{A}_t < 0$ （该动作差于平均）：允许把概率最多缩小到 $1 - \epsilon$ 倍，超过则梯度截断 – 典型 $\epsilon = 0.2$ ，确保每步策略不会更新过激

[NOTE] PPO 在角色动画中的地位

PPO 同时满足：适用连续动作空间、样本效率比纯 on-policy 高、实现简单、超参鲁棒。DeepMimic (Peng et al. 2018) 用 PPO 训练高仿真度运动跟踪控制器，是角色动画 RL 的里程碑；Liu et al. 2016 ControlGraphs、Liu et al. 2018、Peng et al. 2018 DeepMimic 均基于策略梯度方法。

9. 基于模型的强化学习 (Model-Based RL)

9.1 动机与思路

核心矛盾：无模型 RL 样本效率低（需要数百万次仿真交互）；若能学习动力学模型 $\hat{f}(s, a; \theta_f)$ ，就可以在模型上快速优化策略：

$$\min_{\theta_\pi} h(s_0, \pi_{\theta_\pi}(s_0)) + h(\hat{f}(s_0, a_0), \pi_{\theta_\pi}(\cdot)) + \dots \quad (47)$$

通过对已学模型 $\hat{f}$ 反向传播，可得所有需要的梯度 $(d\hat{f}/ds, d\hat{f}/da, d\pi/ds)$ 。

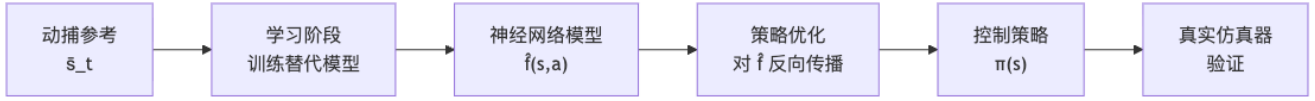
9.2 SuperTrack (Fussell et al. 2021)

核心思路：用神经网络学习物理仿真的替代模型（surrogate model），使控制策略可以通过该模型端到端可微分地优化。

$$\mathcal{L} = \sum_{t=1}^N \|s_t^{\text{pred}} - \bar{s}_t\|^2 \quad (48)$$

其中 s_t^{pred} 是模型自回归滚动预测， $\bar{s}_t$ 是参考动捕状态。

训练流程：



优势：规避物理仿真器不可微问题；训练速度比真实仿真快数十倍。

10. 生成式控制模型

10.1 动机：从”跟踪”到”生成”

前述方法均以某段参考动作为目标。若想让角色学会大量技能并能自由组合，需要一种统一的生成式表示。

10.2 VAE 与控制生成

VAE（变分自编码器）的 ELBO（Evidence Lower Bound）目标：

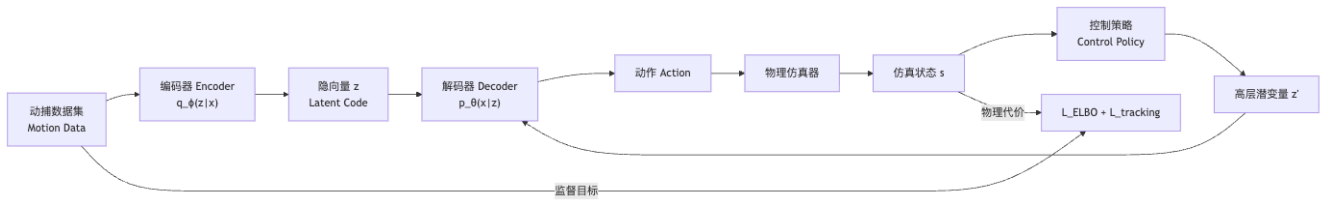
$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{重构损失}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{KL 正则}} \quad (49)$$

- 重构损失：编码再解码后要还原原始运动 x
- KL 散度：编码分布接近标准正态 $\mathcal{N}(0, I)$ ，使潜空间连续可插值、可采样

在角色控制中的意义：把复杂、高维的运动空间压缩为低维连续潜变量 z ，高层策略只在潜空间操作，降低控制维度。

10.3 ControlVAE (Yao et al. 2022)

ControlVAE 将 VAE 与物理仿真深度结合：



- 训练阶段：VAE 损失（式 49）+ 物理跟踪误差联合训练
- 推理阶段：高层策略输出潜变量 z' ，底层控制策略基于 z' 与物理状态生成动作
- 效果：一个 ControlVAE 可以生成多种自然运动技能，并平滑过渡

10.4 VQ-VAE 与离散运动表示

VQ-VAE (Van den Oord et al. 2017)：将连续隐空间替换为有限码本（Codebook）：

$$z_q = e_k, \quad k = \arg \min_j \|z_e - e_j\|_2 \quad (50)$$

- 编码器输出 z_e ，通过最近邻查找映射到离散码字 e_k （直通梯度 straight-through estimator 处理不可微的 argmin）
- 解码器使用离散码字 e_k 重构运动

三项损失函数 (VQ-VAE):

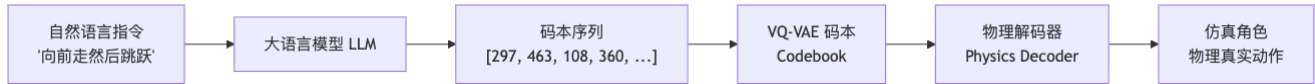
$$\mathcal{L}_{\text{VQ-VAE}} = \underbrace{\|x - \hat{x}\|^2}_{\text{重构}} + \underbrace{\|\text{sg}[z_e] - e\|^2}_{\text{码本更新}} + \beta \underbrace{\|z_e - \text{sg}[e]\|^2}_{\text{commitment}} \quad (51)$$

- $\text{sg}[\cdot]$: 停止梯度 (stop-gradient), 第二项只更新码本 e , 第三项只更新编码器
- β (通常 0.25): 平衡编码器“承诺”靠近码本的程度

为何离散表示更适合与 LLM 对接: 码本中每个码字对应一类运动原语 (如“向前走”=“297”), LLM 可以像操作词汇一样操作运动——这是 MoConVQ 的关键洞察。

10.5 MoConVQ (Yao et al. 2024)

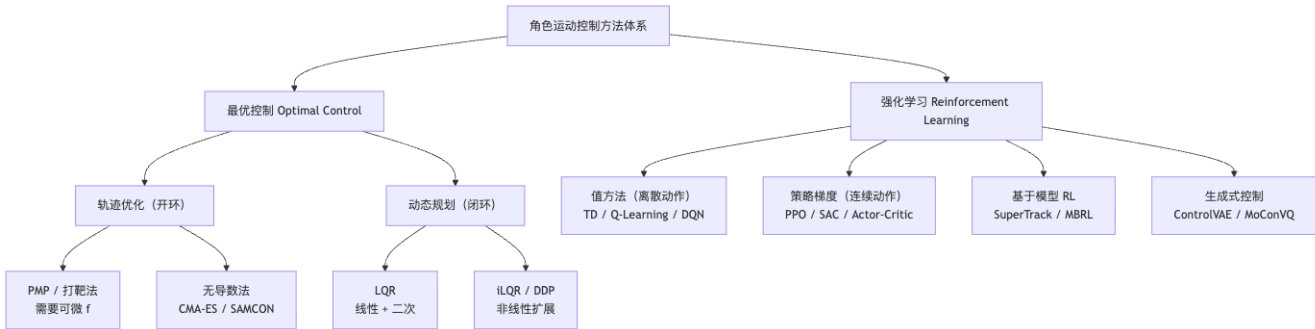
MoConVQ 将离散运动码本与大语言模型 (LLM) 深度对接, 实现文本驱动的物理角色动画:



训练步骤: 1. 用大量动捕数据训练 VQ-VAE, 建立运动码本 2. 物理解码器 (控制策略) 学习将码字序列转化为物理可执行动作 3. 将码字序列视为“语言”, 微调 LLM 使其能用码字描述和规划运动

效果示例: - “向前走绕正方形”→ LLM 推理出“直走 [297,471,...] + 右转 90° [360,...] × 4 次”→ 物理执行 - “捡起钥匙开门然后坐下”→ LLM 分解为 5 个子任务动作序列 → 角色顺序执行

11. 完整方法谱系对比



方法	需要精确模型	动作空间	鲁棒性	代表工作
打靶法 (PMP)	是 (可微)	连续	低	Witkin & Kass 1988
CMA-ES	否 (黑盒)	连续	中	Wampler 2009
SAMCON	否 (采样)	连续	高	Liu et al. 2010, 2015
LQR	是 (线性)	连续	高	经典控制
iLQR / DDP	近似 (可微)	连续	中	Muico et al. 2011
Q-Learning / DQN	否	离散	高	Mnih et al. 2015
PPO	否	连续	高	DeepMimic 2018
ControlVAE	否	连续 + 潜空间	高	Yao et al. 2022
MoConVQ	否	离散码本	高	Yao et al. 2024

Worked Examples (例题解析)

例题 1: LQR 一维系统完整推导

题目: 标量系统 $s_{t+1} = as_t + ba_t$, 代价 $\sum_{t=0}^{T-1} (qs_t^2 + ra_t^2) + q_T s_T^2$ 。求最优控制律及 Riccati 递推。

解:

边界: $P_T = q_T$ (终端代价系数)。

步骤: 对 $t = T-1, T-2, \dots, 0$, Riccati 递推 (标量版):

$$K_t = \frac{b \cdot P_{t+1} \cdot a}{r + b^2 P_{t+1}}, \quad a_t^* = -K_t s_t \quad (\text{E1})$$

$$P_t = q + a^2 P_{t+1} - \frac{a^2 b^2 P_{t+1}^2}{r + b^2 P_{t+1}} = q + \frac{a^2 r P_{t+1}}{r + b^2 P_{t+1}} \quad (\text{E2})$$

无限时域定常解 ($T \rightarrow \infty, a = 1, b = 1, q = r = 1$): P 满足代数 Riccati 方程:

$$P = 1 + \frac{P}{1+P} \Rightarrow P^2 - P - 1 = 0 \Rightarrow P^* = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

(黄金比例!) 增益 $K^* = P^*/(1 + P^*) = 1/\sqrt{5} \approx 0.447$ 。

例题 2: Q-Learning 手算更新

题目: 初始 $Q(s_1, a_R) = 5$ 。观测转移 $(s_1, a_R, r = 10, s_2)$, $Q(s_2, a_L) = 3$, $Q(s_2, a_R) = 7$ 。
 $\alpha = 0.5, \gamma = 0.9$ 。

解:

$$Q(s_1, a_R) \leftarrow 5 + 0.5 \times [10 + 0.9 \times \max(3, 7) - 5] = 5 + 0.5 \times [10 + 6.3 - 5] = 10.65 \quad (\text{E3})$$

例题 3: 策略梯度定理推导 (log-derivative trick)

题目: 证明 $\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t]$, 其中 $G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$ 。

证明:

$$J(\theta) = \int p_{\theta}(\tau) R(\tau) d\tau$$

$$\nabla_{\theta} J = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \underbrace{\frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)}}_{=\nabla_{\theta} \log p_{\theta}(\tau)} R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot R(\tau)] \quad (\text{E4})$$

由于 $\log p_{\theta}(\tau) = \log p(s_0) + \sum_t [\log p(s_{t+1}|s_t, a_t) + \log \pi_{\theta}(a_t|s_t)]$, 对 θ 求导只保留 π_{θ} 项:

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \quad (\text{E5})$$

利用因果性 ($t' < t$ 的奖励与时刻 t 的策略无关, 期望为零), $R(\tau)$ 替换为 G_t , 得证。

例题 4: PPO clip 数值计算

题目: 计算以下两种情形的 clip 目标 ($\epsilon = 0.2$): - 情形 A: $r_t = 1.3$, $\hat{A}_t = 2$ - 情形 B: $r_t = 0.8$, $\hat{A}_t = -1$

解:

情形 A (优势为正, 鼓励增大概率):

$$\text{clip}(1.3, 0.8, 1.2) = 1.2, \quad \min(1.3 \times 2, 1.2 \times 2) = \min(2.6, 2.4) = 2.4$$

clip 生效, 实际更新量被限制, 防止概率增大过多。

情形 B (优势为负, 鼓励减小概率):

$$\text{clip}(0.8, 0.8, 1.2) = 0.8, \quad \min(0.8 \times (-1), 0.8 \times (-1)) = \min(-0.8, -0.8) = -0.8$$

clip 未进一步生效 ($r_t = 0.8$ 已在下界), 正常更新。

例题 5: VQ-VAE 编码过程

题目: 码本 $\{e_1 = (-1, 0), e_2 = (1, 0), e_3 = (0, 1)\}$, 编码器输出 $z_e = (0.8, 0.3)$ 。求量化码字及索引。

解:

$$\|z_e - e_1\| = \sqrt{(1.8)^2 + (0.3)^2} = \sqrt{3.33} \approx 1.825$$

$$\|z_e - e_2\| = \sqrt{(0.2)^2 + (0.3)^2} = \sqrt{0.13} \approx 0.361$$

$$\|z_e - e_3\| = \sqrt{(0.8)^2 + (0.7)^2} = \sqrt{1.13} \approx 1.063$$

$\arg \min = 2$, 量化码字 $z_q = e_2 = (1, 0)$, 对应运动原语索引 2 (例如“向前走”类动作)。

例题 6：伴随变量反向传播验证

题目：二步系统 $f(s, a) = s + a$, 代价 $h(s, a) = s^2 + \frac{a^2}{2}$, 终端代价 $\Phi(s_2) = s_2^2$, 初始 $s_0 = 1$ 。用 PMP 求最优 (a_0^*, a_1^*) 。

解：

由式 (13): $a_t^* = \arg \min_a (a^2/2 + a\lambda_{t+1}) \Rightarrow a_t^* = -\lambda_{t+1}$ 。

由式 (12): $\lambda_2 = \Phi'(s_2) = 2s_2$ 。

由式 (14): $\lambda_1 = h'_s(s_1, a_1) + f'_s\lambda_2 = 2s_1 + \lambda_2$ 。

设 $a_0 = -\lambda_1$, $a_1 = -\lambda_2$ 。状态演化: $s_1 = 1 + a_0 = 1 - \lambda_1$, $s_2 = s_1 + a_1 = s_1 - \lambda_2$ 。

代入 $\lambda_2 = 2s_2$, $\lambda_1 = 2s_1 + 2s_2$, 得联立方程组, 数值解: $a_0^* \approx -0.667$, $a_1^* \approx -0.444$ (具体数值依边界条件求解)。

Anki 卡片 (15+ 张, 复习用)

卡片 1 Q: 轨迹优化 (时空约束) 的标准数学形式? A: $\min \Phi(s_T) + \sum_{t=0}^{T-1} h(s_t, a_t)$, 约束 $s_{t+1} = f(s_t, a_t)$, $0 \leq t < T$ 。

卡片 2 Q: 拉格朗日乘子法的 KKT 等式约束条件 (两个方程)? A: $\partial \mathcal{L} / \partial x = f'(x) + \lambda^T g'(x) = 0$ (最优性: 梯度平行); $\partial \mathcal{L} / \partial \lambda = g(x) = 0$ (约束满足)。

卡片 3 Q: 协态变量 (伴随变量) λ_t 的物理意义是什么? 与什么同构? A: 目标函数对时刻 t 状态 s_t 的灵敏度 (影子价格); 沿时间反向传播, 与神经网络反向传播梯度同构。

卡片 4 Q: PMP 打靶法的三步迭代 (写出三个公式)? A: (1) 前向: $s_{t+1} = f(s_t, a_t)$; (2) 反向: $\lambda_T = \Phi'(s_T)$, $\lambda_t = h'_s + f'^T \lambda_{t+1}$; (3) 更新: $a_t^* = \arg \min_a [h(s_t, a) + f(s_t, a)^T \lambda_{t+1}]$ 。

卡片 5 Q: CMA-ES 的核心循环 (四步)? A: (1) 从 $\mathcal{N}(\mu, \Sigma)$ 采样候选解; (2) 评估代价 (运行仿真); (3) 保留精英样本; (4) 用精英更新 μ 和协方差矩阵 Σ (利用演化路径)。

卡片 6 Q: 贝尔曼最优性方程 (代价最小化版本) 及最优策略公式? A: $V(s) = \min_a [h(s, a) + V(f(s, a))]$; Q 函数 $Q(s, a) = h(s, a) + V(f(s, a))$; 最优策略 $\pi^*(s) = \arg \min_a Q(s, a)$ 。

卡片 7 Q: LQR Riccati 递推的完整公式 (P_t 和 K_t)? A: $P_t = Q_t + A_t^T P_{t+1} A_t - A_t^T P_{t+1} B_t (R_t + B_t^T P_{t+1} B_t)^{-1} B_t^T P_{t+1} A_t$; $K_t = (R_t + B_t^T P_{t+1} B_t)^{-1} B_t^T P_{t+1} A_t$; $a_t^* = -K_t s_t$ 。

卡片 8 Q: LQR 的两个关键性质是什么? iLQR 与 DDP 的区别? A: (1) 值函数始终为二次型; (2) 最优策略始终为线性。iLQR: 动力学一阶 Taylor 展开; DDP: 动力学二阶展开 (精度更高)。

卡片 9 Q: MDP 的五元组是什么? RL 的目标函数? A: $\mathcal{M} = \{S, A, p, r, \gamma\}; J(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_t \gamma^t r(s_t, a_t)]$ (最大化期望折扣回报)。

卡片 10 Q: TD(0) 更新公式, 各符号含义? A: $V(s_t) \leftarrow V(s_t) + \alpha[r_t + \gamma V(s_{t+1}) - V(s_t)]$; α : 学习率; $r_t + \gamma V(s_{t+1}) - V(s_t)$: TD 误差 δ_t 。

卡片 11 Q: Q-Learning 更新规则, 与 TD 的关键区别? A: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$; 区别: TD 误差中用 $\max_{a'} Q$ (最优贪心) 而非当前策略, 是 off-policy 方法。

卡片 12 Q: DQN 的两大稳定性技巧, 各解决什么问题? A: (1) 经验回放: 存历史转移随机采样, 打破序列相关性; (2) 目标网络: 延迟同步参数的固定目标, 避免学习目标随策略频繁变化而发散 (移动靶问题)。

卡片 13 Q: 策略梯度定理的公式及 log-derivative trick 是什么? A: $\nabla_{\theta} J \propto \mathbb{E}[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A^{\pi}(s_t, a_t)]$; trick: $\nabla_{\theta} p = p \cdot \nabla_{\theta} \log p$, 将期望梯度转化为对数概率梯度的期望。

卡片 14 Q: PPO clip 目标函数及 ϵ 的作用? A: $\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t [\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$; ϵ (如 0.2) 限制新旧策略比值范围, 防止更新过大导致策略崩溃。

卡片 15 Q: VAE 的 ELBO 公式, 各项含义? A: $\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{\text{KL}}(q_{\phi}(z|x) \| p(z))$; 第一项: 重构损失 (编码解码后还原运动); 第二项: KL 正则 (隐空间连续, 可采样)。

卡片 16 Q: VQ-VAE 如何量化? 三项损失分别更新什么? A: 量化: $z_q = e_k, k = \arg \min_j \|z_e - e_j\|$ (直通梯度); 重构项更新编码器 + 解码器; 码本项 $\| \text{sg}[z_e] - e \|^2$ 更新码本; commitment 项 $\beta \|z_e - \text{sg}[e]\|^2$ 更新编码器。

卡片 17 Q: MoConVQ 的核心思路是什么? 有何意义? A: 用 VQ-VAE 把运动编码为离散码本 (运动“词汇表”), LLM 以自然语言推理生成码字序列, 物理解码器将码字转化为真实物理控制——实现文本驱动的物理真实角色动画, 打通语言模型与运动控制。

中英术语速查表

中文	English	备注
最优控制	Optimal Control	代价最小化控制策略
轨迹优化	Trajectory Optimization	最优状态-动作序列
时空约束	Spacetime Constraints	Witkin & Kass 1988
开环控制	Open-Loop Control	预计算, 不依赖实时状态
闭环/反馈控制	Closed-Loop / Feedback Control	依赖当前状态
拉格朗日乘子	Lagrange Multiplier	等式约束乘子
KKT 条件	KKT Conditions	最优性一阶必要条件
协态变量 / 伴随变量	Costate / Adjoint Variable	λ_t , 反向传播类比
庞特里亚金极小值原理	Pontryagin's Minimum Principle (PMP)	最优控制核心定理

中文	English	备注
打靶法	Shooting Method	前向 + 反向传递迭代
无导数优化	Derivative-Free Optimization	黑盒优化
协方差矩阵自适应进化策略	Covariance Matrix Adaptation Evolution Strategy (CMA-ES)	经典无导数方法
基于采样的运动控制	Sampling-Based Motion Control (SAMCON)	序贯蒙特卡罗控制
动态规划	Dynamic Programming (DP)	利用最优子结构
贝尔曼最优性原理	Bellman's Principle of Optimality	最优子结构定理
值函数	Value Function $V(s)$	最优代价/奖励的期望
Q 函数 / 状态-动作值函数	Q-Function / State-Action Value Function	$Q(s, a)$
线性二次型调节器	Linear Quadratic Regulator (LQR)	线性 + 二次的解析解
黎卡提方程 / 递推	Riccati Equation / Recursion	LQR 的递推公式
迭代 LQR	iterative LQR (iLQR)	非线性 LQR 推广
微分动态规划	Differential Dynamic Programming (DDP)	iLQR + 二阶动力学展开
马尔可夫决策过程	Markov Decision Process (MDP)	RL 标准框架
折扣回报	Discounted Return	$R = \sum_t \gamma^t r_t$
折扣因子	Discount Factor γ	远期奖励衰减
时序差分学习	Temporal Difference (TD) Learning	在线值函数更新
Q 学习	Q-Learning	Off-policy 最优 Q 学习
深度 Q 网络	Deep Q-Network (DQN)	神经网络参数化 Q
经验回放	Experience Replay	打破时序相关性
目标网络	Target Network	稳定学习目标
策略梯度	Policy Gradient	直接优化策略参数
优势函数	Advantage Function A^π	$Q^\pi - V^\pi$, 减方差基线
演员-评论家	Actor-Critic	策略 + 值函数联合学习
近端策略优化	Proximal Policy Optimization (PPO)	clip 限制更新幅度
变分自编码器	Variational Autoencoder (VAE)	连续隐空间生成模型
证据下界	Evidence Lower Bound (ELBO)	VAE 训练目标
向量量化 VAE	Vector Quantized VAE (VQ-VAE)	离散码本生成模型
码本	Codebook	VQ-VAE 的离散表示
基于模型的强化学习	Model-Based Reinforcement Learning (MBRL)	学习动力学再规划
生成式控制	Generative Control	生成模型 + 物理控制

复习要点

[TIP] 考前必备清单（按重要度排序）

第一优先级（必考）：1. 轨迹优化的标准数学形式（目标函数 + 动力学约束）2. 拉格朗日乘子 KKT 条件（梯度平行 + 约束满足）与几何直觉 3. 协态变量 λ_t 的含义；PMP 打靶法三步流程 4. 贝尔曼方程推导与值函数/Q 函数的定义关系（式 18–20）5. LQR Riccati 递推完整推导（从 P_T 反向到 P_0 ，增益 K_t ，线性策略 $a_t^* = -K_t s_t$ ）6. MDP 五元组；折扣回报；RL 目标 7. TD 误差概念与 Q-Learning 更新规则（式 38–39）8. DQN 两大技巧：经验回放（解决相关性）+ 目标网络（解决移动靶）9. 策略梯度定理 + log-derivative trick 推导 10. PPO clip 目标函数及 ϵ 作用

第二优先级（深入理解）：11. CMA-ES 四步循环；SAMCON 与 CMA-ES 的区别 12. iLQR/DDP 的思路：局部线性化 $\rightarrow$ LQR $\rightarrow$ 迭代收敛 13. Actor-Critic 的双网络结构：Actor 用策略梯度，Critic 用 TD 14. VAE 的 ELBO（重构 + KL），在角色控制中的意义 15. VQ-VAE 量化方式（最近邻）与三项损失的各自更新对象 16. MoConVQ 的核心：离散码本 = 运动语言，LLM 接口实现文本驱动

常见考点陷阱：– 开环 vs 闭环：开环找动作序列，闭环找策略函数 – Q-Learning 是 off-policy ($\max_a Q$)，TD 可以是 on-policy 或 off-policy – LQR 的 Q_t 是代价权重矩阵，与 Q 函数 $Q(s, a)$ 符号相同但概念完全不同 – PPO clip 中 r_t 是新旧策略概率之比，不是奖励

Lecture 12 —肌肉驱动的角色 (Muscle-actuated Characters)

[TIP] 学习指南

本讲核心：把之前“用关节力矩 τ 直接驱动骨架”的抽象动作空间，替换为生物学上更真实的“肌肉激活信号 $a \in [0, 1]$ ”——通过 Hill 型肌肉模型 (Hill-type muscle model) 把神经激活转成肌腱张力，再经过几何映射变成关节力矩，最后驱动多体物理仿真器。

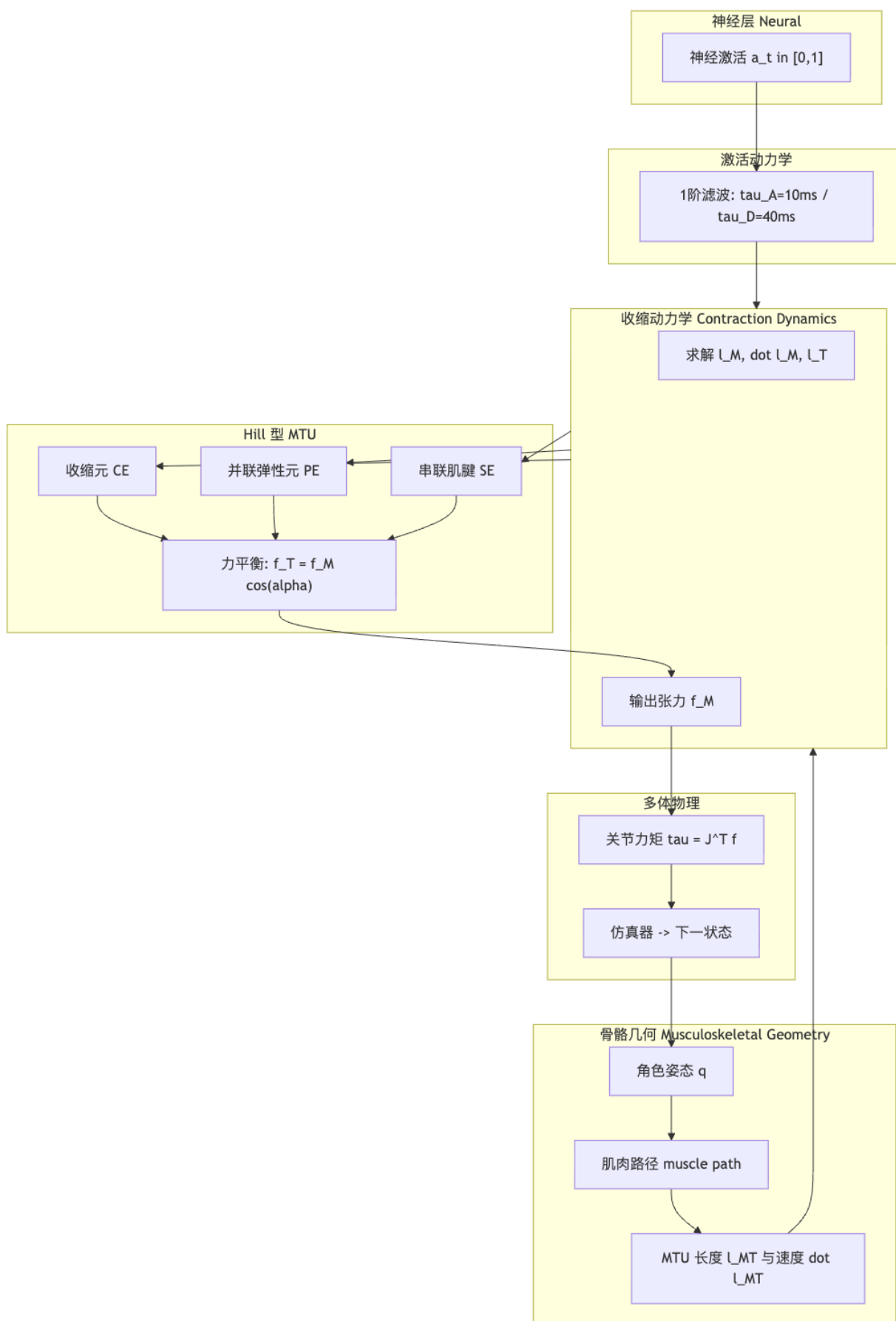
主线脉络（四层递进）：1. **生理结构 (Musculoskeletal anatomy)**：肌肉-肌腱单元 (MTU)、向心/离心收缩、拮抗肌对、羽状角 (pennation angle)。2. **Hill 型力模型**：把 MTU 抽象成“收缩元 CE + 并联弹性元 PE + 串联肌腱”三个力学元件，写出力平衡方程。3. **收缩动力学 (Contraction dynamics)**：给定激活 a_t 和当前姿态，

反解肌纤维长度 l^M 、速度 $\dot{l}^M$ 、肌腱长度 l^T ，再积分推进；引入激活动力学抑制 a 的瞬变。4. **从肌肉力到角色控制**：肌肉路径 $\rightarrow$ 肌腱方向 $\rightarrow$ 关节力矩 $\rightarrow$ 物理仿真；OpenSim 工具、Muscle VAE 学习策略。

前置知识：– lec03–04 多体动力学 (rigid body dynamics)、PD 控制、广义力/力矩；– lec07–08 物理仿真器接口（输入力/力矩，输出下一时刻状态）；– lec11 强化学习与策略网络（用于学习 a_t ）；– 高中生物：肌肉收缩、肌腱、神经冲动；以及一阶常微分方程的显式欧拉积分。

重点掌握清单：1. MTU 的三元件抽象 (CE + PE + 串联肌腱) 与力平衡方程 $f^T = f^M \cos \alpha$ 2. Hill 模型的力归一化拆分： $f^M = \bar{f}_o[a \cdot f_L(l^M)f_V(\dot{l}^M) + f_{PE}(l^M)]$ 3. 用 l^{MT} 、几何约束、 $\dot{l}^M = f_V^{-1}(\cdot)$ 这三条额外方程定解 4. **收缩动力学完整 5 步算法**（已知 $a_t, l_t^M \Rightarrow f^M, l_{t+1}^M$ ）5. **激活动力学**： $\tau_A = 10\text{ms}, \tau_D = 40\text{ms}$ 的一阶滤波 6. **刚性肌腱简化**： $l^T = \bar{l}^T$ 之后无需积分 l^M 7. 肌肉路径如何把标量张力转成关节力矩 8. OpenSim 与 Muscle VAE 的实际应用

0. 全景鸟瞰



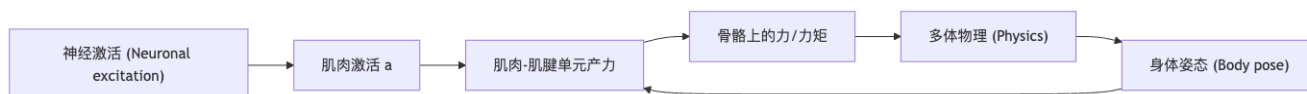
[NOTE] 一句话区分关节力矩驱动 vs 肌肉驱动

– 关节力矩驱动：控制器直接给 τ ，物理上像电机，生物上不真实。– 肌肉驱动：控制器只给“神经命令” $a \in [0, 1]$ ，力的大小受当前肌肉长度、速度、激活水平三重调制，自然产生肌肉疲劳、力速曲线、拮抗肌共激活等生理特性。

1. 角色运动是怎么产生的？

1.1 完整因果链

真实人体运动的生成链路：



肌肉只是这条链路上的一环，把神经信号翻译成可以作用在骨骼上的张力。

1.2 传统仿真控制回路

之前几讲都用“控制器输出关节力矩”这个抽象：

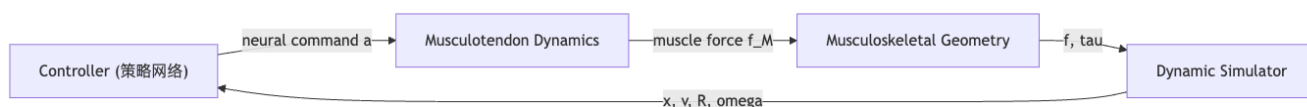


[NOTE] 直觉理解：为什么动画里很多人不用肌肉模型？

因为关节力矩直接可控且解析友好。引入肌肉相当于在控制器和仿真器之间塞了一层非线性“翻译层”，复杂度与参数量都大幅上升。但要研究步态病理、康复、外骨骼、生物力学这些任务，就必须建模到肌肉层。

1.3 肌肉驱动控制回路

把“控制器输出 τ ”换成“控制器输出神经命令 $\rightarrow$ 经 Musculotendon Dynamics $\rightarrow$ 得到肌肉力 $\rightarrow$ 经 Musculoskeletal Geometry $\rightarrow$ 得到 τ ”。



本讲后两节就分别讲清楚 Musculotendon Dynamics（如何由 a 算出 f^M ）和 Musculoskeletal Geometry（如何把张力作用到骨骼）。

2. 肌肉与肌腱：解剖学速通

[NOTE] 定义：肌肉 (Muscle) 与肌腱 (Tendon)

– 肌肉：可主动收缩的生物组织，由许多平行排列的肌纤维 (muscle fiber) 组成。受神经信号刺激时产生张力。
– 肌腱：连接肌肉和骨头的致密结缔组织，本身不能主动收缩，但具有弹性，可以储存和释放弹性能（像橡皮筋）。
在角色动画里：肌肉负责“主动产力”，肌腱负责“传力 + 缓冲”。

2.1 收缩的两种模式

肌肉只能通过收缩产力——不能“推”，只能“拉”。但“收缩”分两种：

模式	英文	长度变化	例子
向心收缩	concentric	缩短	弯举哑铃上举阶段：肱二头肌缩短
离心收缩	eccentric	拉长	弯举哑铃下放阶段：肱二头肌被动拉长但仍发力，控制速度

2.2 拮抗肌对 (Antagonistic Pairs)

骨骼无法被肌肉“推开”，所以人体用成对反向布置的肌肉实现双向运动：

- **主动肌 (agonist)**：正在向心收缩、产生主要动作的肌肉
- **拮抗肌 (antagonist)**：与主动肌作用方向相反，此时放松或被动拉长

例：屈肘时，肱二头肌是主动肌，肱三头肌是拮抗肌；伸肘时角色互换。

[NOTE] 直觉理解：拮抗肌为什么对动画很重要

拮抗肌的“同时激活” (co-contraction) 可以增加关节刚度，让动作更稳但更耗能。表现紧张、负重、平衡时这一现象很明显——是肌肉驱动角色比力矩驱动更“真实”的关键来源之一。

2.3 肌肉-肌腱单元 (MTU)

[NOTE] 定义：MTU (Muscle-Tendon Unit)

把“主动肌纤维 + 串联在其末端的被动肌腱”看作一个力学整体，记为 MTU。在仿真中我们把整个 3D 肌肉抽象成 1 维无质量的中心线 (centroidal path)——它同时代表肌肉段和肌腱段，方便几何计算。

中心线沿“产力轴” (force-generating axis) 排布。这条线遇到关节会沿骨头表面“绕弯”——这就是后面 §6 要讲的肌肉路径。

2.4 羽状角 (Pennation Angle)

肌纤维并不一定与肌腱方向平行——很多肌肉的纤维以一定角度 α 附着到肌腱上（像羽毛的羽枝排在羽轴两侧），称为羽状肌。



关键几何假设：肌纤维等长、平行、共面、被压缩时纤维间距不变——因此当肌肉缩短时羽状角增大（纤维更倾斜），这会影响纤维力到肌腱力的投影。

2.5 肌力是“标量”

肌肉产生的力（张力 tension）是一个标量：

- 沿整个 MTU 中心线均匀——就像一根绷紧的绳子内部张力处处相等
- 由三件事决定：肌纤维长度 l^M 、收缩速度 $\dot{l}^M$ 、激活水平 a

- 方向只在 MTU 的两个端点处由路径切向决定

[WARN] 易错点：肌力的”方向”

肌力的方向不是中间某段决定的；它从路径**两端**沿切向作用到骨上。中间绕弯部分只起到改变路径几何的作用，不”贡献”力的方向。

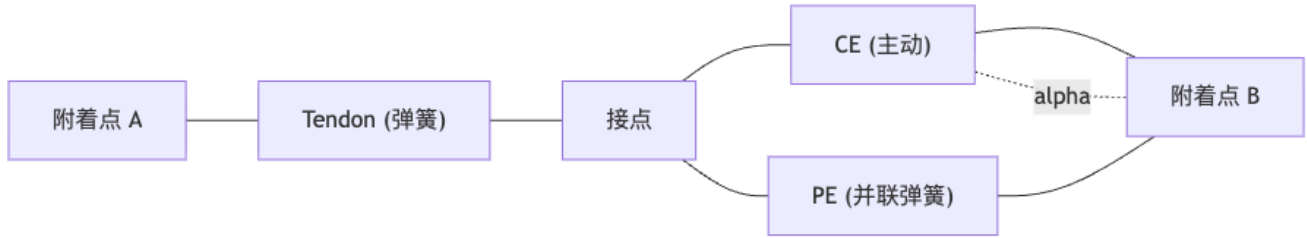
3. Hill 型肌肉模型

3.1 三元件抽象

[NOTE] 定义：Hill 型模型 (Hill-type muscle model)

由生理学家 A. V. Hill 1938 年提出。把肌肉简化为：– **收缩元 (Contractile Element, CE)**：主动产力，张力受激活、长度、速度调制 – **并联弹性元 (Parallel Element, PE)**：被动拉伸时的弹性（肌肉本身的结缔组织）– **串联肌腱 (Series Elastic Tendon)**：与肌肉串联的弹性肌腱
CE 和 PE 并联构成**肌肉部分**，再与肌腱串联构成 MTU。

电路类比图：



3.2 力平衡

肌纤维与肌腱之间有羽状角 α ，所以肌肉部分产生的力 f^M 投影到肌腱轴向才与肌腱力 f^T 平衡：

$$f^T = f^M \cos \alpha \quad (1)$$

CE 与 PE 并联，肌肉力是两者之和：

$$f^M = f^{CE} + f^{PE} \quad (2)$$

3.3 CE 与 PE 的力函数

Hill 模型用归一化无量纲函数描述力学行为。设 $\bar{f}_o$ 为肌肉的最大等长张力（muscle normalization constant，即在最佳长度、静止、满激活时能产生的最大力），则：

$$f^{CE} = \bar{f}_o \cdot a \cdot f_L(l^M) \cdot f_V(\dot{l}^M) \quad (3)$$

$$f^{PE} = \bar{f}_o \cdot f_{PE}(l^M) \quad (4)$$

各符号含义：

符号	含义	取值范围
$\bar{f}_o$	肌肉最大等长张力（材料参数）	> 0 常数
a	激活水平 (activation level)	$[0, 1]$
$f_L(l^M)$	力-长度曲线 (force-length curve): CE 在不同长度下产力能力	钟形, 在 $l^M = l_o^M$ 处取峰值 1
$f_V(\dot{l}^M)$	力-速度曲线 (force-velocity curve)	缩短时 < 1 、拉长时 > 1
$f_{PE}(l^M)$	PE 的被动力-长度曲线	在 $l^M < l_o^M$ 时为 0; 超过自然长度后指数上升

[NOTE] 直觉理解：三条曲线为什么是这个形状

– f_L : 肌节 (sarcomere) 中粗细肌丝最佳重叠时产力最强; 过短粗细丝相互挤压、过长重叠减少, 产力都下降 → 钟形。– f_V : 缩短时分子机器来不及完整循环, 产力下降; 被动拉长时弹性元件被进一步拉紧, 产力反而上升。– f_{PE} : 肌肉静止时无被动力; 被拉过自然长度后结缔组织像非线性弹簧“绷紧”。

3.4 肌腱力函数

肌腱是非线性“弹簧”：当被拉伸超过松弛长度 (slack length) l_s^T 时存储弹性能, 否则不产力。

$$f^T = \bar{f}_o \cdot f_T(l^T) \quad (5)$$

$f_T(l^T)$ 在 $l^T < l_s^T$ 时为 0, 超过后近似分段指数 + 线性。

4. 给定 a 解出肌肉力：收缩动力学

4.1 把所有方程合到一起

代入 (3)(4) 到 (2), 再代入 (1), 得 MTU 内的总力平衡：

$$f^T = \frac{\bar{f}_o [a \cdot f_L(l^M) \cdot f_V(\dot{l}^M) + f_{PE}(l^M)]}{\cos \alpha} \quad (6)$$

把 $f^T = \bar{f}_o f_T(l^T)$ 代进左边, 整理为零形式：

$$\bar{f}_o \left\{ \frac{a \cdot f_L(l^M) \cdot f_V(\dot{l}^M) + f_{PE}(l^M)}{\cos \alpha} - f_T(l^T) \right\} = 0 \quad (7)$$

4.2 未知量与缺方程

未知量: $l^M, \dot{l}^M, l^T$ (共 3 个), 还有 α (但可由几何算出)。已知: 激活 a_t 、角色姿态。方程: 仅 1 条 (7)。需要补 2 条。

4.3 补一: 羽状角的几何约束

把羽状肌简化为”平行四边形”几何: 肌纤维以倾角 α 排列、纤维末端到肌腱轴的垂直距离 d 保持常数 (肌肉收缩时纤维方向旋转但长度方向投影不变):

$$\alpha = \sin^{-1}\left(\frac{d}{l^M}\right) \quad (8)$$

这把 α 表达成了 l^M 的函数, 少一个未知量。

4.4 补二: MTU 总长度由姿态给出

定义 MTU 中心线总长:

$$l^{MT} = l^T + l^M \cos \alpha \quad (9)$$

l^{MT} 完全由当前骨架姿态决定 (从肌肉两端附着点沿肌肉路径的几何长度算出, §6 会讲)。所以 (9) 是一条额外方程, 把 l^T 与 l^M 联系起来。

4.5 还差一条: 把 $\dot{l}^M$ 解出来

至此 (7)(8)(9) 给我们 3 条方程、4 个未知量 ($l^M, \dot{l}^M, l^T, \alpha$)。还差 1 条——让 $\dot{l}^M$ 跟着 l^M 走。

把 (7) 反解 $f_V(\dot{l}^M)$:

$$f_V(\dot{l}^M) = \frac{f_T(l^T) \cos \alpha - f_{PE}(l^M)}{a \cdot f_L(l^M)}$$

再取力-速度曲线的逆函数 f_V^{-1} :

$$\dot{l}^M = f_V^{-1}\left(\frac{f_T(l^T) \cos \alpha - f_{PE}(l^M)}{a \cdot f_L(l^M)}\right) \quad (10)$$

这条等式让 $\dot{l}^M$ 由 (l^M, l^T, a, α) 决定 $\rightarrow$ 系统降为关于 l^M 的一阶常微分方程 (ODE)。

4.6 完整算法: 收缩动力学一步推进

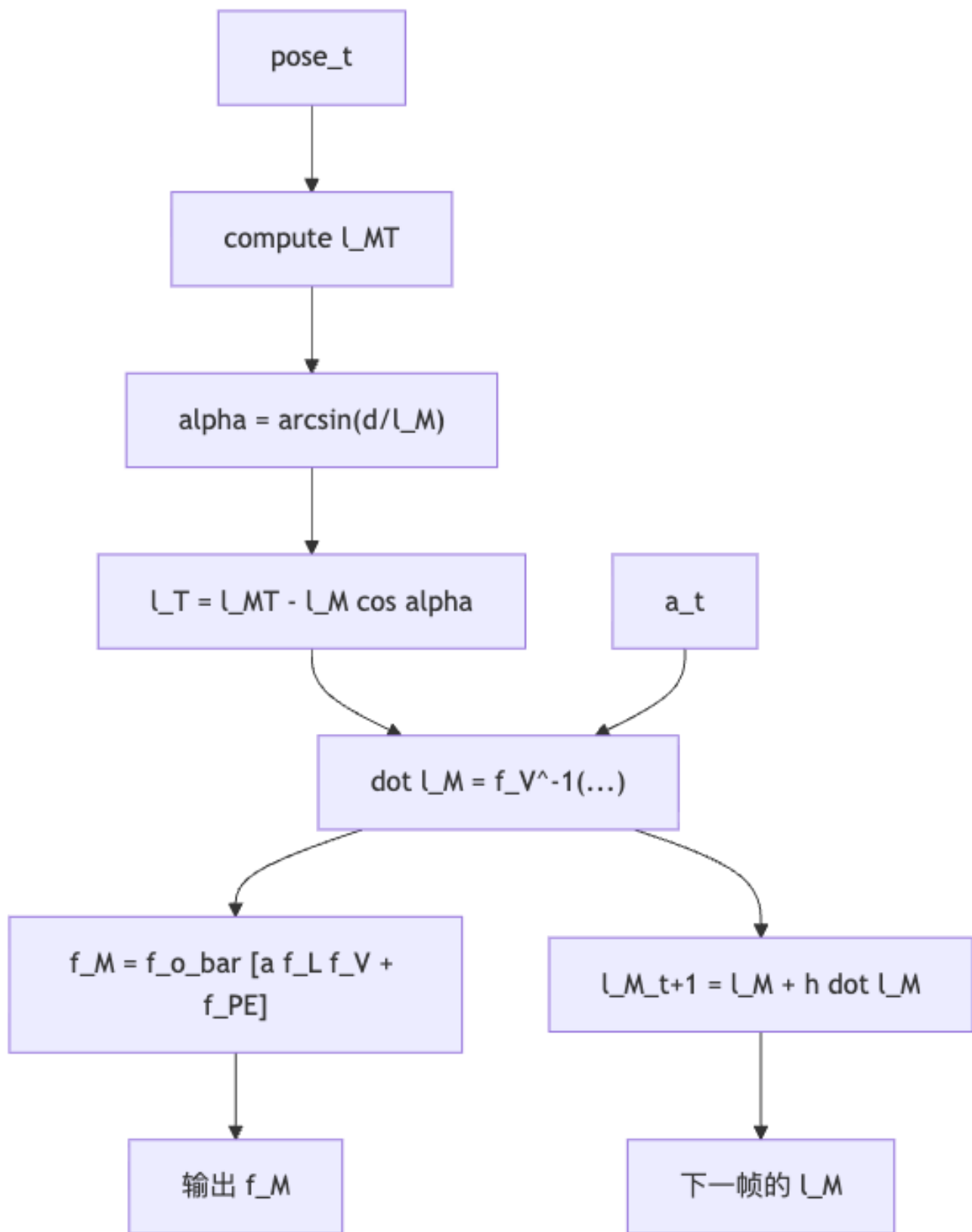
给定初值 l_0^M , 每个时间步 t 做:

```

Algorithm ContractionStep(a_t, l_M_t, pose_t):
  Step 0: l_MT <- 从 pose_t 沿肌肉路径计算总长
  Step 1: alpha <- arcsin( d / l_M_t ) # 由 (8)
  Step 2: l_T <- l_MT - l_M_t * cos(alpha) # 由 (9)
  Step 3: dot_l_M <- f_V_inv( ( f_T(l_T)*cos(alpha) - f_PE(l_M_t) ) / ( a_t * f_L(l_M_t) ) ) # 由 (10)
  Step 4: f_M <- f_o_bar * [ a_t * f_L(l_M_t) * f_V(dot_l_M) + f_PE(l_M_t) ]
  Step 5: l_M_{t+1} <- l_M_t + h * dot_l_M # 显式欧拉积分
  return f_M, l_M_{t+1}

```

复杂度：每条肌肉每时间步 $O(1)$ (只是查曲线 + 几何)。一个人体模型常有 200–700 条肌肉，仍然实时可跑。



[EX] 一根肱二头肌单帧追踪

设 $\bar{f}_o = 600\text{N}$, $l_o^M = 0.12\text{m}$, 当前 $l_t^M = 0.13\text{m}$ (略拉长), $a_t = 0.5$ 。

1. 由姿态算 $l_o^{MT} = 0.30\text{m}$;
 2. $\alpha = \arcsin(0.005/0.13) \approx 2.2^\circ$ (很小);
 3. $l^T \approx 0.30 - 0.13 \times 0.999 \approx 0.170\text{m}$;
 4. 查曲线得 $f_L \approx 0.95$, $f_{PE} \approx 0.02$, $f_T \approx 0.10 \rightarrow$ 反解 $\dot{l}^M$;
 5. 代回得 $f^M \approx 600 \times [0.5 \times 0.95 \times f_V + 0.02] \text{N}$ 。
- 然后用 $f^T = f^M \cos \alpha$ 作用到肌肉两端的骨头上。

5. 激活动力学 (Activation Dynamics)

5.1 为什么 a 不能瞬变

在 §4 我们默默假设 a_t 在每个时间步可任意改变。但真实肌肉的激活水平不能瞬变：

- 神经冲动到达后，肌纤维内的钙离子释放、肌动蛋白-肌球蛋白循环需要时间
- 实测：激活时间常数 $\tau_A \approx 10\text{ms}$ ，失活时间常数 $\tau_D \approx 40\text{ms}$

[WARN] 易错点：a 瞬变带来的数值灾难

如果让控制器直接把 a 在一帧内从 0 跳到 1，方程 (7) 要求 $f_V(\dot{l}^M)$ 立刻“变大”以匹配新激活下的力平衡——这往往需要 $\dot{l}^M$ 取极大值，物理上不真实且数值上爆炸。

5.2 一阶滤波

把 a 看成“想要的激活” $u \in [0, 1]$ 的滤波输出。常用一阶动力学：

$$\dot{a} = \begin{cases} (u - a)/\tau_A, & u \geq a \text{ (正在激活)} \\ (u - a)/\tau_D, & u < a \text{ (正在松弛)} \end{cases} \quad (11)$$

时间常数不对称：激活快、松弛慢。这就解释了为什么真人做高频肢体动作（如颤抖）会发紧——没法及时“松手”。

5.3 控制器输出层

实际系统里：- 策略网络 (RL/VAE) 输出 $u_t \in [0, 1]$ - 由 (11) 一阶滤波得到真实激活 a_t - 把 a_t 喂进 §4 的收缩动力学

[NOTE] 直觉理解：激活动力学和神经控制延迟

一些研究还会在激活动力学外再串一个固定延迟（几十毫秒，模拟脊髓-肌肉传导），让控制器面对的反馈更“真实地”滞后。

6. 刚性肌腱简化 (Rigid Tendon Simplification)

6.1 动机

人体大多数肌腱比肌纤维硬得多：阿喀琉斯腱伸长 4–5% 就接近断裂应变，而肌纤维可以缩短/拉长 30% 以上。于是工程上常用刚性肌腱假设：把肌腱当作不可伸长的“绳”，长度固定为 $\bar{l}^T$ 。

6.2 新的方程组

原系统的未知量是 $\{l^M, \dot{l}^M, l^T\}$ 。现在 $l^T = \bar{l}^T$ 直接消掉一个未知量；从 (9) 直接解 l^M ：

$$\bar{l}^T + l^M \cos \alpha = l^{MT}, \quad \alpha = \sin^{-1}(d/l^M)$$

利用 $\sin^2 \alpha + \cos^2 \alpha = 1$, $l^M \cos \alpha = \sqrt{(l^M)^2 - d^2}$, 所以 $l^{MT} - \bar{l}^T = \sqrt{(l^M)^2 - d^2}$, 解出：

$$l^M = \sqrt{d^2 + (l^{MT} - \bar{l}^T)^2} \quad (12)$$

对时间求导（链式法则， $d^2, \bar{l}^T$ 是常数）：

$$2l^M \dot{l}^M = 2(l^{MT} - \bar{l}^T) \dot{l}^{MT} \Rightarrow \dot{l}^M = \frac{l^{MT} - \bar{l}^T}{l^M} \dot{l}^{MT}$$

用 $\cos \alpha = (l^{MT} - \bar{l}^T)/l^M$ 可改写成更整齐的形式：

$$\dot{l}^M = \dot{l}^{MT} \cdot \cos \alpha = \dot{l}^{MT} \sqrt{1 - \left(\frac{d}{l^M}\right)^2} \quad (13)$$

6.3 简化后的算法

```
Algorithm RigidTendonStep(a_t, pose_t, dot_pose_t):
  Step 0: l_MT, dot l_MT ← 从姿态与速度沿肌肉路径计算
  Step 1: l_M ← sqrt( d^2 + (l_MT - l_T_bar)^2 ) # (12)
  Step 2: dot l_M ← dot l_MT * sqrt( 1 - (d/l_M)^2 ) # (13)
  Step 3: f_M ← f_o_bar * [ a_t * f_L(l_M) * f_V(dot l_M) + f_PE(l_M) ]
  return f_M
```

[NOTE] 直觉理解：为什么简化后更稳

原来要积分 ODE 维护 l^M ，数值误差会累积。刚性肌腱下 l^M 由 l^{MT} 直接代数算出 → 无内部状态，每帧从零开始，鲁棒得多。代价是丢掉了肌腱的弹性储能（不能模拟“弹跳”步态中的能量回收）。

7. 肌肉路径与关节力矩 (Musculoskeletal Geometry)

7.1 肌肉路径 (Muscle Path)

肌肉中心线在骨架上不止两个端点，而是：

- 经过若干**固定附着点 (via points)**，这些点相对某根骨头位置固定
- 在关节附近**包绕 (wrap)** 并滑过圆柱/椭球等几何体（如膝关节后侧腓肠肌绕过股骨髁）



给定骨架姿态 q ，可解析或几何地算出：

- 总路径长度 $l^{MT}(q)$
- 路径长度对姿态的偏导 $\partial l^{MT} / \partial q$ (用于下式)

7.2 从张力到关节力矩

肌力 f^T 是沿路径的标量张力。它在**起点处**沿路径切向作用于骨 A，在**止点处**沿路径反向切向作用于骨 B（中间 via point 给上的力等大反向、互相抵消）。

利用**虚功原理 (principle of virtual work)**：把 MTU 当成一条“线性弹簧 + 主动源”，它对广义坐标 q 做的虚功是

$$\delta W = -f^T \delta l^{MT}$$

广义力（关节力矩） τ^{muscle} 满足 $\delta W = \tau^{\text{muscle}, \top} \delta q$ ，故：

$$\tau^{\text{muscle}} = -f^T \frac{\partial l^{MT}}{\partial q} \quad (14)$$

[NOTE] 直觉理解： $\partial l^{MT} / \partial q$ 就是“力臂”

对单关节单自由度肌肉， $\partial l^{MT} / \partial q$ 退化为传统生物力学里讲的 **moment arm (力臂)** ——肌肉缩短一点时关节转过的角度的倒数。整条肌肉系统的关节力矩，就是各肌肉张力乘各自力臂再求和。

7.3 全身合成

人体一个自由度上常被多条肌肉同时作用，总力矩是所有跨越该关节的肌肉贡献之和：

$$\tau^{\text{total}}(q) = - \sum_{m=1}^M f_m^T \frac{\partial l_m^{MT}}{\partial q}$$

把它代入多体动力学方程 $M(q)\ddot{q} + C(q, \dot{q}) = \tau^{\text{total}} + J^\top \lambda$ 即可推进一步仿真。

[WARN] 易错点：肌肉冗余 (muscle redundancy)

人体自由度远少于肌肉数（如人腿约 7 自由度但有 40+ 条肌肉），由肌肉力反求关节力矩是“一对多”的——同一个关节力矩可对应**无穷多组**肌肉激活方式。这是生物力学中著名的**冗余问题**，传统方法用静态优化（ $\min \sum a_m^2$ ）求解，深度学习方法（Muscle VAE）则让神经网络自己找最优策略。

8. 工具与应用

8.1 OpenSim：肌骨仿真的事实标准

[NOTE] OpenSim

由斯坦福 Delp 团队开发，开源 (<https://simtk.org/projects/opensim>)。提供：– 标准化人体肌骨模型（如 Gait2392、Rajagopal 全身模型）– Hill 型 MTU + 路径包绕 + 多体动力学 – 逆动力学、静态优化、计算肌肉控制 (CMC) 等经典算法
在角色动画中常被当作“参考真相”或仿真后端使用。

8.2 Muscle VAE：把神经控制学进网络

把 §1.3 控制回路里的 Controller 用一个 VAE 学出来：



训练目标：让仿真出的运动跟踪参考动捕，同时正则化 z 与 u （防止激活过激或过度共激）。推理时可以采样新的 z 生成多样运动，或叠加目标条件控制（行走方向、目标位置）。

参考论文 (slide 引用区)：MuscleVAE: Model-Based Controllers of Muscle-Actuated Characters (Feng et al., SIGGRAPH Asia 2023)。

[EX] 应用案例

- 步态合成：用肌肉模型可以学出区分疲劳、负重、伤病的细微步态差异，纯力矩驱动很难表现。
- 外骨骼/假肢设计：在仿真中给角色穿上外骨骼，优化外骨骼输出以最小化某些关键肌群激活。
- 运动康复：模拟肌力下降（降低某条肌肉的 f_o ）后角色如何代偿——为康复方案提供参考。

9. 记号速查表

符号	含义
$a, a_t \in [0, 1]$	肌肉激活水平 (activation)
$u_t \in [0, 1]$	控制器输出的“目标激活”
τ_A, τ_D	激活/失活时间常数 (10ms / 40ms)
f_o	肌肉最大等长张力 (muscle normalization constant)
$l^M, \dot{l}^M$	肌纤维长度、速度
l_o^M	最佳肌纤维长度 (optimal fiber length)
$l^T, \bar{l}^T, l^s_T$	肌腱长度、刚性肌腱定长、肌腱松弛长度
$l^{MT}, \dot{l}^{MT}$	MTU 中心线总长与变化率
α	羽状角 (pennation angle)
d	肌纤维相对于肌腱轴的垂直距离 (常数)
f^{CE}, f^{PE}, f^M, f^T	CE / PE / 肌肉总 / 肌腱 力
$f_L(\cdot), f_V(\cdot), f_{PE}(\cdot), f_T(\cdot)$	归一化力-长度、力-速度、PE、肌腱曲线
f_V^{-1}	力-速度曲线的逆函数
$q, \dot{q}$	广义坐标 (关节角) 与速度
τ^{muscle}	肌肉合成的广义力矩
$\partial l^{MT} / \partial q$	肌肉力臂 (moment arm)

符号	含义
h	仿真时间步长

10. 中英术语对照表

中文	English	备注
肌肉-肌腱单元	Muscle-Tendon Unit (MTU)	仿真基本单元
肌纤维	muscle fiber	1D 直线段抽象
肌腱	tendon	被动弹性
收缩元	Contractile Element (CE)	主动产力
并联弹性元	Parallel Element (PE)	与 CE 并联
串联弹性肌腱	Series Elastic Tendon (SE)	与肌肉串联
Hill 型模型	Hill-type muscle model	A. V. Hill 1938
向心收缩	concentric contraction	缩短发力
离心收缩	eccentric contraction	拉长发力
拮抗肌对	antagonistic pair	agonist / antagonist
共激活	co-contraction	提升关节刚度
羽状角	pennation angle	肌纤维与肌腱夹角
激活水平	activation level / activation	范围 $[0, 1]$
激活动力学	activation dynamics	一阶滤波 τ_A, τ_D
收缩动力学	contraction dynamics	$a \rightarrow f^M$ 的过程
力-长度曲线	force-length curve	钟形
力-速度曲线	force-velocity curve	凹形
肌肉路径	muscle path	含 via point 与 wrap surface
力臂	moment arm	$\partial l^{MT} / \partial q$
肌肉冗余	muscle redundancy	多对一映射
最大等长张力	maximum isometric force	$\bar{f}_o$
最佳纤维长度	optimal fiber length	$\bar{l}_o^M$
松弛长度	slack length	肌腱开始产力的临界长度
刚性肌腱假设	rigid tendon simplification	$\bar{l}^T \equiv \bar{l}^T$
肌骨仿真器	musculoskeletal simulator	OpenSim 等
神经命令	neural command / excitation	控制器输出

[TIP] 期末复习要点

1. Hill 型 MTU 三元件: CE + PE 并联构成“肌肉”, 与肌腱串联; 力平衡 $f^T = f^M \cos \alpha$ 、 $f^M = f^{CE} + f^{PE}$ 。2. 力函数解析式: $f^{CE} = \bar{f}_o a f_L(l^M) f_V(\dot{l}^M)$ 、 $f^{PE} = \bar{f}_o f_{PE}(l^M)$ 、 $f^T = \bar{f}_o f_T(l^T)$ 。3. 三条几何/姿态约束补方程: $\alpha = \arcsin(d/l^M)$; $\bar{l}^{MT} = \bar{l}^T + l^M \cos \alpha$ (由姿态算); $\dot{l}^M = f_V^{-1}(\cdot)$ 让系统降为单 ODE。4. 收缩动力学 5 步: 算 $\bar{l}^{MT} \rightarrow \alpha \rightarrow \bar{l}^T \rightarrow \dot{l}^M \rightarrow f^M \rightarrow$ 显式欧拉更新 $\bar{l}^M$ 。5. 激活动力学: $\tau_A = 10\text{ms}$ (激活快), $\tau_D = 40\text{ms}$ (松弛慢); 防止 a 瞬变引起数值爆炸。6. 刚性肌腱简化: $\bar{l}^T = \bar{l}^T$, 得 $\bar{l}^M = \sqrt{d^2 + (\bar{l}^{MT} - \bar{l}^T)^2}$ 、 $\dot{l}^M = \dot{\bar{l}}^{MT} \cos \alpha$; 无内部状态。7. 从张力到力矩: $\tau^{\text{muscle}} = -f^T \partial \bar{l}^{MT} / \partial q$ (虚功原理), 单关节情形即力臂 $\times$ 张力。8. 肌肉冗余与 OpenSim/Muscle VAE 的实际应用思路。

11. 进一步阅读

- **A. V. Hill (1938).** The heat of shortening and the dynamic constants of muscle. Proc. Royal Soc. B. —Hill 模型原始论文。
- **Zajac, F. E. (1989).** Muscle and tendon: properties, models, scaling, and application to biomechanics and motor control. Critical Reviews in Biomedical Engineering. —Hill 模型在生物力学中的规范化处理。
- **Delp, S. L. et al. (2007).** OpenSim: open-source software to create and analyze dynamic simulations of movement. IEEE TBME. —OpenSim 系统论文。
- **Wang, J. M., Hamner, S. R., Delp, S. L., Koltun, V. (2012).** Optimizing locomotion controllers using biologically-based actuators and objectives. ACM TOG (SIGGRAPH). —把肌肉模型引入图形学的代表作。
- **Lee, S. et al. (2019).** Scalable muscle-actuated human simulation and control. SIGGRAPH. —大规模肌肉驱动角色 RL 控制。
- **Feng, Y. et al. (2023).** MuscleVAE: Model-Based Controllers of Muscle-Actuated Characters. SIGGRAPH Asia. —一本讲最后展示的工作。

Lecture 13 —运动规划与群体仿真 (Motion Planning & Crowds)

[TIP] 学习指南

本讲核心：解决”角色从 A 走到 B”这一看似简单却深刻的问题——把高维构型空间中的避障路径搜索拆解为三类范式（图搜索 / 路标图 / 采样树），再扩展到大规模角色群体的协同运动建模。

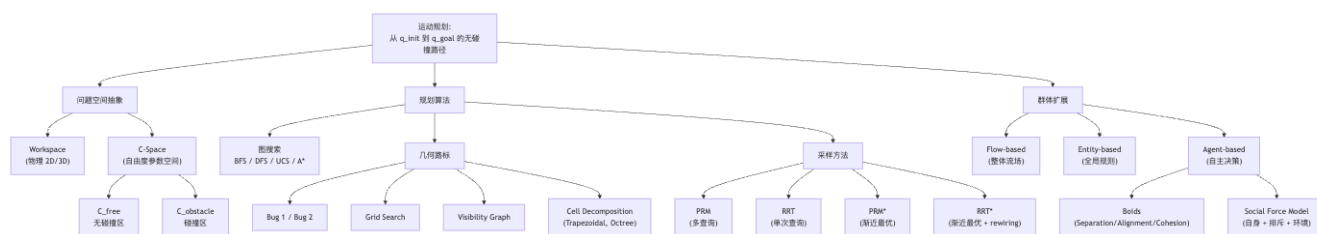
核心主线（五层递进）：1. 问题空间：从工作空间 (workspace) 抽象到构型空间 (C-space)，把”机器人不撞障碍”翻译成”构型点不落入 $C_{obstacle}$ ”。2. 图搜索 (Graph Search)：BFS / DFS / UCS / A*——在已离散化的图上找最短路径，是一切规划的基础工具。3. 几何路标图 (Roadmap)：Bug 算法 / Grid Search / Visibility Graph / Cell Decomposition，用确定性方式刻画 C_{free} 的连通性。4. 采样方法 (Sampling-based)：PRM、RRT、RRT-Connect、PRM*、RRT*——用随机采样克服高维诅咒，把”完备性”放松为”概率完备”与”渐近最优”。5. 群体仿真 (Crowds)：Boids (三规则)、Social Force Model (受力方程)——把单体规划扩展到含社会心理因素的大规模群体。

前置知识（正文遇到时会就地解释）：- 图论基础：节点、边、邻居、优先队列 - 微积分梯度记号 ∇ 、向量内积 - lec03 运动学：关节角度 = 构型 - 概率初步：高斯采样

重点掌握清单：1. C-space 的定义和”为什么要从 workspace 切到 C-space”2. A* 的优先级 $f = g + h$ 与 admissible heuristic 概念 3. Bug 1 / Bug 2 完备性差异 4. Visibility Graph 在 2D 最短路与 3D 失败的几何原因 5. PRM 的两种连接策略 k -NN 与 R -邻域 6. RRT 的”四步循环”伪代码与覆盖性偏置 7. RRT* rewiring 的渐近最优性证明思想 8. Boids 三规则与 Social Force Model 的三力分解

本讲不需要：神经网络、强化学习（属于 lec06 / lec11）；动力学仿真器（属于 lec08）。

0. 全景鸟瞰



[NOTE] 一句话区分三大算法范式

图搜索：图已经给定（如格子地图），只解决”在图上找最短路”。几何路标：自己构造一张能代表 C_{free} 连通性的图，再套图搜索。采样方法：连图都不显式构造，靠随机采样边走边长，适合高维。

1. 运动规划问题：Workspace 与 Configuration Space

1.1 Workspace (工作空间)

[!info] 定义 1.1 (工作空间 Workspace) 机器人/角色物理上所处的空间，通常是 $\mathbb{R}^2$ 或 $\mathbb{R}^3$ ，里面有什么形状的障碍物 $\mathcal{O}$ 和机器人本体 $\mathcal{A}$ 。

但人形机器人有 30+ 个自由度 (关节角度)，在工作空间里画”机器人形状”再做碰撞检测会非常复杂。核心思想：把机器人本身当作一个抽象的”参数点”。

1.2 Configuration Space (构型空间, C-space)

[!info] 定义 1.2 (构型 Configuration) 一个完全描述机器人位姿的参数向量 q 。例如：- 平面位移机器人： $q = (x, y) \in \mathbb{R}^2$ - 平面带朝向： $q = (x, y, \theta) \in \mathbb{R}^2 \times S^1$ - n 关节机械臂： $q = (\theta_1, \dots, \theta_n) \in T^n$ (n -环面) - 人形： $q \in \mathbb{R}^3 \times SO(3) \times \mathbb{R}^{n_{\text{joints}}}$ (根位姿 + 各关节)

[!info] 定义 1.3 (构型空间 C-space) 所有合法构型组成的集合，记作 $\mathcal{C}$ 。它把每个机器人压成 $\mathcal{C}$ 中的一个点——无论机器人有多少个 link、多复杂的形状。

把工作空间里的障碍物”映射”到 C-space：

$$\mathcal{C}_{\text{obstacle}} = \{q \in \mathcal{C} \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\} \quad (1)$$

$$\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obstacle}} \quad (2)$$

其中 $\mathcal{A}(q)$ 是机器人在构型 q 下占据的工作空间区域。

[NOTE] 直觉理解——为什么要切到 C-space

在 workspace 里”机器人形状不能碰墙”是个几何相交检测，每次都烦；而在 C-space 里则简化为”点不能落入 $\mathcal{C}_{\text{obstacle}}$ 区域”，跟一个质点避障完全等价。代价是 $\mathcal{C}_{\text{obstacle}}$ 的形状变复杂了（一般是非凸的，甚至有空洞）。

[EX] 例子 1.4 (圆盘机器人在 2D 障碍场)

Workspace：圆盘半径 r ，障碍是个方形。C-space： $\mathcal{C}_{\text{obstacle}}$ 是方形向外膨胀 r 的”膨胀障碍” (Minkowski sum)， $\mathcal{C}$ 中的”机器人”退化为一个点。

这就是著名的 Minkowski sum 膨胀： $\mathcal{C}_{\text{obstacle}} = \mathcal{O} \oplus (-\mathcal{A})$ 。

[WARN] 易错点

C-space 的维度等于机器人自由度，不是 workspace 的维度。一个二维平面里的 3 关节机械臂，workspace 是 $\mathbb{R}^2$ ，但 C-space 是 T^3 (三维环面)。这就是为什么”高维规划很难”——人形 30+ 自由度，C-space 是 30+ 维的。

1.3 规划问题陈述

[!info] 定义 1.5 (运动规划问题) 给定 $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$ ，起始构型 $q_{\text{init}} \in \mathcal{C}_{\text{free}}$ ，目标构型/目标区 q_{goal} (或 $\mathcal{Q}_{\text{goal}}$)，找一条连续路径

$$\gamma : [0, 1] \rightarrow \mathcal{C}_{\text{free}}, \quad \gamma(0) = q_{\text{init}}, \quad \gamma(1) \in \mathcal{Q}_{\text{goal}}$$

使得整条路径都在 $\mathcal{C}_{\text{free}}$ 中。

1.4 衡量规划算法的三把尺子

属性	含义
Completeness (完备性)	若存在解, 算法 保证 返回解; 若不存在, 能在有限时间内报告“无解”。
Optimality (最优性)	返回的路径 保证 是某个代价 (如长度) 下的全局最优。
Complexity (复杂度)	时间 / 空间开销随 C-space 维度、障碍密度的增长。

弱化版本: – **Resolution-complete**: 当离散网格无限细化时完备 (如 Grid Search) – **Probabilistic-complete**: 当采样数 $n \rightarrow \infty$ 时返回解的概率为 1 (如 PRM、RRT) – **Asymptotically-optimal**: $n \rightarrow \infty$ 时返回的解以概率 1 收敛到全局最优 (如 PRM*, RRT*)

1.5 信息条件分类

- **Uninformed Search**: 算法对环境一无所知, 盲搜——BFS、DFS、UCS。
- **Informed Search**: 能利用启发式 (如“距目标的欧氏距离”)——A*、D*。

2. 图搜索基石

2.1 通用图搜索框架

```
Initialize priority queue Q
Q.insert(x_I) and mark x_I as visited
while Q is not empty:
    x ← Q.popFirst()
    if x in Goal:
        return Success
    for u in edges_of(x):
        x_u ← move(x, u)
        if x_u is not visited:
            mark x_u as visited
            Q.insert(x_u, priority(x_u))
        else:
            resolve_duplicate(x_u)
return Failure
```

所有图搜索的差异只在两件事: 1. $\text{priority}(x_u)$ ——优先级如何计算 2. $\text{resolve_duplicate}(x_u)$ ——重复节点如何处理

算法	优先级
BFS (广度优先)	早访问优先级高 (队列)
DFS (深度优先)	晚访问优先级高 (栈)
UCS (一致代价)	从起点累计代价 $g(x)$
A* (启发式)	$f(x) = g(x) + h(x)$

2.2 A* 算法

[!info] 定义 2.1 (A* 优先级)

$$f(x) = g(x) + h(x) \tag{3}$$

– $g(x)$: 从起点到 x 已确定的最低路径代价 – $h(x)$: 启发函数, 估计从 x 到目标的剩余代价

最优性条件: 若 h 可接受 (admissible), 即

$$h(x) \leq h^*(x) \quad \text{对所有 } x \quad (4)$$

($h^*(x)$ 是真实的最优剩余代价), 则 A^* 返回全局最小代价路径。

[NOTE] 直觉理解——启发式为什么有效

$h(x)$ 像导航软件里“目的地距离”的指针——告诉算法“哪边大方向更有希望”, 避免像 UCS 那样向四面八方均匀扩展。“Admissible”的意思是启发式不准没关系, 但不能高估; 一旦高估, 可能跳过真正的最短路。欧氏距离对最短路径问题永远是 admissible 的 (直线 $\leq$ 任何绕行)。

[EX] 例 2.2 (栅格地图 A^*)

40×40 网格, 起点 $(0, 0)$, 终点 $(39, 39)$, 每格移动代价 1, 斜向 $\sqrt{2}$ 。用 $h(x) = \sqrt{(x.x - 39)^2 + (x.y - 39)^2}$ 即可大幅压缩搜索面积。若改用 $h(x) = 1.5 \cdot \text{Euclidean}$ (高估), 可能更快但不再最优——这叫 weighted A^* , 常见的速度/质量折中。

2.3 在 C-space 中搜索的难题

[WARN] C-space 是连续的

上面伪代码默认图是有限的。但 $\mathcal{C}_{\text{free}}$ 通常是连续空间——这就是后面“离散化” (Grid / Cell / 采样) 的根本动机。

3. 几何路标方法

3.1 Bug 算法——最朴素的“摸着墙走”

[info] 定义 3.1 (Bug 算法家族) 假设机器人只有局部触觉: 能感知“是否撞到障碍”, 并能沿障碍轮廓行走。无需全局地图。

Bug 1

1. 沿直线朝目标走;
2. 一旦撞到障碍: 绕着障碍走一整圈, 记录沿途各点到目标的距离;
3. 回到圈上距目标最近的点, 从该点继续朝目标走。

保证: Bug 1 完备——只要解存在, 必定找到。代价: 每个障碍都要绕完整一圈, 路径明显冗余。

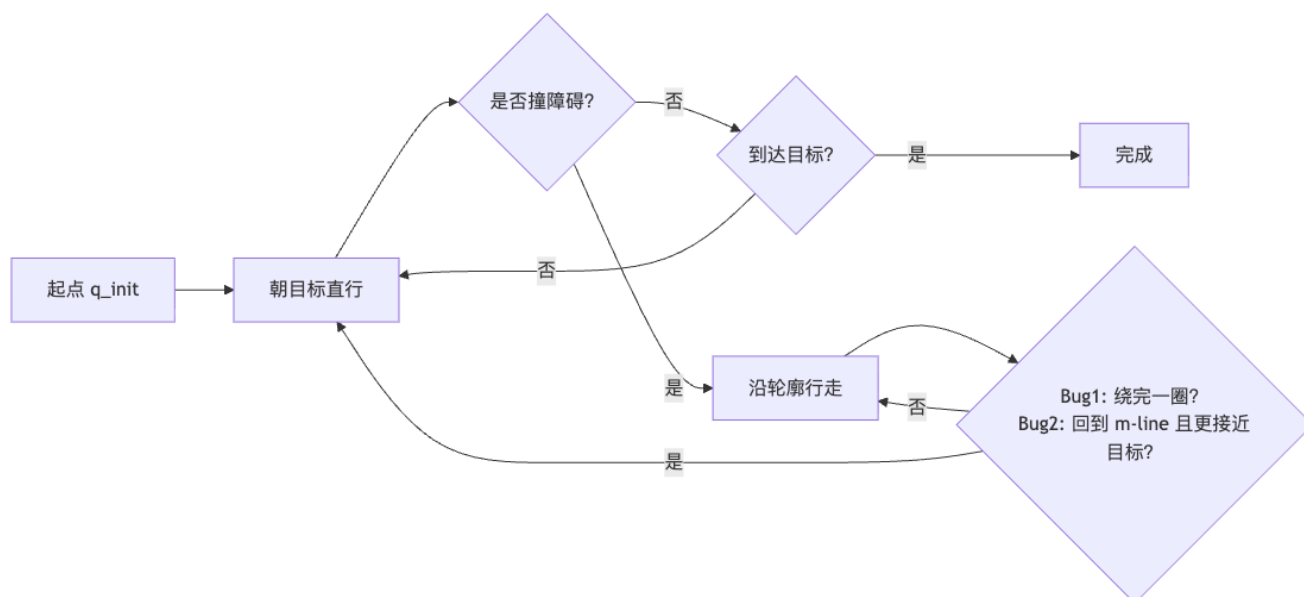
Bug 2 ——“m-line”启发

- 起点到目标的直线叫 m-line。
- 沿 m-line 直走; 撞到障碍后沿轮廓走; 一旦回到 m-line 且比起点更接近目标, 重新沿 m-line 走。

保证: Bug 2 也完备 (当 m-line 与每个障碍轮廓的交点都被正确处理时)。

[WARN] Bug Trap (虫陷阱)

若障碍形成“凹陷”的口袋形（如 U 字），Bug 1 仍能逃出（绕一整圈再选最近点）；而 Bug 2 在“m-line 反复穿越同一障碍”的情形下，需特殊判定（必须新接触点离目标更近），否则可能死循环。课件正是用 Bug Trap 演示了 Bug 2 的脆弱性，并通过“closer to the goal”补丁修复。



3.2 Grid Search (栅格搜索)

把 $\mathcal{C}_{\text{free}}$ 切成均匀网格，每个格子是一个节点，相邻格子有边，调用 A*。

- **Resolution-complete**：网格无限细化时完备；
- 性能完全取决于**格子分辨率 Δq** ：粗了过不去窄通道，细了内存指数爆炸。

[WARN] 维度诅咒

把 d 维 C-space 按每维 n 个格子离散化，总格子数 $= n^d$ 。 $d = 30$ 、 $n = 10$ 时 10^{30} ——远超宇宙原子总数。Grid Search 在 $d \geq 5$ 时基本失效。

3.3 Visibility Graph (可见图)

适用：2D，障碍是多边形。

构造：1. 节点 = 起点 + 终点 + 所有障碍多边形顶点；2. 两节点间有边 这条线段不穿过任何障碍内部。

关键性质：2D 多边形障碍下的最短路径必定沿可见图中的边——证明思路：在自由空间任何最短路径都可以“拉直”直到碰到某个障碍角点，进一步说明最短路径的所有转折点都是障碍顶点。

[WARN] 3D 中失效

在 $\mathbb{R}^3$ 里最短路径可以“绕过棱”，路径中间点不一定是多面体顶点。Visibility Graph 不再包含最短路。这是几何路标在维度上扩展失败的典型例子。

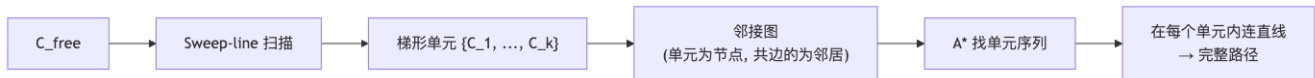
3.4 Cell Decomposition (单元分解)

思想：把 $\mathcal{C}_{\text{free}}$ 切成简单几何形状（如梯形）的不相交“单元”，使得任意单元内两点之间都很容易连线（保证无碰撞）。

Trapezoidal Decomposition (梯形分解)

- 用扫描线算法 (sweep line) 从左到右扫描;
- 在每个障碍顶点处发生四类事件之一:
 - Birth (新增上下两条扫描带)
 - Death/Merge (两条带合并)
 - Split (一条带分裂成两条)
 - Continue (带继续延伸)
- 每两个相邻梯形单元在交界面共享一段线段——成为图的边。

接着在单元邻接图上跑 A*；找到的单元序列再在每个单元内连直线即得真实路径。



Approximate Cell Decomposition (近似分解)

精确分解几何很复杂；近似方法用 Quadtree (2D) / Octree (3D) 等正则八叉树外切自由空间：– 大格子若完全在 C_{free} → 留下；– 若完全在 C_{obstacle} → 舍弃；– 若混合 → 递归细分。

代价是只保留外切空间，可能错过窄通道（变成 resolution-complete）。

3.5 维度诅咒小结

[WARN] 维度诅咒 (Curse of Dimensionality)

1. 体积指数膨胀：单位 d -立方体的 ϵ -覆盖网格点数 = $(1/\epsilon)^d$ ；
 2. 距离失效：高维中“最近邻”与“最远邻”距离差异趋于 0，邻居概念失效；
 3. 均匀采样无效： $d = 10$ 时要覆盖到 0.1 精度需 10^{10} 个样本。
- 这是几何方法的根本天花板——必须改用采样方法。

4. 基于采样的方法 (Sampling-based Approaches)

核心思想：放弃显式构造 C_{free} ，改为随机采样构型 + 判断有效性 + 增量连接。换来：– 高维仍可用 ($d \sim 10$ 量级) – 实现简单 – 概率完备

4.1 Probabilistic Roadmap (PRM)

[info] 定义 4.1 (PRM, Kavraki et al. 1996) 两阶段：1. 离线构图：均匀随机采样 n 个构型 $\{q_i\}$ ；丢弃落入 C_{obstacle} 的；每个保留的 q_i 与“邻居”用局部规划器（通常是直线段连接 + 离散点碰撞检测）尝试连边。2. 在线查询：给定 $(q_{\text{init}}, q_{\text{goal}})$ ，先连接到图中最近的节点，再在图上跑 A*。

两种邻居策略：– k -NN：连接每个点的 k 个最近邻 – R -neighborhood：连接每个点 R 半径内的所有点

Algorithm PRM_Construct(n , NeighborFn):

```
V ← ∅
while |V| < n:
    q ← SampleUniform(C)
    if q ∈ C_free: V.add(q)
E ← ∅
for q in V:
    for q' in NeighborFn(q, V):
        if LocalPlanner(q, q') succeeds:
```

```

        E.add((q, q'))
    return G = (V, E)

```

优势：多查询——同一张图可以反复回答不同 $(q_{\text{init}}, q_{\text{goal}})$ 对。适合静态环境。

Incremental PRM

不预设 n ，而是**边采边查**——每次加少量点，检查 q_{init} 与 q_{goal} 是否在同一连通分量；若是即停。

[NOTE] 直觉理解——PRM 在赌什么

PRM 赌的是“自由空间足够‘胖’，随机撒点能均匀覆盖”。在窄通道处 PRM 退化严重——通道宽度对采样需求是平方甚至立方关系。

4.2 Rapidly-Exploring Random Tree (RRT)

[!info] 定义 4.2 (RRT, LaValle 1998) 单查询树状结构：从 q_{init} 开始，每轮“朝一个随机方向稍微长一点”。

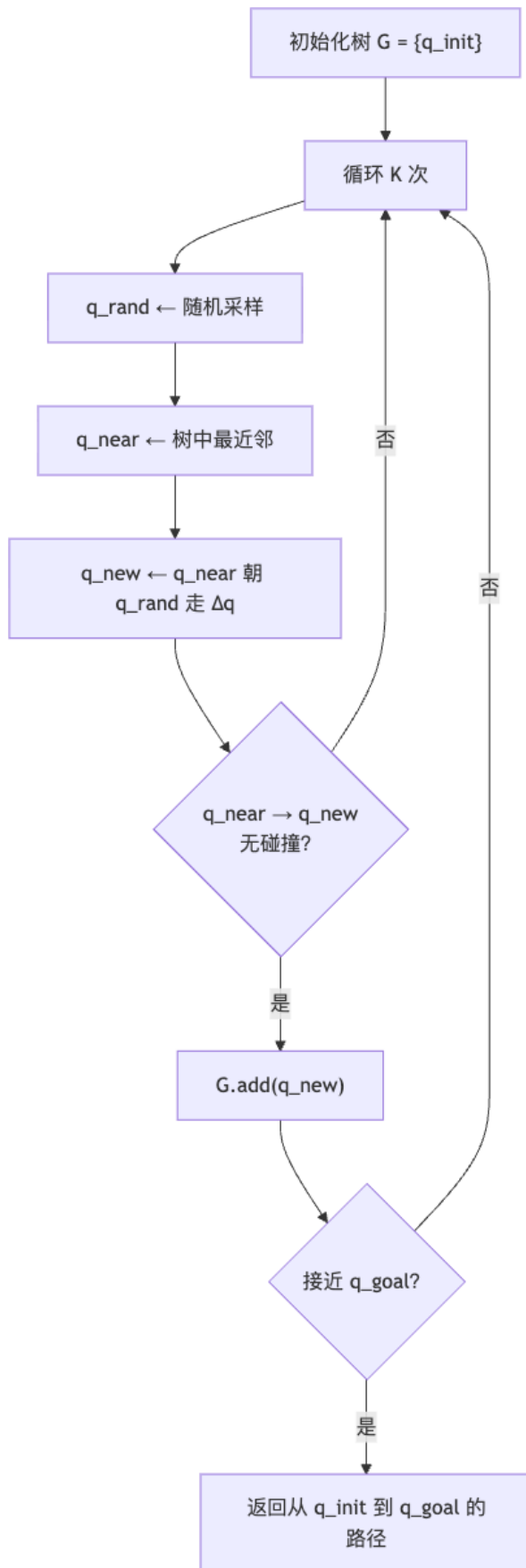
```

Algorithm BuildRRT( $q_{\text{init}}$ ,  $K$ ,  $\Delta q$ ):
    G.init( $q_{\text{init}}$ )
    for  $k = 1$  to  $K$ :
         $q_{\text{rand}} \leftarrow \text{RandConf}()$            # 在  $C$  中随机采样
         $q_{\text{near}} \leftarrow \text{NearestVertex}(q_{\text{rand}}, G)$    # 树中离  $q_{\text{rand}}$  最近的点
         $q_{\text{new}} \leftarrow \text{NewConf}(q_{\text{near}}, q_{\text{rand}}, \Delta q)$  # 从  $q_{\text{near}}$  朝  $q_{\text{rand}}$  走  $\Delta q$ 
        if  $(q_{\text{near}} \rightarrow q_{\text{new}}) \subset C_{\text{free}}$ :
            G.add_vertex( $q_{\text{new}}$ )
            G.add_edge( $q_{\text{near}}, q_{\text{new}}$ )
    return G

```

四步循环：采样 → 找最近邻 → 沿方向小步外推 → 碰撞检测后加边。

关键性质：树的生长会自然偏向最大的未探索区域——证明思路：被采样到的概率正比于在该方向上的 Voronoi 区域面积，越大的未探索区域对应越大的 Voronoi 区域，越容易被选中扩展。



优势： – 无预处理——适合单次查询 – 对非完整约束 (nonholonomic) 友好（如“只能左转的车”——限制 NewConf 即可） – 对机械臂、链状结构等高维问题有效

劣势： – 由“最大未探索区域”偏置——可能在简单区域反复花时间 – 路径远非最优（弯弯曲曲）

[EX] 例 4.3 (RRT 在两类机器人上)

– **Left-turn-only car** (自行车模型)：状态 (x, y, θ) ，控制 = 转向角，NewConf 只允许正向左转。RRT 仍可探索全平面（左转环绕即可）。 – **Articulated robot** (多关节臂)：高维 C-space，RRT 在 ms 内即可探出可行解，传统 Grid Search 完全无法处理。

4.3 RRT-Connect

[!info] 定义 4.4 (RRT-Connect, Kuffner & LaValle 2000) **双向生长**：同时从 q_{init} 和 q_{goal} 各长一棵 RRT；每轮交替“哪棵采样、另一棵尽全力靠拢”，直到两棵相遇。

“尽全力靠拢”：从 $q_{\text{near}}^{(B)}$ 朝 $q_{\text{new}}^{(A)}$ 反复走 Δq 直到碰障碍或到达。

收益： 在多数实际场景下，速度比单向 RRT 提升 5–10 倍。

4.4 路径后处理——Shortcutting

PRM/RRT 给出的路径很弯。**后处理**：1. 随机挑路径上两个点 q_i, q_j ；2. 直线段 $q_i \rightarrow q_j$ 若无碰撞，就用它替换中间所有段；3. 反复迭代。

效果： 在常见走廊场景下能把路径长度削减 30–60%。

4.5 渐近最优规划器

[!info] 定义 4.5 (渐近最优 Asymptotically-Optimal) 当采样数 $n \rightarrow \infty$ 时，规划器返回的路径以概率 1 收敛到全局最优解。

朴素 PRM/RRT 都不是渐近最优： – PRM with k -NN：连接被限死在 k 邻居内，永远无法用“远但更优”的边——非最优。 – PRM with R -neighborhood： R 不变 $\rightarrow$ 邻居数随密度上升，复杂度炸 $\rightarrow$ 实践不可用。 – RRT：一旦加入树就不再换父——子树结构被钉死，无法重连优化。

PRM* (Karaman & Frazzoli 2011)

让连接半径/邻居数随 n 自适应衰减：

$$R^*(n) \propto \left(\frac{\log n}{n} \right)^{1/d} \quad (5)$$

$$k^*(n) \propto \log n \quad (6)$$

其中 d 是 C-space 维度。直觉： – $R^*(n) \rightarrow 0$ ：点变密时半径变小——保证总连接数 $\Theta(n \log n)$ ； – $k^*(n) \rightarrow \infty$ ：但增长慢，单点连接复杂度 $O(\log n)$ 。

[NOTE] 直觉理解—— $R^*(n)$ 中 d 的来源

体积 $\propto R^d$ ；密度 $\propto n$ 。要让“半径 R 内期望邻居数” $\propto \log n$ (保证图连通的临界条件)，需要 $nR^d \propto \log n$ ，解得 $R \propto (\log n/n)^{1/d}$ 。

RRT* (Karaman & Frazzoli 2011)

在 RRT 基础上加两步 rewiring:

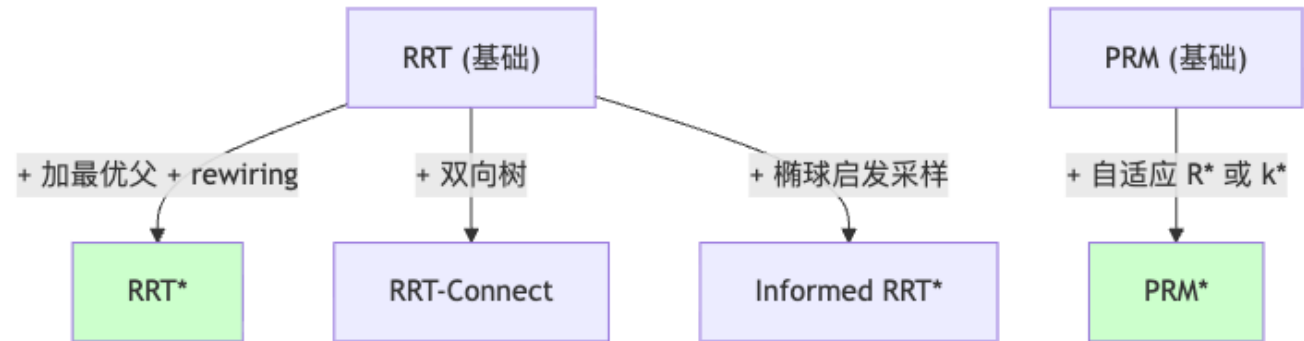
```
Algorithm RRT_Star_Step(G, q_rand, Δq, R*):
    q_near ← NearestVertex(q_rand, G)
    q_new ← NewConf(q_near, q_rand, Δq)
    if collision_free(q_near, q_new):
        Neighbors ← {q in G : ||q - q_new|| ≤ R*(n)}

        # Step A: 给 q_new 选最优父
        q_best ← q_near; c_best ← c(q_near) + d(q_near, q_new)
        for q in Neighbors:
            if collision_free(q, q_new) and c(q) + d(q, q_new) < c_best:
                q_best ← q; c_best ← c(q) + d(q, q_new)
        G.add_edge(q_best, q_new); c(q_new) ← c_best

        # Step B: 用 q_new 重连邻居
        for q in Neighbors:
            if collision_free(q_new, q) and c(q_new) + d(q_new, q) < c(q):
                G.replace_parent(q, q_new)
                c(q) ← c(q_new) + d(q_new, q)
                propagate_cost_to_descendants(q)
```

- Step A: 在邻域里选累积代价最小的父节点;
- Step B: 若通过 q_{new} 能让某个邻居代价更低, 则改父——必须递归更新所有后代的 $c(\cdot)$ 。

结果: RRT* 渐近最优; 每步只增加 $O(\log n)$ 额外代价。



4.6 RRT vs A*

维度	A* (图搜索)	RRT (采样)
适用 C-space	必须先离散化 (栅格/单元)	直接处理连续空间
维度	低维 (≤ 4)	中高维 (~ 10)
完备性	在图上完备/最优	概率完备
最优性	最优 (admissible h)	朴素版不最优; RRT* 渐近最优
单 vs 多查询	多查询友好	单查询友好

5. 在角色动画中的应用

5.1 NavMesh (导航网格)

游戏引擎中的标准实现：– 把可行走地面**预处理**成多边形 mesh，多边形间相邻即邻居；– 等价于一种针对地面拓扑优化的 **Cell Decomposition**；– 运行时跑 A*，配合 **string-pulling**（路径在多边形内拉直）输出折线路径。

[NOTE] NavMesh 是 Cell Decomposition 在游戏里的工程实现

离线生成 mesh，运行时 A* 在多边形邻接图上做规划——基本是 3.4 节内容的真实落地。

5.2 Motion Graph + A*

Motion Graph（运动图，回顾 lec05）：节点 = 动捕片段 (motion clip)，边 = 可平滑过渡的衔接点。

在运动图上跑 A*：– **状态** = (clip ID, 帧 index, 角色根坐标)；– **代价** = 累积位移误差 + 转向偏差；– **启发** = 当前根坐标到目标的欧氏距离。

得到的是一段“由真实动捕拼出来”的运动，而非物理仿真——**几何规划 + 数据驱动动画**的经典结合。

5.3 Yamane 2004 —操作任务运动合成

[EX] 应用：Synthesizing Animations of Human Manipulation Tasks

Yamane 等人用 **RRT in C-space of full-body humanoid** 规划“伸手抓取并搬运”的高维路径，再用 IK 把规划结果精化为人物动画。关键：用 RRT 处理高维 + 用 IK + smoothing 提升美感。

6. 群体仿真 (Crowds)

6.1 群体的本质特征

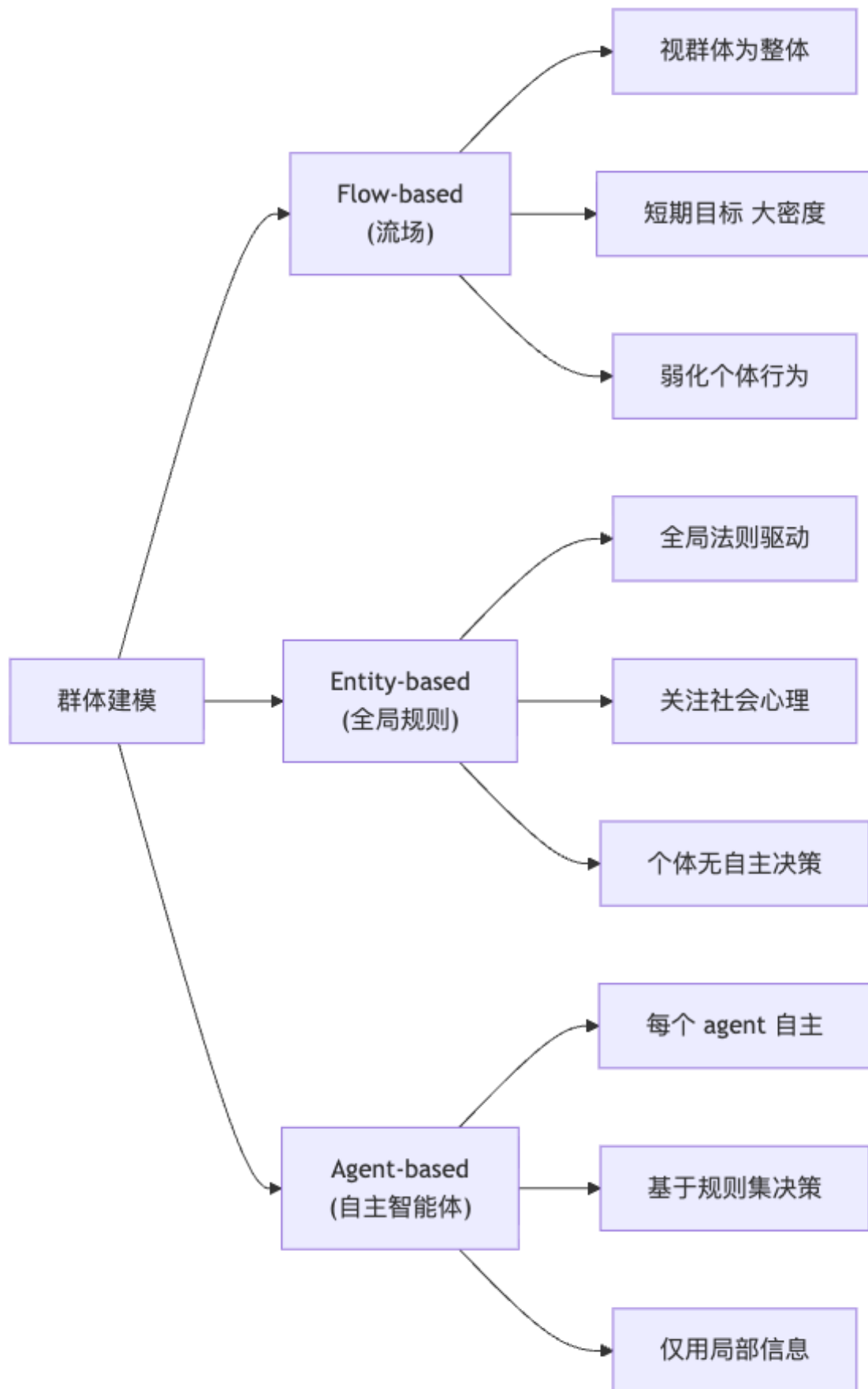
[!info] 定义 6.1 (Crowd) – 共享同一物理环境的多个个体；– 通常受**统一控制策略**（如“逃离火灾出口”）驱动；– 每个体有独特运动表现，但整体涌现可识别的**集体模式**；– 个体行为受**邻居**影响。

三层涌现：Individual Behavior → Group Behavior → Crowd Behavior。

群体仿真的工程目标：– 真实地操控人群，重现人类动态行为；– 应用：虚拟摄影、城市规划、疏散与维稳、军事训练、社会学研究。

核心挑战：– **模型可信度** (model plausibility)；– **速度与可扩展性** (speed & scalability)，百万级实时；– **运动细节与品质** (motion details & quality)。

6.2 三大建模范式



范式	视角	适用	决策能力
Flow-based Entity-based	把人群当流体，模拟流场 全局规则驱动每个实体	大密集人群、短期目标 中小型、研究 jamming / flocking	无个体决策 无自主决策
Agent-based	每个个体自主智能体	真实感人群、复杂行为	基于规则的局部决策

6.3 通用 Agent 仿真框架

每步循环：



每个 agent 的运动 = “目标速度由高层目标决定” + “实际速度受邻居/障碍修正”。

[WARN] 易错点——“I’ve fallen and can’t get up!”

仿真常忽略站起、倒下、跨过等异常行为。Agent 模型对正常行走鲁棒，但跌倒后的恢复行为需要专门的物理/动画支持，否则会卡死帧。

6.4 Boids 模型 (Reynolds 1987)

[!info] 定义 6.2 (Boids) “Bird-oid”——人工生命中最经典的群体行为模型。每个 boid 是一个粒子，每步用三条规则修正速度方向。

三规则

1. Separation (分离)：避开邻居以防碰撞

$$v_{\text{sep}}^{(i)} = - \sum_{j \in N(i)} \frac{p_j - p_i}{\|p_j - p_i\|^2}$$

2. Alignment (对齐)：朝邻居平均速度方向调整

$$v_{\text{ali}}^{(i)} = \frac{1}{|N(i)|} \sum_{j \in N(i)} v_j$$

3. Cohesion (聚合)：朝邻居重心方向移动

$$v_{\text{coh}}^{(i)} = \frac{1}{|N(i)|} \sum_{j \in N(i)} p_j - p_i$$

最终

$$v_{\text{new}}^{(i)} = v^{(i)} + w_s v_{\text{sep}}^{(i)} + w_a v_{\text{ali}}^{(i)} + w_c v_{\text{coh}}^{(i)} \quad (7)$$

[NOTE] 直觉理解——三规则的对称性

Separation 把鸟推开，Cohesion 把鸟拉拢；这两者的张力维持了“群体既不撞也不散”的稳定距离；Alignment 决定群体的朝向一致性——少了它，群体只能像气体一样无序聚合。三条简单规则就能涌现出鸟群、鱼群的复杂集体行为——这是涌现 (emergence) 的标志性例子。

[EX] 例 6.3 (Boids 鸟群飞行)

设 $N = 200$ 个体, 邻域半径 $r = 20$, $(w_s, w_a, w_c) = (1.5, 1.0, 1.0)$, 添加边界折返。运行数秒后即出现波纹状群体转向、临时分裂与重组——和真实棕鸟群高度相似。Wikipedia 的 Boids 演示页是经典参考。

6.5 Social Force Model (SFM)

[!info] 定义 6.4 (Social Force Model, Helbing & Molnár 1995) 把“社会心理影响”建模为个体间的相互作用力, 用牛顿第二定律驱动行人:

$$m_i \frac{dv_i}{dt} = F_i + G_i + H_i \quad (8)$$

– F_i : 自身驱动力 (subjective psychology) – G_i : 他人施加的心理排斥力 – H_i : 环境 (墙壁等) 的影响力

自身驱动力 F_i

$$F_i = m_i \cdot \frac{v_i^0(t) e_i^0(t) - v_i(t)}{\tau_i} \quad (9)$$

符号	含义
$v_i(t)$	行人 i 当前实际速度
$v_i^0(t)$	期望速度大小
$e_i^0(t)$	期望运动方向 (单位向量)
τ_i	加速到期望速度的时间常数
m_i	行人质量

物理含义: 当前速度若偏离期望速度 (方向或大小), 就产生一个把它拽回期望速度的力, 时间常数 τ_i 越小响应越快。

个体间排斥力 G_i

$$G_i = \sum_{j \neq i} f_{ij} = \sum_{j \neq i} (a_{ij} n_{ij} + b_{ij} t_{ij}) \quad (10)$$

其中

$$a_{ij} = A_i e^{\Delta_{ij}/B_i} + k g(\Delta_{ij}) \quad (11)$$

$$b_{ij} = \kappa g(\Delta_{ij}) \Delta v_{ji}^t \quad (12)$$

$$\Delta_{ij} = r_{ij} - d_{ij}, \quad g(x) = \begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$$

符号	含义
n_{ij}	从 j 指向 i 的单位法向量
t_{ij}	与 n_{ij} 正交的切向量

符号	含义
r_{ij}	两人物理半径之和
d_{ij}	两人圆心距
Δ_{ij}	距离差: > 0 表示发生身体接触
A_i, B_i	心理排斥强度与衰减距离
k, κ	物理接触刚度与摩擦系数
Δv_{ji}^t	切向相对速度 (用于身体摩擦)

物理含义: - **指数项** $A_i e^{\Delta_{ij}/B_i}$: 心理“不想靠太近”——指数衰减; - **物理项** $k g(\Delta_{ij})$: 身体接触后的硬弹性挤压; - **切向** b_{ij} : 身体接触时的摩擦阻力。

环境力 H_i

$$H_i = \sum_w f_{iw} = \sum_w (a_{iw} n_{iw} + b_{iw} t_{iw}) \quad (13)$$

其中

$$\begin{aligned} a_{iw} &= A_i e^{\Delta_{iw}/B_i} + k g(\Delta_{iw}) \\ b_{iw} &= -\kappa g(\Delta_{iw}) (v_i \cdot t_{iw}) \\ \Delta_{iw} &= r_i - d_{iw} \end{aligned}$$

结构与 G_i 同构, 仅把“另一人”换成“最近墙点”。

[NOTE] 直觉理解——SFM 为何能复现羊群效应

恐慌状态下 v_i^0 急剧增大 $\rightarrow$ 自身驱动力变强 $\rightarrow$ 推挤接触增多 $\rightarrow$ 切向摩擦 b_{ij} 抑制速度差 $\rightarrow$ 多人同时被卡在同一出口形成**人潮拱形 (arching)**, 进一步阻塞——这正是真实灾难场景的关键现象。Helbing 等 2000 年 Nature 论文正是用 SFM 复现并解释了这种“逃生悖论”。

[EX] 例 6.5 (建筑疏散仿真)

把矩形房间内 $N = 200$ 人初始化到随机位置, 期望方向都指向出口; 运行 SFM 可见: - 低 v^0 (理性慢走): 人群快速通过; - 高 v^0 (恐慌奔跑): 拱形阻塞、踩踏式波动、出口效率反而下降。这就是著名的“**faster-is-slower**”效应。

6.6 高阶话题

高保真群体的方法

- **逆问题求解**: 优化模型参数让仿真匹配实测数据;
- **隐式群体能量**: 用能量泛函刻画群体, 仿真就是能量极小化;
- **数据驱动**: 直接用真实人群轨迹训练模型;
- **视频驱动**: 从监控视频提取行走模式。

大规模实时仿真

- **并行化**: Boids/SFM 在 GPU 上每个 agent 一个线程;
- **群组渲染**: LOD (多分辨率)、impostor、instancing 把百万 agent 实时渲染出来;
- 课件演示了 Golaem Crowd 4 等业界工具, 以及“百万行人云端实时仿真”案例。

Crowd Editing (人群编辑)

- Kim et al. 2014 Interactive manipulation of large-scale crowd animation: 在已有人群序列上做局部时空形变 (drag-and-deform), 保持邻接关系不变。

7. 记号速查表

符号	含义
$\mathcal{C}$	构型空间 (Configuration space)
$\mathcal{C}_{\text{free}}$	自由构型集
$\mathcal{C}_{\text{obstacle}}$	碰撞构型集
q	一个构型 (点)
$q_{\text{init}}, q_{\text{goal}}$	起始 / 目标构型
$\mathcal{A}(q)$	机器人在构型 q 下占据的工作空间
$\mathcal{O}$	工作空间障碍
$g(x)$	A* 中从起点到 x 的已知代价
$h(x)$	A* 启发函数
$f(x) = g + h$	A* 优先级
$h^*(x)$	x 到目标的真实最优代价
Δq	RRT 每步的小步长
$q_{\text{rand}}, q_{\text{near}}, q_{\text{new}}$	RRT 三关键点
$R^*(n), k^*(n)$	PRM* / RRT* 的自适应连接半径 / 邻居数
$c(q)$	RRT* 中节点的累积代价
p_i, v_i	第 i 个 agent 的位置 / 速度
F_i, G_i, H_i	SFM 的自身驱动力 / 个体间排斥力 / 环境力
v_i^0, e_i^0	SFM 期望速度大小与方向
τ_i	SFM 加速时间常数
$\Delta_{ij} = r_{ij} - d_{ij}$	两 agent 半径差与圆心距之差
$g(x)$ (SFM)	ramp 函数 $\max(0, x)$

8. 中英术语对照表

中文	English	备注
工作空间	Workspace	物理 $\mathbb{R}^2/\mathbb{R}^3$ 空间
构型	Configuration	完全描述机器人位姿的参数
构型空间	Configuration Space (C-space)	所有构型的集合
自由空间	Free space, $\mathcal{C}_{\text{free}}$	无碰撞构型集
完备性	Completeness	有解必能找到
最优性	Optimality	解一定全局最优
概率完备	Probabilistic completeness	$n \rightarrow \infty$ 时找到解概率 $\rightarrow 1$
渐近最优	Asymptotically optimal	$n \rightarrow \infty$ 时收敛到最优
启发函数	Heuristic function	$h(x)$
可接受启发	Admissible heuristic	$h(x) \leq h^*(x)$
路标图	Roadmap	表示 $\mathcal{C}_{\text{free}}$ 连通性的图

中文	English	备注
可见图	Visibility Graph	连接互相能“看到”的顶点
单元分解	Cell Decomposition	把自由空间切成简单单元
梯形分解	Trapezoidal Decomposition	扫描线切梯形单元
概率路标图	Probabilistic Roadmap (PRM)	随机采样建图
快速扩展随机树	Rapidly-Exploring Random Tree (RRT)	单查询树方法
双向 RRT	RRT-Connect	起终点同时长树
重连	Rewiring	RRT* 中改父操作
后处理 / 拉直	Shortcutting	随机连段优化路径
维度诅咒	Curse of Dimensionality	高维体积爆炸
导航网格	NavMesh	游戏中的 cell decomposition 落地
运动图	Motion Graph	动捕片段拼接图
末端执行器	End-effector	机械臂末端、手脚等
运动片段	Motion Clip	一段连续动捕数据
群体	Crowd	多个共享环境的个体
流场方法	Flow-based approach	群体整体当流体
智能体方法	Agent-based approach	每个体自主决策
分离/对齐/聚合	Separation / Alignment / Cohesion	Boids 三规则
涌现	Emergence	简单规则产生复杂全局行为
社会力模型	Social Force Model (SFM)	心理力当物理力
期望速度	Desired velocity	$v_i^0 e_i^0$
羊群效应 / 快即是慢	Herding / faster-is-slower	恐慌反而降低疏散效率
拱形阻塞	Arching	出口处人潮形成的拱形堵塞

[TIP] 期末复习要点

1. **C-space** 定义: 能写出 $\mathcal{C}_{\text{obstacle}} = \{q \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}$ 并说明它把机器人变成“点”。2. **A*** 三件套: $f = g + h$ 、admissible 的定义、admissible 最优。3. **Visibility Graph** 在 2D 包含最短路径但 3D 不行, 理由是 3D 中最短路可“沿棱滑行”。4. **PRM** 与 **RRT** 的对比: 多查询 vs 单查询、双策略 (k -NN / R -邻域) 各自的最优性差异。5. **RRT 四步循环 + Voronoi 偏置**: 能写伪代码并解释“为何向最大未探索区生长”。6. **RRT* 的两步 rewiring**: 选最优父 + 用 q_{new} 改善邻居, 渐近最优。7. **PRM*/RRT* 自适应连接半径** $R^*(n) \propto (\log n/n)^{1/d}$ 的维度依赖。8. **Boids 三规则 + SFM 三力分解 + faster-is-slower 现象**。

9. 进一步阅读

几何运动规划 – LaValle, S. M. (2006). Planning Algorithms. Cambridge University Press. <http://planning.cs.uiuc.edu/> 圣经级教材, 免费在线。– Choset, H. et al. (2005). Principles of Robot Motion: Theory, Algorithms, and Implementations. MIT Press.

采样方法的奠基论文 – Kavraki, L. E., Svestka, P., Latombe, J.-C., & Overmars, M. H. (1996). Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces. IEEE T-RA. —PRM 原作。– LaValle, S. M. (1998). Rapidly-Exploring Random Trees: A New Tool for Path Planning. TR 98-11. —RRT 原作。– Kuffner, J. & LaValle, S. M. (2000). RRT-Connect: An Efficient Approach to Single-Query Path Planning. ICRA. —双向 RRT。– Karaman, S. & Frazzoli, E. (2011). Sampling-based Algorithms for Optimal Motion Planning. IJRR. —PRM* 与 RRT* 渐近最优证明。– Gammell, J. D., Srinivasa, S. S. & Barfoot, T. D. (2014). Informed RRT*. IROS. —用椭圆启发缩小采样域。

角色动画中的运动规划 – Yamane, K., Kuffner, J. & Hodgins, J. K. (2004). Synthesizing Animations of Human Manipulation Tasks. SIGGRAPH. —RRT 在全身人形操作任务上的应用。– Pettré, J., Laumond, J.-P. & Thal-

mann, D. (2005). A Navigation Graph for Real-time Crowd Animation on Multilayered and Uneven Terrain. VCROWDS. – Snook, G. (2000). Simplified 3D Movement and Pathfinding Using Navigation Meshes. Game Programming Gems.

群体仿真 – Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. SIGGRAPH. —Boids 原作。 – Helbing, D. & Molnár, P. (1995). Social Force Model for Pedestrian Dynamics. Phys. Rev. E 51(5), 4282–4286. —SFM 原作。 – Helbing, D., Farkas, I. & Vicsek, T. (2000). Simulating Dynamical Features of Escape Panic. Nature 407, 487–490. —faster-is-slower 经典实验。 – Treuille, A., Cooper, S. & Popovic, Z. (2006). Continuum Crowds. SIGGRAPH. —Flow-based 群体的代表作。 – van den Berg, J., Lin, M. & Manocha, D. (2008). Reciprocal Velocity Obstacles for Real-time Multi-agent Navigation. ICRA. —RVO, 主流避碰算法。 – Kim, J., Seol, Y., Kwon, T. & Lee, J. (2014). Interactive Manipulation of Large-Scale Crowd Animation. SIGGRAPH.

工程实践与工具 – Recast & Detour: 开源 NavMesh 库。 – Golaem Crowd / Massive Software: 影视特效中的群体仿真主流工具。